

Gravitational Lensing forward modeling

<https://gemini.google.com/app/c7b37f2e70207265>

User prompt: Write a new simulation of an astronomical phenomenon in HTML + vanilla Javascript, with all code embedded in the HTML file. This is a simulation of strong gravitational lensing by a galaxy cluster. This will illustrate the path followed by multiple rays of light from a distant galaxy passing by an intermediate cluster of galaxies. Mimic the appearance and the exaggerated scales used in <https://foothillastrosims.github.io/gravitational-lensing/>. But instead of showing only the rays that go to Earth, show ten rays all roughly going from the distant galaxy to Earth, but spread across a range of (exaggerated) angles.

Response: Here is a complete, self-contained HTML file containing the gravitational lensing simulation. It uses vanilla JavaScript and HTML5 Canvas to model the exaggerated paths of ten light rays passing from a background galaxy, bending around an intermediate galaxy cluster, and traveling toward Earth.

Save the code below as an .html file (e.g., gravitational_lensing.html) and open it in any modern web browser.

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Strong Gravitational Lensing Simulation</title>
  <style>
    :root {
      --bg-color: #0b0e14;
      --panel-bg: #151922;
      --panel-border: #2a3142;
      --accent-color: #00f0ff;
      --text-main: #e1e7ec;
      --text-muted: #8c9bae;
      --ray-color: #38bdf8;
      --ray-hit-color: #facc15;
      --virtual-color: #ef4444;
    }

    * {
      box-sizing: border-box;
      margin: 0;
      padding: 0;
    }

    body {
      font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial, sans-serif;
      background-color: var(--bg-color);
      color: var(--text-main);
      display: flex;
      flex-direction: column;
      min-height: 100vh;
      overflow-x: hidden;
    }

    header {
      background-color: var(--panel-bg);
      border-bottom: 1px solid var(--panel-border);
      padding: 1rem 1.5rem;
      text-align: center;
    }

    header h1 {
      font-size: 1.4rem;
      font-weight: 600;
      color: #ffff;
      letter-spacing: 0.5px;
    }

    header p {
      font-size: 0.85rem;
      color: var(--text-muted);
      margin-top: 0.25rem;
    }

    .main-container {
      display: flex;
      flex: 1;
      flex-direction: column;
  
```

```

    gap: 1rem;
    padding: 1rem;
    max-width: 1400px;
    margin: 0 auto;
    width: 100%;
}

@media (min-width: 1024px) {
  .main-container {
    flex-direction: row;
  }
}

.canvas-card {
  flex: 1;
  background-color: var(--panel-bg);
  border: 1px solid var(--panel-border);
  border-radius: 8px;
  padding: 1rem;
  display: flex;
  flex-direction: column;
  gap: 0.5rem;
  box-shadow: 0 4px 20px rgba(0, 0, 0, 0.4);
}

.canvas-wrapper {
  position: relative;
  width: 100%;
  height: 620px;
  background-color: #04060a;
  border-radius: 6px;
  overflow: hidden;
  border: 1px solid rgba(255, 255, 255, 0.05);
}

canvas {
  width: 100%;
  height: 100%;
  display: block;
}

.controls-panel {
  width: 100%;
  max-width: 360px;
  background-color: var(--panel-bg);
  border: 1px solid var(--panel-border);
  border-radius: 8px;
  padding: 1.25rem;
  display: flex;
  flex-direction: column;
  gap: 1.25rem;
  box-shadow: 0 4px 20px rgba(0, 0, 0, 0.4);
}

.control-group {
  display: flex;
  flex-direction: column;
  gap: 0.4rem;
}

.control-group label {
  font-size: 0.85rem;
  font-weight: 500;
  color: var(--text-main);
  display: flex;
  justify-content: space-between;
}

.control-group label span {
  color: var(--accent-color);
  font-family: monospace;
}

input[type="range"] {

```

```
-webkit-appearance: none;
width: 100%;
height: 6px;
border-radius: 3px;
background: #2a3142;
outline: none;
}

input[type="range"]::-webkit-slider-thumb {
  -webkit-appearance: none;
  width: 16px;
  height: 16px;
  border-radius: 50%;
  background: var(--accent-color);
  cursor: pointer;
  box-shadow: 0 0 8px var(--accent-color);
  transition: transform 0.1s ease;
}

input[type="range"]::-webkit-slider-thumb:hover {
  transform: scale(1.2);
}

.preset-buttons {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 0.5rem;
}

button {
  background-color: #1f2636;
  color: var(--text-main);
  border: 1px solid var(--panel-border);
  padding: 0.6rem 0.8rem;
  border-radius: 4px;
  font-size: 0.8rem;
  font-weight: 500;
  cursor: pointer;
  transition: all 0.2s ease;
}

button:hover {
  background-color: #2a334a;
  border-color: var(--accent-color);
  color: #fff;
}

button.active {
  background-color: rgba(0, 240, 255, 0.15);
  border-color: var(--accent-color);
  color: var(--accent-color);
}

.legend {
  border-top: 1px solid var(--panel-border);
  padding-top: 1rem;
  display: flex;
  flex-direction: column;
  gap: 0.5rem;
  font-size: 0.8rem;
}

.legend-item {
  display: flex;
  align-items: center;
  gap: 0.6rem;
  color: var(--text-muted);
}

.legend-color {
  width: 14px;
  height: 4px;
  border-radius: 2px;
}
```

```

.toggle-row {
  display: flex;
  align-items: center;
  justify-content: space-between;
  font-size: 0.85rem;
  color: var(--text-main);
}

.switch {
  position: relative;
  display: inline-block;
  width: 40px;
  height: 20px;
}

.switch input {
  opacity: 0;
  width: 0;
  height: 0;
}

.slider {
  position: absolute;
  cursor: pointer;
  top: 0; left: 0; right: 0; bottom: 0;
  background-color: #2a3142;
  transition: .2s;
  border-radius: 20px;
}

.slider:before {
  position: absolute;
  content: "";
  height: 14px;
  width: 14px;
  left: 3px;
  bottom: 3px;
  background-color: white;
  transition: .2s;
  border-radius: 50%;
}

input:checked + .slider {
  background-color: var(--accent-color);
}

input:checked + .slider:before {
  transform: translateX(20px);
}
</style>
</head>
<body>

<header>
  <h1>Gravitational Lensing by a Galaxy Cluster</h1>
  <p>Exaggerated ray tracing of light deflection caused by curved spacetime</p>
</header>

<div class="main-container">
  <div class="canvas-card">
    <div class="canvas-wrapper">
      <canvas id="simCanvas"></canvas>
    </div>
  </div>

  <div class="controls-panel">
    <div class="control-group">
      <label for="sliderMass">Cluster Mass ($M$): <span id="valMass">1.0 × 1015 M⊙</span></label>
      <input type="range" id="sliderMass" min="0" max="2.5" step="0.05" value="1.2">
    </div>

    <div class="control-group">
      <label for="sliderLensDist">Cluster Distance ($d_L$): <span id="valLensDist">4.0 Gly</span></label>

```

```

    <input type="range" id="sliderLensDist" min="1.5" max="7.0" step="0.1" value="4.0">
  </div>

  <div class="control-group">
    <label for="sliderSourceOffset">Galaxy Alignment Offset: <span id="valSourceOffset">0.05 Gly</span></label>
    <input type="range" id="sliderSourceOffset" min="-0.8" max="0.8" step="0.02" value="0.05">
  </div>

  <div class="control-group">
    <label for="sliderFanSpread">Ray Fan Angular Spread: <span id="valFanSpread">28°</span></label>
    <input type="range" id="sliderFanSpread" min="10" max="50" step="1" value="28">
  </div>

  <div class="control-group">
    <label>Presets</label>
    <div class="preset-buttons">
      <button id="btnEinstein">Einstein Ring</button>
      <button id="btnDoubleArc">Double Arcs</button>
      <button id="btnOffAxis">Off-Axis Lensing</button>
      <button id="btnNoMass">No Cluster</button>
    </div>
  </div>

  <div class="toggle-row">
    <span>Animate Photons</span>
    <label class="switch">
      <input type="checkbox" id="toggleParticles" checked>
      <span class="slider"></span>
    </label>
  </div>

  <div class="toggle-row">
    <span>Show Virtual Tangents</span>
    <label class="switch">
      <input type="checkbox" id="toggleTangents" checked>
      <span class="slider"></span>
    </label>
  </div>

  <div class="legend">
    <div class="legend-item">
      <div class="legend-color" style="background: #38bdf8;"></div>
      <span>Light rays bypassing Earth</span>
    </div>
    <div class="legend-item">
      <div class="legend-color" style="background: #facc15;"></div>
      <span>Light rays captured by Earth</span>
    </div>
    <div class="legend-item">
      <div class="legend-color" style="border: 1px dashed #ef4444; background: transparent;"></div>
      <span>Apparent straight-line paths (Virtual Images)</span>
    </div>
  </div>
</div>

<script>
const canvas = document.getElementById('simCanvas');
const ctx = canvas.getContext('2d');

// Controls
const sliderMass = document.getElementById('sliderMass');
const sliderLensDist = document.getElementById('sliderLensDist');
const sliderSourceOffset = document.getElementById('sliderSourceOffset');
const sliderFanSpread = document.getElementById('sliderFanSpread');

const valMass = document.getElementById('valMass');
const valLensDist = document.getElementById('valLensDist');
const valSourceOffset = document.getElementById('valSourceOffset');
const valFanSpread = document.getElementById('valFanSpread');

const toggleParticles = document.getElementById('toggleParticles');
const toggleTangents = document.getElementById('toggleTangents');

```

```

const btnEinstein = document.getElementById('btnEinstein');
const btnDoubleArc = document.getElementById('btnDoubleArc');
const btnOffAxis = document.getElementById('btnOffAxis');
const btnNoMass = document.getElementById('btnNoMass');

// Physics & Layout Constants
const NUM_RAYS = 10;
const TOTAL_DISTANCE_GLY = 8.0; // Distance Earth to Distant Galaxy in Giga-light-years
let animFrame = null;
let particleOffset = 0;

// Canvas Dimensions
let width = 0;
let height = 0;
let sideViewHeight = 0;
let skyViewHeight = 0;

// Starfield Background
const stars = [];
function generateStars(count) {
  stars.length = 0;
  for (let i = 0; i < count; i++) {
    stars.push({
      x: Math.random(),
      y: Math.random(),
      size: Math.random() * 1.5 + 0.3,
      alpha: Math.random() * 0.7 + 0.3
    });
  }
}
generateStars(180);

function resizeCanvas() {
  const rect = canvas.parentElement.getBoundingClientRect();
  const dpr = window.devicePixelRatio || 1;
  width = rect.width;
  height = rect.height;

  canvas.width = width * dpr;
  canvas.height = height * dpr;
  ctx.scale(dpr, dpr);

  sideViewHeight = height * 0.65;
  skyViewHeight = height - sideViewHeight;
}

window.addEventListener('resize', resizeCanvas);
resizeCanvas();

// Simulation Math / Calculation
function computeTrajectories() {
  const M = parseFloat(sliderMass.value);
  const dL = parseFloat(sliderLensDist.value);
  const sourceYOffsetGly = parseFloat(sliderSourceOffset.value);
  const fanSpreadDeg = parseFloat(sliderFanSpread.value);

  // Coordinates mapping for Side View
  const marginX = 80;
  const centerY = sideViewHeight / 2;

  const earthX = marginX;
  const earthY = centerY;

  const sourceX = width - marginX;
  // Convert source Y offset (Gly) to pixels
  const pxPerGly = (sourceX - earthX) / TOTAL_DISTANCE_GLY;
  const sourceY = centerY + (sourceYOffsetGly * pxPerGly);

  const lensX = earthX + (dL * pxPerGly);
  const lensY = centerY;

  // Telescope aperture size at Earth
  const earthRadius = 18;

```

```

const rays = [];
const fanSpreadRad = (fanSpreadDeg * Math.PI / 180);

// Calculate initial direction towards Earth center point
const baseAngle = Math.atan2(earthY - sourceY, earthX - sourceX);

for (let i = 0; i < NUM_RAYS; i++) {
  // Uniformly distribute angles across fan spread
  const frac = (i / (NUM_RAYS - 1)) - 0.5;
  const angle = baseAngle + (frac * fanSpreadRad);

  const rayPath = [];
  let currX = sourceX;
  let currY = sourceY;

  let vx = Math.cos(angle);
  let vy = Math.sin(angle);

  rayPath.push({ x: currX, y: currY });

  const dt = 3.0; // Step size
  let hitsEarth = false;
  let missed = false;

  // Trace light ray through galaxy cluster potential
  while (currX > earthX && !missed) {
    // Gravitational Deflection calculation
    if (M > 0) {
      const dx = currX - lensX;
      const dy = currY - lensY;
      const r2 = dx * dx + dy * dy;

      // Softened center mass halo acceleration
      const coreRadius2 = 1200;
      const softFactor = Math.pow(r2 + coreRadius2, 1.2);

      // Deflection strength (scaled for visual exaggeration)
      const G_scale = 180000 * M;
      const ax = -G_scale * dx / softFactor;
      const ay = -G_scale * dy / softFactor;

      vx += ax * dt;
      vy += ay * dt;

      // Re-normalize velocity to maintain constant speed of light
      const speed = Math.hypot(vx, vy);
      vx /= speed;
      vy /= speed;
    }

    currX += vx * dt;
    currY += vy * dt;
    rayPath.push({ x: currX, y: currY });

    // Check boundary conditions
    if (currY < -100 || currY > sideViewHeight + 100) {
      missed = true;
    }
  }

  // Final intersection at Earth plane
  const finalPoint = rayPath[rayPath.length - 1];
  const distToEarthCenter = Math.abs(finalPoint.y - earthY);
  if (distToEarthCenter <= earthRadius) {
    hitsEarth = true;
  }

  // Calculate apparent virtual image back-projection along final tangent vector
  const finalVx = vx;
  const finalVy = vy;

  // Extend line backward to the Source plane (x = sourceX)
  const tVirtual = (sourceX - finalPoint.x) / finalVx;
  const virtualY = finalPoint.y + finalVy * tVirtual;

```

```

    rays.push({
      path: rayPath,
      hitsEarth: hitsEarth,
      finalAngle: Math.atan2(finalVy, finalVx),
      virtualY: virtualY,
      endPoint: finalPoint
    });
  }

  return {
    earthX, earthY, earthRadius,
    lensX, lensY, dL,
    sourceX, sourceY, sourceYOffsetGly,
    rays, M
  };
}

// Rendering Logic
function draw() {
  ctx.clearRect(0, 0, width, height);

  const sim = computeTrajectories();

  // 1. Draw Starfield Background
  ctx.fillStyle = "#ffffff";
  stars.forEach(s => {
    ctx.globalAlpha = s.alpha;
    ctx.fillRect(s.x * width, s.y * height, s.size, s.size);
  });
  ctx.globalAlpha = 1.0;

  // 2. Draw Side-View Simulation Boundary & Separator
  ctx.strokeStyle = "#2a3142";
  ctx.lineWidth = 1;
  ctx.beginPath();
  ctx.moveTo(0, sideViewHeight);
  ctx.lineTo(width, sideViewHeight);
  ctx.stroke();

  // Side View Label
  ctx.fillStyle = "#8c9bae";
  ctx.font = "12px sans-serif";
  ctx.fillText("SIDE VIEW (CROSS-SECTION)", 15, 22);

  // --- DRAW OBJECTS IN SIDE VIEW ---

  // A. Distant Galaxy (Source)
  ctx.save();
  ctx.translate(sim.sourceX, sim.sourceY);

  // Outer Glow
  const galGlow = ctx.createRadialGradient(0, 0, 2, 0, 0, 30);
  galGlow.addColorStop(0, "rgba(255, 200, 100, 0.9)");
  galGlow.addColorStop(0.4, "rgba(238, 100, 30, 0.4)");
  galGlow.addColorStop(1, "rgba(0, 0, 0, 0)");
  ctx.fillStyle = galGlow;
  ctx.beginPath();
  ctx.arc(0, 0, 30, 0, Math.PI * 2);
  ctx.fill();

  // Spiral Galaxy Ellipse Body
  ctx.rotate(-0.3);
  ctx.fillStyle = "#ffd59e";
  ctx.beginPath();
  ctx.ellipse(0, 0, 22, 7, 0, 0, Math.PI * 2);
  ctx.fill();

  ctx.fillStyle = "#ffffff";
  ctx.beginPath();
  ctx.arc(0, 0, 4, 0, Math.PI * 2);
  ctx.fill();
  ctx.restore();

```



```

ctx.fillStyle = "#e1e7ec";
ctx.font = "12px sans-serif";
ctx.textAlign = "center";
ctx.fillText("Distant Galaxy", sim.sourceX, sim.sourceY + 38);
ctx.fillStyle = "#8c9bae";
ctx.font = "11px monospace";
ctx.fillText(`(8.0 Gly)`, sim.sourceX, sim.sourceY + 52);

// B. Intermediate Galaxy Cluster (Lens)
if (sim.M > 0) {
  ctx.save();
  ctx.translate(sim.lensX, sim.lensY);

  // Dark Matter Halo (Glowing Purple/Violet)
  const haloRadius = 40 + sim.M * 25;
  const haloGlow = ctx.createRadialGradient(0, 0, 5, 0, 0, haloRadius);
  haloGlow.addColorStop(0, "rgba(168, 85, 247, 0.45)");
  haloGlow.addColorStop(0.5, "rgba(126, 34, 206, 0.2)");
  haloGlow.addColorStop(1, "rgba(0, 0, 0, 0)");
  ctx.fillStyle = haloGlow;
  ctx.beginPath();
  ctx.arc(0, 0, haloRadius, 0, Math.PI * 2);
  ctx.fill();

  // Cluster Galaxies (Member Galaxies)
  const clusterMemberCoords = [
    {x: 0, y: 0, r: 8},
    {x: -15, y: -10, r: 5},
    {x: 18, y: 12, r: 6},
    {x: 12, y: -16, r: 4.5},
    {x: -18, y: 14, r: 5},
    {x: -6, y: 20, r: 4}
  ];

  clusterMemberCoords.forEach(m => {
    const mGlow = ctx.createRadialGradient(m.x, m.y, 0, m.x, m.y, m.r * 2.5);
    mGlow.addColorStop(0, "#fff5d6");
    mGlow.addColorStop(0.5, "rgba(245, 158, 11, 0.6)");
    mGlow.addColorStop(1, "rgba(0, 0, 0, 0)");
    ctx.fillStyle = mGlow;
    ctx.beginPath();
    ctx.arc(m.x, m.y, m.r * 2.5, 0, Math.PI * 2);
    ctx.fill();

    ctx.fillStyle = "#ffffff";
    ctx.beginPath();
    ctx.arc(m.x, m.y, m.r * 0.7, 0, Math.PI * 2);
    ctx.fill();
  });

  ctx.restore();

  ctx.fillStyle = "#e1e7ec";
  ctx.font = "12px sans-serif";
  ctx.fillText("Galaxy Cluster", sim.lensX, sim.lensY + haloRadius + 18);
  ctx.fillStyle = "#8c9bae";
  ctx.font = "11px monospace";
  ctx.fillText(`(${sim.dL.toFixed(1)} Gly)`, sim.lensX, sim.lensY + haloRadius + 32);
}

// C. Earth / Telescope
ctx.save();
ctx.translate(sim.earthX, sim.earthY);

// Earth Globe
const earthGrad = ctx.createRadialGradient(-3, -3, 1, 0, 0, 12);
earthGrad.addColorStop(0, "#60a5fa");
earthGrad.addColorStop(0.7, "#1d4ed8");
earthGrad.addColorStop(1, "#1e3a8a");
ctx.fillStyle = earthGrad;
ctx.beginPath();
ctx.arc(0, 0, 12, 0, Math.PI * 2);
ctx.fill();

```

```

// Earth Atmosphere Glow
ctx.strokeStyle = "rgba(96, 165, 250, 0.5)";
ctx.lineWidth = 2;
ctx.beginPath();
ctx.arc(0, 0, 13.5, 0, Math.PI * 2);
ctx.stroke();

// Detection aperture ring
ctx.strokeStyle = "rgba(250, 204, 21, 0.4)";
ctx.setLineDash([3, 3]);
ctx.beginPath();
ctx.arc(0, 0, sim.earthRadius, 0, Math.PI * 2);
ctx.stroke();
ctx.setLineDash([]);

ctx.restore();

ctx.fillStyle = "#e1e7ec";
ctx.font = "12px sans-serif";
ctx.fillText("Earth / Observer", sim.earthX, sim.earthY + 38);

// --- DRAW 10 LIGHT RAYS & VIRTUAL TANGENTS ---

sim.rays.forEach((ray, idx) => {
  const isHit = ray.hitsEarth;

  // 1. Draw Virtual Tangent Lines (Apparent straight back-projection)
  if (toggleTangents.checked && isHit) {
    ctx.strokeStyle = "rgba(239, 68, 68, 0.6)";
    ctx.lineWidth = 1.2;
    ctx.setLineDash([4, 4]);

    ctx.beginPath();
    ctx.moveTo(sim.earthX, sim.earthY);
    ctx.lineTo(sim.sourceX, ray.virtualY);
    ctx.stroke();
    ctx.setLineDash([]);

    // Virtual Image Dot at Source Plane
    ctx.fillStyle = "#ef4444";
    ctx.beginPath();
    ctx.arc(sim.sourceX, ray.virtualY, 3.5, 0, Math.PI * 2);
    ctx.fill();
  }

  // 2. Draw Curved Light Ray Path
  ctx.beginPath();
  ctx.moveTo(ray.path[0].x, ray.path[0].y);
  for (let p = 1; p < ray.path.length; p++) {
    ctx.lineTo(ray.path[p].x, ray.path[p].y);
  }

  if (isHit) {
    ctx.strokeStyle = "#facc15"; // Bright Yellow for rays hitting Earth
    ctx.lineWidth = 2.2;
    ctx.shadowColor = "#facc15";
    ctx.shadowBlur = 8;
  } else {
    ctx.strokeStyle = "rgba(56, 189, 248, 0.45)"; // Soft Cyan for bypassed rays
    ctx.lineWidth = 1.2;
    ctx.shadowBlur = 0;
  }
  ctx.stroke();
  ctx.shadowBlur = 0; // Reset glow

  // 3. Photon Particles Moving along Rays
  if (toggleParticles.checked) {
    const pathLen = ray.path.length;
    // Offset each ray's particle slightly
    const progress = (particleOffset + (idx / NUM_RAYS)) % 1.0;
    const pointIdx = Math.floor(progress * (pathLen - 1));
    const pt = ray.path[pointIdx];

    if (pt) {

```

```

        ctx.fillStyle = isHit ? "#ffffff" : "#00f0ff";
        ctx.beginPath();
        ctx.arc(pt.x, pt.y, isHit ? 2.8 : 2.0, 0, Math.PI * 2);
        ctx.fill();
    }
}
});

// --- DRAW SKY VIEW (OBSERVER'S TELESCOPE VIEW AT BOTTOM) ---

const skyViewY = sideViewHeight;

// Background for sky view
ctx.fillStyle = "#020305";
ctx.fillRect(0, skyViewY, width, skyViewHeight);

ctx.fillStyle = "#8c9bae";
ctx.font = "12px sans-serif";
ctx.textAlign = "left";
ctx.fillText("EARTH TELESCOPE SKY VIEW (WHAT OBSERVER SEES)", 15, skyViewY + 22);

const skyCenterX = width / 2;
const skyCenterY = skyViewY + (skyViewHeight / 2) + 5;
const skyScale = 0.85;

// Telescope Field of View Circle
const fovRadius = Math.min(skyViewHeight / 2 - 20, 90);
ctx.strokeStyle = "#2a3142";
ctx.lineWidth = 1.5;
ctx.beginPath();
ctx.arc(skyCenterX, skyCenterY, fovRadius, 0, Math.PI * 2);
ctx.stroke();

// Crosshairs
ctx.strokeStyle = "rgba(255, 255, 255, 0.08)";
ctx.setLineDash([2, 4]);
ctx.beginPath();
ctx.moveTo(skyCenterX - fovRadius - 10, skyCenterY);
ctx.lineTo(skyCenterX + fovRadius + 10, skyCenterY);
ctx.moveTo(skyCenterX, skyCenterY - fovRadius - 10);
ctx.lineTo(skyCenterX, skyCenterY + fovRadius + 10);
ctx.stroke();
ctx.setLineDash([]);

// Draw Cluster in Sky Center
if (sim.M > 0) {
    ctx.fillStyle = "rgba(245, 158, 11, 0.8)";
    ctx.beginPath();
    ctx.arc(skyCenterX, skyCenterY, 7, 0, Math.PI * 2);
    ctx.fill();

    ctx.fillStyle = "rgba(245, 158, 11, 0.5)";
    ctx.beginPath();
    ctx.arc(skyCenterX - 12, skyCenterY - 8, 4, 0, Math.PI * 2);
    ctx.arc(skyCenterX + 14, skyCenterY + 10, 5, 0, Math.PI * 2);
    ctx.arc(skyCenterX + 8, skyCenterY - 12, 3.5, 0, Math.PI * 2);
    ctx.fill();
}

// Draw Unlensed True Position (Faint Dashed Reference Circle)
const trueYOffset = (sim.sourceY - (sideViewHeight / 2)) * skyScale;
ctx.strokeStyle = "rgba(255, 255, 255, 0.2)";
ctx.setLineDash([2, 2]);
ctx.beginPath();
ctx.arc(skyCenterX, skyCenterY + trueYOffset, 6, 0, Math.PI * 2);
ctx.stroke();
ctx.setLineDash([]);

// Draw Lensed Arcs / Images formed by rays reaching Earth
const capturedRays = sim.rays.filter(r => r.hitsEarth);

if (capturedRays.length > 0) {
    capturedRays.forEach(r => {

```

```

const apparentYOffset = (r.virtualY - (sideViewHeight / 2)) * skyScale;
const imageY = skyCenterY + apparentYOffset;

// Compute arc radius from center
const dy = imageY - skyCenterY;
const arcR = Math.abs(dy);

ctx.save();
ctx.strokeStyle = "#facc15";
ctx.fillStyle = "#ffd59e";
ctx.lineWidth = 3.5;
ctx.shadowColor = "#facc15";
ctx.shadowBlur = 10;

if (arcR < 3) {
  // Einstein Ring Case
  ctx.beginPath();
  ctx.arc(skyCenterX, skyCenterY, 20, 0, Math.PI * 2);
  ctx.stroke();
} else {
  // Gravitational Arc
  const isTop = dy < 0;
  const startAng = isTop ? -Math.PI * 0.85 : Math.PI * 0.15;
  const endAng = isTop ? -Math.PI * 0.15 : Math.PI * 0.85;

  ctx.beginPath();
  ctx.arc(skyCenterX, skyCenterY, arcR, startAng, endAng);
  ctx.stroke();
}

ctx.restore();
});

// Apparent Image count display
ctx.fillStyle = "#facc15";
ctx.font = "11px monospace";
ctx.textAlign = "right";
ctx.fillText(`Captured Rays: ${capturedRays.length} / 10`, width - 20, skyViewY + 22);
ctx.fillText(`Images Formed: ${capturedRays.length > 0 ? (Math.abs(sim.sourceYOffsetGly) < 0.08 ? "Eir"
} else {
  ctx.fillStyle = "#8c9bae";
  ctx.font = "11px monospace";
  ctx.textAlign = "right";
  ctx.fillText(`Captured Rays: 0 / 10 (No rays hit Earth)`, width - 20, skyViewY + 22);
}

// Update Slider Value Displays
valMass.textContent = `${sim.M.toFixed(2)} × 1015 M⊙`;
valLensDist.textContent = `${sim.dL.toFixed(1)} Gly`;
valSourceOffset.textContent = `${sim.sourceYOffsetGly.toFixed(2)} Gly`;
valFanSpread.textContent = `${sliderFanSpread.value}°`;

// Particle animation step
if (toggleParticles.checked) {
  particleOffset = (particleOffset + 0.008) % 1.0;
}
}

// Animation Loop
function animate() {
  draw();
  animFrame = requestAnimationFrame(animate);
}

// Preset Handlers
btnEinstein.addEventListener('click', () => {
  sliderMass.value = 1.4;
  sliderLensDist.value = 4.0;
  sliderSourceOffset.value = 0.0;
  sliderFanSpread.value = 28;
  updateActivePreset(btnEinstein);
});

btnDoubleArc.addEventListener('click', () => {

```

```

        sliderMass.value = 1.5;
        sliderLensDist.value = 4.0;
        sliderSourceOffset.value = 0.12;
        sliderFanSpread.value = 32;
        updateActivePreset(btnDoubleArc);
    });

    btnOffAxis.addEventListener('click', () => {
        sliderMass.value = 1.8;
        sliderLensDist.value = 4.5;
        sliderSourceOffset.value = 0.45;
        sliderFanSpread.value = 35;
        updateActivePreset(btnOffAxis);
    });

    btnNoMass.addEventListener('click', () => {
        sliderMass.value = 0;
        sliderSourceOffset.value = 0;
        sliderFanSpread.value = 25;
        updateActivePreset(btnNoMass);
    });

    function updateActivePreset(activeBtn) {
        [btnEinstein, btnDoubleArc, btnOffAxis, btnNoMass].forEach(b => b.classList.remove('active'));
        if (activeBtn) activeBtn.classList.add('active');
    }

    // Input listeners
    [sliderMass, sliderLensDist, sliderSourceOffset, sliderFanSpread].forEach(elem => {
        elem.addEventListener('input', () => {
            updateActivePreset(null);
        });
    });

    // Initialize Simulation
    btnDoubleArc.click();
    animate();
</script>
</body>
</html>

```

Key Interactive Features

- **Ten Fan Rays:** Light rays are emitted across a 2D angular fan from the background galaxy towards Earth.
- **Spacetime Deflection:** Rays curve continuously as they pass through the softened gravitational potential of the galaxy cluster.
- **Ray Highlights & Virtual Tangents:** Rays that fall within Earth's telescope aperture turn **yellow**, while red dashed tangent lines project backward to show where observers perceive virtual images.
- **Observer Sky View:** The bottom panel visualizes what an observer on Earth sees through a telescope (e.g., Einstein rings or arcs).

Sources

1. [Foothill AstroSims Gravitational Lensing Simulator](#)

User prompt: This page does not load when I try to run it locally. Any ideas what the problem could be?

Response: The primary technical reason this page fails to load or freezes your browser tab is an **unbounded loop in the ray tracing physics function**. If a light ray is deflected too strongly by high gravity, its direction vector reverses ($v_x > 0$), causing the while (`currX > earthX`) condition to never resolve and freezing the main thread instantly.

Here are the four most common reasons local HTML files fail to render, along with the fixed code below:

Common Local Execution Problems

1. **Infinite Loop in Ray Tracing (Fixed Below):** Without a maximum step safety check, strongly curved rays can loop indefinitely during the initial calculation step before the page finishes rendering.
2. **Hidden File Extension (.html.txt):** Default Windows and macOS settings hide file extensions. Notepad or TextEdit often saves files as `lensing.html.txt`. Ensure "File name extensions" is checked in your file explorer view settings.
3. **Rich Text Formatting (macOS TextEdit):** TextEdit defaults to RTF format, replacing raw quotes with smart quotes (“ ”) and embedding hidden formatting tags. Use a plain text editor (VS Code, Sublime Text, Notepad++, or set TextEdit to *Format > Make Plain Text*).

4. **Canvas Zero-Sizing:** If opened before DOM dimensions initialize, canvas contexts can collapse.

Fixed & Fully Guarded HTML Code

The updated code below includes a step-limit guard (`maxSteps = 3000`) and spatial boundary bounds to prevent ray calculations from locking up the browser.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Strong Gravitational Lensing Simulation</title>
  <style>
    :root {
      --bg-color: #0b0e14;
      --panel-bg: #151922;
      --panel-border: #2a3142;
      --accent-color: #00f0ff;
      --text-main: #e1e7ec;
      --text-muted: #8c9bae;
    }

    * {
      box-sizing: border-box;
      margin: 0;
      padding: 0;
    }

    body {
      font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial, sans-serif;
      background-color: var(--bg-color);
      color: var(--text-main);
      display: flex;
      flex-direction: column;
      min-height: 100vh;
      overflow-x: hidden;
    }

    header {
      background-color: var(--panel-bg);
      border-bottom: 1px solid var(--panel-border);
      padding: 1rem 1.5rem;
      text-align: center;
    }

    header h1 {
      font-size: 1.4rem;
      font-weight: 600;
      color: #fff;
    }

    header p {
      font-size: 0.85rem;
      color: var(--text-muted);
      margin-top: 0.25rem;
    }

    .main-container {
      display: flex;
      flex: 1;
      flex-direction: column;
      gap: 1rem;
      padding: 1rem;
      max-width: 1400px;
      margin: 0 auto;
      width: 100%;
    }

    @media (min-width: 1024px) {
      .main-container {
        flex-direction: row;
      }
    }
  </style>
</head>
<body>
```

```
.canvas-card {
  flex: 1;
  background-color: var(--panel-bg);
  border: 1px solid var(--panel-border);
  border-radius: 8px;
  padding: 1rem;
  display: flex;
  flex-direction: column;
  gap: 0.5rem;
  box-shadow: 0 4px 20px rgba(0, 0, 0, 0.4);
}

.canvas-wrapper {
  position: relative;
  width: 100%;
  height: 620px;
  background-color: #04060a;
  border-radius: 6px;
  overflow: hidden;
  border: 1px solid rgba(255, 255, 255, 0.05);
}

canvas {
  width: 100%;
  height: 100%;
  display: block;
}

.controls-panel {
  width: 100%;
  max-width: 360px;
  background-color: var(--panel-bg);
  border: 1px solid var(--panel-border);
  border-radius: 8px;
  padding: 1.25rem;
  display: flex;
  flex-direction: column;
  gap: 1.25rem;
  box-shadow: 0 4px 20px rgba(0, 0, 0, 0.4);
}

.control-group {
  display: flex;
  flex-direction: column;
  gap: 0.4rem;
}

.control-group label {
  font-size: 0.85rem;
  font-weight: 500;
  color: var(--text-main);
  display: flex;
  justify-content: space-between;
}

.control-group label span {
  color: var(--accent-color);
  font-family: monospace;
}

input[type="range"] {
  -webkit-appearance: none;
  width: 100%;
  height: 6px;
  border-radius: 3px;
  background: #2a3142;
  outline: none;
}

input[type="range"]::-webkit-slider-thumb {
  -webkit-appearance: none;
  width: 16px;
  height: 16px;
}
```

```
border-radius: 50%;
background: var(--accent-color);
cursor: pointer;
}

.preset-buttons {
display: grid;
grid-template-columns: 1fr 1fr;
gap: 0.5rem;
}

button {
background-color: #1f2636;
color: var(--text-main);
border: 1px solid var(--panel-border);
padding: 0.6rem 0.8rem;
border-radius: 4px;
font-size: 0.8rem;
font-weight: 500;
cursor: pointer;
}

button:hover {
background-color: #2a334a;
border-color: var(--accent-color);
color: #fff;
}

button.active {
background-color: rgba(0, 240, 255, 0.15);
border-color: var(--accent-color);
color: var(--accent-color);
}

.legend {
border-top: 1px solid var(--panel-border);
padding-top: 1rem;
display: flex;
flex-direction: column;
gap: 0.5rem;
font-size: 0.8rem;
}

.legend-item {
display: flex;
align-items: center;
gap: 0.6rem;
color: var(--text-muted);
}

.legend-color {
width: 14px;
height: 4px;
border-radius: 2px;
}

.toggle-row {
display: flex;
align-items: center;
justify-content: space-between;
font-size: 0.85rem;
color: var(--text-main);
}

.switch {
position: relative;
display: inline-block;
width: 40px;
height: 20px;
}

.switch input {
opacity: 0;
width: 0;
}
```



```

    height: 0;
  }

  .slider {
    position: absolute;
    cursor: pointer;
    top: 0; left: 0; right: 0; bottom: 0;
    background-color: #2a3142;
    transition: .2s;
    border-radius: 20px;
  }

  .slider:before {
    position: absolute;
    content: "";
    height: 14px;
    width: 14px;
    left: 3px;
    bottom: 3px;
    background-color: white;
    transition: .2s;
    border-radius: 50%;
  }

  input:checked + .slider {
    background-color: var(--accent-color);
  }

  input:checked + .slider:before {
    transform: translateX(20px);
  }
</style>
</head>
<body>

<header>
  <h1>Gravitational Lensing by a Galaxy Cluster</h1>
  <p>Exaggerated ray tracing of light deflection caused by curved spacetime</p>
</header>

<div class="main-container">
  <div class="canvas-card">
    <div class="canvas-wrapper">
      <canvas id="simCanvas"></canvas>
    </div>
  </div>

  <div class="controls-panel">
    <div class="control-group">
      <label for="sliderMass">Cluster Mass (M): <span id="valMass">1.20 × 1015 M⊙</span></label>
      <input type="range" id="sliderMass" min="0" max="2.5" step="0.05" value="1.2">
    </div>

    <div class="control-group">
      <label for="sliderLensDist">Cluster Distance (dL): <span id="valLensDist">4.0 Gly</span></label>
      <input type="range" id="sliderLensDist" min="1.5" max="7.0" step="0.1" value="4.0">
    </div>

    <div class="control-group">
      <label for="sliderSourceOffset">Galaxy Alignment Offset: <span id="valSourceOffset">0.05 Gly</span></label>
      <input type="range" id="sliderSourceOffset" min="-0.8" max="0.8" step="0.02" value="0.05">
    </div>

    <div class="control-group">
      <label for="sliderFanSpread">Ray Fan Angular Spread: <span id="valFanSpread">28°</span></label>
      <input type="range" id="sliderFanSpread" min="10" max="50" step="1" value="28">
    </div>

    <div class="control-group">
      <label>Presets</label>
      <div class="preset-buttons">
        <button id="btnEinstein">Einstein Ring</button>
        <button id="btnDoubleArc" class="active">Double Arcs</button>
        <button id="btnOffAxis">Off-Axis Lensing</button>
      </div>
    </div>
  </div>
</div>

```

```

        <button id="btnNoMass">No Cluster</button>
    </div>
</div>

<div class="toggle-row">
    <span>Animate Photons</span>
    <label class="switch">
        <input type="checkbox" id="toggleParticles" checked>
        <span class="slider"></span>
    </label>
</div>

<div class="toggle-row">
    <span>Show Virtual Tangents</span>
    <label class="switch">
        <input type="checkbox" id="toggleTangents" checked>
        <span class="slider"></span>
    </label>
</div>

<div class="legend">
    <div class="legend-item">
        <div class="legend-color" style="background: #38bdf8;"></div>
        <span>Light rays bypassing Earth</span>
    </div>
    <div class="legend-item">
        <div class="legend-color" style="background: #facc15;"></div>
        <span>Light rays captured by Earth</span>
    </div>
    <div class="legend-item">
        <div class="legend-color" style="border: 1px dashed #ef4444; background: transparent;"></div>
        <span>Apparent straight-line paths (Virtual Images)</span>
    </div>
</div>
</div>
</div>

<script>
const canvas = document.getElementById('simCanvas');
const ctx = canvas.getContext('2d');

const sliderMass = document.getElementById('sliderMass');
const sliderLensDist = document.getElementById('sliderLensDist');
const sliderSourceOffset = document.getElementById('sliderSourceOffset');
const sliderFanSpread = document.getElementById('sliderFanSpread');

const valMass = document.getElementById('valMass');
const valLensDist = document.getElementById('valLensDist');
const valSourceOffset = document.getElementById('valSourceOffset');
const valFanSpread = document.getElementById('valFanSpread');

const toggleParticles = document.getElementById('toggleParticles');
const toggleTangents = document.getElementById('toggleTangents');

const btnEinstein = document.getElementById('btnEinstein');
const btnDoubleArc = document.getElementById('btnDoubleArc');
const btnOffAxis = document.getElementById('btnOffAxis');
const btnNoMass = document.getElementById('btnNoMass');

const NUM_RAYS = 10;
const TOTAL_DISTANCE_GLY = 8.0;
let particleOffset = 0;

let width = 0;
let height = 0;
let sideViewHeight = 0;
let skyViewHeight = 0;

const stars = [];
function generateStars(count) {
    stars.length = 0;
    for (let i = 0; i < count; i++) {
        stars.push({
            x: Math.random(),

```

```

        y: Math.random(),
        size: Math.random() * 1.5 + 0.3,
        alpha: Math.random() * 0.7 + 0.3
    });
}
}
generateStars(180);

function resizeCanvas() {
    const rect = canvas.parentElement.getBoundingClientRect();
    const dpr = window.devicePixelRatio || 1;
    width = rect.width || 800;
    height = rect.height || 620;

    canvas.width = width * dpr;
    canvas.height = height * dpr;
    ctx.scale(dpr, dpr);

    sideViewHeight = height * 0.65;
    skyViewHeight = height - sideViewHeight;
}

window.addEventListener('resize', resizeCanvas);
resizeCanvas();

function computeTrajectories() {
    const M = parseFloat(sliderMass.value);
    const dL = parseFloat(sliderLensDist.value);
    const sourceYOffsetGly = parseFloat(sliderSourceOffset.value);
    const fanSpreadDeg = parseFloat(sliderFanSpread.value);

    const marginX = 80;
    const centerY = sideViewHeight / 2;

    const earthX = marginX;
    const earthY = centerY;

    const sourceX = width - marginX;
    const pxPerGly = (sourceX - earthX) / TOTAL_DISTANCE_GLY;
    const sourceY = centerY + (sourceYOffsetGly * pxPerGly);

    const lensX = earthX + (dL * pxPerGly);
    const lensY = centerY;

    const earthRadius = 18;
    const rays = [];
    const fanSpreadRad = (fanSpreadDeg * Math.PI / 180);

    const baseAngle = Math.atan2(earthY - sourceY, earthX - sourceX);

    for (let i = 0; i < NUM_RAYS; i++) {
        const frac = (i / (NUM_RAYS - 1)) - 0.5;
        const angle = baseAngle + (frac * fanSpreadRad);

        const rayPath = [];
        let currX = sourceX;
        let currY = sourceY;

        let vx = Math.cos(angle);
        let vy = Math.sin(angle);

        rayPath.push({ x: currX, y: currY });

        const dt = 3.0;
        let hitsEarth = false;
        let missed = false;

        let steps = 0;
        const maxSteps = 2500; // Hard cap preventing freeze

        while (currX > earthX && !missed && steps < maxSteps) {
            steps++;

            if (M > 0) {

```

```

    const dx = currX - lensX;
    const dy = currY - lensY;
    const r2 = dx * dx + dy * dy;

    const coreRadius2 = 1200;
    const softFactor = Math.pow(r2 + coreRadius2, 1.2);

    const G_scale = 180000 * M;
    const ax = -G_scale * dx / softFactor;
    const ay = -G_scale * dy / softFactor;

    vx += ax * dt;
    vy += ay * dt;

    const speed = Math.hypot(vx, vy);
    if (speed > 0) {
        vx /= speed;
        vy /= speed;
    }
}

currX += vx * dt;
currY += vy * dt;
rayPath.push({ x: currX, y: currY });

if (currY < -100 || currY > sideViewHeight + 100 || currX > width + 200) {
    missed = true;
}
}

const finalPoint = rayPath[rayPath.length - 1];
const distToEarthCenter = Math.abs(finalPoint.y - earthY);
if (distToEarthCenter <= earthRadius) {
    hitsEarth = true;
}

const finalVx = vx;
const finalVy = vy;

const tVirtual = (sourceX - finalPoint.x) / (finalVx || -1);
const virtualY = finalPoint.y + finalVy * tVirtual;

rays.push({
    path: rayPath,
    hitsEarth: hitsEarth,
    finalAngle: Math.atan2(finalVy, finalVx),
    virtualY: virtualY,
    endPoint: finalPoint
});
}

return {
    earthX, earthY, earthRadius,
    lensX, lensY, dL,
    sourceX, sourceY, sourceYOffsetGly,
    rays, M
};
}

function draw() {
    ctx.clearRect(0, 0, width, height);

    const sim = computeTrajectories();

    // Background
    ctx.fillStyle = "#ffffff";
    stars.forEach(s => {
        ctx.globalAlpha = s.alpha;
        ctx.fillRect(s.x * width, s.y * height, s.size, s.size);
    });
    ctx.globalAlpha = 1.0;

    // Divider
    ctx.strokeStyle = "#2a3142";

```

```

ctx.lineWidth = 1;
ctx.beginPath();
ctx.moveTo(0, sideViewHeight);
ctx.lineTo(width, sideViewHeight);
ctx.stroke();

ctx.fillStyle = "#8c9bae";
ctx.font = "12px sans-serif";
ctx.fillText("SIDE VIEW (CROSS-SECTION)", 15, 22);

// Distant Galaxy
ctx.save();
ctx.translate(sim.sourceX, sim.sourceY);

const galGlow = ctx.createRadialGradient(0, 0, 2, 0, 0, 30);
galGlow.addColorStop(0, "rgba(255, 200, 100, 0.9)");
galGlow.addColorStop(0.4, "rgba(238, 100, 30, 0.4)");
galGlow.addColorStop(1, "rgba(0, 0, 0, 0)");
ctx.fillStyle = galGlow;
ctx.beginPath();
ctx.arc(0, 0, 30, 0, Math.PI * 2);
ctx.fill();

ctx.rotate(-0.3);
ctx.fillStyle = "#ffd59e";
ctx.beginPath();
ctx.ellipse(0, 0, 22, 7, 0, 0, Math.PI * 2);
ctx.fill();

ctx.fillStyle = "ffffff";
ctx.beginPath();
ctx.arc(0, 0, 4, 0, Math.PI * 2);
ctx.fill();
ctx.restore();

ctx.fillStyle = "#e1e7ec";
ctx.font = "12px sans-serif";
ctx.textAlign = "center";
ctx.fillText("Distant Galaxy", sim.sourceX, sim.sourceY + 38);

// Galaxy Cluster
if (sim.M > 0) {
  ctx.save();
  ctx.translate(sim.lensX, sim.lensY);

  const haloRadius = 40 + sim.M * 25;
  const haloGlow = ctx.createRadialGradient(0, 0, 5, 0, 0, haloRadius);
  haloGlow.addColorStop(0, "rgba(168, 85, 247, 0.45)");
  haloGlow.addColorStop(0.5, "rgba(126, 34, 206, 0.2)");
  haloGlow.addColorStop(1, "rgba(0, 0, 0, 0)");
  ctx.fillStyle = haloGlow;
  ctx.beginPath();
  ctx.arc(0, 0, haloRadius, 0, Math.PI * 2);
  ctx.fill();

  const clusterMembers = [
    {x: 0, y: 0, r: 8}, {x: -15, y: -10, r: 5},
    {x: 18, y: 12, r: 6}, {x: 12, y: -16, r: 4.5},
    {x: -18, y: 14, r: 5}, {x: -6, y: 20, r: 4}
  ];

  clusterMembers.forEach(m => {
    const mGlow = ctx.createRadialGradient(m.x, m.y, 0, m.x, m.y, m.r * 2.5);
    mGlow.addColorStop(0, "ffff5d6");
    mGlow.addColorStop(0.5, "rgba(245, 158, 11, 0.6)");
    mGlow.addColorStop(1, "rgba(0, 0, 0, 0)");
    ctx.fillStyle = mGlow;
    ctx.beginPath();
    ctx.arc(m.x, m.y, m.r * 2.5, 0, Math.PI * 2);
    ctx.fill();
  });

  ctx.restore();
}

```

```

    ctx.fillStyle = "#e1e7ec";
    ctx.font = "12px sans-serif";
    ctx.fillText("Galaxy Cluster", sim.lensX, sim.lensY + haloRadius + 18);
}

// Earth
ctx.save();
ctx.translate(sim.earthX, sim.earthY);

const earthGrad = ctx.createRadialGradient(-3, -3, 1, 0, 0, 12);
earthGrad.addColorStop(0, "#60a5fa");
earthGrad.addColorStop(0.7, "#1d4ed8");
earthGrad.addColorStop(1, "#1e3a8a");
ctx.fillStyle = earthGrad;
ctx.beginPath();
ctx.arc(0, 0, 12, 0, Math.PI * 2);
ctx.fill();

ctx.strokeStyle = "rgba(250, 204, 21, 0.4)";
ctx.setLineDash([3, 3]);
ctx.beginPath();
ctx.arc(0, 0, sim.earthRadius, 0, Math.PI * 2);
ctx.stroke();
ctx.setLineDash([]);
ctx.restore();

ctx.fillStyle = "#e1e7ec";
ctx.font = "12px sans-serif";
ctx.fillText("Earth / Observer", sim.earthX, sim.earthY + 38);

// Rays
sim.rays.forEach((ray, idx) => {
    const isHit = ray.hitsEarth;

    if (toggleTangents.checked && isHit) {
        ctx.strokeStyle = "rgba(239, 68, 68, 0.6)";
        ctx.lineWidth = 1.2;
        ctx.setLineDash([4, 4]);

        ctx.beginPath();
        ctx.moveTo(sim.earthX, sim.earthY);
        ctx.lineTo(sim.sourceX, ray.virtualY);
        ctx.stroke();
        ctx.setLineDash([]);

        ctx.fillStyle = "#ef4444";
        ctx.beginPath();
        ctx.arc(sim.sourceX, ray.virtualY, 3.5, 0, Math.PI * 2);
        ctx.fill();
    }

    ctx.beginPath();
    ctx.moveTo(ray.path[0].x, ray.path[0].y);
    for (let p = 1; p < ray.path.length; p++) {
        ctx.lineTo(ray.path[p].x, ray.path[p].y);
    }

    if (isHit) {
        ctx.strokeStyle = "#facc15";
        ctx.lineWidth = 2.2;
    } else {
        ctx.strokeStyle = "rgba(56, 189, 248, 0.45)";
        ctx.lineWidth = 1.2;
    }
    ctx.stroke();

    if (toggleParticles.checked) {
        const pathLen = ray.path.length;
        const progress = (particleOffset + (idx / NUM_RAYS)) % 1.0;
        const pointIdx = Math.floor(progress * (pathLen - 1));
        const pt = ray.path[pointIdx];

        if (pt) {
            ctx.fillStyle = isHit ? "ffffff" : "#00f0ff";

```

```

        ctx.beginPath();
        ctx.arc(pt.x, pt.y, isHit ? 2.8 : 2.0, 0, Math.PI * 2);
        ctx.fill();
    }
}
});

// Sky View
const skyViewY = sideViewHeight;
ctx.fillStyle = "#020305";
ctx.fillRect(0, skyViewY, width, skyViewHeight);

ctx.fillStyle = "#8c9bae";
ctx.font = "12px sans-serif";
ctx.textAlign = "left";
ctx.fillText("EARTH TELESCOPE SKY VIEW", 15, skyViewY + 22);

const skyCenterX = width / 2;
const skyCenterY = skyViewY + (skyViewHeight / 2) + 5;
const fovRadius = Math.min(skyViewHeight / 2 - 20, 90);

ctx.strokeStyle = "#2a3142";
ctx.lineWidth = 1.5;
ctx.beginPath();
ctx.arc(skyCenterX, skyCenterY, fovRadius, 0, Math.PI * 2);
ctx.stroke();

const capturedRays = sim.rays.filter(r => r.hitsEarth);

if (capturedRays.length > 0) {
    capturedRays.forEach(r => {
        const apparentYOffset = (r.virtualY - (sideViewHeight / 2)) * 0.85;
        const imageY = skyCenterY + apparentYOffset;
        const arcR = Math.abs(imageY - skyCenterY);

        ctx.strokeStyle = "#facc15";
        ctx.lineWidth = 3.5;

        if (arcR < 3) {
            ctx.beginPath();
            ctx.arc(skyCenterX, skyCenterY, 20, 0, Math.PI * 2);
            ctx.stroke();
        } else {
            const isTop = (imageY - skyCenterY) < 0;
            ctx.beginPath();
            ctx.arc(skyCenterX, skyCenterY, arcR, isTop ? -Math.PI * 0.8 : Math.PI * 0.2, isTop ? -Math.PI * 0.2 : Math.PI * 0.8);
            ctx.stroke();
        }
    });
}

valMass.textContent = `${sim.M.toFixed(2)} × 1015 M⊙`;
valLensDist.textContent = `${sim.dL.toFixed(1)} Gly`;
valSourceOffset.textContent = `${sim.sourceYOffsetGly.toFixed(2)} Gly`;
valFanSpread.textContent = `${sliderFanSpread.value}°`;

if (toggleParticles.checked) {
    particleOffset = (particleOffset + 0.008) % 1.0;
}

function animate() {
    draw();
    requestAnimationFrame(animate);
}

btnEinstein.addEventListener('click', () => {
    sliderMass.value = 1.4;
    sliderLensDist.value = 4.0;
    sliderSourceOffset.value = 0.0;
    sliderFanSpread.value = 28;
    updateActivePreset(btnEinstein);
});

```

```

btnDoubleArc.addEventListener('click', () => {
  sliderMass.value = 1.5;
  sliderLensDist.value = 4.0;
  sliderSourceOffset.value = 0.12;
  sliderFanSpread.value = 32;
  updateActivePreset(btnDoubleArc);
});

btnOffAxis.addEventListener('click', () => {
  sliderMass.value = 1.8;
  sliderLensDist.value = 4.5;
  sliderSourceOffset.value = 0.45;
  sliderFanSpread.value = 35;
  updateActivePreset(btnOffAxis);
});

btnNoMass.addEventListener('click', () => {
  sliderMass.value = 0;
  sliderSourceOffset.value = 0;
  sliderFanSpread.value = 25;
  updateActivePreset(btnNoMass);
});

function updateActivePreset(activeBtn) {
  [btnEinstein, btnDoubleArc, btnOffAxis, btnNoMass].forEach(b => b.classList.remove('active'));
  if (activeBtn) activeBtn.classList.add('active');
}

animate();
</script>
</body>
</html>

```

To run this locally, open your browser's Developer Tools (**F12** or **Right Click > Inspect > Console**) when opening the file to check if any specific script errors are logged.

User prompt: This is a good start. In the next iteration: 1) Remove the buttons for Einstein Ring, Double Arcs, and Off-Axis lensing. The purpose of this sim is just to show what happens to light passing by the intermediate cluster. 2) Let's make the angle exaggeration explicit. Add a slider for "angle exaggeration", default to 3600 (range 1000 to 6000). Then the Ray Fan angular spread will be in units of arcseconds, and what is displayed will be multiplied by the exaggeration factor. 3) remove the lower window Earth Telescope view

Response: Here is the updated complete HTML file.

The lower Earth Telescope view and preset buttons have been removed. The full canvas now displays the side-view ray-tracing diagram, and explicit controls have been added for the **Angle Exaggeration** factor (1000× to 6000×, default 3600×) and **Ray Fan Angular Spread** in arcseconds (").

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Strong Gravitational Lensing Simulation</title>
  <style>
    :root {
      --bg-color: #0b0e14;
      --panel-bg: #151922;
      --panel-border: #2a3142;
      --accent-color: #00f0ff;
      --text-main: #e1e7ec;
      --text-muted: #8c9bae;
    }

    * {
      box-sizing: border-box;
      margin: 0;
      padding: 0;
    }

    body {
      font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial, sans-serif;
      background-color: var(--bg-color);
      color: var(--text-main);
      display: flex;

```



```
    flex-direction: column;
    min-height: 100vh;
    overflow-x: hidden;
  }

  header {
    background-color: var(--panel-bg);
    border-bottom: 1px solid var(--panel-border);
    padding: 1rem 1.5rem;
    text-align: center;
  }

  header h1 {
    font-size: 1.4rem;
    font-weight: 600;
    color: #fff;
  }

  header p {
    font-size: 0.85rem;
    color: var(--text-muted);
    margin-top: 0.25rem;
  }

  .main-container {
    display: flex;
    flex: 1;
    flex-direction: column;
    gap: 1rem;
    padding: 1rem;
    max-width: 1400px;
    margin: 0 auto;
    width: 100%;
  }

  @media (min-width: 1024px) {
    .main-container {
      flex-direction: row;
    }
  }

  .canvas-card {
    flex: 1;
    background-color: var(--panel-bg);
    border: 1px solid var(--panel-border);
    border-radius: 8px;
    padding: 1rem;
    display: flex;
    flex-direction: column;
    gap: 0.5rem;
    box-shadow: 0 4px 20px rgba(0, 0, 0, 0.4);
  }

  .canvas-wrapper {
    position: relative;
    width: 100%;
    height: 620px;
    background-color: #04060a;
    border-radius: 6px;
    overflow: hidden;
    border: 1px solid rgba(255, 255, 255, 0.05);
  }

  canvas {
    width: 100%;
    height: 100%;
    display: block;
  }

  .controls-panel {
    width: 100%;
    max-width: 380px;
    background-color: var(--panel-bg);
    border: 1px solid var(--panel-border);
```

```
border-radius: 8px;
padding: 1.25rem;
display: flex;
flex-direction: column;
gap: 1.25rem;
box-shadow: 0 4px 20px rgba(0, 0, 0, 0.4);
}

.control-group {
  display: flex;
  flex-direction: column;
  gap: 0.4rem;
}

.control-group label {
  font-size: 0.85rem;
  font-weight: 500;
  color: var(--text-main);
  display: flex;
  justify-content: space-between;
}

.control-group label span {
  color: var(--accent-color);
  font-family: monospace;
}

input[type="range"] {
  -webkit-appearance: none;
  width: 100%;
  height: 6px;
  border-radius: 3px;
  background: #2a3142;
  outline: none;
}

input[type="range"]::-webkit-slider-thumb {
  -webkit-appearance: none;
  width: 16px;
  height: 16px;
  border-radius: 50%;
  background: var(--accent-color);
  cursor: pointer;
  box-shadow: 0 0 8px var(--accent-color);
  transition: transform 0.1s ease;
}

input[type="range"]::-webkit-slider-thumb:hover {
  transform: scale(1.2);
}

.legend {
  border-top: 1px solid var(--panel-border);
  padding-top: 1rem;
  display: flex;
  flex-direction: column;
  gap: 0.5rem;
  font-size: 0.8rem;
}

.legend-item {
  display: flex;
  align-items: center;
  gap: 0.6rem;
  color: var(--text-muted);
}

.legend-color {
  width: 14px;
  height: 4px;
  border-radius: 2px;
}

.toggle-row {
```

```

    display: flex;
    align-items: center;
    justify-content: space-between;
    font-size: 0.85rem;
    color: var(--text-main);
  }

  .switch {
    position: relative;
    display: inline-block;
    width: 40px;
    height: 20px;
  }

  .switch input {
    opacity: 0;
    width: 0;
    height: 0;
  }

  .slider {
    position: absolute;
    cursor: pointer;
    top: 0; left: 0; right: 0; bottom: 0;
    background-color: #2a3142;
    transition: .2s;
    border-radius: 20px;
  }

  .slider:before {
    position: absolute;
    content: "";
    height: 14px;
    width: 14px;
    left: 3px;
    bottom: 3px;
    background-color: white;
    transition: .2s;
    border-radius: 50%;
  }

  input:checked + .slider {
    background-color: var(--accent-color);
  }

  input:checked + .slider:before {
    transform: translateX(20px);
  }
</style>
</head>
<body>

<header>
  <h1>Gravitational Lensing by a Galaxy Cluster</h1>
  <p>Ray tracing of light deflection caused by curved spacetime (with exaggerated angular scale)</p>
</header>

<div class="main-container">
  <div class="canvas-card">
    <div class="canvas-wrapper">
      <canvas id="simCanvas"></canvas>
    </div>
  </div>

  <div class="controls-panel">
    <div class="control-group">
      <label for="sliderMass">Cluster Mass ($M$): <span id="valMass">1.20 × 1015 M☉</span></label>
      <input type="range" id="sliderMass" min="0" max="2.5" step="0.05" value="1.2">
    </div>

    <div class="control-group">
      <label for="sliderLensDist">Cluster Distance ($d_L$): <span id="valLensDist">4.0 Gly</span></label>
      <input type="range" id="sliderLensDist" min="1.5" max="7.0" step="0.1" value="4.0">
    </div>
  </div>
</div>

```

```

<div class="control-group">
  <label for="sliderSourceOffset">Galaxy Alignment Offset: <span id="valSourceOffset">0.05 Gly</span></label>
  <input type="range" id="sliderSourceOffset" min="-0.8" max="0.8" step="0.02" value="0.05">
</div>

<div class="control-group">
  <label for="sliderFanSpread">Ray Fan Angular Spread: <span id="valFanSpread">28" (Vis: 28.0°)</span></label>
  <input type="range" id="sliderFanSpread" min="5" max="60" step="1" value="28">
</div>

<div class="control-group">
  <label for="sliderExaggeration">Angle Exaggeration Factor: <span id="valExaggeration">3600x</span></label>
  <input type="range" id="sliderExaggeration" min="1000" max="6000" step="100" value="3600">
</div>

<div class="toggle-row">
  <span>Animate Photons</span>
  <label class="switch">
    <input type="checkbox" id="toggleParticles" checked>
    <span class="slider"></span>
  </label>
</div>

<div class="toggle-row">
  <span>Show Virtual Tangents</span>
  <label class="switch">
    <input type="checkbox" id="toggleTangents" checked>
    <span class="slider"></span>
  </label>
</div>

<div class="legend">
  <div class="legend-item">
    <div class="legend-color" style="background: #38bdf8;"></div>
    <span>Light rays bypassing Earth</span>
  </div>
  <div class="legend-item">
    <div class="legend-color" style="background: #facc15;"></div>
    <span>Light rays captured by Earth</span>
  </div>
  <div class="legend-item">
    <div class="legend-color" style="border: 1px dashed #ef4444; background: transparent;"></div>
    <span>Apparent straight-line paths (Virtual Images)</span>
  </div>
</div>
</div>
</div>

<script>
const canvas = document.getElementById('simCanvas');
const ctx = canvas.getContext('2d');

// Control Elements
const sliderMass = document.getElementById('sliderMass');
const sliderLensDist = document.getElementById('sliderLensDist');
const sliderSourceOffset = document.getElementById('sliderSourceOffset');
const sliderFanSpread = document.getElementById('sliderFanSpread');
const sliderExaggeration = document.getElementById('sliderExaggeration');

const valMass = document.getElementById('valMass');
const valLensDist = document.getElementById('valLensDist');
const valSourceOffset = document.getElementById('valSourceOffset');
const valFanSpread = document.getElementById('valFanSpread');
const valExaggeration = document.getElementById('valExaggeration');

const toggleParticles = document.getElementById('toggleParticles');
const toggleTangents = document.getElementById('toggleTangents');

// Simulation Constants
const NUM_RAYS = 10;
const TOTAL_DISTANCE_GLY = 8.0; // Distance Earth to Distant Galaxy in Giga-light-years
let particleOffset = 0;

```

```

let width = 0;
let height = 0;

// Background Stars
const stars = [];
function generateStars(count) {
  stars.length = 0;
  for (let i = 0; i < count; i++) {
    stars.push({
      x: Math.random(),
      y: Math.random(),
      size: Math.random() * 1.5 + 0.3,
      alpha: Math.random() * 0.7 + 0.3
    });
  }
}
generateStars(220);

function resizeCanvas() {
  const rect = canvas.parentElement.getBoundingClientRect();
  const dpr = window.devicePixelRatio || 1;
  width = rect.width || 800;
  height = rect.height || 620;

  canvas.width = width * dpr;
  canvas.height = height * dpr;
  ctx.scale(dpr, dpr);
}

window.addEventListener('resize', resizeCanvas);
resizeCanvas();

// Core Ray Tracing Calculation
function computeTrajectories() {
  const M = parseFloat(sliderMass.value);
  const dL = parseFloat(sliderLensDist.value);
  const sourceYOffsetGly = parseFloat(sliderSourceOffset.value);
  const fanSpreadArcsec = parseFloat(sliderFanSpread.value);
  const exaggeration = parseFloat(sliderExaggeration.value);

  const marginX = 90;
  const centerY = height / 2;

  const earthX = marginX;
  const earthY = centerY;

  const sourceX = width - marginX;
  const pxPerGly = (sourceX - earthX) / TOTAL_DISTANCE_GLY;
  const sourceY = centerY + (sourceYOffsetGly * pxPerGly);

  const lensX = earthX + (dL * pxPerGly);
  const lensY = centerY;

  const earthRadius = 18;
  const rays = [];

  // Calculate visual fan spread angle in radians
  // 1 arcsec = 1/3600 degree. Multiply by exaggeration factor.
  const visualSpreadDegrees = (fanSpreadArcsec * exaggeration) / 3600.0;
  const visualFanSpreadRad = visualSpreadDegrees * (Math.PI / 180.0);

  const baseAngle = Math.atan2(earthY - sourceY, earthX - sourceX);

  // Scale gravitational deflection proportionally with angle exaggeration
  const G_scale = 180000 * M * (exaggeration / 3600.0);

  for (let i = 0; i < NUM_RAYS; i++) {
    const frac = (i / (NUM_RAYS - 1)) - 0.5;
    const angle = baseAngle + (frac * visualFanSpreadRad);

    const rayPath = [];
    let currX = sourceX;
    let currY = sourceY;

```

```

let vx = Math.cos(angle);
let vy = Math.sin(angle);

rayPath.push({ x: currX, y: currY });

const dt = 3.0;
let hitsEarth = false;
let missed = false;

let steps = 0;
const maxSteps = 3000; // Safety guard against infinite loops

while (currX > earthX && !missed && steps < maxSteps) {
  steps++;

  if (M > 0) {
    const dx = currX - lensX;
    const dy = currY - lensY;
    const r2 = dx * dx + dy * dy;

    const coreRadius2 = 1200;
    const softFactor = Math.pow(r2 + coreRadius2, 1.2);

    const ax = -G_scale * dx / softFactor;
    const ay = -G_scale * dy / softFactor;

    vx += ax * dt;
    vy += ay * dt;

    const speed = Math.hypot(vx, vy);
    if (speed > 0) {
      vx /= speed;
      vy /= speed;
    }
  }

  currX += vx * dt;
  currY += vy * dt;
  rayPath.push({ x: currX, y: currY });

  if (currY < -100 || currY > height + 100 || currX > width + 200) {
    missed = true;
  }
}

const finalPoint = rayPath[rayPath.length - 1];
const distToEarthCenter = Math.abs(finalPoint.y - earthY);
if (distToEarthCenter <= earthRadius) {
  hitsEarth = true;
}

const finalVx = vx;
const finalVy = vy;

// Linear back-projection to source plane
const tVirtual = (sourceX - finalPoint.x) / (finalVx || -1);
const virtualY = finalPoint.y + finalVy * tVirtual;

rays.push({
  path: rayPath,
  hitsEarth: hitsEarth,
  finalAngle: Math.atan2(finalVy, finalVx),
  virtualY: virtualY,
  endPoint: finalPoint
});
}

return {
  earthX, earthY, earthRadius,
  lensX, lensY, dL,
  sourceX, sourceY, sourceYOffsetGly,
  rays, M, visualSpreadDegrees, exaggeration, fanSpreadArcsec
};
}

```

```

// Render Canvas Frame
function draw() {
  ctx.clearRect(0, 0, width, height);

  const sim = computeTrajectories();

  // 1. Draw Starfield Background
  ctx.fillStyle = "#ffffff";
  stars.forEach(s => {
    ctx.globalAlpha = s.alpha;
    ctx.fillRect(s.x * width, s.y * height, s.size, s.size);
  });
  ctx.globalAlpha = 1.0;

  // Diagram Label
  ctx.fillStyle = "#8c9bae";
  ctx.font = "12px sans-serif";
  ctx.fillText("SIDE VIEW (CROSS-SECTION)", 15, 22);

  // 2. Distant Galaxy (Source)
  ctx.save();
  ctx.translate(sim.sourceX, sim.sourceY);

  const galGlow = ctx.createRadialGradient(0, 0, 2, 0, 0, 30);
  galGlow.addColorStop(0, "rgba(255, 200, 100, 0.9)");
  galGlow.addColorStop(0.4, "rgba(238, 100, 30, 0.4)");
  galGlow.addColorStop(1, "rgba(0, 0, 0, 0)");
  ctx.fillStyle = galGlow;
  ctx.beginPath();
  ctx.arc(0, 0, 30, 0, Math.PI * 2);
  ctx.fill();

  ctx.rotate(-0.3);
  ctx.fillStyle = "#ffd59e";
  ctx.beginPath();
  ctx.ellipse(0, 0, 22, 7, 0, 0, Math.PI * 2);
  ctx.fill();

  ctx.fillStyle = "#ffffff";
  ctx.beginPath();
  ctx.arc(0, 0, 4, 0, Math.PI * 2);
  ctx.fill();
  ctx.restore();

  ctx.fillStyle = "#e1e7ec";
  ctx.font = "12px sans-serif";
  ctx.textAlign = "center";
  ctx.fillText("Distant Galaxy", sim.sourceX, sim.sourceY + 38);
  ctx.fillStyle = "#8c9bae";
  ctx.font = "11px monospace";
  ctx.fillText("(8.0 Gly)", sim.sourceX, sim.sourceY + 52);

  // 3. Galaxy Cluster (Lens)
  if (sim.M > 0) {
    ctx.save();
    ctx.translate(sim.lensX, sim.lensY);

    const haloRadius = 40 + sim.M * 25;
    const haloGlow = ctx.createRadialGradient(0, 0, 5, 0, 0, haloRadius);
    haloGlow.addColorStop(0, "rgba(168, 85, 247, 0.45)");
    haloGlow.addColorStop(0.5, "rgba(126, 34, 206, 0.2)");
    haloGlow.addColorStop(1, "rgba(0, 0, 0, 0)");
    ctx.fillStyle = haloGlow;
    ctx.beginPath();
    ctx.arc(0, 0, haloRadius, 0, Math.PI * 2);
    ctx.fill();

    const clusterMembers = [
      {x: 0, y: 0, r: 8}, {x: -15, y: -10, r: 5},
      {x: 18, y: 12, r: 6}, {x: 12, y: -16, r: 4.5},
      {x: -18, y: 14, r: 5}, {x: -6, y: 20, r: 4}
    ];
  }
}

```

```

clusterMembers.forEach(m => {
  const mGlow = ctx.createRadialGradient(m.x, m.y, 0, m.x, m.y, m.r * 2.5);
  mGlow.addColorStop(0, "#fff5d6");
  mGlow.addColorStop(0.5, "rgba(245, 158, 11, 0.6)");
  mGlow.addColorStop(1, "rgba(0, 0, 0, 0)");
  ctx.fillStyle = mGlow;
  ctx.beginPath();
  ctx.arc(m.x, m.y, m.r * 2.5, 0, Math.PI * 2);
  ctx.fill();

  ctx.fillStyle = "#ffffff";
  ctx.beginPath();
  ctx.arc(m.x, m.y, m.r * 0.7, 0, Math.PI * 2);
  ctx.fill();
});

ctx.restore();

ctx.fillStyle = "#e1e7ec";
ctx.font = "12px sans-serif";
ctx.fillText("Galaxy Cluster", sim.lensX, sim.lensY + haloRadius + 18);
ctx.fillStyle = "#8c9bae";
ctx.font = "11px monospace";
ctx.fillText(`${sim.dL.toFixed(1)} Gly`, sim.lensX, sim.lensY + haloRadius + 32);
}

// 4. Earth / Observer
ctx.save();
ctx.translate(sim.earthX, sim.earthY);

const earthGrad = ctx.createRadialGradient(-3, -3, 1, 0, 0, 12);
earthGrad.addColorStop(0, "#60a5fa");
earthGrad.addColorStop(0.7, "#1d4ed8");
earthGrad.addColorStop(1, "#1e3a8a");
ctx.fillStyle = earthGrad;
ctx.beginPath();
ctx.arc(0, 0, 12, 0, Math.PI * 2);
ctx.fill();

// Detection aperture boundary
ctx.strokeStyle = "rgba(250, 204, 21, 0.4)";
ctx.setLineDash([3, 3]);
ctx.beginPath();
ctx.arc(0, 0, sim.earthRadius, 0, Math.PI * 2);
ctx.stroke();
ctx.setLineDash([]);

ctx.restore();

ctx.fillStyle = "#e1e7ec";
ctx.font = "12px sans-serif";
ctx.textAlign = "center";
ctx.fillText("Earth / Observer", sim.earthX, sim.earthY + 38);

// 5. Draw 10 Light Rays & Virtual Tangents
sim.rays.forEach((ray, idx) => {
  const isHit = ray.hitsEarth;

  // Virtual back-projections
  if (toggleTangents.checked && isHit) {
    ctx.strokeStyle = "rgba(239, 68, 68, 0.6)";
    ctx.lineWidth = 1.2;
    ctx.setLineDash([4, 4]);

    ctx.beginPath();
    ctx.moveTo(sim.earthX, sim.earthY);
    ctx.lineTo(sim.sourceX, ray.virtualY);
    ctx.stroke();
    ctx.setLineDash([]);

    ctx.fillStyle = "#ef4444";
    ctx.beginPath();
    ctx.arc(sim.sourceX, ray.virtualY, 3.5, 0, Math.PI * 2);
    ctx.fill();
  }
});

```



```

    }

    // Light ray path
    ctx.beginPath();
    ctx.moveTo(ray.path[0].x, ray.path[0].y);
    for (let p = 1; p < ray.path.length; p++) {
        ctx.lineTo(ray.path[p].x, ray.path[p].y);
    }

    if (isHit) {
        ctx.strokeStyle = "#facc15";
        ctx.lineWidth = 2.2;
        ctx.shadowColor = "#facc15";
        ctx.shadowBlur = 8;
    } else {
        ctx.strokeStyle = "rgba(56, 189, 248, 0.45)";
        ctx.lineWidth = 1.2;
        ctx.shadowBlur = 0;
    }
    ctx.stroke();
    ctx.shadowBlur = 0;

    // Animated photons
    if (toggleParticles.checked) {
        const pathLen = ray.path.length;
        const progress = (particleOffset + (idx / NUM_RAYS)) % 1.0;
        const pointIdx = Math.floor(progress * (pathLen - 1));
        const pt = ray.path[pointIdx];

        if (pt) {
            ctx.fillStyle = isHit ? "#ffffff" : "#00f0ff";
            ctx.beginPath();
            ctx.arc(pt.x, pt.y, isHit ? 2.8 : 2.0, 0, Math.PI * 2);
            ctx.fill();
        }
    }
});

// Update control text labels
valMass.textContent = `${sim.M.toFixed(2)} × 1015 M⊙`;
valLensDist.textContent = `${sim.dL.toFixed(1)} Gly`;
valSourceOffset.textContent = `${sim.sourceYOffsetGly.toFixed(2)} Gly`;
valFanSpread.textContent = `${sim.fanSpreadArcsec}` (Vis: ${sim.visualSpreadDegrees.toFixed(1)}°);
valExaggeration.textContent = `${sim.exaggeration}×`;

if (toggleParticles.checked) {
    particleOffset = (particleOffset + 0.008) % 1.0;
}

function animate() {
    draw();
    requestAnimationFrame(animate);
}

animate();
</script>
</body>
</html>

```

User prompt: Why are the rays not displaying at highly exaggerated angles (i.e., I should see a fan of rays across the image)

Response: Rays fail to fan out at wide, exaggerated angles primarily due to **directional light assumptions**, **camera perspective settings**, or **screen-space shader parameters**.

Here are the primary technical reasons why your rays appear parallel or narrow instead of forming a wide visual fan:

1. Light Source Configuration (Directional vs. Point/Spot)

- **Directional Lights (Sunlight):** Directional lights cast completely parallel rays in 3D space. They do not naturally diverge.
- **Point or Spot Lights:** These cast rays radially from a single point. If you want physically diverging rays without relying on perspective tricks, switch from a directional light to a point or spot light positioned close to the scene objects.

2. Camera Field of View (FOV) and Distance

Crepuscular rays ("god rays") in nature are actually parallel; they only *appear* to fan out due to perspective projection (the same way train tracks converge at a horizon vanishing point).

- **Narrow FOV / Telephoto Lens:** A narrow camera FOV flattens perspective, making divergent or perspective-based rays appear parallel across the frame. Increasing the FOV exaggerates the angle of the rays across the screen edges.
- **Distance to Light Source:** If a point light is placed extremely far away from the occluders or camera, the light vectors entering your camera frustum become effectively parallel, neutralizing the fanning effect.

3. Screen-Space Shader Calculations (Post-Processing God Rays)

If you are using a screen-space volumetric light shader (e.g., radial blur along light vectors):

- **Light Position Clipping/Off-Screen Limits:** Screen-space rays calculate vectors from pixels toward the light's projected 2D screen coordinate (x, y) . If the light source is too far off-screen, the directional vectors across the viewport become almost uniform rather than radial, collapsing the fan.
- **Decay and Density Tuning:** High decay rates or low sample density extinguish the light samples before they travel across the full width of the image.

4. Refraction and Optical Simulation (Physics)

If you are simulating refraction through glass, lenses, or prisms:

- **Index of Refraction (IOR):** An IOR too close to 1.0 (air) creates minimal light bending.
- **Small Angle Approximations:** If the simulation relies on paraxial approximations ($\sin \theta \approx \theta$), extreme wide-angle refractions will be suppressed.

Are you experiencing this in a specific 3D engine (e.g., Unreal, Unity, Blender), a custom post-processing shader, or an optical physics simulation?

User prompt: Let's simplify. Let's show a single ray from the distant galaxy passing by the intervening cluster. We have the following controls: 1) distance from Earth to the emitting galaxy (from 2 to 13 billion light-years. Must always be at least 1 billion light-years further than the cluster) 2) distance from Earth to the intermediate cluster (from 1 to 12 billion light-years. Must always be at least 1 billion light-years closer than the galaxy) 3) mass of the intermediate cluster (from 5 to 1000 trillion solar masses) 4) initial angle of the light ray, relative to the horizontal line across the screen. This is in arc-seconds, from -40 to +40. This angle is shown on screen exaggerated by a factor of 3600. The distance in light-years and the initial angle in arc-seconds are used to calculate the impact parameter of the light-ray (how far off to the side it is passing by the cluster). The impact parameter and the mass of the cluster are used to calculate the angle that the beam is deflected by. The beam deflection angle is also shown with an exaggeration of 3600. Add a box to the screen that shows the numerical results of these calculations, so that I can double check.

```
Response: <!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Gravitational Lensing: Single Ray Deflection</title>
  <style>
    :root {
      --bg-color: #0b0e14;
      --panel-bg: #151922;
      --panel-border: #2a3142;
      --accent-color: #00f0ff;
      --accent-amber: #facc15;
      --accent-red: #ef4444;
      --text-main: #e1e7ec;
      --text-muted: #8c9bae;
    }

    * {
      box-sizing: border-box;
      margin: 0;
      padding: 0;
    }

    body {
      font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial, sans-serif;
      background-color: var(--bg-color);
      color: var(--text-main);
      display: flex;
      flex-direction: column;
      min-height: 100vh;
```

```
    overflow-x: hidden;
}

header {
    background-color: var(--panel-bg);
    border-bottom: 1px solid var(--panel-border);
    padding: 0.85rem 1.5rem;
    text-align: center;
}

header h1 {
    font-size: 1.3rem;
    font-weight: 600;
    color: #fff;
}

header p {
    font-size: 0.85rem;
    color: var(--text-muted);
    margin-top: 0.2rem;
}

.main-container {
    display: flex;
    flex: 1;
    flex-direction: column;
    gap: 1rem;
    padding: 1rem;
    max-width: 1400px;
    margin: 0 auto;
    width: 100%;
}

@media (min-width: 1024px) {
    .main-container {
        flex-direction: row;
    }
}

.canvas-card {
    flex: 1;
    background-color: var(--panel-bg);
    border: 1px solid var(--panel-border);
    border-radius: 8px;
    padding: 1rem;
    display: flex;
    flex-direction: column;
    gap: 0.5rem;
    box-shadow: 0 4px 20px rgba(0, 0, 0, 0.4);
    position: relative;
}

.canvas-wrapper {
    position: relative;
    width: 100%;
    height: 600px;
    background-color: #04060a;
    border-radius: 6px;
    overflow: hidden;
    border: 1px solid rgba(255, 255, 255, 0.05);
}

canvas {
    width: 100%;
    height: 100%;
    display: block;
}

.calc-box {
    position: absolute;
    top: 1.5rem;
    left: 1.5rem;
    background: rgba(15, 20, 30, 0.88);
    border: 1px solid var(--panel-border);
```

```
    backdrop-filter: blur(8px);
    border-radius: 8px;
    padding: 0.9rem 1.1rem;
    font-family: SFMono-Regular, Consolas, "Liberation Mono", Menlo, monospace;
    font-size: 0.8rem;
    line-height: 1.45;
    color: #d1d5db;
    box-shadow: 0 4px 15px rgba(0, 0, 0, 0.5);
    max-width: 340px;
    z-index: 10;
    pointer-events: none;
}

.calc-box h3 {
    font-size: 0.85rem;
    text-transform: uppercase;
    letter-spacing: 0.05em;
    color: var(--accent-color);
    margin-bottom: 0.5rem;
    border-bottom: 1px solid rgba(255, 255, 255, 0.1);
    padding-bottom: 0.3rem;
    font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;
}

.calc-row {
    display: flex;
    justify-content: space-between;
    margin-bottom: 0.25rem;
}

.calc-row .label {
    color: var(--text-muted);
}

.calc-row .value {
    color: #fff;
    font-weight: 600;
}

.calc-row .highlight {
    color: var(--accent-amber);
}

.calc-divider {
    border-top: 1px dashed rgba(255, 255, 255, 0.15);
    margin: 0.4rem 0;
}

.controls-panel {
    width: 100%;
    max-width: 380px;
    background-color: var(--panel-bg);
    border: 1px solid var(--panel-border);
    border-radius: 8px;
    padding: 1.25rem;
    display: flex;
    flex-direction: column;
    gap: 1.25rem;
    box-shadow: 0 4px 20px rgba(0, 0, 0, 0.4);
}

.control-group {
    display: flex;
    flex-direction: column;
    gap: 0.4rem;
}

.control-group label {
    font-size: 0.85rem;
    font-weight: 500;
    color: var(--text-main);
    display: flex;
    justify-content: space-between;
}
```

```
.control-group label span {
  color: var(--accent-color);
  font-family: monospace;
}

input[type="range"] {
  -webkit-appearance: none;
  width: 100%;
  height: 6px;
  border-radius: 3px;
  background: #2a3142;
  outline: none;
}

input[type="range"]::-webkit-slider-thumb {
  -webkit-appearance: none;
  width: 16px;
  height: 16px;
  border-radius: 50%;
  background: var(--accent-color);
  cursor: pointer;
  box-shadow: 0 0 8px var(--accent-color);
  transition: transform 0.1s ease;
}

input[type="range"]::-webkit-slider-thumb:hover {
  transform: scale(1.2);
}

.toggle-row {
  display: flex;
  align-items: center;
  justify-content: space-between;
  font-size: 0.85rem;
  color: var(--text-main);
}

.switch {
  position: relative;
  display: inline-block;
  width: 40px;
  height: 20px;
}

.switch input {
  opacity: 0;
  width: 0;
  height: 0;
}

.slider {
  position: absolute;
  cursor: pointer;
  top: 0; left: 0; right: 0; bottom: 0;
  background-color: #2a3142;
  transition: .2s;
  border-radius: 20px;
}

.slider:before {
  position: absolute;
  content: "";
  height: 14px;
  width: 14px;
  left: 3px;
  bottom: 3px;
  background-color: white;
  transition: .2s;
  border-radius: 50%;
}

input:checked + .slider {
  background-color: var(--accent-color);
}
```

```

}

input:checked + .slider:before {
  transform: translateX(20px);
}

.legend {
  border-top: 1px solid var(--panel-border);
  padding-top: 0.8rem;
  display: flex;
  flex-direction: column;
  gap: 0.4rem;
  font-size: 0.8rem;
}

.legend-item {
  display: flex;
  align-items: center;
  gap: 0.6rem;
  color: var(--text-muted);
}

.legend-color {
  width: 14px;
  height: 4px;
  border-radius: 2px;
}
</style>
</head>
<body>

<header>
  <h1>Gravitational Deflection of Light (Single Ray)</h1>
  <p>Simulating photon path bending near a massive galaxy cluster with 3600× angle exaggeration</p>
</header>

<div class="main-container">
  <div class="canvas-card">
    <div class="canvas-wrapper">
      <canvas id="simCanvas"></canvas>
    <div class="calc-box">
      <h3>Numerical Verification</h3>
      <div class="calc-row">
        <span class="label">Galaxy Dist (d<sub>G</sub>):</span>
        <span class="value" id="boxDistG">8.0 Gly</span>
      </div>
      <div class="calc-row">
        <span class="label">Cluster Dist (d<sub>C</sub>):</span>
        <span class="value" id="boxDistC">4.0 Gly</span>
      </div>
      <div class="calc-row">
        <span class="label">d<sub>GC</sub> = d<sub>G</sub> - d<sub>C</sub>:</span>
        <span class="value" id="boxDistGC">4.0 Gly</span>
      </div>
      <div class="calc-divider"></div>
      <div class="calc-row">
        <span class="label">Cluster Mass (M):</span>
        <span class="value" id="boxMass">200 × 1012 M⊙</span>
      </div>
      <div class="calc-row">
        <span class="label">Initial Angle (θ<sub>init</sub>):</span>
        <span class="value" id="boxThetaInit">+15.0°</span>
      </div>
      <div class="calc-row">
        <span class="label">Visual Angle (3600×):</span>
        <span class="value" id="boxThetaInitVis">+15.00°</span>
      </div>
      <div class="calc-divider"></div>
      <div class="calc-row">
        <span class="label">Impact Param (b):</span>
        <span class="value highlight" id="boxImpact">290.89 kly</span>
      </div>
      <div class="calc-row">
        <span class="label">Formula:</span>

```

```

        <span class="value" style="font-size: 0.75rem; color: #9ca3af;"> $\alpha = 4GM / (c^2b)$ </span>
    </div>
    <div class="calc-row">
        <span class="label">Deflection Angle ( $\alpha$ ):</span>
        <span class="value highlight" id="boxAlpha">88.54</span>
    </div>
    <div class="calc-row">
        <span class="label">Visual Deflect (3600 $\times$ ):</span>
        <span class="value highlight" id="boxAlphaVis">88.54</span>
    </div>
    <div class="calc-divider"></div>
    <div class="calc-row">
        <span class="label">Final Ray Angle:</span>
        <span class="value" id="boxThetaFinal">-73.54</span>
    </div>
</div>
</div>
</div>
<div class="controls-panel">
    <div class="control-group">
        <label for="sliderDistG">
            1. Distance to Galaxy (dG):
            <span id="valDistG">8.0 Gly</span>
        </label>
        <input type="range" id="sliderDistG" min="2.0" max="13.0" step="0.1" value="8.0">
    </div>

    <div class="control-group">
        <label for="sliderDistC">
            2. Distance to Cluster (dC):
            <span id="valDistC">4.0 Gly</span>
        </label>
        <input type="range" id="sliderDistC" min="1.0" max="12.0" step="0.1" value="4.0">
    </div>

    <div class="control-group">
        <label for="sliderMass">
            3. Cluster Mass (M):
            <span id="valMass">200  $\times 10^{12}$  M⊙</span>
        </label>
        <input type="range" id="sliderMass" min="5" max="1000" step="5" value="200">
    </div>

    <div class="control-group">
        <label for="sliderTheta">
            4. Initial Ray Angle ( $\theta_{\text{init}}$ ):
            <span id="valTheta">+15.0</span>
        </label>
        <input type="range" id="sliderTheta" min="-40.0" max="40.0" step="0.5" value="15.0">
    </div>

    <div class="toggle-row">
        <span>Animate Photon</span>
        <label class="switch">
            <input type="checkbox" id="togglePhoton" checked>
            <span class="slider"></span>
        </label>
    </div>

    <div class="toggle-row">
        <span>Show Virtual Tangent Line</span>
        <label class="switch">
            <input type="checkbox" id="toggleTangent" checked>
            <span class="slider"></span>
        </label>
    </div>

    <div class="legend">
        <div class="legend-item">
            <div class="legend-color" style="background: #facc15;"></div>
            <span>Actual deflected light ray</span>
        </div>
        <div class="legend-item">

```

```
<div class="legend-color" style="border: 1px dashed #ef4444; background: transparent;"></div>
<span>Apparent straight-line path (as seen by observer)</span>
</div>
<div class="legend-item">
  <div class="legend-color" style="background: #00f0ff;"></div>
  <span>Impact parameter (b) & Lens plane</span>
</div>
</div>
</div>
</div>
```



```

    let dC = parseFloat(sliderDistC.value);
    if (dG < dC + 1.0) {
        dC = Math.max(1.0, dG - 1.0);
        sliderDistC.value = dC.toFixed(1);
    }
});

sliderDistC.addEventListener('input', () => {
    let dG = parseFloat(sliderDistG.value);
    let dC = parseFloat(sliderDistC.value);
    if (dC > dG - 1.0) {
        dG = Math.min(13.0, dC + 1.0);
        sliderDistG.value = dG.toFixed(1);
    }
});

function calculatePhysics() {
    const d_G_Gly = parseFloat(sliderDistG.value);
    const d_C_Gly = parseFloat(sliderDistC.value);
    const d_GC_Gly = d_G_Gly - d_C_Gly;
    const M_tril = parseFloat(sliderMass.value); // Trillion solar masses
    const theta_init_arcsec = parseFloat(sliderTheta.value);

    // Conversion constants:
    // 1 arcsec in radians = 1 / 206264.806247
    const ARCSEC_TO_RAD = Math.PI / (180.0 * 3600.0);

    // 1. Impact parameter b = d_GC * theta_init
    // d_GC in light-years
    const d_GC_ly = d_GC_Gly * 1e9;
    const theta_init_rad = theta_init_arcsec * ARCSEC_TO_RAD;
    const b_ly = d_GC_ly * theta_init_rad;
    const abs_b_ly = Math.abs(b_ly);

    // 2. Deflection angle alpha = 4 G M / (c^2 * b)
    // For 1 trillion solar masses (10^12 M_sun), 4GM/c^2 = 0.6243363 light-years
    const scale_4GM_c2_ly = 0.6243363 * M_tril;

    let alpha_arcsec = 0;
    if (abs_b_ly > 1e-4) {
        const alpha_rad = scale_4GM_c2_ly / abs_b_ly;
        alpha_arcsec = alpha_rad / ARCSEC_TO_RAD;
    }

    // Gravitational force pulls towards the lens plane (y = 0).
    // If ray is above (theta_init > 0), gravity bends it downwards (-alpha).
    // If ray is below (theta_init < 0), gravity bends it upwards (+alpha).
    const theta_after_arcsec = theta_init_arcsec > 0
        ? theta_init_arcsec - alpha_arcsec
        : theta_init_arcsec + alpha_arcsec;

    return {
        d_G_Gly,
        d_C_Gly,
        d_GC_Gly,
        M_tril,
        theta_init_arcsec,
        b_ly,
        abs_b_ly,
        alpha_arcsec,
        theta_after_arcsec
    };
};

function draw() {
    ctx.clearRect(0, 0, width, height);

    const phys = calculatePhysics();

    // Update control text values
    valDistG.textContent = `${phys.d_G_Gly.toFixed(1)} Gly`;
    valDistC.textContent = `${phys.d_C_Gly.toFixed(1)} Gly`;
    valMass.textContent = `${phys.M_tril} × 1012 M☉`;
    valTheta.textContent = `${phys.theta_init_arcsec} ≥ 0 ? '+' : '-'`;
    valTheta.textContent += `${phys.theta_init_arcsec.toFixed(1)}";

```

```

// Update Verification Box
boxDistG.textContent = `${phys.d_G_Gly.toFixed(1)} Gly`;
boxDistC.textContent = `${phys.d_C_Gly.toFixed(1)} Gly`;
boxDistGC.textContent = `${phys.d_GC_Gly.toFixed(1)} Gly`;
boxMass.textContent = `${phys.M_tril} × 1012 M⊙`;
boxThetaInit.textContent = `${phys.theta_init_arcsec >= 0 ? '+' : ''}${phys.theta_init_arcsec.toFixed(1)}`;
boxThetaInitVis.textContent = `${phys.theta_init_arcsec >= 0 ? '+' : ''}${phys.theta_init_arcsec.toFixed(1)}`;

boxImpact.textContent = `${(phys.abs_b_ly / 1000.0).toFixed(2)} kly (${Math.round(phys.abs_b_ly).toLocaleString()})`;
boxAlpha.textContent = `${phys.alpha_arcsec.toFixed(2)}°`;
boxAlphaVis.textContent = `${phys.alpha_arcsec.toFixed(2)}°`;
boxThetaFinal.textContent = `${phys.theta_after_arcsec >= 0 ? '+' : ''}${phys.theta_after_arcsec.toFixed(1)}`;

// Visual Geometry setup
const marginX = 80;
const centerY = height / 2;

const earthX = marginX;
const galaxyX = width - marginX;
const totalWidthPx = galaxyX - earthX;

// Position along x-axis is proportional to light-year distances
const pxPerGly = totalWidthPx / phys.d_G_Gly;
const clusterX = earthX + (phys.d_C_Gly * pxPerGly);

// Exaggeration: 1 arcsec = 1 degree on screen display
const DEG_TO_RAD = Math.PI / 180.0;
const theta_vis_rad = phys.theta_init_arcsec * DEG_TO_RAD;
const alpha_vis_rad = phys.alpha_arcsec * DEG_TO_RAD;

// 1. Draw Starfield
ctx.fillStyle = "#ffffff";
stars.forEach(s => {
  ctx.globalAlpha = s.alpha;
  ctx.fillRect(s.x * width, s.y * height, s.size, s.size);
});
ctx.globalAlpha = 1.0;

// 2. Optical Axis (Horizontal Baseline)
ctx.strokeStyle = "rgba(255, 255, 255, 0.15)";
ctx.lineWidth = 1;
ctx.setLineDash([6, 6]);
ctx.beginPath();
ctx.moveTo(earthX - 20, centerY);
ctx.lineTo(galaxyX + 20, centerY);
ctx.stroke();
ctx.setLineDash([]);

// 3. Cluster Plane (Vertical Dashed Line)
ctx.strokeStyle = "rgba(0, 240, 255, 0.25)";
ctx.beginPath();
ctx.moveTo(clusterX, 40);
ctx.lineTo(clusterX, height - 40);
ctx.stroke();

// 4. Calculate Ray Points on Canvas
// Point A: Distant Galaxy center
const ptA = { x: galaxyX, y: centerY };

// Point B: Ray position at Cluster Plane
// Note: canvas y goes DOWN, so positive angle means ray goes UP (smaller y)
const distGC_px = galaxyX - clusterX;
const yB_offset_px = distGC_px * Math.tan(theta_vis_rad);
const ptB = { x: clusterX, y: centerY - yB_offset_px };

// Direction after cluster plane
const theta_after_vis_rad = phys.theta_init_arcsec > 0
  ? theta_vis_rad - alpha_vis_rad
  : theta_vis_rad + alpha_vis_rad;

const distC_px = clusterX - earthX;
const yC_offset_px = yB_offset_px + distC_px * Math.tan(theta_after_vis_rad);
const ptC = { x: earthX, y: centerY - yC_offset_px };

```

```

// 5. Draw Virtual Straight Line / Tangent (Apparent Path)
if (toggleTangent.checked) {
  ctx.strokeStyle = "rgba(239, 68, 68, 0.7)";
  ctx.lineWidth = 1.5;
  ctx.setLineDash([4, 4]);

  // Tangent back-projection along incoming ray vector at Earth
  ctx.beginPath();
  ctx.moveTo(ptC.x, ptC.y);
  ctx.lineTo(ptB.x, ptB.y);
  const virtualY_at_galaxy = ptB.y + distGC_px * Math.tan(theta_after_vis_rad);
  ctx.lineTo(galaxyX, virtualY_at_galaxy);
  ctx.stroke();
  ctx.setLineDash([]);

  // Virtual Galaxy Image Dot
  ctx.fillStyle = "rgba(239, 68, 68, 0.9)";
  ctx.beginPath();
  ctx.arc(galaxyX, virtualY_at_galaxy, 5, 0, Math.PI * 2);
  ctx.fill();

  ctx.fillStyle = "#ef4444";
  ctx.font = "11px sans-serif";
  ctx.textAlign = "left";
  ctx.fillText("Apparent Image", galaxyX + 8, virtualY_at_galaxy + 4);
}

// 6. Draw Unbent Path Reference (Dashed light grey)
ctx.strokeStyle = "rgba(255, 255, 255, 0.15)";
ctx.lineWidth = 1;
ctx.setLineDash([3, 3]);
ctx.beginPath();
ctx.moveTo(ptA.x, ptA.y);
ctx.lineTo(ptB.x, ptB.y);
const unbentY_at_earth = ptB.y - distC_px * Math.tan(theta_vis_rad);
ctx.lineTo(earthX, unbentY_at_earth);
ctx.stroke();
ctx.setLineDash([]);

// 7. Draw Actual Deflected Light Ray
ctx.strokeStyle = "#facc15";
ctx.lineWidth = 2.5;
ctx.shadowColor = "#facc15";
ctx.shadowBlur = 10;

ctx.beginPath();
ctx.moveTo(ptA.x, ptA.y);
ctx.lineTo(ptB.x, ptB.y);
ctx.lineTo(ptC.x, ptC.y);
ctx.stroke();
ctx.shadowBlur = 0;

// 8. Impact Parameter (b) Dimension Indicator
if (Math.abs(yB_offset_px) > 3) {
  ctx.strokeStyle = "#00f0ff";
  ctx.lineWidth = 1.5;
  ctx.beginPath();
  ctx.moveTo(clusterX, centerY);
  ctx.lineTo(clusterX, ptB.y);
  ctx.stroke();

  ctx.fillStyle = "#00f0ff";
  ctx.beginPath();
  ctx.arc(clusterX, ptB.y, 3, 0, Math.PI * 2);
  ctx.arc(clusterX, centerY, 3, 0, Math.PI * 2);
  ctx.fill();

  ctx.font = "11px monospace";
  ctx.textAlign = "right";
  const labelY = (centerY + ptB.y) / 2;
  ctx.fillText(`b = ${phys.abs_b_ly/1000}.toFixed(1)} kly`, clusterX - 6, labelY + 4);
}

```

```

// 9. Deflection Angle Arc Indicator (at Point B)
if (phys.alpha_arcsec > 0.5) {
  ctx.save();
  ctx.translate(ptB.x, ptB.y);

  const startAngle = Math.PI - theta_vis_rad;
  const endAngle = Math.PI - theta_after_vis_rad;

  ctx.strokeStyle = "#ef4444";
  ctx.lineWidth = 1.8;
  ctx.beginPath();
  ctx.arc(0, 0, 38, startAngle, endAngle, phys.theta_init_arcsec < 0);
  ctx.stroke();

  ctx.fillStyle = "#ef4444";
  ctx.font = "11px sans-serif";
  ctx.textAlign = "right";
  ctx.fillText(`α = ${phys.alpha_arcsec.toFixed(1)}"`, -44, 4);
  ctx.restore();
}

// 10. Draw Animated Photon
if (togglePhoton.checked) {
  // Calculate position along segment 1 or 2
  let px = 0, py = 0;
  if (photonAnimProgress <= 0.5) {
    const t = photonAnimProgress / 0.5;
    px = ptA.x + t * (ptB.x - ptA.x);
    py = ptA.y + t * (ptB.y - ptA.y);
  } else {
    const t = (photonAnimProgress - 0.5) / 0.5;
    px = ptB.x + t * (ptC.x - ptB.x);
    py = ptB.y + t * (ptC.y - ptB.y);
  }

  ctx.fillStyle = "#ffffff";
  ctx.shadowColor = "#00f0ff";
  ctx.shadowBlur = 12;
  ctx.beginPath();
  ctx.arc(px, py, 4, 0, Math.PI * 2);
  ctx.fill();
  ctx.shadowBlur = 0;

  photonAnimProgress = (photonAnimProgress + 0.005) % 1.0;
}

// 11. Render Objects (Galaxy, Cluster, Earth)
// Galaxy
ctx.save();
ctx.translate(ptA.x, ptA.y);
const galGlow = ctx.createRadialGradient(0, 0, 2, 0, 0, 22);
galGlow.addColorStop(0, "rgba(255, 200, 100, 0.9)");
galGlow.addColorStop(0.5, "rgba(238, 100, 30, 0.3)");
galGlow.addColorStop(1, "rgba(0, 0, 0, 0)");
ctx.fillStyle = galGlow;
ctx.beginPath();
ctx.arc(0, 0, 22, 0, Math.PI * 2);
ctx.fill();

ctx.fillStyle = "#ffd59e";
ctx.beginPath();
ctx.ellipse(0, 0, 14, 5, -0.3, 0, Math.PI * 2);
ctx.fill();
ctx.restore();

ctx.fillStyle = "#e1e7ec";
ctx.font = "12px sans-serif";
ctx.textAlign = "center";
ctx.fillText("Distant Galaxy", ptA.x, ptA.y + 30);

// Cluster
ctx.save();
ctx.translate(clusterX, centerY);
const haloRadius = 18 + (phys.M_tril / 1000) * 22;

```

```

const clusterGlow = ctx.createRadialGradient(0, 0, 2, 0, 0, haloRadius);
clusterGlow.addColorStop(0, "rgba(168, 85, 247, 0.6)");
clusterGlow.addColorStop(0.6, "rgba(126, 34, 206, 0.2)");
clusterGlow.addColorStop(1, "rgba(0, 0, 0, 0)");
ctx.fillStyle = clusterGlow;
ctx.beginPath();
ctx.arc(0, 0, haloRadius, 0, Math.PI * 2);
ctx.fill();

ctx.fillStyle = "#ffffff";
ctx.beginPath();
ctx.arc(0, 0, 4, 0, Math.PI * 2);
ctx.arc(-8, -6, 2.5, 0, Math.PI * 2);
ctx.arc(10, 5, 3, 0, Math.PI * 2);
ctx.fill();
ctx.restore();

ctx.fillStyle = "#e1e7ec";
ctx.font = "12px sans-serif";
ctx.textAlign = "center";
ctx.fillText("Intermediate Cluster", clusterX, centerY + haloRadius + 16);

// Earth / Observer
ctx.save();
ctx.translate(earthX, centerY);
const earthGrad = ctx.createRadialGradient(-2, -2, 1, 0, 0, 10);
earthGrad.addColorStop(0, "#60a5fa");
earthGrad.addColorStop(0.8, "#1d4ed8");
earthGrad.addColorStop(1, "#1e3a8a");
ctx.fillStyle = earthGrad;
ctx.beginPath();
ctx.arc(0, 0, 9, 0, Math.PI * 2);
ctx.fill();
ctx.restore();

ctx.fillStyle = "#e1e7ec";
ctx.font = "12px sans-serif";
ctx.textAlign = "center";
ctx.fillText("Earth", earthX, centerY + 25);
}

function animate() {
  draw();
  requestAnimationFrame(animate);
}

animate();
</script>
</body>
</html>

```

User prompt: Use the same functions to now draw a number of rays (controlled by a slider, from 1 to 10), spread across a fan of 10 to 40 arc-seconds (controlled by a slider). The direction slider is inactive if the number of rays is greater than 1. If the number of rays is set to 1, the direction slider becomes active, and the fan slider becomes inactive.

Response: <!DOCTYPE html>
 <html lang="en">
 <head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Gravitational Lensing: Multi-Ray Fan Deflection</title>
 <style>
 :root {
 --bg-color: #0b0e14;
 --panel-bg: #151922;
 --panel-border: #2a3142;
 --accent-color: #00f0ff;
 --accent-amber: #facc15;
 --accent-red: #ef4444;
 --text-main: #e1e7ec;
 --text-muted: #8c9bae;
 }

```
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial, sans-serif;
  background-color: var(--bg-color);
  color: var(--text-main);
  display: flex;
  flex-direction: column;
  min-height: 100vh;
  overflow-x: hidden;
}

header {
  background-color: var(--panel-bg);
  border-bottom: 1px solid var(--panel-border);
  padding: 0.85rem 1.5rem;
  text-align: center;
}

header h1 {
  font-size: 1.3rem;
  font-weight: 600;
  color: #fff;
}

header p {
  font-size: 0.85rem;
  color: var(--text-muted);
  margin-top: 0.2rem;
}

.main-container {
  display: flex;
  flex: 1;
  flex-direction: column;
  gap: 1rem;
  padding: 1rem;
  max-width: 1400px;
  margin: 0 auto;
  width: 100%;
}

@media (min-width: 1024px) {
  .main-container {
    flex-direction: row;
  }
}

.canvas-card {
  flex: 1;
  background-color: var(--panel-bg);
  border: 1px solid var(--panel-border);
  border-radius: 8px;
  padding: 1rem;
  display: flex;
  flex-direction: column;
  gap: 0.5rem;
  box-shadow: 0 4px 20px rgba(0, 0, 0, 0.4);
  position: relative;
}

.canvas-wrapper {
  position: relative;
  width: 100%;
  height: 620px;
  background-color: #04060a;
  border-radius: 6px;
  overflow: hidden;
  border: 1px solid rgba(255, 255, 255, 0.05);
}
```

```
}

canvas {
  width: 100%;
  height: 100%;
  display: block;
}

.calc-box {
  position: absolute;
  top: 1.25rem;
  left: 1.25rem;
  background: rgba(15, 20, 30, 0.90);
  border: 1px solid var(--panel-border);
  backdrop-filter: blur(8px);
  border-radius: 8px;
  padding: 0.85rem 1.1rem;
  font-family: SFMono-Regular, Consolas, "Liberation Mono", Menlo, monospace;
  font-size: 0.78rem;
  line-height: 1.45;
  color: #d1d5db;
  box-shadow: 0 4px 18px rgba(0, 0, 0, 0.5);
  width: 350px;
  z-index: 10;
  pointer-events: none;
}

.calc-box h3 {
  font-size: 0.85rem;
  text-transform: uppercase;
  letter-spacing: 0.05em;
  color: var(--accent-color);
  margin-bottom: 0.5rem;
  border-bottom: 1px solid rgba(255, 255, 255, 0.1);
  padding-bottom: 0.3rem;
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;
}

.calc-row {
  display: flex;
  justify-content: space-between;
  margin-bottom: 0.25rem;
}

.calc-row .label {
  color: var(--text-muted);
}

.calc-row .value {
  color: #fff;
  font-weight: 600;
}

.calc-row .highlight {
  color: var(--accent-amber);
}

.calc-divider {
  border-top: 1px dashed rgba(255, 255, 255, 0.15);
  margin: 0.4rem 0;
}

.controls-panel {
  width: 100%;
  max-width: 380px;
  background-color: var(--panel-bg);
  border: 1px solid var(--panel-border);
  border-radius: 8px;
  padding: 1.25rem;
  display: flex;
  flex-direction: column;
  gap: 1.1rem;
  box-shadow: 0 4px 20px rgba(0, 0, 0, 0.4);
}
```

```
.control-group {
  display: flex;
  flex-direction: column;
  gap: 0.4rem;
  transition: opacity 0.2s ease;
}

.control-group.disabled-control {
  opacity: 0.35;
}

.control-group label {
  font-size: 0.83rem;
  font-weight: 500;
  color: var(--text-main);
  display: flex;
  justify-content: space-between;
}

.control-group label span {
  color: var(--accent-color);
  font-family: monospace;
}

input[type="range"] {
  -webkit-appearance: none;
  width: 100%;
  height: 6px;
  border-radius: 3px;
  background: #2a3142;
  outline: none;
}

input[type="range"]:disabled {
  cursor: not-allowed;
}

input[type="range"]::-webkit-slider-thumb {
  -webkit-appearance: none;
  width: 16px;
  height: 16px;
  border-radius: 50%;
  background: var(--accent-color);
  cursor: pointer;
  box-shadow: 0 0 8px var(--accent-color);
  transition: transform 0.1s ease;
}

input[type="range"]:disabled::-webkit-slider-thumb {
  background: #6b7280;
  box-shadow: none;
  cursor: not-allowed;
}

input[type="range"]::-webkit-slider-thumb:hover:not(:disabled) {
  transform: scale(1.2);
}

.toggle-row {
  display: flex;
  align-items: center;
  justify-content: space-between;
  font-size: 0.83rem;
  color: var(--text-main);
}

.switch {
  position: relative;
  display: inline-block;
  width: 40px;
  height: 20px;
}
```



```

.switch input {
  opacity: 0;
  width: 0;
  height: 0;
}

.slider {
  position: absolute;
  cursor: pointer;
  top: 0; left: 0; right: 0; bottom: 0;
  background-color: #2a3142;
  transition: .2s;
  border-radius: 20px;
}

.slider:before {
  position: absolute;
  content: "";
  height: 14px;
  width: 14px;
  left: 3px;
  bottom: 3px;
  background-color: white;
  transition: .2s;
  border-radius: 50%;
}

input:checked + .slider {
  background-color: var(--accent-color);
}

input:checked + .slider:before {
  transform: translateX(20px);
}

.legend {
  border-top: 1px solid var(--panel-border);
  padding-top: 0.8rem;
  display: flex;
  flex-direction: column;
  gap: 0.4rem;
  font-size: 0.8rem;
}

.legend-item {
  display: flex;
  align-items: center;
  gap: 0.6rem;
  color: var(--text-muted);
}

.legend-color {
  width: 14px;
  height: 4px;
  border-radius: 2px;
}
</style>
</head>
<body>

<header>
  <h1>Gravitational Deflection of Light Fan</h1>
  <p>Simulating multiple light rays deflected by an intermediate galaxy cluster (3600× angle exaggeration)</p>
</header>

<div class="main-container">
  <div class="canvas-card">
    <div class="canvas-wrapper">
      <canvas id="simCanvas"></canvas>
      <div class="calc-box">
        <h3>Numerical Results</h3>
        <div class="calc-row">
          <span class="label">Galaxy Dist (d<sub>G</sub>)</span>
          <span class="value" id="boxDistG">8.0 Gly</span>
        </div>
      </div>
    </div>
  </div>
</div>

```

```

</div>
<div class="calc-row">
  <span class="label">Cluster Dist ( $d_C$ ):</span>
  <span class="value" id="boxDistC">4.0 Gly</span>
</div>
<div class="calc-row">
  <span class="label"> $d_{GC} = d_G - d_C$ :</span>
  <span class="value" id="boxDistGC">4.0 Gly</span>
</div>
<div class="calc-row">
  <span class="label">Cluster Mass (M):</span>
  <span class="value" id="boxMass">200 × 1012 M⊙</span>
</div>
<div class="calc-divider"></div>
<div class="calc-row">
  <span class="label">Ray Mode:</span>
  <span class="value" id="boxMode">Multi-Ray Fan (5 rays)</span>
</div>
<div class="calc-row" id="rowBoxAngle">
  <span class="label">Initial Angle Spread:</span>
  <span class="value" id="boxAngleVal">20.0° (-10.0° to +10.0°)</span>
</div>
<div class="calc-divider"></div>
<div class="calc-row">
  <span class="label">Impact Param (b) Range:</span>
  <span class="value highlight" id="boxImpactRange">0.0 to 193.9 kly</span>
</div>
<div class="calc-row">
  <span class="label">Deflection Angle (α):</span>
  <span class="value highlight" id="boxAlphaRange">66.4° max</span>
</div>
<div class="calc-row">
  <span class="label">Formula:</span>
  <span class="value" style="font-size: 0.72rem; color: #9ca3af;"> $\alpha = 4GM / (c^2 b)$ </span>
</div>
</div>
</div>
</div>
<div class="controls-panel">
  <div class="control-group">
    <label for="sliderDistG">
      1. Distance to Galaxy ( $d_G$ ):
      <span id="valDistG">8.0 Gly</span>
    </label>
    <input type="range" id="sliderDistG" min="2.0" max="13.0" step="0.1" value="8.0">
  </div>
  <div class="control-group">
    <label for="sliderDistC">
      2. Distance to Cluster ( $d_C$ ):
      <span id="valDistC">4.0 Gly</span>
    </label>
    <input type="range" id="sliderDistC" min="1.0" max="12.0" step="0.1" value="4.0">
  </div>
  <div class="control-group">
    <label for="sliderMass">
      3. Cluster Mass (M):
      <span id="valMass">200 × 1012 M⊙</span>
    </label>
    <input type="range" id="sliderMass" min="5" max="1000" step="5" value="200">
  </div>
  <div class="control-group">
    <label for="sliderNumRays">
      4. Number of Light Rays:
      <span id="valNumRays">5</span>
    </label>
    <input type="range" id="sliderNumRays" min="1" max="10" step="1" value="5">
  </div>
  <div class="control-group" id="grpFanSpread">
    <label for="sliderFanSpread">

```

```

    5. Ray Fan Spread:
    <span id="valFanSpread">20</span>
  </label>
  <input type="range" id="sliderFanSpread" min="10" max="40" step="1" value="20">
</div>

<div class="control-group" id="grpTheta">
  <label for="sliderTheta">
    6. Initial Ray Direction (1 ray):
    <span id="valTheta">+15.0</span>
  </label>
  <input type="range" id="sliderTheta" min="-40.0" max="40.0" step="0.5" value="15.0">
</div>

<div class="toggle-row">
  <span>Animate Photons</span>
  <label class="switch">
    <input type="checkbox" id="togglePhoton" checked>
    <span class="slider"></span>
  </label>
</div>

<div class="toggle-row">
  <span>Show Virtual Tangent Lines</span>
  <label class="switch">
    <input type="checkbox" id="toggleTangent" checked>
    <span class="slider"></span>
  </label>
</div>

<div class="legend">
  <div class="legend-item">
    <div class="legend-color" style="background: #facc15;"></div>
    <span>Actual deflected light rays</span>
  </div>
  <div class="legend-item">
    <div class="legend-color" style="border: 1px dashed #ef4444; background: transparent;"></div>
    <span>Apparent straight-line paths (virtual images)</span>
  </div>
  <div class="legend-item">
    <div class="legend-color" style="background: #00f0ff;"></div>
    <span>Lens plane / Impact parameter (b)</span>
  </div>
</div>
</div>
</div>

<script>
const canvas = document.getElementById('simCanvas');
const ctx = canvas.getContext('2d');

// Controls
const sliderDistG = document.getElementById('sliderDistG');
const sliderDistC = document.getElementById('sliderDistC');
const sliderMass = document.getElementById('sliderMass');
const sliderNumRays = document.getElementById('sliderNumRays');
const sliderFanSpread = document.getElementById('sliderFanSpread');
const sliderTheta = document.getElementById('sliderTheta');

const grpFanSpread = document.getElementById('grpFanSpread');
const grpTheta = document.getElementById('grpTheta');

const valDistG = document.getElementById('valDistG');
const valDistC = document.getElementById('valDistC');
const valMass = document.getElementById('valMass');
const valNumRays = document.getElementById('valNumRays');
const valFanSpread = document.getElementById('valFanSpread');
const valTheta = document.getElementById('valTheta');

const togglePhoton = document.getElementById('togglePhoton');
const toggleTangent = document.getElementById('toggleTangent');

// Calculation Box Elements
const boxDistG = document.getElementById('boxDistG');

```

```

const boxDistC = document.getElementById('boxDistC');
const boxDistGC = document.getElementById('boxDistGC');
const boxMass = document.getElementById('boxMass');
const boxMode = document.getElementById('boxMode');
const boxAngleVal = document.getElementById('boxAngleVal');
const boxImpactRange = document.getElementById('boxImpactRange');
const boxAlphaRange = document.getElementById('boxAlphaRange');

let width = 0;
let height = 0;
let photonAnimProgress = 0;

// Background Stars
const stars = [];
for (let i = 0; i < 220; i++) {
  stars.push({
    x: Math.random(),
    y: Math.random(),
    size: Math.random() * 1.5 + 0.3,
    alpha: Math.random() * 0.7 + 0.3
  });
}

function resizeCanvas() {
  const rect = canvas.parentElement.getBoundingClientRect();
  const dpr = window.devicePixelRatio || 1;
  width = rect.width || 800;
  height = rect.height || 620;

  canvas.width = width * dpr;
  canvas.height = height * dpr;
  ctx.scale(dpr, dpr);
}

window.addEventListener('resize', resizeCanvas);
resizeCanvas();

// Enforce distance constraints: d_G >= d_C + 1.0 Gly
sliderDistG.addEventListener('input', () => {
  let dG = parseFloat(sliderDistG.value);
  let dC = parseFloat(sliderDistC.value);
  if (dG < dC + 1.0) {
    dC = Math.max(1.0, dG - 1.0);
    sliderDistC.value = dC.toFixed(1);
  }
  updateControlStates();
});

sliderDistC.addEventListener('input', () => {
  let dG = parseFloat(sliderDistG.value);
  let dC = parseFloat(sliderDistC.value);
  if (dC > dG - 1.0) {
    dG = Math.min(13.0, dC + 1.0);
    sliderDistG.value = dG.toFixed(1);
  }
  updateControlStates();
});

function updateControlStates() {
  const numRays = parseInt(sliderNumRays.value);

  if (numRays > 1) {
    // Multi-ray mode: Angle/Direction slider inactive, Fan Spread active
    sliderTheta.disabled = true;
    grpTheta.classList.add('disabled-control');

    sliderFanSpread.disabled = false;
    grpFanSpread.classList.remove('disabled-control');
  } else {
    // Single-ray mode: Angle/Direction slider active, Fan Spread inactive
    sliderTheta.disabled = false;
    grpTheta.classList.remove('disabled-control');

    sliderFanSpread.disabled = true;
  }
}

```

```

    grpFanSpread.classList.add('disabled-control');
  }
}

sliderNumRays.addEventListener('input', updateControlStates);
updateControlStates();

function calculateRays() {
  const d_G_Gly = parseFloat(sliderDistG.value);
  const d_C_Gly = parseFloat(sliderDistC.value);
  const d_GC_Gly = d_G_Gly - d_C_Gly;
  const M_tril = parseFloat(sliderMass.value);
  const numRays = parseInt(sliderNumRays.value);
  const fanSpreadArcsec = parseFloat(sliderFanSpread.value);
  const thetaInitArcsec = parseFloat(sliderTheta.value);

  const ARCSEC_TO_RAD = Math.PI / (180.0 * 3600.0);
  const d_GC_ly = d_GC_Gly * 1e9;

  // Scale factor for deflection:  $4GM/c^2 = 0.6243363$  light-years per  $10^{12} M_{\odot}$ 
  const scale_4GM_c2_ly = 0.6243363 * M_tril;

  const rays = [];

  for (let i = 0; i < numRays; i++) {
    let initialAngle = 0;

    if (numRays === 1) {
      initialAngle = thetaInitArcsec;
    } else {
      // Spread fan evenly from -fanSpread/2 to +fanSpread/2
      const frac = (i / (numRays - 1)) - 0.5;
      initialAngle = frac * fanSpreadArcsec;
    }

    const theta_init_rad = initialAngle * ARCSEC_TO_RAD;
    const b_ly = d_GC_ly * theta_init_rad;
    const abs_b_ly = Math.abs(b_ly);

    let alpha_arcsec = 0;
    if (abs_b_ly > 1e-3) {
      const alpha_rad = scale_4GM_c2_ly / abs_b_ly;
      alpha_arcsec = alpha_rad / ARCSEC_TO_RAD;
    }

    // Deflection direction: pulls toward lens plane (center line)
    let theta_after_arcsec = initialAngle;
    if (initialAngle > 0) {
      theta_after_arcsec = initialAngle - alpha_arcsec;
    } else if (initialAngle < 0) {
      theta_after_arcsec = initialAngle + alpha_arcsec;
    }

    rays.push({
      index: i,
      theta_init_arcsec: initialAngle,
      b_ly: b_ly,
      abs_b_ly: abs_b_ly,
      alpha_arcsec: alpha_arcsec,
      theta_after_arcsec: theta_after_arcsec
    });
  }

  return {
    d_G_Gly,
    d_C_Gly,
    d_GC_Gly,
    M_tril,
    numRays,
    fanSpreadArcsec,
    thetaInitArcsec,
    rays
  };
}

```

```

function draw() {
  ctx.clearRect(0, 0, width, height);

  const sim = calculateRays();

  // Label Updates
  valDistG.textContent = `${sim.d_G_Gly.toFixed(1)} Gly`;
  valDistC.textContent = `${sim.d_C_Gly.toFixed(1)} Gly`;
  valMass.textContent = `${sim.M_tril} × 1012 M⊙`;
  valNumRays.textContent = `${sim.numRays}`;
  valFanSpread.textContent = `${sim.fanSpreadArcsec}`;
  valTheta.textContent = `${sim.thetaInitArcsec >= 0 ? '+' : ''}${sim.thetaInitArcsec.toFixed(1)}°`;

  // Update Numerical Box
  boxDistG.textContent = `${sim.d_G_Gly.toFixed(1)} Gly`;
  boxDistC.textContent = `${sim.d_C_Gly.toFixed(1)} Gly`;
  boxDistGC.textContent = `${sim.d_GC_Gly.toFixed(1)} Gly`;
  boxMass.textContent = `${sim.M_tril} × 1012 M⊙`;

  if (sim.numRays === 1) {
    const r = sim.rays[0];
    boxMode.textContent = `Single Ray`;
    boxAngleVal.textContent = `${r.theta_init_arcsec >= 0 ? '+' : ''}${r.theta_init_arcsec.toFixed(1)}°`;
    boxImpactRange.textContent = `${(r.abs_b_ly / 1000.0).toFixed(2)} kly (${Math.round(r.abs_b_ly).toLocaleString()})`;
    boxAlphaRange.textContent = `${r.alpha_arcsec.toFixed(2)}°`;
  } else {
    boxMode.textContent = `Fan (${sim.numRays} rays)`;
    boxAngleVal.textContent = `${sim.fanSpreadArcsec} (−${(sim.fanSpreadArcsec/2).toFixed(1)}° to +${(sim.fanSpreadArcsec/2).toFixed(1)}°)`;

    const bValues = sim.rays.map(r => r.abs_b_ly / 1000.0);
    const alphaValues = sim.rays.map(r => r.alpha_arcsec).filter(a => isFinite(a));

    const minB = Math.min(...bValues);
    const maxB = Math.max(...bValues);
    const maxAlpha = Math.max(...alphaValues);

    boxImpactRange.textContent = `${minB.toFixed(1)} to ${maxB.toFixed(1)} kly`;
    boxAlphaRange.textContent = `${maxAlpha.toFixed(2)}° max`;
  }

  // Visual Layout Dimensions
  const marginX = 80;
  const centerY = height / 2;

  const earthX = marginX;
  const galaxyX = width - marginX;
  const totalWidthPx = galaxyX - earthX;

  const pxPerGly = totalWidthPx / sim.d_G_Gly;
  const clusterX = earthX + (sim.d_C_Gly * pxPerGly);

  const DEG_TO_RAD = Math.PI / 180.0;
  const distGC_px = galaxyX - clusterX;
  const distC_px = clusterX - earthX;

  // 1. Starfield Background
  ctx.fillStyle = "#ffffff";
  stars.forEach(s => {
    ctx.globalAlpha = s.alpha;
    ctx.fillRect(s.x * width, s.y * height, s.size, s.size);
  });
  ctx.globalAlpha = 1.0;

  // 2. Baseline Axis
  ctx.strokeStyle = "rgba(255, 255, 255, 0.15)";
  ctx.lineWidth = 1;
  ctx.setLineDash([6, 6]);
  ctx.beginPath();
  ctx.moveTo(earthX - 20, centerY);
  ctx.lineTo(galaxyX + 20, centerY);
  ctx.stroke();
  ctx.setLineDash([]);

```

```

// 3. Cluster Plane Line
ctx.strokeStyle = "rgba(0, 240, 255, 0.25)";
ctx.beginPath();
ctx.moveTo(clusterX, 40);
ctx.lineTo(clusterX, height - 40);
ctx.stroke();

const ptA = { x: galaxyX, y: centerY };

// 4. Render Light Rays & Virtual Tangents
sim.rays.forEach((ray, idx) => {
  // Angles scaled by 3600 factor (1 arcsec = 1 deg)
  const theta_vis_rad = ray.theta_init_arcsec * DEG_TO_RAD;
  const alpha_vis_rad = ray.alpha_arcsec * DEG_TO_RAD;

  const yB_offset_px = distGC_px * Math.tan(theta_vis_rad);
  const ptB = { x: clusterX, y: centerY - yB_offset_px };

  let theta_after_vis_rad = theta_vis_rad;
  if (ray.theta_init_arcsec > 0) {
    theta_after_vis_rad = theta_vis_rad - alpha_vis_rad;
  } else if (ray.theta_init_arcsec < 0) {
    theta_after_vis_rad = theta_vis_rad + alpha_vis_rad;
  }

  const yC_offset_px = yB_offset_px + distC_px * Math.tan(theta_after_vis_rad);
  const ptC = { x: earthX, y: centerY - yC_offset_px };

  // Draw Virtual Tangent Lines
  if (toggleTangent.checked && Math.abs(ray.alpha_arcsec) > 0.1) {
    ctx.strokeStyle = "rgba(239, 68, 68, 0.5)";
    ctx.lineWidth = 1.2;
    ctx.setLineDash([3, 3]);

    ctx.beginPath();
    ctx.moveTo(ptC.x, ptC.y);
    ctx.lineTo(ptB.x, ptB.y);
    const virtualY_at_galaxy = ptB.y + distGC_px * Math.tan(theta_after_vis_rad);
    ctx.lineTo(galaxyX, virtualY_at_galaxy);
    ctx.stroke();
    ctx.setLineDash([]);

    ctx.fillStyle = "rgba(239, 68, 68, 0.8)";
    ctx.beginPath();
    ctx.arc(galaxyX, virtualY_at_galaxy, 3.5, 0, Math.PI * 2);
    ctx.fill();
  }

  // Draw Actual Light Path
  ctx.strokeStyle = "#facc15";
  ctx.lineWidth = sim.numRays === 1 ? 2.5 : 1.8;
  ctx.shadowColor = "#facc15";
  ctx.shadowBlur = sim.numRays === 1 ? 8 : 4;

  ctx.beginPath();
  ctx.moveTo(ptA.x, ptA.y);
  ctx.lineTo(ptB.x, ptB.y);
  ctx.lineTo(ptC.x, ptC.y);
  ctx.stroke();
  ctx.shadowBlur = 0;

  // Single Ray Impact Parameter visual indicator
  if (sim.numRays === 1 && Math.abs(yB_offset_px) > 3) {
    ctx.strokeStyle = "#00f0ff";
    ctx.lineWidth = 1.5;
    ctx.beginPath();
    ctx.moveTo(clusterX, centerY);
    ctx.lineTo(clusterX, ptB.y);
    ctx.stroke();

    ctx.fillStyle = "#00f0ff";
    ctx.beginPath();
    ctx.arc(clusterX, ptB.y, 3, 0, Math.PI * 2);
    ctx.arc(clusterX, centerY, 3, 0, Math.PI * 2);
  }
});

```

```

    ctx.fill();

    ctx.font = "11px monospace";
    ctx.textAlign = "right";
    ctx.fillText(`b = ${((ray.abs_b_ly / 1000).toFixed(1))} kly`, clusterX - 6, (centerY + ptB.y) / 2 + 4
  }

  // Animated Photons along Rays
  if (togglePhoton.checked) {
    const rayOffset = (idx / sim.numRays);
    const progress = (photonAnimProgress + rayOffset) % 1.0;
    let px = 0, py = 0;

    if (progress <= 0.5) {
      const t = progress / 0.5;
      px = ptA.x + t * (ptB.x - ptA.x);
      py = ptA.y + t * (ptB.y - ptA.y);
    } else {
      const t = (progress - 0.5) / 0.5;
      px = ptB.x + t * (ptC.x - ptB.x);
      py = ptB.y + t * (ptC.y - ptB.y);
    }

    ctx.fillStyle = "#ffffff";
    ctx.shadowColor = "#00f0ff";
    ctx.shadowBlur = 8;
    ctx.beginPath();
    ctx.arc(px, py, 3.5, 0, Math.PI * 2);
    ctx.fill();
    ctx.shadowBlur = 0;
  }
});

if (togglePhoton.checked) {
  photonAnimProgress = (photonAnimProgress + 0.006) % 1.0;
}

// 5. Draw Celestial Objects
// Galaxy
ctx.save();
ctx.translate(ptA.x, ptA.y);
const galGlow = ctx.createRadialGradient(0, 0, 2, 0, 0, 22);
galGlow.addColorStop(0, "rgba(255, 200, 100, 0.9)");
galGlow.addColorStop(0.5, "rgba(238, 100, 30, 0.3)");
galGlow.addColorStop(1, "rgba(0, 0, 0, 0)");
ctx.fillStyle = galGlow;
ctx.beginPath();
ctx.arc(0, 0, 22, 0, Math.PI * 2);
ctx.fill();

ctx.fillStyle = "#ffd59e";
ctx.beginPath();
ctx.ellipse(0, 0, 14, 5, -0.3, 0, Math.PI * 2);
ctx.fill();
ctx.restore();

ctx.fillStyle = "#e1e7ec";
ctx.font = "12px sans-serif";
ctx.textAlign = "center";
ctx.fillText("Distant Galaxy", ptA.x, ptA.y + 32);

// Intermediate Cluster
ctx.save();
ctx.translate(clusterX, centerY);
const haloRadius = 18 + (sim.M_tril / 1000) * 22;
const clusterGlow = ctx.createRadialGradient(0, 0, 2, 0, 0, haloRadius);
clusterGlow.addColorStop(0, "rgba(168, 85, 247, 0.65)");
clusterGlow.addColorStop(0.6, "rgba(126, 34, 206, 0.2)");
clusterGlow.addColorStop(1, "rgba(0, 0, 0, 0)");
ctx.fillStyle = clusterGlow;
ctx.beginPath();
ctx.arc(0, 0, haloRadius, 0, Math.PI * 2);
ctx.fill();

```



```

    ctx.fillStyle = "#ffffff";
    ctx.beginPath();
    ctx.arc(0, 0, 4, 0, Math.PI * 2);
    ctx.arc(-8, -6, 2.5, 0, Math.PI * 2);
    ctx.arc(10, 5, 3, 0, Math.PI * 2);
    ctx.fill();
    ctx.restore();

    ctx.fillStyle = "#e1e7ec";
    ctx.font = "12px sans-serif";
    ctx.textAlign = "center";
    ctx.fillText("Intermediate Cluster", cClusterX, centerY + haloRadius + 16);

    // Earth / Observer
    ctx.save();
    ctx.translate(earthX, centerY);
    const earthGrad = ctx.createRadialGradient(-2, -2, 1, 0, 0, 10);
    earthGrad.addColorStop(0, "#60a5fa");
    earthGrad.addColorStop(0.8, "#1d4ed8");
    earthGrad.addColorStop(1, "#1e3a8a");
    ctx.fillStyle = earthGrad;
    ctx.beginPath();
    ctx.arc(0, 0, 9, 0, Math.PI * 2);
    ctx.fill();
    ctx.restore();

    ctx.fillStyle = "#e1e7ec";
    ctx.font = "12px sans-serif";
    ctx.textAlign = "center";
    ctx.fillText("Earth", earthX, centerY + 25);
}

function animate() {
    draw();
    requestAnimationFrame(animate);
}

animate();
</script>
</body>
</html>

```

User prompt: move the numerical outputs to a box below the graphic

```

Response: <!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Gravitational Lensing: Multi-Ray Fan Deflection</title>
  <style>
    :root {
      --bg-color: #0b0e14;
      --panel-bg: #151922;
      --panel-border: #2a3142;
      --accent-color: #00f0ff;
      --accent-amber: #facc15;
      --accent-red: #ef4444;
      --text-main: #e1e7ec;
      --text-muted: #8c9bae;
    }

    * {
      box-sizing: border-box;
      margin: 0;
      padding: 0;
    }

    body {
      font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial, sans-serif;
      background-color: var(--bg-color);
      color: var(--text-main);

```

```

    display: flex;
    flex-direction: column;
    min-height: 100vh;
    overflow-x: hidden;
}

header {
    background-color: var(--panel-bg);
    border-bottom: 1px solid var(--panel-border);
    padding: 0.85rem 1.5rem;
    text-align: center;
}

header h1 {
    font-size: 1.3rem;
    font-weight: 600;
    color: #fff;
}

header p {
    font-size: 0.85rem;
    color: var(--text-muted);
    margin-top: 0.2rem;
}

.main-container {
    display: flex;
    flex: 1;
    flex-direction: column;
    gap: 1rem;
    padding: 1rem;
    max-width: 1400px;
    margin: 0 auto;
    width: 100%;
}

@media (min-width: 1024px) {
    .main-container {
        flex-direction: row;
    }
}

.canvas-card {
    flex: 1;
    background-color: var(--panel-bg);
    border: 1px solid var(--panel-border);
    border-radius: 8px;
    padding: 1rem;
    display: flex;
    flex-direction: column;
    gap: 1rem;
    box-shadow: 0 4px 20px rgba(0, 0, 0, 0.4);
}

.canvas-wrapper {
    position: relative;
    width: 100%;
    height: 480px;
    background-color: #04060a;
    border-radius: 6px;
    overflow: hidden;
    border: 1px solid rgba(255, 255, 255, 0.05);
}

canvas {
    width: 100%;
    height: 100%;
    display: block;
}

/* Numerical Box located below graphic */
.calc-box {
    background: rgba(10, 14, 22, 0.7);
    border: 1px solid var(--panel-border);
}

```

```
border-radius: 6px;
padding: 1rem 1.25rem;
font-family: SFMono-Regular, Consolas, "Liberation Mono", Menlo, monospace;
font-size: 0.8rem;
color: #d1d5db;
}

.calc-box h3 {
  font-size: 0.85rem;
  text-transform: uppercase;
  letter-spacing: 0.05em;
  color: var(--accent-color);
  margin-bottom: 0.75rem;
  border-bottom: 1px solid rgba(255, 255, 255, 0.1);
  padding-bottom: 0.4rem;
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;
}

.calc-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(190px, 1fr));
  gap: 0.85rem 1.25rem;
}

.calc-item {
  display: flex;
  flex-direction: column;
  gap: 0.25rem;
}

.calc-item .label {
  color: var(--text-muted);
  font-size: 0.75rem;
}

.calc-item .value {
  color: #fff;
  font-weight: 600;
  font-size: 0.85rem;
}

.calc-item .highlight {
  color: var(--accent-amber);
}

.calc-footer {
  margin-top: 0.85rem;
  padding-top: 0.6rem;
  border-top: 1px dashed rgba(255, 255, 255, 0.15);
  display: flex;
  justify-content: space-between;
  align-items: center;
  font-size: 0.75rem;
}

.calc-footer .formula {
  color: #9ca3af;
}

.controls-panel {
  width: 100%;
  max-width: 380px;
  background-color: var(--panel-bg);
  border: 1px solid var(--panel-border);
  border-radius: 8px;
  padding: 1.25rem;
  display: flex;
  flex-direction: column;
  gap: 1.1rem;
  box-shadow: 0 4px 20px rgba(0, 0, 0, 0.4);
}

.control-group {
  display: flex;
```

```
    flex-direction: column;
    gap: 0.4rem;
    transition: opacity 0.2s ease;
  }

  .control-group.disabled-control {
    opacity: 0.35;
  }

  .control-group label {
    font-size: 0.83rem;
    font-weight: 500;
    color: var(--text-main);
    display: flex;
    justify-content: space-between;
  }

  .control-group label span {
    color: var(--accent-color);
    font-family: monospace;
  }

  input[type="range"] {
    -webkit-appearance: none;
    width: 100%;
    height: 6px;
    border-radius: 3px;
    background: #2a3142;
    outline: none;
  }

  input[type="range"]:disabled {
    cursor: not-allowed;
  }

  input[type="range"]::-webkit-slider-thumb {
    -webkit-appearance: none;
    width: 16px;
    height: 16px;
    border-radius: 50%;
    background: var(--accent-color);
    cursor: pointer;
    box-shadow: 0 0 8px var(--accent-color);
    transition: transform 0.1s ease;
  }

  input[type="range"]:disabled::-webkit-slider-thumb {
    background: #6b7280;
    box-shadow: none;
    cursor: not-allowed;
  }

  input[type="range"]::-webkit-slider-thumb:hover:not(:disabled) {
    transform: scale(1.2);
  }

  .toggle-row {
    display: flex;
    align-items: center;
    justify-content: space-between;
    font-size: 0.83rem;
    color: var(--text-main);
  }

  .switch {
    position: relative;
    display: inline-block;
    width: 40px;
    height: 20px;
  }

  .switch input {
    opacity: 0;
    width: 0;
  }
```

```

    height: 0;
  }

  .slider {
    position: absolute;
    cursor: pointer;
    top: 0; left: 0; right: 0; bottom: 0;
    background-color: #2a3142;
    transition: .2s;
    border-radius: 20px;
  }

  .slider:before {
    position: absolute;
    content: "";
    height: 14px;
    width: 14px;
    left: 3px;
    bottom: 3px;
    background-color: white;
    transition: .2s;
    border-radius: 50%;
  }

  input:checked + .slider {
    background-color: var(--accent-color);
  }

  input:checked + .slider:before {
    transform: translateX(20px);
  }

  .legend {
    border-top: 1px solid var(--panel-border);
    padding-top: 0.8rem;
    display: flex;
    flex-direction: column;
    gap: 0.4rem;
    font-size: 0.8rem;
  }

  .legend-item {
    display: flex;
    align-items: center;
    gap: 0.6rem;
    color: var(--text-muted);
  }

  .legend-color {
    width: 14px;
    height: 4px;
    border-radius: 2px;
  }
</style>
</head>
<body>

<header>
  <h1>Gravitational Deflection of Light Fan</h1>
  <p>Simulating multiple light rays deflected by an intermediate galaxy cluster (3600× angle exaggeration)</p>
</header>

<div class="main-container">
  <div class="canvas-card">
    <div class="canvas-wrapper">
      <canvas id="simCanvas"></canvas>
    </div>

    <!-- Numerical Output Box below Graphic -->
    <div class="calc-box">
      <h3>Numerical Results</h3>
      <div class="calc-grid">
        <div class="calc-item">
          <span class="label">Galaxy Distance (d<sub>G</sub>):</span>

```

```

        <span class="value" id="boxDistG">8.0 Gly</span>
    </div>
    <div class="calc-item">
        <span class="label">Cluster Distance (d<sub>C</sub>):</span>
        <span class="value" id="boxDistC">4.0 Gly</span>
    </div>
    <div class="calc-item">
        <span class="label">d<sub>GC</sub> = d<sub>G</sub> - d<sub>C</sub>:</span>
        <span class="value" id="boxDistGC">4.0 Gly</span>
    </div>
    <div class="calc-item">
        <span class="label">Cluster Mass (M):</span>
        <span class="value" id="boxMass">200 × 1012 M⊙</span>
    </div>
    <div class="calc-item">
        <span class="label">Ray Mode:</span>
        <span class="value" id="boxMode">Multi-Ray Fan (5 rays)</span>
    </div>
    <div class="calc-item">
        <span class="label">Initial Angle / Spread:</span>
        <span class="value" id="boxAngleVal">20.0° (-10.0° to +10.0°)</span>
    </div>
    <div class="calc-item">
        <span class="label">Impact Parameter (b):</span>
        <span class="value highlight" id="boxImpactRange">0.0 to 193.9 kly</span>
    </div>
    <div class="calc-item">
        <span class="label">Deflection Angle (α):</span>
        <span class="value highlight" id="boxAlphaRange">66.4° max</span>
    </div>
</div>
<div class="calc-footer">
    <span class="label">Theoretical Formula:</span>
    <span class="formula">α = 4GM / (c2b)</span>
</div>
</div>
</div>
<div class="controls-panel">
    <div class="control-group">
        <label for="sliderDistG">
            1. Distance to Galaxy (d<sub>G</sub>):
            <span id="valDistG">8.0 Gly</span>
        </label>
        <input type="range" id="sliderDistG" min="2.0" max="13.0" step="0.1" value="8.0">
    </div>

    <div class="control-group">
        <label for="sliderDistC">
            2. Distance to Cluster (d<sub>C</sub>):
            <span id="valDistC">4.0 Gly</span>
        </label>
        <input type="range" id="sliderDistC" min="1.0" max="12.0" step="0.1" value="4.0">
    </div>

    <div class="control-group">
        <label for="sliderMass">
            3. Cluster Mass (M):
            <span id="valMass">200 × 1012 M⊙</span>
        </label>
        <input type="range" id="sliderMass" min="5" max="1000" step="5" value="200">
    </div>

    <div class="control-group">
        <label for="sliderNumRays">
            4. Number of Light Rays:
            <span id="valNumRays">5</span>
        </label>
        <input type="range" id="sliderNumRays" min="1" max="10" step="1" value="5">
    </div>

    <div class="control-group" id="grpFanSpread">
        <label for="sliderFanSpread">
            5. Ray Fan Spread:

```

```

    <span id="valFanSpread">20</span>
  </label>
  <input type="range" id="sliderFanSpread" min="10" max="40" step="1" value="20">
</div>

<div class="control-group" id="grpTheta">
  <label for="sliderTheta">
    6. Initial Ray Direction (1 ray):
    <span id="valTheta">+15.0</span>
  </label>
  <input type="range" id="sliderTheta" min="-40.0" max="40.0" step="0.5" value="15.0">
</div>

<div class="toggle-row">
  <span>Animate Photons</span>
  <label class="switch">
    <input type="checkbox" id="togglePhoton" checked>
    <span class="slider"></span>
  </label>
</div>

<div class="toggle-row">
  <span>Show Virtual Tangent Lines</span>
  <label class="switch">
    <input type="checkbox" id="toggleTangent" checked>
    <span class="slider"></span>
  </label>
</div>

<div class="legend">
  <div class="legend-item">
    <div class="legend-color" style="background: #facc15;"></div>
    <span>Actual deflected light rays</span>
  </div>
  <div class="legend-item">
    <div class="legend-color" style="border: 1px dashed #ef4444; background: transparent;"></div>
    <span>Apparent straight-line paths (virtual images)</span>
  </div>
  <div class="legend-item">
    <div class="legend-color" style="background: #00f0ff;"></div>
    <span>Lens plane / Impact parameter (b)</span>
  </div>
</div>
</div>
</div>

<script>
const canvas = document.getElementById('simCanvas');
const ctx = canvas.getContext('2d');

// Controls
const sliderDistG = document.getElementById('sliderDistG');
const sliderDistC = document.getElementById('sliderDistC');
const sliderMass = document.getElementById('sliderMass');
const sliderNumRays = document.getElementById('sliderNumRays');
const sliderFanSpread = document.getElementById('sliderFanSpread');
const sliderTheta = document.getElementById('sliderTheta');

const grpFanSpread = document.getElementById('grpFanSpread');
const grpTheta = document.getElementById('grpTheta');

const valDistG = document.getElementById('valDistG');
const valDistC = document.getElementById('valDistC');
const valMass = document.getElementById('valMass');
const valNumRays = document.getElementById('valNumRays');
const valFanSpread = document.getElementById('valFanSpread');
const valTheta = document.getElementById('valTheta');

const togglePhoton = document.getElementById('togglePhoton');
const toggleTangent = document.getElementById('toggleTangent');

// Calculation Box Elements
const boxDistG = document.getElementById('boxDistG');
const boxDistC = document.getElementById('boxDistC');

```

```

const boxDistGC = document.getElementById('boxDistGC');
const boxMass = document.getElementById('boxMass');
const boxMode = document.getElementById('boxMode');
const boxAngleVal = document.getElementById('boxAngleVal');
const boxImpactRange = document.getElementById('boxImpactRange');
const boxAlphaRange = document.getElementById('boxAlphaRange');

let width = 0;
let height = 0;
let photonAnimProgress = 0;

// Background Stars
const stars = [];
for (let i = 0; i < 220; i++) {
  stars.push({
    x: Math.random(),
    y: Math.random(),
    size: Math.random() * 1.5 + 0.3,
    alpha: Math.random() * 0.7 + 0.3
  });
}

function resizeCanvas() {
  const rect = canvas.parentElement.getBoundingClientRect();
  const dpr = window.devicePixelRatio || 1;
  width = rect.width || 800;
  height = rect.height || 480;

  canvas.width = width * dpr;
  canvas.height = height * dpr;
  ctx.scale(dpr, dpr);
}

window.addEventListener('resize', resizeCanvas);
resizeCanvas();

// Distance constraints: d_G >= d_C + 1.0 Gly
sliderDistG.addEventListener('input', () => {
  let dG = parseFloat(sliderDistG.value);
  let dC = parseFloat(sliderDistC.value);
  if (dG < dC + 1.0) {
    dC = Math.max(1.0, dG - 1.0);
    sliderDistC.value = dC.toFixed(1);
  }
  updateControlStates();
});

sliderDistC.addEventListener('input', () => {
  let dG = parseFloat(sliderDistG.value);
  let dC = parseFloat(sliderDistC.value);
  if (dC > dG - 1.0) {
    dG = Math.min(13.0, dC + 1.0);
    sliderDistG.value = dG.toFixed(1);
  }
  updateControlStates();
});

function updateControlStates() {
  const numRays = parseInt(sliderNumRays.value);

  if (numRays > 1) {
    // Multi-ray mode: Angle/Direction slider inactive, Fan Spread active
    sliderTheta.disabled = true;
    grpTheta.classList.add('disabled-control');

    sliderFanSpread.disabled = false;
    grpFanSpread.classList.remove('disabled-control');
  } else {
    // Single-ray mode: Angle/Direction slider active, Fan Spread inactive
    sliderTheta.disabled = false;
    grpTheta.classList.remove('disabled-control');

    sliderFanSpread.disabled = true;
    grpFanSpread.classList.add('disabled-control');
  }
}

```



```

    }
  }

  sliderNumRays.addEventListener('input', updateControlStates);
  updateControlStates();

  function calculateRays() {
    const d_G_Gly = parseFloat(sliderDistG.value);
    const d_C_Gly = parseFloat(sliderDistC.value);
    const d_GC_Gly = d_G_Gly - d_C_Gly;
    const M_tril = parseFloat(sliderMass.value);
    const numRays = parseInt(sliderNumRays.value);
    const fanSpreadArcsec = parseFloat(sliderFanSpread.value);
    const thetaInitArcsec = parseFloat(sliderTheta.value);

    const ARCSEC_TO_RAD = Math.PI / (180.0 * 3600.0);
    const d_GC_ly = d_GC_Gly * 1e9;

    const scale_4GM_c2_ly = 0.6243363 * M_tril;

    const rays = [];

    for (let i = 0; i < numRays; i++) {
      let initialAngle = 0;

      if (numRays === 1) {
        initialAngle = thetaInitArcsec;
      } else {
        const frac = (i / (numRays - 1)) - 0.5;
        initialAngle = frac * fanSpreadArcsec;
      }

      const theta_init_rad = initialAngle * ARCSEC_TO_RAD;
      const b_ly = d_GC_ly * theta_init_rad;
      const abs_b_ly = Math.abs(b_ly);

      let alpha_arcsec = 0;
      if (abs_b_ly > 1e-3) {
        const alpha_rad = scale_4GM_c2_ly / abs_b_ly;
        alpha_arcsec = alpha_rad / ARCSEC_TO_RAD;
      }

      let theta_after_arcsec = initialAngle;
      if (initialAngle > 0) {
        theta_after_arcsec = initialAngle - alpha_arcsec;
      } else if (initialAngle < 0) {
        theta_after_arcsec = initialAngle + alpha_arcsec;
      }

      rays.push({
        index: i,
        theta_init_arcsec: initialAngle,
        b_ly: b_ly,
        abs_b_ly: abs_b_ly,
        alpha_arcsec: alpha_arcsec,
        theta_after_arcsec: theta_after_arcsec
      });
    }

    return {
      d_G_Gly,
      d_C_Gly,
      d_GC_Gly,
      M_tril,
      numRays,
      fanSpreadArcsec,
      thetaInitArcsec,
      rays
    };
  }

  function draw() {
    ctx.clearRect(0, 0, width, height);
  }

```

```

const sim = calculateRays();

// Label Updates
valDistG.textContent = `${sim.d_G_Gly.toFixed(1)} Gly`;
valDistC.textContent = `${sim.d_C_Gly.toFixed(1)} Gly`;
valMass.textContent = `${sim.M_tril} × 1012 M☉`;
valNumRays.textContent = `${sim.numRays}`;
valFanSpread.textContent = `${sim.fanSpreadArcsec}`;
valTheta.textContent = `${sim.thetaInitArcsec >= 0 ? '+' : ''}${sim.thetaInitArcsec.toFixed(1)}°`;

// Update Numerical Box
boxDistG.textContent = `${sim.d_G_Gly.toFixed(1)} Gly`;
boxDistC.textContent = `${sim.d_C_Gly.toFixed(1)} Gly`;
boxDistGC.textContent = `${sim.d_GC_Gly.toFixed(1)} Gly`;
boxMass.textContent = `${sim.M_tril} × 1012 M☉`;

if (sim.numRays === 1) {
  const r = sim.rays[0];
  boxMode.textContent = `Single Ray`;
  boxAngleVal.textContent = `${r.theta_init_arcsec >= 0 ? '+' : ''}${r.theta_init_arcsec.toFixed(1)}°`;
  boxImpactRange.textContent = `${(r.abs_b_ly / 1000.0).toFixed(2)} kly (${Math.round(r.abs_b_ly).toLocaleString()})`;
  boxAlphaRange.textContent = `${r.alpha_arcsec.toFixed(2)}°`;
} else {
  boxMode.textContent = `Fan (${sim.numRays} rays)`;
  boxAngleVal.textContent = `${sim.fanSpreadArcsec}` + ` (−${(sim.fanSpreadArcsec/2).toFixed(1)}° to +${(sim.fanSpreadArcsec/2).toFixed(1)}°)`;

  const bValues = sim.rays.map(r => r.abs_b_ly / 1000.0);
  const alphaValues = sim.rays.map(r => r.alpha_arcsec).filter(a => isFinite(a));

  const minB = Math.min(...bValues);
  const maxB = Math.max(...bValues);
  const maxAlpha = Math.max(...alphaValues);

  boxImpactRange.textContent = `${minB.toFixed(1)} to ${maxB.toFixed(1)} kly`;
  boxAlphaRange.textContent = `${maxAlpha.toFixed(2)}° max`;
}

// Visual Layout Dimensions
const marginX = 80;
const centerY = height / 2;

const earthX = marginX;
const galaxyX = width - marginX;
const totalWidthPx = galaxyX - earthX;

const pxPerGly = totalWidthPx / sim.d_G_Gly;
const clusterX = earthX + (sim.d_C_Gly * pxPerGly);

const DEG_TO_RAD = Math.PI / 180.0;
const distGC_px = galaxyX - clusterX;
const distC_px = clusterX - earthX;

// Starfield Background
ctx.fillStyle = "#ffffff";
stars.forEach(s => {
  ctx.globalAlpha = s.alpha;
  ctx.fillRect(s.x * width, s.y * height, s.size, s.size);
});
ctx.globalAlpha = 1.0;

// Baseline Axis
ctx.strokeStyle = "rgba(255, 255, 255, 0.15)";
ctx.lineWidth = 1;
ctx.setLineDash([6, 6]);
ctx.beginPath();
ctx.moveTo(earthX - 20, centerY);
ctx.lineTo(galaxyX + 20, centerY);
ctx.stroke();
ctx.setLineDash([]);

// Cluster Plane Line
ctx.strokeStyle = "rgba(0, 240, 255, 0.25)";
ctx.beginPath();
ctx.moveTo(clusterX, 30);

```

```

ctx.lineTo(clusterX, height - 30);
ctx.stroke();

const ptA = { x: galaxyX, y: centerY };

// Render Light Rays & Virtual Tangents
sim.rays.forEach((ray, idx) => {
  const theta_vis_rad = ray.theta_init_arcsec * DEG_TO_RAD;
  const alpha_vis_rad = ray.alpha_arcsec * DEG_TO_RAD;

  const yB_offset_px = distGC_px * Math.tan(theta_vis_rad);
  const ptB = { x: clusterX, y: centerY - yB_offset_px };

  let theta_after_vis_rad = theta_vis_rad;
  if (ray.theta_init_arcsec > 0) {
    theta_after_vis_rad = theta_vis_rad - alpha_vis_rad;
  } else if (ray.theta_init_arcsec < 0) {
    theta_after_vis_rad = theta_vis_rad + alpha_vis_rad;
  }

  const yC_offset_px = yB_offset_px + distC_px * Math.tan(theta_after_vis_rad);
  const ptC = { x: earthX, y: centerY - yC_offset_px };

  // Draw Virtual Tangent Lines
  if (toggleTangent.checked && Math.abs(ray.alpha_arcsec) > 0.1) {
    ctx.strokeStyle = "rgba(239, 68, 68, 0.5)";
    ctx.lineWidth = 1.2;
    ctx.setLineDash([3, 3]);

    ctx.beginPath();
    ctx.moveTo(ptC.x, ptC.y);
    ctx.lineTo(ptB.x, ptB.y);
    const virtualY_at_galaxy = ptB.y + distGC_px * Math.tan(theta_after_vis_rad);
    ctx.lineTo(galaxyX, virtualY_at_galaxy);
    ctx.stroke();
    ctx.setLineDash([]);

    ctx.fillStyle = "rgba(239, 68, 68, 0.8)";
    ctx.beginPath();
    ctx.arc(galaxyX, virtualY_at_galaxy, 3.5, 0, Math.PI * 2);
    ctx.fill();
  }

  // Draw Actual Light Path
  ctx.strokeStyle = "#facc15";
  ctx.lineWidth = sim.numRays === 1 ? 2.5 : 1.8;
  ctx.shadowColor = "#facc15";
  ctx.shadowBlur = sim.numRays === 1 ? 8 : 4;

  ctx.beginPath();
  ctx.moveTo(ptA.x, ptA.y);
  ctx.lineTo(ptB.x, ptB.y);
  ctx.lineTo(ptC.x, ptC.y);
  ctx.stroke();
  ctx.shadowBlur = 0;

  // Single Ray Impact Parameter visual indicator
  if (sim.numRays === 1 && Math.abs(yB_offset_px) > 3) {
    ctx.strokeStyle = "#00f0ff";
    ctx.lineWidth = 1.5;
    ctx.beginPath();
    ctx.moveTo(clusterX, centerY);
    ctx.lineTo(clusterX, ptB.y);
    ctx.stroke();

    ctx.fillStyle = "#00f0ff";
    ctx.beginPath();
    ctx.arc(clusterX, ptB.y, 3, 0, Math.PI * 2);
    ctx.arc(clusterX, centerY, 3, 0, Math.PI * 2);
    ctx.fill();

    ctx.font = "11px monospace";
    ctx.textAlign = "right";
    ctx.fillText('b = ${(ray.abs_b_ly / 1000).toFixed(1)} kly ` , clusterX - 6, (centerY + ptB.y) / 2 + 4

```

```

}

// Animated Photons
if (togglePhoton.checked) {
  const rayOffset = (idx / sim.numRays);
  const progress = (photonAnimProgress + rayOffset) % 1.0;
  let px = 0, py = 0;

  if (progress <= 0.5) {
    const t = progress / 0.5;
    px = ptA.x + t * (ptB.x - ptA.x);
    py = ptA.y + t * (ptB.y - ptA.y);
  } else {
    const t = (progress - 0.5) / 0.5;
    px = ptB.x + t * (ptC.x - ptB.x);
    py = ptB.y + t * (ptC.y - ptB.y);
  }

  ctx.fillStyle = "#ffffff";
  ctx.shadowColor = "#00f0ff";
  ctx.shadowBlur = 8;
  ctx.beginPath();
  ctx.arc(px, py, 3.5, 0, Math.PI * 2);
  ctx.fill();
  ctx.shadowBlur = 0;
}
});

if (togglePhoton.checked) {
  photonAnimProgress = (photonAnimProgress + 0.006) % 1.0;
}

// Draw Celestial Objects
// Galaxy
ctx.save();
ctx.translate(ptA.x, ptA.y);
const galGlow = ctx.createRadialGradient(0, 0, 2, 0, 0, 22);
galGlow.addColorStop(0, "rgba(255, 200, 100, 0.9)");
galGlow.addColorStop(0.5, "rgba(238, 100, 30, 0.3)");
galGlow.addColorStop(1, "rgba(0, 0, 0, 0)");
ctx.fillStyle = galGlow;
ctx.beginPath();
ctx.arc(0, 0, 22, 0, Math.PI * 2);
ctx.fill();

ctx.fillStyle = "#ffd59e";
ctx.beginPath();
ctx.ellipse(0, 0, 14, 5, -0.3, 0, Math.PI * 2);
ctx.fill();
ctx.restore();

ctx.fillStyle = "#e1e7ec";
ctx.font = "12px sans-serif";
ctx.textAlign = "center";
ctx.fillText("Distant Galaxy", ptA.x, ptA.y + 32);

// Intermediate Cluster
ctx.save();
ctx.translate(clusterX, centerY);
const haloRadius = 18 + (sim.M_tril / 1000) * 22;
const clusterGlow = ctx.createRadialGradient(0, 0, 2, 0, 0, haloRadius);
clusterGlow.addColorStop(0, "rgba(168, 85, 247, 0.65)");
clusterGlow.addColorStop(0.6, "rgba(126, 34, 206, 0.2)");
clusterGlow.addColorStop(1, "rgba(0, 0, 0, 0)");
ctx.fillStyle = clusterGlow;
ctx.beginPath();
ctx.arc(0, 0, haloRadius, 0, Math.PI * 2);
ctx.fill();

ctx.fillStyle = "#ffffff";
ctx.beginPath();
ctx.arc(0, 0, 4, 0, Math.PI * 2);
ctx.arc(-8, -6, 2.5, 0, Math.PI * 2);
ctx.arc(10, 5, 3, 0, Math.PI * 2);

```

```

    ctx.fill();
    ctx.restore();

    ctx.fillStyle = "#e1e7ec";
    ctx.font = "12px sans-serif";
    ctx.textAlign = "center";
    ctx.fillText("Intermediate Cluster", clusterX, centerY + haloRadius + 16);

    // Earth / Observer
    ctx.save();
    ctx.translate(earthX, centerY);
    const earthGrad = ctx.createRadialGradient(-2, -2, 1, 0, 0, 10);
    earthGrad.addColorStop(0, "#60a5fa");
    earthGrad.addColorStop(0.8, "#1d4ed8");
    earthGrad.addColorStop(1, "#1e3a8a");
    ctx.fillStyle = earthGrad;
    ctx.beginPath();
    ctx.arc(0, 0, 9, 0, Math.PI * 2);
    ctx.fill();
    ctx.restore();

    ctx.fillStyle = "#e1e7ec";
    ctx.font = "12px sans-serif";
    ctx.textAlign = "center";
    ctx.fillText("Earth", earthX, centerY + 25);
}

function animate() {
    draw();
    requestAnimationFrame(animate);
}

animate();
</script>
</body>
</html>

```

User prompt: default to 1 ray, and add back in the ability to shift the position of the distant galaxy up / down from the direct horizontal across the screen from 500 to -500 thousand light-years. Fix the size of the canvas to be 15 billion light-years wide, so that the distant galaxy appears closer / further from us as we change its distance.

Response: <!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Gravitational Lensing: Fixed 15 Gly Canvas</title>
<style>
:root {
--bg-color: #0b0e14;
--panel-bg: #151922;
--panel-border: #2a3142;
--accent-color: #00f0ff;
--accent-amber: #facc15;
--accent-red: #ef4444;
--text-main: #e1e7ec;
--text-muted: #8c9bae;
}
* {
box-sizing: border-box;
margin: 0;
padding: 0;
}
body {
font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial, sans-serif;
background-color: var(--bg-color);
color: var(--text-main);
display: flex;
flex-direction: column;
min-height: 100vh;

```
    overflow-x: hidden;
}

header {
    background-color: var(--panel-bg);
    border-bottom: 1px solid var(--panel-border);
    padding: 0.85rem 1.5rem;
    text-align: center;
}

header h1 {
    font-size: 1.3rem;
    font-weight: 600;
    color: #fff;
}

header p {
    font-size: 0.85rem;
    color: var(--text-muted);
    margin-top: 0.2rem;
}

.main-container {
    display: flex;
    flex: 1;
    flex-direction: column;
    gap: 1rem;
    padding: 1rem;
    max-width: 1400px;
    margin: 0 auto;
    width: 100%;
}

@media (min-width: 1024px) {
    .main-container {
        flex-direction: row;
    }
}

.canvas-card {
    flex: 1;
    background-color: var(--panel-bg);
    border: 1px solid var(--panel-border);
    border-radius: 8px;
    padding: 1rem;
    display: flex;
    flex-direction: column;
    gap: 1rem;
    box-shadow: 0 4px 20px rgba(0, 0, 0, 0.4);
}

.canvas-wrapper {
    position: relative;
    width: 100%;
    height: 480px;
    background-color: #04060a;
    border-radius: 6px;
    overflow: hidden;
    border: 1px solid rgba(255, 255, 255, 0.05);
}

canvas {
    width: 100%;
    height: 100%;
    display: block;
}

.calc-box {
    background: rgba(10, 14, 22, 0.7);
    border: 1px solid var(--panel-border);
    border-radius: 6px;
    padding: 1rem 1.25rem;
    font-family: SFMono-Regular, Consolas, "Liberation Mono", Menlo, monospace;
    font-size: 0.8rem;
```

```
    color: #d1d5db;
  }

  .calc-box h3 {
    font-size: 0.85rem;
    text-transform: uppercase;
    letter-spacing: 0.05em;
    color: var(--accent-color);
    margin-bottom: 0.75rem;
    border-bottom: 1px solid rgba(255, 255, 255, 0.1);
    padding-bottom: 0.4rem;
    font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;
  }

  .calc-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(190px, 1fr));
    gap: 0.85rem 1.25rem;
  }

  .calc-item {
    display: flex;
    flex-direction: column;
    gap: 0.25rem;
  }

  .calc-item .label {
    color: var(--text-muted);
    font-size: 0.75rem;
  }

  .calc-item .value {
    color: #fff;
    font-weight: 600;
    font-size: 0.85rem;
  }

  .calc-item .highlight {
    color: var(--accent-amber);
  }

  .calc-footer {
    margin-top: 0.85rem;
    padding-top: 0.6rem;
    border-top: 1px dashed rgba(255, 255, 255, 0.15);
    display: flex;
    justify-content: space-between;
    align-items: center;
    font-size: 0.75rem;
  }

  .calc-footer .formula {
    color: #9ca3af;
  }

  .controls-panel {
    width: 100%;
    max-width: 380px;
    background-color: var(--panel-bg);
    border: 1px solid var(--panel-border);
    border-radius: 8px;
    padding: 1.25rem;
    display: flex;
    flex-direction: column;
    gap: 1.1rem;
    box-shadow: 0 4px 20px rgba(0, 0, 0, 0.4);
  }

  .control-group {
    display: flex;
    flex-direction: column;
    gap: 0.4rem;
    transition: opacity 0.2s ease;
  }
```

```
.control-group.disabled-control {
  opacity: 0.35;
}

.control-group label {
  font-size: 0.83rem;
  font-weight: 500;
  color: var(--text-main);
  display: flex;
  justify-content: space-between;
}

.control-group label span {
  color: var(--accent-color);
  font-family: monospace;
}

input[type="range"] {
  -webkit-appearance: none;
  width: 100%;
  height: 6px;
  border-radius: 3px;
  background: #2a3142;
  outline: none;
}

input[type="range"]:disabled {
  cursor: not-allowed;
}

input[type="range"]::-webkit-slider-thumb {
  -webkit-appearance: none;
  width: 16px;
  height: 16px;
  border-radius: 50%;
  background: var(--accent-color);
  cursor: pointer;
  box-shadow: 0 0 8px var(--accent-color);
  transition: transform 0.1s ease;
}

input[type="range"]:disabled::-webkit-slider-thumb {
  background: #6b7280;
  box-shadow: none;
  cursor: not-allowed;
}

input[type="range"]::-webkit-slider-thumb:hover:not(:disabled) {
  transform: scale(1.2);
}

.toggle-row {
  display: flex;
  align-items: center;
  justify-content: space-between;
  font-size: 0.83rem;
  color: var(--text-main);
}

.switch {
  position: relative;
  display: inline-block;
  width: 40px;
  height: 20px;
}

.switch input {
  opacity: 0;
  width: 0;
  height: 0;
}

.slider {
```



```

    position: absolute;
    cursor: pointer;
    top: 0; left: 0; right: 0; bottom: 0;
    background-color: #2a3142;
    transition: .2s;
    border-radius: 20px;
}

.slider:before {
    position: absolute;
    content: "";
    height: 14px;
    width: 14px;
    left: 3px;
    bottom: 3px;
    background-color: white;
    transition: .2s;
    border-radius: 50%;
}

input:checked + .slider {
    background-color: var(--accent-color);
}

input:checked + .slider:before {
    transform: translateX(20px);
}

.legend {
    border-top: 1px solid var(--panel-border);
    padding-top: 0.8rem;
    display: flex;
    flex-direction: column;
    gap: 0.4rem;
    font-size: 0.8rem;
}

.legend-item {
    display: flex;
    align-items: center;
    gap: 0.6rem;
    color: var(--text-muted);
}

.legend-color {
    width: 14px;
    height: 4px;
    border-radius: 2px;
}
</style>
</head>
<body>

<header>
<h1>Gravitational Deflection of Light Fan</h1>
<p>Fixed 15 Gly Field Simulation (3600× Angle Exaggeration)</p>
</header>

<div class="main-container">
<div class="canvas-card">
<div class="canvas-wrapper">
<canvas id="simCanvas"></canvas>
</div>

<!-- Numerical Output Box below Graphic -->
<div class="calc-box">
<h3>Numerical Results</h3>
<div class="calc-grid">
<div class="calc-item">
<span class="label">Galaxy Distance (d<sub>G</sub>):</span>
<span class="value" id="boxDistG">8.0 Gly</span>
</div>
<div class="calc-item">
<span class="label">Galaxy Offset (y<sub>G</sub>):</span>

```

```

        <span class="value" id="boxOffsetY">0 kly</span>
    </div>
    <div class="calc-item">
        <span class="label">Cluster Distance ( $d_{\text{C}}$ ):</span>
        <span class="value" id="boxDistC">4.0 Gly</span>
    </div>
    <div class="calc-item">
        <span class="label"> $d_{\text{GC}} = d_{\text{G}} - d_{\text{C}}$ :</span>
        <span class="value" id="boxDistGC">4.0 Gly</span>
    </div>
    <div class="calc-item">
        <span class="label">Cluster Mass (M):</span>
        <span class="value" id="boxMass"> $200 \times 10^{12} M_{\odot}$ </span>
    </div>
    <div class="calc-item">
        <span class="label">Ray Mode:</span>
        <span class="value" id="boxMode">Single Ray</span>
    </div>
    <div class="calc-item">
        <span class="label">Initial Angle / Spread:</span>
        <span class="value" id="boxAngleVal"> $+15.0^{\circ}$ </span>
    </div>
    <div class="calc-item">
        <span class="label">Impact Parameter (b):</span>
        <span class="value highlight" id="boxImpactRange">290.9 kly</span>
    </div>
    <div class="calc-item">
        <span class="label">Deflection Angle ( $\hat{\alpha}$ ):</span>
        <span class="value highlight" id="boxAlphaRange"> $44.3^{\circ}$ </span>
    </div>
    </div>
    <div class="calc-footer">
        <span class="label">Theoretical Formula:</span>
        <span class="formula"> $\hat{\alpha} = 4GM / (c^2 b)$ </span>
    </div>
</div>
<div class="controls-panel">
    <div class="control-group">
        <label for="sliderDistG">
            1. Distance to Galaxy ( $d_{\text{G}}$ ):
            <span id="valDistG">8.0 Gly</span>
        </label>
        <input type="range" id="sliderDistG" min="2.0" max="13.0" step="0.1" value="8.0">
    </div>
    <div class="control-group">
        <label for="sliderOffsetY">
            2. Galaxy Vertical Offset ( $y_{\text{G}}$ ):
            <span id="valOffsetY">0 kly</span>
        </label>
        <input type="range" id="sliderOffsetY" min="-500" max="500" step="10" value="0">
    </div>
    <div class="control-group">
        <label for="sliderDistC">
            3. Distance to Cluster ( $d_{\text{C}}$ ):
            <span id="valDistC">4.0 Gly</span>
        </label>
        <input type="range" id="sliderDistC" min="1.0" max="12.0" step="0.1" value="4.0">
    </div>
    <div class="control-group">
        <label for="sliderMass">
            4. Cluster Mass (M):
            <span id="valMass"> $200 \times 10^{12} M_{\odot}$ </span>
        </label>
        <input type="range" id="sliderMass" min="5" max="1000" step="5" value="200">
    </div>
    <div class="control-group">
        <label for="sliderNumRays">
            5. Number of Light Rays:

```

```

    <span id="valNumRays">1</span>
  </label>
  <input type="range" id="sliderNumRays" min="1" max="10" step="1" value="1">
</div>

<div class="control-group" id="grpFanSpread">
  <label for="sliderFanSpread">
    6. Ray Fan Spread:
    <span id="valFanSpread">20</span>
  </label>
  <input type="range" id="sliderFanSpread" min="10" max="40" step="1" value="20">
</div>

<div class="control-group" id="grpTheta">
  <label for="sliderTheta">
    7. Initial Ray Direction (1 ray):
    <span id="valTheta">+15.0</span>
  </label>
  <input type="range" id="sliderTheta" min="-40.0" max="40.0" step="0.5" value="15.0">
</div>

<div class="toggle-row">
  <span>Animate Photons</span>
  <label class="switch">
    <input type="checkbox" id="togglePhoton" checked>
    <span class="slider"></span>
  </label>
</div>

<div class="toggle-row">
  <span>Show Virtual Tangent Lines</span>
  <label class="switch">
    <input type="checkbox" id="toggleTangent" checked>
    <span class="slider"></span>
  </label>
</div>

<div class="legend">
  <div class="legend-item">
    <div class="legend-color" style="background: #facc15;"></div>
    <span>Actual deflected light rays</span>
  </div>
  <div class="legend-item">
    <div class="legend-color" style="border: 1px dashed #ef4444; background: transparent;"></div>
    <span>Apparent straight-line paths (virtual images)</span>
  </div>
  <div class="legend-item">
    <div class="legend-color" style="background: #00f0ff;"></div>
    <span>Lens plane / Impact parameter (b)</span>
  </div>
</div>
</div>
</div>

<script>
const canvas = document.getElementById('simCanvas');
const ctx = canvas.getContext('2d');

// Canvas Fixed Total Width Scale (15 Billion Light-Years)
const CANVAS_TOTAL_GLY = 15.0;

// Controls
const sliderDistG = document.getElementById('sliderDistG');
const sliderOffsetY = document.getElementById('sliderOffsetY');
const sliderDistC = document.getElementById('sliderDistC');
const sliderMass = document.getElementById('sliderMass');
const sliderNumRays = document.getElementById('sliderNumRays');
const sliderFanSpread = document.getElementById('sliderFanSpread');
const sliderTheta = document.getElementById('sliderTheta');

const grpFanSpread = document.getElementById('grpFanSpread');
const grpTheta = document.getElementById('grpTheta');

const valDistG = document.getElementById('valDistG');
```

```

const valOffsetY = document.getElementById('valOffsetY');
const valDistC = document.getElementById('valDistC');
const valMass = document.getElementById('valMass');
const valNumRays = document.getElementById('valNumRays');
const valFanSpread = document.getElementById('valFanSpread');
const valTheta = document.getElementById('valTheta');

const togglePhoton = document.getElementById('togglePhoton');
const toggleTangent = document.getElementById('toggleTangent');

// Calculation Box Elements
const boxDistG = document.getElementById('boxDistG');
const boxOffsetY = document.getElementById('boxOffsetY');
const boxDistC = document.getElementById('boxDistC');
const boxDistGC = document.getElementById('boxDistGC');
const boxMass = document.getElementById('boxMass');
const boxMode = document.getElementById('boxMode');
const boxAngleVal = document.getElementById('boxAngleVal');
const boxImpactRange = document.getElementById('boxImpactRange');
const boxAlphaRange = document.getElementById('boxAlphaRange');

let width = 0;
let height = 0;
let photonAnimProgress = 0;

// Background Stars
const stars = [];
for (let i = 0; i < 220; i++) {
  stars.push({
    x: Math.random(),
    y: Math.random(),
    size: Math.random() * 1.5 + 0.3,
    alpha: Math.random() * 0.7 + 0.3
  });
}

function resizeCanvas() {
  const rect = canvas.parentElement.getBoundingClientRect();
  const dpr = window.devicePixelRatio || 1;
  width = rect.width || 800;
  height = rect.height || 480;

  canvas.width = width * dpr;
  canvas.height = height * dpr;
  ctx.scale(dpr, dpr);
}

window.addEventListener('resize', resizeCanvas);
resizeCanvas();

// Distance constraints: d_G >= d_C + 1.0 Gly
sliderDistG.addEventListener('input', () => {
  let dG = parseFloat(sliderDistG.value);
  let dC = parseFloat(sliderDistC.value);
  if (dG < dC + 1.0) {
    dC = Math.max(1.0, dG - 1.0);
    sliderDistC.value = dC.toFixed(1);
  }
  updateControlStates();
});

sliderDistC.addEventListener('input', () => {
  let dG = parseFloat(sliderDistG.value);
  let dC = parseFloat(sliderDistC.value);
  if (dC > dG - 1.0) {
    dG = Math.min(13.0, dC + 1.0);
    sliderDistG.value = dG.toFixed(1);
  }
  updateControlStates();
});

function updateControlStates() {
  const numRays = parseInt(sliderNumRays.value);

```

```

if (numRays > 1) {
  // Multi-ray mode: Angle/Direction slider inactive, Fan Spread active
  sliderTheta.disabled = true;
  grpTheta.classList.add('disabled-control');

  sliderFanSpread.disabled = false;
  grpFanSpread.classList.remove('disabled-control');
} else {
  // Single-ray mode: Angle/Direction slider active, Fan Spread inactive
  sliderTheta.disabled = false;
  grpTheta.classList.remove('disabled-control');

  sliderFanSpread.disabled = true;
  grpFanSpread.classList.add('disabled-control');
}

sliderNumRays.addEventListener('input', updateControlStates);
updateControlStates();

function calculateRays() {
  const d_G_Gly = parseFloat(sliderDistG.value);
  const y_G_kly = parseFloat(sliderOffsetY.value);
  const d_C_Gly = parseFloat(sliderDistC.value);
  const d_GC_Gly = d_G_Gly - d_C_Gly;
  const M_tril = parseFloat(sliderMass.value);
  const numRays = parseInt(sliderNumRays.value);
  const fanSpreadArcsec = parseFloat(sliderFanSpread.value);
  const thetaInitArcsec = parseFloat(sliderTheta.value);

  const ARCSEC_TO_RAD = Math.PI / (180.0 * 3600.0);
  const d_GC_ly = d_GC_Gly * 1e9;
  const y_G_ly = y_G_kly * 1e3;

  const scale_4GM_c2_ly = 0.6243363 * M_tril;

  const rays = [];

  for (let i = 0; i < numRays; i++) {
    let initialAngle = 0;

    if (numRays === 1) {
      initialAngle = thetaInitArcsec;
    } else {
      const frac = (i / (numRays - 1)) - 0.5;
      initialAngle = frac * fanSpreadArcsec;
    }

    const theta_init_rad = initialAngle * ARCSEC_TO_RAD;

    // Impact parameter b relative to lens center line (y=0)
    // b = y_G + d_GC * tan(theta_init)
    const b_ly = y_G_ly + d_GC_ly * Math.tan(theta_init_rad);
    const abs_b_ly = Math.abs(b_ly);

    let alpha_arcsec = 0;
    if (abs_b_ly > 1e-3) {
      const alpha_rad = scale_4GM_c2_ly / abs_b_ly;
      alpha_arcsec = alpha_rad / ARCSEC_TO_RAD;
    }

    // Deflection angle direction pulls toward lens center (y=0)
    let theta_after_arcsec = initialAngle;
    if (b_ly > 0) {
      theta_after_arcsec = initialAngle - alpha_arcsec;
    } else if (b_ly < 0) {
      theta_after_arcsec = initialAngle + alpha_arcsec;
    }

    rays.push({
      index: i,
      theta_init_arcsec: initialAngle,
      b_ly: b_ly,
      abs_b_ly: abs_b_ly,
    });
  }
}

```

```

    alpha_arcsec: alpha_arcsec,
    theta_after_arcsec: theta_after_arcsec
  });
}

return {
  d_G_Gly,
  y_G_kly,
  d_C_Gly,
  d_GC_Gly,
  M_tril,
  numRays,
  fanSpreadArcsec,
  thetaInitArcsec,
  rays
};
}

function draw() {
  ctx.clearRect(0, 0, width, height);

  const sim = calculateRays();

  // Label Updates
  valDistG.textContent = `${sim.d_G_Gly.toFixed(1)} Gly`;
  valOffsetY.textContent = `${sim.y_G_kly >= 0 ? '+' : ''}${sim.y_G_kly} kly`;
  valDistC.textContent = `${sim.d_C_Gly.toFixed(1)} Gly`;
  valMass.textContent = `${sim.M_tril} × 1012 M⊙`;
  valNumRays.textContent = `${sim.numRays}`;
  valFanSpread.textContent = `${sim.fanSpreadArcsec}`;
  valTheta.textContent = `${sim.thetaInitArcsec >= 0 ? '+' : ''}${sim.thetaInitArcsec.toFixed(1)}°`;

  // Update Numerical Box
  boxDistG.textContent = `${sim.d_G_Gly.toFixed(1)} Gly`;
  boxOffsetY.textContent = `${sim.y_G_kly >= 0 ? '+' : ''}${sim.y_G_kly} kly`;
  boxDistC.textContent = `${sim.d_C_Gly.toFixed(1)} Gly`;
  boxDistGC.textContent = `${sim.d_GC_Gly.toFixed(1)} Gly`;
  boxMass.textContent = `${sim.M_tril} × 1012 M⊙`;

  if (sim.numRays === 1) {
    const r = sim.rays[0];
    boxMode.textContent = `Single Ray`;
    boxAngleVal.textContent = `${r.theta_init_arcsec >= 0 ? '+' : ''}${r.theta_init_arcsec.toFixed(1)}°`;
    boxImpactRange.textContent = `${(r.abs_b_ly / 1000.0).toFixed(2)} kly (${Math.round(r.abs_b_ly).toLocaleString()})`;
    boxAlphaRange.textContent = `${r.alpha_arcsec.toFixed(2)}°`;
  } else {
    boxMode.textContent = `Fan (${sim.numRays} rays)`;
    boxAngleVal.textContent = `${sim.fanSpreadArcsec} (−${(sim.fanSpreadArcsec/2).toFixed(1)}° to +${(sim.fanSpreadArcsec/2).toFixed(1)}°)`;

    const bValues = sim.rays.map(r => r.abs_b_ly / 1000.0);
    const alphaValues = sim.rays.map(r => r.alpha_arcsec).filter(a => isFinite(a));

    const minB = Math.min(...bValues);
    const maxB = Math.max(...bValues);
    const maxAlpha = Math.max(...alphaValues);

    boxImpactRange.textContent = `${minB.toFixed(1)} to ${maxB.toFixed(1)} kly`;
    boxAlphaRange.textContent = `${maxAlpha.toFixed(2)}° max`;
  }

  // Visual Layout Dimensions
  const marginX = 80;
  const centerY = height / 2;

  const earthX = marginX;
  // Fixed canvas scale: the printable width span (width - 2*marginX) represents 15.0 Gly
  const drawableWidth = width - 2 * marginX;
  const pxPerGly = drawableWidth / CANVAS_TOTAL_GLY;

  const galaxyX = earthX + (sim.d_G_Gly * pxPerGly);
  const clusterX = earthX + (sim.d_C_Gly * pxPerGly);

  // Exaggerated Y scale for vertical galaxy offset: 500 kly maps to 120 pixels visually
  const pxPerKly = 120.0 / 500.0;

```

```

const galaxyY = centerY - (sim.y_G_kly * pxPerKly);

const DEG_TO_RAD = Math.PI / 180.0;
const distGC_px = galaxyX - clusterX;
const distC_px = clusterX - earthX;

// Starfield Background
ctx.fillStyle = "#ffffff";
stars.forEach(s => {
  ctx.globalAlpha = s.alpha;
  ctx.fillRect(s.x * width, s.y * height, s.size, s.size);
});
ctx.globalAlpha = 1.0;

// Baseline Axis (0 Gly to 15 Gly full canvas span)
ctx.strokeStyle = "rgba(255, 255, 255, 0.15)";
ctx.lineWidth = 1;
ctx.setLineDash([6, 6]);
ctx.beginPath();
ctx.moveTo(earthX - 20, centerY);
ctx.lineTo(earthX + CANVAS_TOTAL_GLY * pxPerGly + 20, centerY);
ctx.stroke();
ctx.setLineDash([]);

// Cluster Plane Line
ctx.strokeStyle = "rgba(0, 240, 255, 0.25)";
ctx.beginPath();
ctx.moveTo(clusterX, 30);
ctx.lineTo(clusterX, height - 30);
ctx.stroke();

const ptA = { x: galaxyX, y: galaxyY };

// Render Light Rays & Virtual Tangents
sim.rays.forEach((ray, idx) => {
  const theta_vis_rad = ray.theta_init_arcsec * DEG_TO_RAD;
  const alpha_vis_rad = ray.alpha_arcsec * DEG_TO_RAD;

  const yB_offset_px = (ptA.y - centerY) + distGC_px * Math.tan(theta_vis_rad);
  const ptB = { x: clusterX, y: centerY + yB_offset_px };

  let theta_after_vis_rad = theta_vis_rad;
  if (ray.b_ly > 0) {
    theta_after_vis_rad = theta_vis_rad - alpha_vis_rad;
  } else if (ray.b_ly < 0) {
    theta_after_vis_rad = theta_vis_rad + alpha_vis_rad;
  }

  const yC_offset_px = yB_offset_px + distC_px * Math.tan(theta_after_vis_rad);
  const ptC = { x: earthX, y: centerY + yC_offset_px };

  // Draw Virtual Tangent Lines
  if (toggleTangent.checked && Math.abs(ray.alpha_arcsec) > 0.1) {
    ctx.strokeStyle = "rgba(239, 68, 68, 0.5)";
    ctx.lineWidth = 1.2;
    ctx.setLineDash([3, 3]);

    ctx.beginPath();
    ctx.moveTo(ptC.x, ptC.y);
    ctx.lineTo(ptB.x, ptB.y);
    const virtualY_at_galaxy = ptB.y - distGC_px * Math.tan(theta_after_vis_rad);
    ctx.lineTo(galaxyX, virtualY_at_galaxy);
    ctx.stroke();
    ctx.setLineDash([]);

    ctx.fillStyle = "rgba(239, 68, 68, 0.8)";
    ctx.beginPath();
    ctx.arc(galaxyX, virtualY_at_galaxy, 3.5, 0, Math.PI * 2);
    ctx.fill();
  }

  // Draw Actual Light Path
  ctx.strokeStyle = "#facc15";
  ctx.lineWidth = sim.numRays === 1 ? 2.5 : 1.8;

```

```

ctx.shadowColor = "#facc15";
ctx.shadowBlur = sim.numRays === 1 ? 8 : 4;

ctx.beginPath();
ctx.moveTo(ptA.x, ptA.y);
ctx.lineTo(ptB.x, ptB.y);
ctx.lineTo(ptC.x, ptC.y);
ctx.stroke();
ctx.shadowBlur = 0;

// Single Ray Impact Parameter visual indicator
if (sim.numRays === 1 && Math.abs(ptB.y - centerY) > 3) {
  ctx.strokeStyle = "#00f0ff";
  ctx.lineWidth = 1.5;
  ctx.beginPath();
  ctx.moveTo(clusterX, centerY);
  ctx.lineTo(clusterX, ptB.y);
  ctx.stroke();

  ctx.fillStyle = "#00f0ff";
  ctx.beginPath();
  ctx.arc(clusterX, ptB.y, 3, 0, Math.PI * 2);
  ctx.arc(clusterX, centerY, 3, 0, Math.PI * 2);
  ctx.fill();

  ctx.font = "11px monospace";
  ctx.textAlign = "right";
  ctx.fillText(`b = ${ray.abs_b_ly / 1000}.toFixed(1)} kly`, clusterX - 6, (centerY + ptB.y) / 2 + 4
}

// Animated Photons
if (togglePhoton.checked) {
  const rayOffset = (idx / sim.numRays);
  const progress = (photonAnimProgress + rayOffset) % 1.0;
  let px = 0, py = 0;

  if (progress <= 0.5) {
    const t = progress / 0.5;
    px = ptA.x + t * (ptB.x - ptA.x);
    py = ptA.y + t * (ptB.y - ptA.y);
  } else {
    const t = (progress - 0.5) / 0.5;
    px = ptB.x + t * (ptC.x - ptB.x);
    py = ptB.y + t * (ptC.y - ptB.y);
  }

  ctx.fillStyle = "#ffffff";
  ctx.shadowColor = "#00f0ff";
  ctx.shadowBlur = 8;
  ctx.beginPath();
  ctx.arc(px, py, 3.5, 0, Math.PI * 2);
  ctx.fill();
  ctx.shadowBlur = 0;
}
});

if (togglePhoton.checked) {
  photonAnimProgress = (photonAnimProgress + 0.006) % 1.0;
}

// Draw Celestial Objects
// Distant Galaxy
ctx.save();
ctx.translate(ptA.x, ptA.y);
const galGlow = ctx.createRadialGradient(0, 0, 2, 0, 0, 22);
galGlow.addColorStop(0, "rgba(255, 200, 100, 0.9)");
galGlow.addColorStop(0.5, "rgba(238, 100, 30, 0.3)");
galGlow.addColorStop(1, "rgba(0, 0, 0, 0)");
ctx.fillStyle = galGlow;
ctx.beginPath();
ctx.arc(0, 0, 22, 0, Math.PI * 2);
ctx.fill();

ctx.fillStyle = "#ffd59e";

```



```

ctx.beginPath();
ctx.ellipse(0, 0, 14, 5, -0.3, 0, Math.PI * 2);
ctx.fill();
ctx.restore();

ctx.fillStyle = "#e1e7ec";
ctx.font = "12px sans-serif";
ctx.textAlign = "center";
ctx.fillText("Distant Galaxy", ptA.x, ptA.y + 32);

// Intermediate Cluster
ctx.save();
ctx.translate(clusterX, centerY);
const haloRadius = 18 + (sim.M_tril / 1000) * 22;
const clusterGlow = ctx.createRadialGradient(0, 0, 2, 0, 0, haloRadius);
clusterGlow.addColorStop(0, "rgba(168, 85, 247, 0.65)");
clusterGlow.addColorStop(0.6, "rgba(126, 34, 206, 0.2)");
clusterGlow.addColorStop(1, "rgba(0, 0, 0, 0)");
ctx.fillStyle = clusterGlow;
ctx.beginPath();
ctx.arc(0, 0, haloRadius, 0, Math.PI * 2);
ctx.fill();

ctx.fillStyle = "#ffffff";
ctx.beginPath();
ctx.arc(0, 0, 4, 0, Math.PI * 2);
ctx.arc(-8, -6, 2.5, 0, Math.PI * 2);
ctx.arc(10, 5, 3, 0, Math.PI * 2);
ctx.fill();
ctx.restore();

ctx.fillStyle = "#e1e7ec";
ctx.font = "12px sans-serif";
ctx.textAlign = "center";
ctx.fillText("Intermediate Cluster", clusterX, centerY + haloRadius + 16);

// Earth / Observer
ctx.save();
ctx.translate(earthX, centerY);
const earthGrad = ctx.createRadialGradient(-2, -2, 1, 0, 0, 10);
earthGrad.addColorStop(0, "#60a5fa");
earthGrad.addColorStop(0.8, "#1d4ed8");
earthGrad.addColorStop(1, "#1e3a8a");
ctx.fillStyle = earthGrad;
ctx.beginPath();
ctx.arc(0, 0, 9, 0, Math.PI * 2);
ctx.fill();
ctx.restore();

ctx.fillStyle = "#e1e7ec";
ctx.font = "12px sans-serif";
ctx.textAlign = "center";
ctx.fillText("Earth", earthX, centerY + 25);
}

function animate() {
  draw();
  requestAnimationFrame(animate);
}

animate();
</script>
</body>
</html>

```

User prompt: Now we need to adjust how the rays are drawn as they proceed from the cluster to Earth. For each ray, calculate what its impact parameter will be relative to Earth. Then draw the ray from where it passes by the cluster to the distance where it will pass by Earth, using the same vertical scale as for the galaxy placement.

Response: <!DOCTYPE html>
 <html lang="en">
 <head>

```

<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Gravitational Lensing: Fixed 15 Gly Field Simulation</title>
<style>
:root {
  --bg-color: #0b0e14;
  --panel-bg: #151922;
  --panel-border: #2a3142;
  --accent-color: #00f0ff;
  --accent-amber: #facc15;
  --accent-red: #ef4444;
  --text-main: #e1e7ec;
  --text-muted: #8c9bae;
}

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial, sans-serif;
  background-color: var(--bg-color);
  color: var(--text-main);
  display: flex;
  flex-direction: column;
  min-height: 100vh;
  overflow-x: hidden;
}

header {
  background-color: var(--panel-bg);
  border-bottom: 1px solid var(--panel-border);
  padding: 0.85rem 1.5rem;
  text-align: center;
}

header h1 {
  font-size: 1.3rem;
  font-weight: 600;
  color: #fff;
}

header p {
  font-size: 0.85rem;
  color: var(--text-muted);
  margin-top: 0.2rem;
}

.main-container {
  display: flex;
  flex: 1;
  flex-direction: column;
  gap: 1rem;
  padding: 1rem;
  max-width: 1400px;
  margin: 0 auto;
  width: 100%;
}

@media (min-width: 1024px) {
  .main-container {
    flex-direction: row;
  }
}

.canvas-card {
  flex: 1;
  background-color: var(--panel-bg);
  border: 1px solid var(--panel-border);
  border-radius: 8px;
  padding: 1rem;
  display: flex;

```

```
    flex-direction: column;
    gap: 1rem;
    box-shadow: 0 4px 20px rgba(0, 0, 0, 0.4);
}

.canvas-wrapper {
    position: relative;
    width: 100%;
    height: 480px;
    background-color: #04060a;
    border-radius: 6px;
    overflow: hidden;
    border: 1px solid rgba(255, 255, 255, 0.05);
}

canvas {
    width: 100%;
    height: 100%;
    display: block;
}

.calc-box {
    background: rgba(10, 14, 22, 0.7);
    border: 1px solid var(--panel-border);
    border-radius: 6px;
    padding: 1rem 1.25rem;
    font-family: SFMono-Regular, Consolas, "Liberation Mono", Menlo, monospace;
    font-size: 0.8rem;
    color: #d1d5db;
}

.calc-box h3 {
    font-size: 0.85rem;
    text-transform: uppercase;
    letter-spacing: 0.05em;
    color: var(--accent-color);
    margin-bottom: 0.75rem;
    border-bottom: 1px solid rgba(255, 255, 255, 0.1);
    padding-bottom: 0.4rem;
    font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;
}

.calc-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(190px, 1fr));
    gap: 0.85rem 1.25rem;
}

.calc-item {
    display: flex;
    flex-direction: column;
    gap: 0.25rem;
}

.calc-item .label {
    color: var(--text-muted);
    font-size: 0.75rem;
}

.calc-item .value {
    color: #fff;
    font-weight: 600;
    font-size: 0.85rem;
}

.calc-item .highlight {
    color: var(--accent-amber);
}

.calc-footer {
    margin-top: 0.85rem;
    padding-top: 0.6rem;
    border-top: 1px dashed rgba(255, 255, 255, 0.15);
    display: flex;
```

```
    justify-content: space-between;
    align-items: center;
    font-size: 0.75rem;
}

.calc-footer .formula {
    color: #9ca3af;
}

.controls-panel {
    width: 100%;
    max-width: 380px;
    background-color: var(--panel-bg);
    border: 1px solid var(--panel-border);
    border-radius: 8px;
    padding: 1.25rem;
    display: flex;
    flex-direction: column;
    gap: 1.1rem;
    box-shadow: 0 4px 20px rgba(0, 0, 0, 0.4);
}

.control-group {
    display: flex;
    flex-direction: column;
    gap: 0.4rem;
    transition: opacity 0.2s ease;
}

.control-group.disabled-control {
    opacity: 0.35;
}

.control-group label {
    font-size: 0.83rem;
    font-weight: 500;
    color: var(--text-main);
    display: flex;
    justify-content: space-between;
}

.control-group label span {
    color: var(--accent-color);
    font-family: monospace;
}

input[type="range"] {
    -webkit-appearance: none;
    width: 100%;
    height: 6px;
    border-radius: 3px;
    background: #2a3142;
    outline: none;
}

input[type="range"]:disabled {
    cursor: not-allowed;
}

input[type="range"]::-webkit-slider-thumb {
    -webkit-appearance: none;
    width: 16px;
    height: 16px;
    border-radius: 50%;
    background: var(--accent-color);
    cursor: pointer;
    box-shadow: 0 0 8px var(--accent-color);
    transition: transform 0.1s ease;
}

input[type="range"]:disabled::-webkit-slider-thumb {
    background: #6b7280;
    box-shadow: none;
    cursor: not-allowed;
}
```

```
}

input[type="range"]::-webkit-slider-thumb:hover:not(:disabled) {
  transform: scale(1.2);
}

.toggle-row {
  display: flex;
  align-items: center;
  justify-content: space-between;
  font-size: 0.83rem;
  color: var(--text-main);
}

.switch {
  position: relative;
  display: inline-block;
  width: 40px;
  height: 20px;
}

.switch input {
  opacity: 0;
  width: 0;
  height: 0;
}

.slider {
  position: absolute;
  cursor: pointer;
  top: 0; left: 0; right: 0; bottom: 0;
  background-color: #2a3142;
  transition: .2s;
  border-radius: 20px;
}

.slider:before {
  position: absolute;
  content: "";
  height: 14px;
  width: 14px;
  left: 3px;
  bottom: 3px;
  background-color: white;
  transition: .2s;
  border-radius: 50%;
}

input:checked + .slider {
  background-color: var(--accent-color);
}

input:checked + .slider:before {
  transform: translateX(20px);
}

.legend {
  border-top: 1px solid var(--panel-border);
  padding-top: 0.8rem;
  display: flex;
  flex-direction: column;
  gap: 0.4rem;
  font-size: 0.8rem;
}

.legend-item {
  display: flex;
  align-items: center;
  gap: 0.6rem;
  color: var(--text-muted);
}

.legend-color {
  width: 14px;
}
```

```

    height: 4px;
    border-radius: 2px;
  }
</style>
</head>
<body>

<header>
  <h1>Gravitational Deflection of Light Fan</h1>
  <p>Fixed 15 Gly Field Simulation (Consistent Vertical Scale & Precise Earth Impact Parameter)</p>
</header>

<div class="main-container">
  <div class="canvas-card">
    <div class="canvas-wrapper">
      <canvas id="simCanvas"></canvas>
    </div>

    <!-- Numerical Output Box below Graphic -->
    <div class="calc-box">
      <h3>Numerical Results</h3>
      <div class="calc-grid">
        <div class="calc-item">
          <span class="label">Galaxy Distance (d<sub>G</sub>):</span>
          <span class="value" id="boxDistG">8.0 Gly</span>
        </div>
        <div class="calc-item">
          <span class="label">Galaxy Offset (y<sub>G</sub>):</span>
          <span class="value" id="boxOffsetY">0 kly</span>
        </div>
        <div class="calc-item">
          <span class="label">Cluster Distance (d<sub>C</sub>):</span>
          <span class="value" id="boxDistC">4.0 Gly</span>
        </div>
        <div class="calc-item">
          <span class="label">d<sub>GC</sub> = d<sub>G</sub> - d<sub>C</sub>:</span>
          <span class="value" id="boxDistGC">4.0 Gly</span>
        </div>
        <div class="calc-item">
          <span class="label">Cluster Mass (M):</span>
          <span class="value" id="boxMass">200 × 1012 M⊙</span>
        </div>
        <div class="calc-item">
          <span class="label">Ray Mode:</span>
          <span class="value" id="boxMode">Single Ray</span>
        </div>
        <div class="calc-item">
          <span class="label">Cluster Impact (b<sub>C</sub>):</span>
          <span class="value highlight" id="boxImpactRange">290.9 kly</span>
        </div>
        <div class="calc-item">
          <span class="label">Earth Impact (b<sub>E</sub>):</span>
          <span class="value highlight" id="boxEarthImpactRange">581.8 kly</span>
        </div>
        <div class="calc-item">
          <span class="label">Deflection Angle (α):</span>
          <span class="value highlight" id="boxAlphaRange">44.3°</span>
        </div>
      </div>
      <div class="calc-footer">
        <span class="label">Theoretical Formula:</span>
        <span class="formula">α = 4GM / (c2b<sub>C</sub>) &nbsp;|&nbsp;b<sub>E</sub> = b<sub>C</sub> - d<sub>GC</sub>
      </div>
    </div>

    <div class="controls-panel">
      <div class="control-group">
        <label for="sliderDistG">
          1. Distance to Galaxy (d<sub>G</sub>):
          <span id="valDistG">8.0 Gly</span>
        </label>
        <input type="range" id="sliderDistG" min="2.0" max="13.0" step="0.1" value="8.0">
      </div>
    </div>
  </div>
</div>

```

```

<div class="control-group">
  <label for="sliderOffsetY">
    2. Galaxy Vertical Offset ( $y_G$ ):
    <span id="valOffsetY">0 kly</span>
  </label>
  <input type="range" id="sliderOffsetY" min="-500" max="500" step="10" value="0">
</div>

<div class="control-group">
  <label for="sliderDistC">
    3. Distance to Cluster ( $d_C$ ):
    <span id="valDistC">4.0 Gly</span>
  </label>
  <input type="range" id="sliderDistC" min="1.0" max="12.0" step="0.1" value="4.0">
</div>

<div class="control-group">
  <label for="sliderMass">
    4. Cluster Mass ( $M$ ):
    <span id="valMass">200  $\times 10^{12} M_\odot$ </span>
  </label>
  <input type="range" id="sliderMass" min="5" max="1000" step="5" value="200">
</div>

<div class="control-group">
  <label for="sliderNumRays">
    5. Number of Light Rays:
    <span id="valNumRays">1</span>
  </label>
  <input type="range" id="sliderNumRays" min="1" max="10" step="1" value="1">
</div>

<div class="control-group" id="grpFanSpread">
  <label for="sliderFanSpread">
    6. Ray Fan Spread:
    <span id="valFanSpread">20°</span>
  </label>
  <input type="range" id="sliderFanSpread" min="10" max="40" step="1" value="20">
</div>

<div class="control-group" id="grpTheta">
  <label for="sliderTheta">
    7. Initial Ray Direction (1 ray):
    <span id="valTheta">+15.0°</span>
  </label>
  <input type="range" id="sliderTheta" min="-40.0" max="40.0" step="0.5" value="15.0">
</div>

<div class="toggle-row">
  <span>Animate Photons</span>
  <label class="switch">
    <input type="checkbox" id="togglePhoton" checked>
    <span class="slider"></span>
  </label>
</div>

<div class="toggle-row">
  <span>Show Virtual Tangent Lines</span>
  <label class="switch">
    <input type="checkbox" id="toggleTangent" checked>
    <span class="slider"></span>
  </label>
</div>

<div class="legend">
  <div class="legend-item">
    <div class="legend-color" style="background: #facc15;"></div>
    <span>Actual deflected light rays</span>
  </div>
  <div class="legend-item">
    <div class="legend-color" style="border: 1px dashed #ef4444; background: transparent;"></div>
    <span>Apparent straight-line paths (virtual images)</span>
  </div>
</div>

```

```

    <div class="legend-item">
      <div class="legend-color" style="background: #00f0ff;"></div>
      <span>Cluster impact parameter (b<sub>C</sub>) & Earth impact (b<sub>E</sub>)</span>
    </div>
  </div>
</div>

<script>
  const canvas = document.getElementById('simCanvas');
  const ctx = canvas.getContext('2d');

  // Canvas Fixed Total Width Scale (15 Billion Light-Years)
  const CANVAS_TOTAL_GLY = 15.0;

  // Controls
  const sliderDistG = document.getElementById('sliderDistG');
  const sliderOffsetY = document.getElementById('sliderOffsetY');
  const sliderDistC = document.getElementById('sliderDistC');
  const sliderMass = document.getElementById('sliderMass');
  const sliderNumRays = document.getElementById('sliderNumRays');
  const sliderFanSpread = document.getElementById('sliderFanSpread');
  const sliderTheta = document.getElementById('sliderTheta');

  const grpFanSpread = document.getElementById('grpFanSpread');
  const grpTheta = document.getElementById('grpTheta');

  const valDistG = document.getElementById('valDistG');
  const valOffsetY = document.getElementById('valOffsetY');
  const valDistC = document.getElementById('valDistC');
  const valMass = document.getElementById('valMass');
  const valNumRays = document.getElementById('valNumRays');
  const valFanSpread = document.getElementById('valFanSpread');
  const valTheta = document.getElementById('valTheta');

  const togglePhoton = document.getElementById('togglePhoton');
  const toggleTangent = document.getElementById('toggleTangent');

  // Calculation Box Elements
  const boxDistG = document.getElementById('boxDistG');
  const boxOffsetY = document.getElementById('boxOffsetY');
  const boxDistC = document.getElementById('boxDistC');
  const boxDistGC = document.getElementById('boxDistGC');
  const boxMass = document.getElementById('boxMass');
  const boxMode = document.getElementById('boxMode');
  const boxImpactRange = document.getElementById('boxImpactRange');
  const boxEarthImpactRange = document.getElementById('boxEarthImpactRange');
  const boxAlphaRange = document.getElementById('boxAlphaRange');

  let width = 0;
  let height = 0;
  let photonAnimProgress = 0;

  // Background Stars
  const stars = [];
  for (let i = 0; i < 220; i++) {
    stars.push({
      x: Math.random(),
      y: Math.random(),
      size: Math.random() * 1.5 + 0.3,
      alpha: Math.random() * 0.7 + 0.3
    });
  }

  function resizeCanvas() {
    const rect = canvas.parentElement.getBoundingClientRect();
    const dpr = window.devicePixelRatio || 1;
    width = rect.width || 800;
    height = rect.height || 480;

    canvas.width = width * dpr;
    canvas.height = height * dpr;
    ctx.scale(dpr, dpr);
  }

```



```

window.addEventListener('resize', resizeCanvas);
resizeCanvas();

// Distance constraints: d_G >= d_C + 1.0 Gly
sliderDistG.addEventListener('input', () => {
  let dG = parseFloat(sliderDistG.value);
  let dC = parseFloat(sliderDistC.value);
  if (dG < dC + 1.0) {
    dC = Math.max(1.0, dG - 1.0);
    sliderDistC.value = dC.toFixed(1);
  }
  updateControlStates();
});

sliderDistC.addEventListener('input', () => {
  let dG = parseFloat(sliderDistG.value);
  let dC = parseFloat(sliderDistC.value);
  if (dC > dG - 1.0) {
    dG = Math.min(13.0, dC + 1.0);
    sliderDistG.value = dG.toFixed(1);
  }
  updateControlStates();
});

function updateControlStates() {
  const numRays = parseInt(sliderNumRays.value);

  if (numRays > 1) {
    // Multi-ray mode: Angle/Direction slider inactive, Fan Spread active
    sliderTheta.disabled = true;
    grpTheta.classList.add('disabled-control');

    sliderFanSpread.disabled = false;
    grpFanSpread.classList.remove('disabled-control');
  } else {
    // Single-ray mode: Angle/Direction slider active, Fan Spread inactive
    sliderTheta.disabled = false;
    grpTheta.classList.remove('disabled-control');

    sliderFanSpread.disabled = true;
    grpFanSpread.classList.add('disabled-control');
  }
}

sliderNumRays.addEventListener('input', updateControlStates);
updateControlStates();

function calculateRays() {
  const d_G_Gly = parseFloat(sliderDistG.value);
  const y_G_kly = parseFloat(sliderOffsetY.value);
  const d_C_Gly = parseFloat(sliderDistC.value);
  const d_GC_Gly = d_G_Gly - d_C_Gly;
  const M_tril = parseFloat(sliderMass.value);
  const numRays = parseInt(sliderNumRays.value);
  const fanSpreadArcsec = parseFloat(sliderFanSpread.value);
  const thetaInitArcsec = parseFloat(sliderTheta.value);

  const ARCSEC_TO_RAD = Math.PI / (180.0 * 3600.0);
  const d_GC_ly = d_GC_Gly * 1e9;
  const d_C_ly = d_C_Gly * 1e9;
  const y_G_ly = y_G_kly * 1e3;

  const scale_4GM_c2_ly = 0.6243363 * M_tril;

  const rays = [];

  for (let i = 0; i < numRays; i++) {
    let initialAngle = 0;

    if (numRays === 1) {
      initialAngle = thetaInitArcsec;
    } else {
      const frac = (i / (numRays - 1)) - 0.5;

```

```

    initialAngle = frac * fanSpreadArcsec;
  }

  const theta_init_rad = initialAngle * ARCSEC_TO_RAD;

  // Impact parameter b_C at cluster relative to horizontal baseline (y=0)
  // b_C = y_G + d_GC * tan(theta_init)
  const b_C_ly = y_G_ly + d_GC_ly * Math.tan(theta_init_rad);
  const abs_b_C_ly = Math.abs(b_C_ly);

  let alpha_arcsec = 0;
  let alpha_rad = 0;
  if (abs_b_C_ly > 1e-3) {
    alpha_rad = scale_4GM_c2_ly / abs_b_C_ly;
    alpha_arcsec = alpha_rad / ARCSEC_TO_RAD;
  }

  // Deflection angle angle_after relative to horizontal axis
  let theta_after_rad = theta_init_rad;
  if (b_C_ly > 0) {
    theta_after_rad = theta_init_rad - alpha_rad;
  } else if (b_C_ly < 0) {
    theta_after_rad = theta_init_rad + alpha_rad;
  }
  const theta_after_arcsec = theta_after_rad / ARCSEC_TO_RAD;

  // Impact parameter b_E at Earth (d = 0)
  // Light travels from cluster (x = d_C) towards Earth (x = 0) at angle theta_after
  const b_E_ly = b_C_ly - d_C_ly * Math.tan(theta_after_rad);
  const abs_b_E_ly = Math.abs(b_E_ly);

  rays.push({
    index: i,
    theta_init_arcsec: initialAngle,
    b_C_ly: b_C_ly,
    abs_b_C_ly: abs_b_C_ly,
    b_E_ly: b_E_ly,
    abs_b_E_ly: abs_b_E_ly,
    alpha_arcsec: alpha_arcsec,
    theta_after_arcsec: theta_after_arcsec
  });
}

return {
  d_G_Gly,
  y_G_kly,
  d_C_Gly,
  d_GC_Gly,
  M_tril,
  numRays,
  fanSpreadArcsec,
  thetaInitArcsec,
  rays
};
}

function draw() {
  ctx.clearRect(0, 0, width, height);

  const sim = calculateRays();

  // Label Updates
  valDistG.textContent = `${sim.d_G_Gly.toFixed(1)} Gly`;
  valOffsetY.textContent = `${sim.y_G_kly >= 0 ? '+' : ''}${sim.y_G_kly} kly`;
  valDistC.textContent = `${sim.d_C_Gly.toFixed(1)} Gly`;
  valMass.textContent = `${sim.M_tril} × 1012 M⊙`;
  valNumRays.textContent = `${sim.numRays}`;
  valFanSpread.textContent = `${sim.fanSpreadArcsec}`;
  valTheta.textContent = `${sim.thetaInitArcsec >= 0 ? '+' : ''}${sim.thetaInitArcsec.toFixed(1)}°`;

  // Update Numerical Box
  boxDistG.textContent = `${sim.d_G_Gly.toFixed(1)} Gly`;
  boxOffsetY.textContent = `${sim.y_G_kly >= 0 ? '+' : ''}${sim.y_G_kly} kly`;
  boxDistC.textContent = `${sim.d_C_Gly.toFixed(1)} Gly`;

```

```

boxDistGC.textContent = `${sim.d_GC_Gly.toFixed(1)} Gly`;
boxMass.textContent = `${sim.M_tril} × 1012 M⊙`;

if (sim.numRays === 1) {
  const r = sim.rays[0];
  boxMode.textContent = `Single Ray`;
  boxImpactRange.textContent = `${(r.abs_b_C_ly / 1000.0).toFixed(2)} kly (${Math.round(r.abs_b_C_ly).toFixed(1)} kly)`;
  boxEarthImpactRange.textContent = `${(r.abs_b_E_ly / 1000.0).toFixed(2)} kly (${Math.round(r.abs_b_E_ly).toFixed(1)} kly)`;
  boxAlphaRange.textContent = `${r.alpha_arcsec.toFixed(2)}°`;
} else {
  boxMode.textContent = `Fan (${sim.numRays} rays)`;

  const bCValues = sim.rays.map(r => r.abs_b_C_ly / 1000.0);
  const bEValues = sim.rays.map(r => r.abs_b_E_ly / 1000.0);
  const alphaValues = sim.rays.map(r => r.alpha_arcsec).filter(a => isFinite(a));

  const minBC = Math.min(...bCValues);
  const maxBC = Math.max(...bCValues);
  const minBE = Math.min(...bEValues);
  const maxBE = Math.max(...bEValues);
  const maxAlpha = Math.max(...alphaValues);

  boxImpactRange.textContent = `${minBC.toFixed(1)} to ${maxBC.toFixed(1)} kly`;
  boxEarthImpactRange.textContent = `${minBE.toFixed(1)} to ${maxBE.toFixed(1)} kly`;
  boxAlphaRange.textContent = `${maxAlpha.toFixed(2)}° max`;
}

// Visual Layout Dimensions
const marginX = 80;
const centerY = height / 2;

const earthX = marginX;
// Fixed canvas scale: printable width span (width - 2*marginX) represents 15.0 Gly
const drawableWidth = width - 2 * marginX;
const pxPerGly = drawableWidth / CANVAS_TOTAL_GLY;

const galaxyX = earthX + (sim.d_G_Gly * pxPerGly);
const clusterX = earthX + (sim.d_C_Gly * pxPerGly);

// Uniform Vertical Physical Scale across Galaxy, Cluster, and Earth:
// 500 kly is mapped to 120 pixels visually on screen.
const pxPerKly = 120.0 / 500.0;
const galaxyY = centerY - (sim.y_G_kly * pxPerKly);

const ptA = { x: galaxyX, y: galaxyY };

// Starfield Background
ctx.fillStyle = "#ffffff";
stars.forEach(s => {
  ctx.globalAlpha = s.alpha;
  ctx.fillRect(s.x * width, s.y * height, s.size, s.size);
});
ctx.globalAlpha = 1.0;

// Baseline Axis (0 Gly to 15 Gly full canvas span)
ctx.strokeStyle = "rgba(255, 255, 255, 0.15)";
ctx.lineWidth = 1;
ctx.setLineDash([6, 6]);
ctx.beginPath();
ctx.moveTo(earthX - 20, centerY);
ctx.lineTo(earthX + CANVAS_TOTAL_GLY * pxPerGly + 20, centerY);
ctx.stroke();
ctx.setLineDash([]);

// Cluster Plane Line
ctx.strokeStyle = "rgba(0, 240, 255, 0.25)";
ctx.beginPath();
ctx.moveTo(clusterX, 30);
ctx.lineTo(clusterX, height - 30);
ctx.stroke();

// Earth Plane Line (for impact parameter b_E)
ctx.strokeStyle = "rgba(0, 240, 255, 0.15)";
ctx.beginPath();

```

```

ctx.moveTo(earthX, 30);
ctx.lineTo(earthX, height - 30);
ctx.stroke();

// Render Light Rays & Virtual Tangents
sim.rays.forEach((ray, idx) => {
  // Calculate physical pixel Y coordinates directly from light path geometry using pxPerKly
  const clusterY = centerY - (ray.b_C_ly / 1000.0) * pxPerKly;
  const earthY = centerY - (ray.b_E_ly / 1000.0) * pxPerKly;

  const ptB = { x: clusterX, y: clusterY };
  const ptC = { x: earthX, y: earthY };

  // Draw Virtual Tangent Line (Extrapolated straight back from Earth)
  if (toggleTangent.checked && Math.abs(ray.alpha_arcsec) > 0.1) {
    ctx.strokeStyle = "rgba(239, 68, 68, 0.5)";
    ctx.lineWidth = 1.2;
    ctx.setLineDash([3, 3]);

    // Slope of deflected ray from Cluster to Earth
    const slope = (ptC.y - ptB.y) / (ptC.x - ptB.x);
    const virtualY_at_galaxy = ptB.y + slope * (galaxyX - clusterX);

    ctx.beginPath();
    ctx.moveTo(ptC.x, ptC.y);
    ctx.lineTo(galaxyX, virtualY_at_galaxy);
    ctx.stroke();
    ctx.setLineDash([]);

    ctx.fillStyle = "rgba(239, 68, 68, 0.8)";
    ctx.beginPath();
    ctx.arc(galaxyX, virtualY_at_galaxy, 3.5, 0, Math.PI * 2);
    ctx.fill();
  }

  // Draw Actual Light Path (Galaxy -> Cluster -> Earth Impact Plane)
  ctx.strokeStyle = "#facc15";
  ctx.lineWidth = sim.numRays === 1 ? 2.5 : 1.8;
  ctx.shadowColor = "#facc15";
  ctx.shadowBlur = sim.numRays === 1 ? 8 : 4;

  ctx.beginPath();
  ctx.moveTo(ptA.x, ptA.y);
  ctx.lineTo(ptB.x, ptB.y);
  ctx.lineTo(ptC.x, ptC.y);
  ctx.stroke();
  ctx.shadowBlur = 0;

  // Visual Indicators for Impact Parameters in Single-Ray Mode
  if (sim.numRays === 1) {
    // Cluster Impact Parameter (b_C)
    if (Math.abs(ptB.y - centerY) > 3) {
      ctx.strokeStyle = "#00f0ff";
      ctx.lineWidth = 1.5;
      ctx.beginPath();
      ctx.moveTo(clusterX, centerY);
      ctx.lineTo(clusterX, ptB.y);
      ctx.stroke();

      ctx.fillStyle = "#00f0ff";
      ctx.beginPath();
      ctx.arc(clusterX, ptB.y, 3, 0, Math.PI * 2);
      ctx.arc(clusterX, centerY, 3, 0, Math.PI * 2);
      ctx.fill();

      ctx.font = "11px monospace";
      ctx.textAlign = "right";
      ctx.fillText(`b_C = ${ (ray.abs_b_C_ly / 1000).toFixed(1) } kly`, clusterX - 6, (centerY + ptB.y) / 2);
    }

    // Earth Impact Parameter (b_E)
    if (Math.abs(ptC.y - centerY) > 3) {
      ctx.strokeStyle = "#00f0ff";
      ctx.lineWidth = 1.5;
    }
  }
});

```

```

    ctx.beginPath();
    ctx.moveTo(earthX, centerY);
    ctx.lineTo(earthX, ptC.y);
    ctx.stroke();

    ctx.fillStyle = "#00f0ff";
    ctx.beginPath();
    ctx.arc(earthX, ptC.y, 3, 0, Math.PI * 2);
    ctx.arc(earthX, centerY, 3, 0, Math.PI * 2);
    ctx.fill();

    ctx.font = "11px monospace";
    ctx.textAlign = "left";
    ctx.fillText(` b_E = ${ (ray.abs_b_E_ly / 1000).toFixed(1) } kly`, earthX + 6, (centerY + ptC.y) / 2
  }
}

// Animated Photons
if (togglePhoton.checked) {
  const rayOffset = (idx / sim.numRays);
  const progress = (photonAnimProgress + rayOffset) % 1.0;
  let px = 0, py = 0;

  if (progress <= 0.5) {
    const t = progress / 0.5;
    px = ptA.x + t * (ptB.x - ptA.x);
    py = ptA.y + t * (ptB.y - ptA.y);
  } else {
    const t = (progress - 0.5) / 0.5;
    px = ptB.x + t * (ptC.x - ptB.x);
    py = ptB.y + t * (ptC.y - ptB.y);
  }

  ctx.fillStyle = "#ffffff";
  ctx.shadowColor = "#00f0ff";
  ctx.shadowBlur = 8;
  ctx.beginPath();
  ctx.arc(px, py, 3.5, 0, Math.PI * 2);
  ctx.fill();
  ctx.shadowBlur = 0;
}
});

if (togglePhoton.checked) {
  photonAnimProgress = (photonAnimProgress + 0.006) % 1.0;
}

// Draw Celestial Objects
// Distant Galaxy
ctx.save();
ctx.translate(ptA.x, ptA.y);
const galGlow = ctx.createRadialGradient(0, 0, 2, 0, 0, 22);
galGlow.addColorStop(0, "rgba(255, 200, 100, 0.9)");
galGlow.addColorStop(0.5, "rgba(238, 100, 30, 0.3)");
galGlow.addColorStop(1, "rgba(0, 0, 0, 0)");
ctx.fillStyle = galGlow;
ctx.beginPath();
ctx.arc(0, 0, 22, 0, Math.PI * 2);
ctx.fill();

ctx.fillStyle = "#ffd59e";
ctx.beginPath();
ctx.ellipse(0, 0, 14, 5, -0.3, 0, Math.PI * 2);
ctx.fill();
ctx.restore();

ctx.fillStyle = "#e1e7ec";
ctx.font = "12px sans-serif";
ctx.textAlign = "center";
ctx.fillText("Distant Galaxy", ptA.x, ptA.y + 32);

// Intermediate Cluster
ctx.save();
ctx.translate(clusterX, centerY);

```

```

const haloRadius = 18 + (sim.M_tril / 1000) * 22;
const clusterGlow = ctx.createRadialGradient(0, 0, 2, 0, 0, haloRadius);
clusterGlow.addColorStop(0, "rgba(168, 85, 247, 0.65)");
clusterGlow.addColorStop(0.6, "rgba(126, 34, 206, 0.2)");
clusterGlow.addColorStop(1, "rgba(0, 0, 0, 0)");
ctx.fillStyle = clusterGlow;
ctx.beginPath();
ctx.arc(0, 0, haloRadius, 0, Math.PI * 2);
ctx.fill();

ctx.fillStyle = "#ffffff";
ctx.beginPath();
ctx.arc(0, 0, 4, 0, Math.PI * 2);
ctx.arc(-8, -6, 2.5, 0, Math.PI * 2);
ctx.arc(10, 5, 3, 0, Math.PI * 2);
ctx.fill();
ctx.restore();

ctx.fillStyle = "#e1e7ec";
ctx.font = "12px sans-serif";
ctx.textAlign = "center";
ctx.fillText("Intermediate Cluster", clusterX, centerY + haloRadius + 16);

// Earth / Observer
ctx.save();
ctx.translate(earthX, centerY);
const earthGrad = ctx.createRadialGradient(-2, -2, 1, 0, 0, 10);
earthGrad.addColorStop(0, "#60a5fa");
earthGrad.addColorStop(0.8, "#1d4ed8");
earthGrad.addColorStop(1, "#1e3a8a");
ctx.fillStyle = earthGrad;
ctx.beginPath();
ctx.arc(0, 0, 9, 0, Math.PI * 2);
ctx.fill();
ctx.restore();

ctx.fillStyle = "#e1e7ec";
ctx.font = "12px sans-serif";
ctx.textAlign = "center";
ctx.fillText("Earth", earthX, centerY + 25);
}

function animate() {
  draw();
  requestAnimationFrame(animate);
}

animate();
</script>
</body>
</html>

```

User prompt: You have the direction of the deflection backwards. For example, if the distant galaxy is offset in the positive direction, and the ray has an angle of 0, the resulting deflected ray should be below horizontal, not moving upward.

```

Response: <!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Gravitational Lensing: Fixed 15 Gly Field Simulation</title>
<style>
  :root {
    --bg-color: #0b0e14;
    --panel-bg: #151922;
    --panel-border: #2a3142;
    --accent-color: #00f0ff;
    --accent-amber: #facc15;
    --accent-red: #ef4444;
    --text-main: #e1e7ec;
    --text-muted: #8c9bae;
  }

```

```
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial, sans-serif;
  background-color: var(--bg-color);
  color: var(--text-main);
  display: flex;
  flex-direction: column;
  min-height: 100vh;
  overflow-x: hidden;
}

header {
  background-color: var(--panel-bg);
  border-bottom: 1px solid var(--panel-border);
  padding: 0.85rem 1.5rem;
  text-align: center;
}

header h1 {
  font-size: 1.3rem;
  font-weight: 600;
  color: #fff;
}

header p {
  font-size: 0.85rem;
  color: var(--text-muted);
  margin-top: 0.2rem;
}

.main-container {
  display: flex;
  flex: 1;
  flex-direction: column;
  gap: 1rem;
  padding: 1rem;
  max-width: 1400px;
  margin: 0 auto;
  width: 100%;
}

@media (min-width: 1024px) {
  .main-container {
    flex-direction: row;
  }
}

.canvas-card {
  flex: 1;
  background-color: var(--panel-bg);
  border: 1px solid var(--panel-border);
  border-radius: 8px;
  padding: 1rem;
  display: flex;
  flex-direction: column;
  gap: 1rem;
  box-shadow: 0 4px 20px rgba(0, 0, 0, 0.4);
}

.canvas-wrapper {
  position: relative;
  width: 100%;
  height: 480px;
  background-color: #04060a;
  border-radius: 6px;
  overflow: hidden;
  border: 1px solid rgba(255, 255, 255, 0.05);
}
```

```
canvas {
  width: 100%;
  height: 100%;
  display: block;
}

.calc-box {
  background: rgba(10, 14, 22, 0.7);
  border: 1px solid var(--panel-border);
  border-radius: 6px;
  padding: 1rem 1.25rem;
  font-family: SFMono-Regular, Consolas, "Liberation Mono", Menlo, monospace;
  font-size: 0.8rem;
  color: #d1d5db;
}

.calc-box h3 {
  font-size: 0.85rem;
  text-transform: uppercase;
  letter-spacing: 0.05em;
  color: var(--accent-color);
  margin-bottom: 0.75rem;
  border-bottom: 1px solid rgba(255, 255, 255, 0.1);
  padding-bottom: 0.4rem;
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;
}

.calc-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(190px, 1fr));
  gap: 0.85rem 1.25rem;
}

.calc-item {
  display: flex;
  flex-direction: column;
  gap: 0.25rem;
}

.calc-item .label {
  color: var(--text-muted);
  font-size: 0.75rem;
}

.calc-item .value {
  color: #fff;
  font-weight: 600;
  font-size: 0.85rem;
}

.calc-item .highlight {
  color: var(--accent-amber);
}

.calc-footer {
  margin-top: 0.85rem;
  padding-top: 0.6rem;
  border-top: 1px dashed rgba(255, 255, 255, 0.15);
  display: flex;
  justify-content: space-between;
  align-items: center;
  font-size: 0.75rem;
}

.calc-footer .formula {
  color: #9ca3af;
}

.controls-panel {
  width: 100%;
  max-width: 380px;
  background-color: var(--panel-bg);
  border: 1px solid var(--panel-border);
}
```



```
border-radius: 8px;
padding: 1.25rem;
display: flex;
flex-direction: column;
gap: 1.1rem;
box-shadow: 0 4px 20px rgba(0, 0, 0, 0.4);
}

.control-group {
display: flex;
flex-direction: column;
gap: 0.4rem;
transition: opacity 0.2s ease;
}

.control-group.disabled-control {
opacity: 0.35;
}

.control-group label {
font-size: 0.83rem;
font-weight: 500;
color: var(--text-main);
display: flex;
justify-content: space-between;
}

.control-group label span {
color: var(--accent-color);
font-family: monospace;
}

input[type="range"] {
-webkit-appearance: none;
width: 100%;
height: 6px;
border-radius: 3px;
background: #2a3142;
outline: none;
}

input[type="range"]:disabled {
cursor: not-allowed;
}

input[type="range"]::-webkit-slider-thumb {
-webkit-appearance: none;
width: 16px;
height: 16px;
border-radius: 50%;
background: var(--accent-color);
cursor: pointer;
box-shadow: 0 0 8px var(--accent-color);
transition: transform 0.1s ease;
}

input[type="range"]:disabled::-webkit-slider-thumb {
background: #6b7280;
box-shadow: none;
cursor: not-allowed;
}

input[type="range"]::-webkit-slider-thumb:hover:not(:disabled) {
transform: scale(1.2);
}

.toggle-row {
display: flex;
align-items: center;
justify-content: space-between;
font-size: 0.83rem;
color: var(--text-main);
}
```

```

.switch {
  position: relative;
  display: inline-block;
  width: 40px;
  height: 20px;
}

.switch input {
  opacity: 0;
  width: 0;
  height: 0;
}

.slider {
  position: absolute;
  cursor: pointer;
  top: 0; left: 0; right: 0; bottom: 0;
  background-color: #2a3142;
  transition: .2s;
  border-radius: 20px;
}

.slider:before {
  position: absolute;
  content: "";
  height: 14px;
  width: 14px;
  left: 3px;
  bottom: 3px;
  background-color: white;
  transition: .2s;
  border-radius: 50%;
}

input:checked + .slider {
  background-color: var(--accent-color);
}

input:checked + .slider:before {
  transform: translateX(20px);
}

.legend {
  border-top: 1px solid var(--panel-border);
  padding-top: 0.8rem;
  display: flex;
  flex-direction: column;
  gap: 0.4rem;
  font-size: 0.8rem;
}

.legend-item {
  display: flex;
  align-items: center;
  gap: 0.6rem;
  color: var(--text-muted);
}

.legend-color {
  width: 14px;
  height: 4px;
  border-radius: 2px;
}
</style>
</head>
<body>

<header>
  <h1>Gravitational Deflection of Light Fan</h1>
  <p>Fixed 15 Gly Field Simulation (Attractive Deflection Geometry)</p>
</header>

<div class="main-container">
  <div class="canvas-card">

```

```

<div class="canvas-wrapper">
  <canvas id="simCanvas"></canvas>
</div>

<!-- Numerical Output Box below Graphic -->
<div class="calc-box">
  <h3>Numerical Results</h3>
  <div class="calc-grid">
    <div class="calc-item">
      <span class="label">Galaxy Distance ( $d_G$ ):</span>
      <span class="value" id="boxDistG">8.0 Gly</span>
    </div>
    <div class="calc-item">
      <span class="label">Galaxy Offset ( $y_G$ ):</span>
      <span class="value" id="boxOffsetY">0 kly</span>
    </div>
    <div class="calc-item">
      <span class="label">Cluster Distance ( $d_C$ ):</span>
      <span class="value" id="boxDistC">4.0 Gly</span>
    </div>
    <div class="calc-item">
      <span class="label"> $d_{GC} = d_G - d_C$ :</span>
      <span class="value" id="boxDistGC">4.0 Gly</span>
    </div>
    <div class="calc-item">
      <span class="label">Cluster Mass (M):</span>
      <span class="value" id="boxMass"> $200 \times 10^{12} M_\odot$ </span>
    </div>
    <div class="calc-item">
      <span class="label">Ray Mode:</span>
      <span class="value" id="boxMode">Single Ray</span>
    </div>
    <div class="calc-item">
      <span class="label">Cluster Impact ( $b_C$ ):</span>
      <span class="value highlight" id="boxImpactRange">290.9 kly</span>
    </div>
    <div class="calc-item">
      <span class="label">Earth Impact ( $b_E$ ):</span>
      <span class="value highlight" id="boxEarthImpactRange">581.8 kly</span>
    </div>
    <div class="calc-item">
      <span class="label">Deflection Angle ( $\alpha$ ):</span>
      <span class="value highlight" id="boxAlphaRange">44.3" </span>
    </div>
  </div>
  <div class="calc-footer">
    <span class="label">Theoretical Formula:</span>
    <span class="formula"> $\alpha = 4GM / (c^2 |b_C|) \quad \&nbps; \quad b_E = b_C + d$ </span>
  </div>
</div>

<div class="controls-panel">
  <div class="control-group">
    <label for="sliderDistG">
      1. Distance to Galaxy ( $d_G$ ):
      <span id="valDistG">8.0 Gly</span>
    </label>
    <input type="range" id="sliderDistG" min="2.0" max="13.0" step="0.1" value="8.0">
  </div>

  <div class="control-group">
    <label for="sliderOffsetY">
      2. Galaxy Vertical Offset ( $y_G$ ):
      <span id="valOffsetY">0 kly</span>
    </label>
    <input type="range" id="sliderOffsetY" min="-500" max="500" step="10" value="0">
  </div>

  <div class="control-group">
    <label for="sliderDistC">
      3. Distance to Cluster ( $d_C$ ):
      <span id="valDistC">4.0 Gly</span>
    </label>
  </div>
</div>

```

```

    <input type="range" id="sliderDistC" min="1.0" max="12.0" step="0.1" value="4.0">
  </div>

  <div class="control-group">
    <label for="sliderMass">
      4. Cluster Mass (M):
      <span id="valMass">200 × 1012 M⊙</span>
    </label>
    <input type="range" id="sliderMass" min="5" max="1000" step="5" value="200">
  </div>

  <div class="control-group">
    <label for="sliderNumRays">
      5. Number of Light Rays:
      <span id="valNumRays">1</span>
    </label>
    <input type="range" id="sliderNumRays" min="1" max="10" step="1" value="1">
  </div>

  <div class="control-group" id="grpFanSpread">
    <label for="sliderFanSpread">
      6. Ray Fan Spread:
      <span id="valFanSpread">20°</span>
    </label>
    <input type="range" id="sliderFanSpread" min="10" max="40" step="1" value="20">
  </div>

  <div class="control-group" id="grpTheta">
    <label for="sliderTheta">
      7. Initial Ray Direction (1 ray):
      <span id="valTheta">+15.0°</span>
    </label>
    <input type="range" id="sliderTheta" min="-40.0" max="40.0" step="0.5" value="15.0">
  </div>

  <div class="toggle-row">
    <span>Animate Photons</span>
    <label class="switch">
      <input type="checkbox" id="togglePhoton" checked>
      <span class="slider"></span>
    </label>
  </div>

  <div class="toggle-row">
    <span>Show Virtual Tangent Lines</span>
    <label class="switch">
      <input type="checkbox" id="toggleTangent" checked>
      <span class="slider"></span>
    </label>
  </div>

  <div class="legend">
    <div class="legend-item">
      <div class="legend-color" style="background: #facc15;"></div>
      <span>Actual deflected light rays</span>
    </div>
    <div class="legend-item">
      <div class="legend-color" style="border: 1px dashed #ef4444; background: transparent;"></div>
      <span>Apparent straight-line paths (virtual images)</span>
    </div>
    <div class="legend-item">
      <div class="legend-color" style="background: #00f0ff;"></div>
      <span>Cluster impact parameter (b<sub>C</sub>) & Earth impact (b<sub>E</sub>)</span>
    </div>
  </div>
</div>
</div>

<script>
  const canvas = document.getElementById('simCanvas');
  const ctx = canvas.getContext('2d');

  // Canvas Fixed Total Width Scale (15 Billion Light-Years)
  const CANVAS_TOTAL_GLY = 15.0;

```

```

// Controls
const sliderDistG = document.getElementById('sliderDistG');
const sliderOffsetY = document.getElementById('sliderOffsetY');
const sliderDistC = document.getElementById('sliderDistC');
const sliderMass = document.getElementById('sliderMass');
const sliderNumRays = document.getElementById('sliderNumRays');
const sliderFanSpread = document.getElementById('sliderFanSpread');
const sliderTheta = document.getElementById('sliderTheta');

const grpFanSpread = document.getElementById('grpFanSpread');
const grpTheta = document.getElementById('grpTheta');

const valDistG = document.getElementById('valDistG');
const valOffsetY = document.getElementById('valOffsetY');
const valDistC = document.getElementById('valDistC');
const valMass = document.getElementById('valMass');
const valNumRays = document.getElementById('valNumRays');
const valFanSpread = document.getElementById('valFanSpread');
const valTheta = document.getElementById('valTheta');

const togglePhoton = document.getElementById('togglePhoton');
const toggleTangent = document.getElementById('toggleTangent');

// Calculation Box Elements
const boxDistG = document.getElementById('boxDistG');
const boxOffsetY = document.getElementById('boxOffsetY');
const boxDistC = document.getElementById('boxDistC');
const boxDistGC = document.getElementById('boxDistGC');
const boxMass = document.getElementById('boxMass');
const boxMode = document.getElementById('boxMode');
const boxImpactRange = document.getElementById('boxImpactRange');
const boxEarthImpactRange = document.getElementById('boxEarthImpactRange');
const boxAlphaRange = document.getElementById('boxAlphaRange');

let width = 0;
let height = 0;
let photonAnimProgress = 0;

// Background Stars
const stars = [];
for (let i = 0; i < 220; i++) {
  stars.push({
    x: Math.random(),
    y: Math.random(),
    size: Math.random() * 1.5 + 0.3,
    alpha: Math.random() * 0.7 + 0.3
  });
}

function resizeCanvas() {
  const rect = canvas.parentElement.getBoundingClientRect();
  const dpr = window.devicePixelRatio || 1;
  width = rect.width || 800;
  height = rect.height || 480;

  canvas.width = width * dpr;
  canvas.height = height * dpr;
  ctx.scale(dpr, dpr);
}

window.addEventListener('resize', resizeCanvas);
resizeCanvas();

// Distance constraints: d_G >= d_C + 1.0 Gly
sliderDistG.addEventListener('input', () => {
  let dG = parseFloat(sliderDistG.value);
  let dC = parseFloat(sliderDistC.value);
  if (dG < dC + 1.0) {
    dC = Math.max(1.0, dG - 1.0);
    sliderDistC.value = dC.toFixed(1);
  }
  updateControlStates();
});

```

```

sliderDistC.addEventListener('input', () => {
  let dG = parseFloat(sliderDistG.value);
  let dC = parseFloat(sliderDistC.value);
  if (dC > dG - 1.0) {
    dG = Math.min(13.0, dC + 1.0);
    sliderDistG.value = dG.toFixed(1);
  }
  updateControlStates();
});

function updateControlStates() {
  const numRays = parseInt(sliderNumRays.value);

  if (numRays > 1) {
    // Multi-ray mode: Angle/Direction slider inactive, Fan Spread active
    sliderTheta.disabled = true;
    grpTheta.classList.add('disabled-control');

    sliderFanSpread.disabled = false;
    grpFanSpread.classList.remove('disabled-control');
  } else {
    // Single-ray mode: Angle/Direction slider active, Fan Spread inactive
    sliderTheta.disabled = false;
    grpTheta.classList.remove('disabled-control');

    sliderFanSpread.disabled = true;
    grpFanSpread.classList.add('disabled-control');
  }
}

sliderNumRays.addEventListener('input', updateControlStates);
updateControlStates();

function calculateRays() {
  const d_G_Gly = parseFloat(sliderDistG.value);
  const y_G_kly = parseFloat(sliderOffsetY.value);
  const d_C_Gly = parseFloat(sliderDistC.value);
  const d_GC_Gly = d_G_Gly - d_C_Gly;
  const M_tril = parseFloat(sliderMass.value);
  const numRays = parseInt(sliderNumRays.value);
  const fanSpreadArcsec = parseFloat(sliderFanSpread.value);
  const thetaInitArcsec = parseFloat(sliderTheta.value);

  const ARCSEC_TO_RAD = Math.PI / (180.0 * 3600.0);
  const d_GC_ly = d_GC_Gly * 1e9;
  const d_C_ly = d_C_Gly * 1e9;
  const y_G_ly = y_G_kly * 1e3;

  const scale_4GM_c2_ly = 0.6243363 * M_tril;

  const rays = [];

  for (let i = 0; i < numRays; i++) {
    let initialAngle = 0;

    if (numRays === 1) {
      initialAngle = thetaInitArcsec;
    } else {
      const frac = (i / (numRays - 1)) - 0.5;
      initialAngle = frac * fanSpreadArcsec;
    }

    const theta_init_rad = initialAngle * ARCSEC_TO_RAD;

    // Impact parameter b_C at cluster relative to horizontal baseline (y=0)
    // Light travels leftward from Galaxy (d_G) to Cluster (d_C).
    // Positive theta_init_rad aims upwards towards +y.
    const b_C_ly = y_G_ly + d_GC_ly * Math.tan(theta_init_rad);
    const abs_b_C_ly = Math.abs(b_C_ly);

    let alpha_arcsec = 0;
    let alpha_rad = 0;
    if (abs_b_C_ly > 1e-3) {

```

```

    alpha_rad = scale_4GM_c2_ly / abs_b_C_ly;
    alpha_arcsec = alpha_rad / ARCSEC_TO_RAD;
}

// Gravity is ATTRACTIVE towards y = 0:
// - If b_C > 0 (ray is above cluster), gravity bends ray DOWNWARDS (towards smaller y, decreasing angle)
// - If b_C < 0 (ray is below cluster), gravity bends ray UPWARDS (towards larger y, increasing angle)
let theta_after_rad = theta_init_rad;
if (b_C_ly > 0) {
    theta_after_rad = theta_init_rad - alpha_rad;
} else if (b_C_ly < 0) {
    theta_after_rad = theta_init_rad + alpha_rad;
}
const theta_after_arcsec = theta_after_rad / ARCSEC_TO_RAD;

// Impact parameter b_E at Earth (d = 0)
// As light travels leftward from cluster (x = d_C) to Earth (x = 0),
// vertical position changes by d_C * tan(theta_after_rad)
const b_E_ly = b_C_ly + d_C_ly * Math.tan(theta_after_rad);
const abs_b_E_ly = Math.abs(b_E_ly);

rays.push({
    index: i,
    theta_init_arcsec: initialAngle,
    b_C_ly: b_C_ly,
    abs_b_C_ly: abs_b_C_ly,
    b_E_ly: b_E_ly,
    abs_b_E_ly: abs_b_E_ly,
    alpha_arcsec: alpha_arcsec,
    theta_after_arcsec: theta_after_arcsec
});
}

return {
    d_G_Gly,
    y_G_kly,
    d_C_Gly,
    d_GC_Gly,
    M_tril,
    numRays,
    fanSpreadArcsec,
    thetaInitArcsec,
    rays
};
}

function draw() {
    ctx.clearRect(0, 0, width, height);

    const sim = calculateRays();

    // Label Updates
    valDistG.textContent = `${sim.d_G_Gly.toFixed(1)} Gly`;
    valOffsetY.textContent = `${sim.y_G_kly >= 0 ? '+' : ''}${sim.y_G_kly} kly`;
    valDistC.textContent = `${sim.d_C_Gly.toFixed(1)} Gly`;
    valMass.textContent = `${sim.M_tril} × 1012 M⊙`;
    valNumRays.textContent = `${sim.numRays}`;
    valFanSpread.textContent = `${sim.fanSpreadArcsec}`;
    valTheta.textContent = `${sim.thetaInitArcsec >= 0 ? '+' : ''}${sim.thetaInitArcsec.toFixed(1)}°`;

    // Update Numerical Box
    boxDistG.textContent = `${sim.d_G_Gly.toFixed(1)} Gly`;
    boxOffsetY.textContent = `${sim.y_G_kly >= 0 ? '+' : ''}${sim.y_G_kly} kly`;
    boxDistC.textContent = `${sim.d_C_Gly.toFixed(1)} Gly`;
    boxDistGC.textContent = `${sim.d_GC_Gly.toFixed(1)} Gly`;
    boxMass.textContent = `${sim.M_tril} × 1012 M⊙`;

    if (sim.numRays === 1) {
        const r = sim.rays[0];
        boxMode.textContent = `Single Ray`;
        boxImpactRange.textContent = `${(r.abs_b_C_ly / 1000.0).toFixed(2)} kly (${Math.round(r.abs_b_C_ly).toLocaleString()} pc)`;
        boxEarthImpactRange.textContent = `${(r.abs_b_E_ly / 1000.0).toFixed(2)} kly (${Math.round(r.abs_b_E_ly).toLocaleString()} pc)`;
        boxAlphaRange.textContent = `${r.alpha_arcsec.toFixed(2)}°`;
    } else {

```

```

boxMode.textContent = `Fan (${sim.numRays} rays)`;

const bcValues = sim.rays.map(r => r.abs_b_C_ly / 1000.0);
const bEValues = sim.rays.map(r => r.abs_b_E_ly / 1000.0);
const alphaValues = sim.rays.map(r => r.alpha_arcsec).filter(a => isFinite(a));

const minBC = Math.min(...bcValues);
const maxBC = Math.max(...bcValues);
const minBE = Math.min(...bEValues);
const maxBE = Math.max(...bEValues);
const maxAlpha = Math.max(...alphaValues);

boxImpactRange.textContent = `${minBC.toFixed(1)} to ${maxBC.toFixed(1)} kly`;
boxEarthImpactRange.textContent = `${minBE.toFixed(1)} to ${maxBE.toFixed(1)} kly`;
boxAlphaRange.textContent = `${maxAlpha.toFixed(2)}" max`;
}

// Visual Layout Dimensions
const marginX = 80;
const centerY = height / 2;

const earthX = marginX;
// Fixed canvas scale: printable width span (width - 2*marginX) represents 15.0 Gly
const drawableWidth = width - 2 * marginX;
const pxPerGly = drawableWidth / CANVAS_TOTAL_GLY;

const galaxyX = earthX + (sim.d_G_Gly * pxPerGly);
const clusterX = earthX + (sim.d_C_Gly * pxPerGly);

// Uniform Vertical Physical Scale across Galaxy, Cluster, and Earth:
// 500 kly is mapped to 120 pixels visually on screen.
// Note: In Canvas Y coordinate space, smaller Y is UP, larger Y is DOWN.
const pxPerKly = 120.0 / 500.0;
const galaxyY = centerY - (sim.y_G_kly * pxPerKly);

const ptA = { x: galaxyX, y: galaxyY };

// Starfield Background
ctx.fillStyle = "#ffffff";
stars.forEach(s => {
  ctx.globalAlpha = s.alpha;
  ctx.fillRect(s.x * width, s.y * height, s.size, s.size);
});
ctx.globalAlpha = 1.0;

// Baseline Axis (0 Gly to 15 Gly full canvas span)
ctx.strokeStyle = "rgba(255, 255, 255, 0.15)";
ctx.lineWidth = 1;
ctx.setLineDash([6, 6]);
ctx.beginPath();
ctx.moveTo(earthX - 20, centerY);
ctx.lineTo(earthX + CANVAS_TOTAL_GLY * pxPerGly + 20, centerY);
ctx.stroke();
ctx.setLineDash([]);

// Cluster Plane Line
ctx.strokeStyle = "rgba(0, 240, 255, 0.25)";
ctx.beginPath();
ctx.moveTo(clusterX, 30);
ctx.lineTo(clusterX, height - 30);
ctx.stroke();

// Earth Plane Line (for impact parameter b_E)
ctx.strokeStyle = "rgba(0, 240, 255, 0.15)";
ctx.beginPath();
ctx.moveTo(earthX, 30);
ctx.lineTo(earthX, height - 30);
ctx.stroke();

// Render Light Rays & Virtual Tangents
sim.rays.forEach((ray, idx) => {
  // Physical pixel Y coordinates: physical positive y is UP (subtract from centerY in Canvas)
  const clusterY = centerY - (ray.b_C_ly / 1000.0) * pxPerGly;
  const earthY = centerY - (ray.b_E_ly / 1000.0) * pxPerKly;

```



```

const ptB = { x: clusterX, y: clusterY };
const ptC = { x: earthX, y: earthY };

// Draw Virtual Tangent Line (Extrapolated straight line from Earth direction back to Galaxy plane)
if (toggleTangent.checked && Math.abs(ray.alpha_arcsec) > 0.1) {
  ctx.strokeStyle = "rgba(239, 68, 68, 0.5)";
  ctx.lineWidth = 1.2;
  ctx.setLineDash([3, 3]);

  // Slope of deflected ray arriving at Earth (from cluster to Earth)
  const slope = (ptC.y - ptB.y) / (ptC.x - ptB.x);
  const virtualY_at_galaxy = ptB.y + slope * (galaxyX - clusterX);

  ctx.beginPath();
  ctx.moveTo(ptC.x, ptC.y);
  ctx.lineTo(galaxyX, virtualY_at_galaxy);
  ctx.stroke();
  ctx.setLineDash([]);

  ctx.fillStyle = "rgba(239, 68, 68, 0.8)";
  ctx.beginPath();
  ctx.arc(galaxyX, virtualY_at_galaxy, 3.5, 0, Math.PI * 2);
  ctx.fill();
}

// Draw Actual Light Path (Galaxy -> Cluster -> Earth Impact Plane)
ctx.strokeStyle = "#facc15";
ctx.lineWidth = sim.numRays === 1 ? 2.5 : 1.8;
ctx.shadowColor = "#facc15";
ctx.shadowBlur = sim.numRays === 1 ? 8 : 4;

ctx.beginPath();
ctx.moveTo(ptA.x, ptA.y);
ctx.lineTo(ptB.x, ptB.y);
ctx.lineTo(ptC.x, ptC.y);
ctx.stroke();
ctx.shadowBlur = 0;

// Visual Indicators for Impact Parameters in Single-Ray Mode
if (sim.numRays === 1) {
  // Cluster Impact Parameter (b_C)
  if (Math.abs(ptB.y - centerY) > 3) {
    ctx.strokeStyle = "#00f0ff";
    ctx.lineWidth = 1.5;
    ctx.beginPath();
    ctx.moveTo(clusterX, centerY);
    ctx.lineTo(clusterX, ptB.y);
    ctx.stroke();

    ctx.fillStyle = "#00f0ff";
    ctx.beginPath();
    ctx.arc(clusterX, ptB.y, 3, 0, Math.PI * 2);
    ctx.arc(clusterX, centerY, 3, 0, Math.PI * 2);
    ctx.fill();

    ctx.font = "11px monospace";
    ctx.textAlign = "right";
    ctx.fillText(`b_C = ${ray.abs_b_C_ly / 1000}.toFixed(1)} kly`, clusterX - 6, (centerY + ptB.y) / 2);
  }

  // Earth Impact Parameter (b_E)
  if (Math.abs(ptC.y - centerY) > 3) {
    ctx.strokeStyle = "#00f0ff";
    ctx.lineWidth = 1.5;
    ctx.beginPath();
    ctx.moveTo(earthX, centerY);
    ctx.lineTo(earthX, ptC.y);
    ctx.stroke();

    ctx.fillStyle = "#00f0ff";
    ctx.beginPath();
    ctx.arc(earthX, ptC.y, 3, 0, Math.PI * 2);
    ctx.arc(earthX, centerY, 3, 0, Math.PI * 2);
    ctx.fill();
  }
}

```

```

    ctx.fill();

    ctx.font = "11px monospace";
    ctx.textAlign = "left";
    ctx.fillText(` b_E = ${ (ray.abs_b_E_ly / 1000).toFixed(1) } kly`, earthX + 6, (centerY + ptC.y) / 2
  }
}

// Animated Photons
if (togglePhoton.checked) {
  const rayOffset = (idx / sim.numRays);
  const progress = (photonAnimProgress + rayOffset) % 1.0;
  let px = 0, py = 0;

  if (progress <= 0.5) {
    const t = progress / 0.5;
    px = ptA.x + t * (ptB.x - ptA.x);
    py = ptA.y + t * (ptB.y - ptA.y);
  } else {
    const t = (progress - 0.5) / 0.5;
    px = ptB.x + t * (ptC.x - ptB.x);
    py = ptB.y + t * (ptC.y - ptB.y);
  }

  ctx.fillStyle = "#ffffff";
  ctx.shadowColor = "#00f0ff";
  ctx.shadowBlur = 8;
  ctx.beginPath();
  ctx.arc(px, py, 3.5, 0, Math.PI * 2);
  ctx.fill();
  ctx.shadowBlur = 0;
}
});

if (togglePhoton.checked) {
  photonAnimProgress = (photonAnimProgress + 0.006) % 1.0;
}

// Draw Celestial Objects
// Distant Galaxy
ctx.save();
ctx.translate(ptA.x, ptA.y);
const galGlow = ctx.createRadialGradient(0, 0, 2, 0, 0, 22);
galGlow.addColorStop(0, "rgba(255, 200, 100, 0.9)");
galGlow.addColorStop(0.5, "rgba(238, 100, 30, 0.3)");
galGlow.addColorStop(1, "rgba(0, 0, 0, 0)");
ctx.fillStyle = galGlow;
ctx.beginPath();
ctx.arc(0, 0, 22, 0, Math.PI * 2);
ctx.fill();

ctx.fillStyle = "#ffd59e";
ctx.beginPath();
ctx.ellipse(0, 0, 14, 5, -0.3, 0, Math.PI * 2);
ctx.fill();
ctx.restore();

ctx.fillStyle = "#e1e7ec";
ctx.font = "12px sans-serif";
ctx.textAlign = "center";
ctx.fillText("Distant Galaxy", ptA.x, ptA.y + 32);

// Intermediate Cluster
ctx.save();
ctx.translate(clusterX, centerY);
const haloRadius = 18 + (sim.M_tril / 1000) * 22;
const clusterGlow = ctx.createRadialGradient(0, 0, 2, 0, 0, haloRadius);
clusterGlow.addColorStop(0, "rgba(168, 85, 247, 0.65)");
clusterGlow.addColorStop(0.6, "rgba(126, 34, 206, 0.2)");
clusterGlow.addColorStop(1, "rgba(0, 0, 0, 0)");
ctx.fillStyle = clusterGlow;
ctx.beginPath();
ctx.arc(0, 0, haloRadius, 0, Math.PI * 2);
ctx.fill();

```

```

    ctx.fillStyle = "#ffffff";
    ctx.beginPath();
    ctx.arc(0, 0, 4, 0, Math.PI * 2);
    ctx.arc(-8, -6, 2.5, 0, Math.PI * 2);
    ctx.arc(10, 5, 3, 0, Math.PI * 2);
    ctx.fill();
    ctx.restore();

    ctx.fillStyle = "#e1e7ec";
    ctx.font = "12px sans-serif";
    ctx.textAlign = "center";
    ctx.fillText("Intermediate Cluster", clusterX, centerY + haloRadius + 16);

    // Earth / Observer
    ctx.save();
    ctx.translate(earthX, centerY);
    const earthGrad = ctx.createRadialGradient(-2, -2, 1, 0, 0, 10);
    earthGrad.addColorStop(0, "#60a5fa");
    earthGrad.addColorStop(0.8, "#1d4ed8");
    earthGrad.addColorStop(1, "#1e3a8a");
    ctx.fillStyle = earthGrad;
    ctx.beginPath();
    ctx.arc(0, 0, 9, 0, Math.PI * 2);
    ctx.fill();
    ctx.restore();

    ctx.fillStyle = "#e1e7ec";
    ctx.font = "12px sans-serif";
    ctx.textAlign = "center";
    ctx.fillText("Earth", earthX, centerY + 25);
}

function animate() {
    draw();
    requestAnimationFrame(animate);
}

animate();
</script>
</body>
</html>

```

User prompt: Re-arrange the graphic so that it is drawn vertically, with the distant galaxy at the top of the view and Earth at the bottom. On the mass slider for the cluster, allow the mass to go down to zero - at that point, deactivate the cluster distance slider.

Response: <!DOCTYPE html>
 <html lang="en">
 <head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Gravitational Lensing: Vertical Field Simulation</title>
 <style>
 :root {
 --bg-color: #0b0e14;
 --panel-bg: #151922;
 --panel-border: #2a3142;
 --accent-color: #00f0ff;
 --accent-amber: #facc15;
 --accent-red: #ef4444;
 --text-main: #e1e7ec;
 --text-muted: #8c9bae;
 }

 * {
 box-sizing: border-box;
 margin: 0;
 padding: 0;
 }

 body {
 font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial, sans-serif;

```
background-color: var(--bg-color);
color: var(--text-main);
display: flex;
flex-direction: column;
min-height: 100vh;
overflow-x: hidden;
}

header {
background-color: var(--panel-bg);
border-bottom: 1px solid var(--panel-border);
padding: 0.85rem 1.5rem;
text-align: center;
}

header h1 {
font-size: 1.3rem;
font-weight: 600;
color: #fff;
}

header p {
font-size: 0.85rem;
color: var(--text-muted);
margin-top: 0.2rem;
}

.main-container {
display: flex;
flex: 1;
flex-direction: column;
gap: 1rem;
padding: 1rem;
max-width: 1400px;
margin: 0 auto;
width: 100%;
}

@media (min-width: 1024px) {
.main-container {
flex-direction: row;
}
}

.canvas-card {
flex: 1;
background-color: var(--panel-bg);
border: 1px solid var(--panel-border);
border-radius: 8px;
padding: 1rem;
display: flex;
flex-direction: column;
gap: 1rem;
box-shadow: 0 4px 20px rgba(0, 0, 0, 0.4);
}

.canvas-wrapper {
position: relative;
width: 100%;
height: 560px;
background-color: #04060a;
border-radius: 6px;
overflow: hidden;
border: 1px solid rgba(255, 255, 255, 0.05);
}

canvas {
width: 100%;
height: 100%;
display: block;
}

.calc-box {
background: rgba(10, 14, 22, 0.7);
```

```
border: 1px solid var(--panel-border);
border-radius: 6px;
padding: 1rem 1.25rem;
font-family: SFMono-Regular, Consolas, "Liberation Mono", Menlo, monospace;
font-size: 0.8rem;
color: #d1d5db;
}

.calc-box h3 {
  font-size: 0.85rem;
  text-transform: uppercase;
  letter-spacing: 0.05em;
  color: var(--accent-color);
  margin-bottom: 0.75rem;
  border-bottom: 1px solid rgba(255, 255, 255, 0.1);
  padding-bottom: 0.4rem;
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;
}

.calc-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(190px, 1fr));
  gap: 0.85rem 1.25rem;
}

.calc-item {
  display: flex;
  flex-direction: column;
  gap: 0.25rem;
}

.calc-item .label {
  color: var(--text-muted);
  font-size: 0.75rem;
}

.calc-item .value {
  color: #fff;
  font-weight: 600;
  font-size: 0.85rem;
}

.calc-item .highlight {
  color: var(--accent-amber);
}

.calc-footer {
  margin-top: 0.85rem;
  padding-top: 0.6rem;
  border-top: 1px dashed rgba(255, 255, 255, 0.15);
  display: flex;
  justify-content: space-between;
  align-items: center;
  font-size: 0.75rem;
}

.calc-footer .formula {
  color: #9ca3af;
}

.controls-panel {
  width: 100%;
  max-width: 380px;
  background-color: var(--panel-bg);
  border: 1px solid var(--panel-border);
  border-radius: 8px;
  padding: 1.25rem;
  display: flex;
  flex-direction: column;
  gap: 1.1rem;
  box-shadow: 0 4px 20px rgba(0, 0, 0, 0.4);
}

.control-group {
```

```
    display: flex;
    flex-direction: column;
    gap: 0.4rem;
    transition: opacity 0.2s ease;
  }

  .control-group.disabled-control {
    opacity: 0.35;
  }

  .control-group label {
    font-size: 0.83rem;
    font-weight: 500;
    color: var(--text-main);
    display: flex;
    justify-content: space-between;
  }

  .control-group label span {
    color: var(--accent-color);
    font-family: monospace;
  }

  input[type="range"] {
    -webkit-appearance: none;
    width: 100%;
    height: 6px;
    border-radius: 3px;
    background: #2a3142;
    outline: none;
  }

  input[type="range"]:disabled {
    cursor: not-allowed;
  }

  input[type="range"]::-webkit-slider-thumb {
    -webkit-appearance: none;
    width: 16px;
    height: 16px;
    border-radius: 50%;
    background: var(--accent-color);
    cursor: pointer;
    box-shadow: 0 0 8px var(--accent-color);
    transition: transform 0.1s ease;
  }

  input[type="range"]:disabled::-webkit-slider-thumb {
    background: #6b7280;
    box-shadow: none;
    cursor: not-allowed;
  }

  input[type="range"]::-webkit-slider-thumb:hover:not(:disabled) {
    transform: scale(1.2);
  }

  .toggle-row {
    display: flex;
    align-items: center;
    justify-content: space-between;
    font-size: 0.83rem;
    color: var(--text-main);
  }

  .switch {
    position: relative;
    display: inline-block;
    width: 40px;
    height: 20px;
  }

  .switch input {
    opacity: 0;
  }
```

```

    width: 0;
    height: 0;
  }

  .slider {
    position: absolute;
    cursor: pointer;
    top: 0; left: 0; right: 0; bottom: 0;
    background-color: #2a3142;
    transition: .2s;
    border-radius: 20px;
  }

  .slider:before {
    position: absolute;
    content: "";
    height: 14px;
    width: 14px;
    left: 3px;
    bottom: 3px;
    background-color: white;
    transition: .2s;
    border-radius: 50%;
  }

  input:checked + .slider {
    background-color: var(--accent-color);
  }

  input:checked + .slider:before {
    transform: translateX(20px);
  }

  .legend {
    border-top: 1px solid var(--panel-border);
    padding-top: 0.8rem;
    display: flex;
    flex-direction: column;
    gap: 0.4rem;
    font-size: 0.8rem;
  }

  .legend-item {
    display: flex;
    align-items: center;
    gap: 0.6rem;
    color: var(--text-muted);
  }

  .legend-color {
    width: 14px;
    height: 4px;
    border-radius: 2px;
  }
</style>
</head>
<body>

<header>
  <h1>Gravitational Deflection of Light Fan</h1>
  <p>Vertical Propagation View (Distant Galaxy at Top, Earth at Bottom)</p>
</header>

<div class="main-container">
  <div class="canvas-card">
    <div class="canvas-wrapper">
      <canvas id="simCanvas"></canvas>
    </div>

    <!-- Numerical Output Box below Graphic -->
    <div class="calc-box">
      <h3>Numerical Results</h3>
      <div class="calc-grid">
        <div class="calc-item">

```

```

    <span class="label">Galaxy Distance ( $d_{\text{G}}$ ):</span>
    <span class="value" id="boxDistG">8.0 Gly</span>
  </div>
  <div class="calc-item">
    <span class="label">Galaxy Offset ( $x_{\text{G}}$ ):</span>
    <span class="value" id="boxOffsetX">0 kly</span>
  </div>
  <div class="calc-item">
    <span class="label">Cluster Distance ( $d_{\text{C}}$ ):</span>
    <span class="value" id="boxDistC">4.0 Gly</span>
  </div>
  <div class="calc-item">
    <span class="label"> $d_{\text{GC}} = d_{\text{G}} - d_{\text{C}}$ :</span>
    <span class="value" id="boxDistGC">4.0 Gly</span>
  </div>
  <div class="calc-item">
    <span class="label">Cluster Mass (M):</span>
    <span class="value" id="boxMass"> $200 \times 10^{12} M_{\odot}$ </span>
  </div>
  <div class="calc-item">
    <span class="label">Ray Mode:</span>
    <span class="value" id="boxMode">Single Ray</span>
  </div>
  <div class="calc-item">
    <span class="label">Cluster Impact ( $b_{\text{C}}$ ):</span>
    <span class="value highlight" id="boxImpactRange">290.9 kly</span>
  </div>
  <div class="calc-item">
    <span class="label">Earth Impact ( $b_{\text{E}}$ ):</span>
    <span class="value highlight" id="boxEarthImpactRange">581.8 kly</span>
  </div>
  <div class="calc-item">
    <span class="label">Deflection Angle ( $\alpha$ ):</span>
    <span class="value highlight" id="boxAlphaRange">44.3" </span>
  </div>
</div>
<div class="calc-footer">
  <span class="label">Theoretical Formula:</span>
  <span class="formula"> $\alpha = 4GM / (c^2 |b_{\text{C}}|) \quad \& \quad b_{\text{E}} = b_{\text{C}} + d_{\text{C}}$ </span>
</div>
</div>

<div class="controls-panel">
  <div class="control-group">
    <label for="sliderDistG">
      1. Distance to Galaxy ( $d_{\text{G}}$ ):
      <span id="valDistG">8.0 Gly</span>
    </label>
    <input type="range" id="sliderDistG" min="2.0" max="13.0" step="0.1" value="8.0">
  </div>

  <div class="control-group">
    <label for="sliderOffsetX">
      2. Galaxy Horizontal Offset ( $x_{\text{G}}$ ):
      <span id="valOffsetX">0 kly</span>
    </label>
    <input type="range" id="sliderOffsetX" min="-500" max="500" step="10" value="0">
  </div>

  <div class="control-group" id="grpDistC">
    <label for="sliderDistC">
      3. Distance to Cluster ( $d_{\text{C}}$ ):
      <span id="valDistC">4.0 Gly</span>
    </label>
    <input type="range" id="sliderDistC" min="1.0" max="12.0" step="0.1" value="4.0">
  </div>

  <div class="control-group">
    <label for="sliderMass">
      4. Cluster Mass (M):
      <span id="valMass"> $200 \times 10^{12} M_{\odot}$ </span>
    </label>
    <input type="range" id="sliderMass" min="0" max="1000" step="5" value="200">
  </div>
</div>

```



```

</div>

<div class="control-group">
  <label for="sliderNumRays">
    5. Number of Light Rays:
    <span id="valNumRays">1</span>
  </label>
  <input type="range" id="sliderNumRays" min="1" max="10" step="1" value="1">
</div>

<div class="control-group" id="grpFanSpread">
  <label for="sliderFanSpread">
    6. Ray Fan Spread:
    <span id="valFanSpread">20</span>
  </label>
  <input type="range" id="sliderFanSpread" min="10" max="40" step="1" value="20">
</div>

<div class="control-group" id="grpTheta">
  <label for="sliderTheta">
    7. Initial Ray Direction (1 ray):
    <span id="valTheta">+15.0</span>
  </label>
  <input type="range" id="sliderTheta" min="-40.0" max="40.0" step="0.5" value="15.0">
</div>

<div class="toggle-row">
  <span>Animate Photons</span>
  <label class="switch">
    <input type="checkbox" id="togglePhoton" checked>
    <span class="slider"></span>
  </label>
</div>

<div class="toggle-row">
  <span>Show Virtual Tangent Lines</span>
  <label class="switch">
    <input type="checkbox" id="toggleTangent" checked>
    <span class="slider"></span>
  </label>
</div>

<div class="legend">
  <div class="legend-item">
    <div class="legend-color" style="background: #facc15;"></div>
    <span>Actual deflected light rays</span>
  </div>
  <div class="legend-item">
    <div class="legend-color" style="border: 1px dashed #ef4444; background: transparent;"></div>
    <span>Apparent straight-line paths (virtual images)</span>
  </div>
  <div class="legend-item">
    <div class="legend-color" style="background: #00f0ff;"></div>
    <span>Cluster impact parameter (b<sub>C</sub>) & Earth impact (b<sub>E</sub>)</span>
  </div>
</div>
</div>
</div>

<script>
const canvas = document.getElementById('simCanvas');
const ctx = canvas.getContext('2d');

// Canvas Fixed Total Height Scale (15 Billion Light-Years)
const CANVAS_TOTAL_GLY = 15.0;

// Controls
const sliderDistG = document.getElementById('sliderDistG');
const sliderOffsetX = document.getElementById('sliderOffsetX');
const sliderDistC = document.getElementById('sliderDistC');
const sliderMass = document.getElementById('sliderMass');
const sliderNumRays = document.getElementById('sliderNumRays');
const sliderFanSpread = document.getElementById('sliderFanSpread');
const sliderTheta = document.getElementById('sliderTheta');

```

```

const grpDistC = document.getElementById('grpDistC');
const grpFanSpread = document.getElementById('grpFanSpread');
const grpTheta = document.getElementById('grpTheta');

const valDistG = document.getElementById('valDistG');
const valOffsetX = document.getElementById('valOffsetX');
const valDistC = document.getElementById('valDistC');
const valMass = document.getElementById('valMass');
const valNumRays = document.getElementById('valNumRays');
const valFanSpread = document.getElementById('valFanSpread');
const valTheta = document.getElementById('valTheta');

const togglePhoton = document.getElementById('togglePhoton');
const toggleTangent = document.getElementById('toggleTangent');

// Calculation Box Elements
const boxDistG = document.getElementById('boxDistG');
const boxOffsetX = document.getElementById('boxOffsetX');
const boxDistC = document.getElementById('boxDistC');
const boxDistGC = document.getElementById('boxDistGC');
const boxMass = document.getElementById('boxMass');
const boxMode = document.getElementById('boxMode');
const boxImpactRange = document.getElementById('boxImpactRange');
const boxEarthImpactRange = document.getElementById('boxEarthImpactRange');
const boxAlphaRange = document.getElementById('boxAlphaRange');

let width = 0;
let height = 0;
let photonAnimProgress = 0;

// Background Stars
const stars = [];
for (let i = 0; i < 220; i++) {
  stars.push({
    x: Math.random(),
    y: Math.random(),
    size: Math.random() * 1.5 + 0.3,
    alpha: Math.random() * 0.7 + 0.3
  });
}

function resizeCanvas() {
  const rect = canvas.parentElement.getBoundingClientRect();
  const dpr = window.devicePixelRatio || 1;
  width = rect.width || 800;
  height = rect.height || 560;

  canvas.width = width * dpr;
  canvas.height = height * dpr;
  ctx.scale(dpr, dpr);
}

window.addEventListener('resize', resizeCanvas);
resizeCanvas();

// Distance constraints: d_G >= d_C + 1.0 Gly
sliderDistG.addEventListener('input', () => {
  let dG = parseFloat(sliderDistG.value);
  let dC = parseFloat(sliderDistC.value);
  if (dG < dC + 1.0) {
    dC = Math.max(1.0, dG - 1.0);
    sliderDistC.value = dC.toFixed(1);
  }
  updateControlStates();
});

sliderDistC.addEventListener('input', () => {
  let dG = parseFloat(sliderDistG.value);
  let dC = parseFloat(sliderDistC.value);
  if (dC > dG - 1.0) {
    dG = Math.min(13.0, dC + 1.0);
    sliderDistG.value = dG.toFixed(1);
  }
});

```

```

    updateControlStates();
  });

sliderMass.addEventListener('input', updateControlStates);
sliderNumRays.addEventListener('input', updateControlStates);

function updateControlStates() {
  const numRays = parseInt(sliderNumRays.value);
  const mass = parseFloat(sliderMass.value);

  // Mass = 0 -> Deactivate Cluster Distance Slider
  if (mass === 0) {
    sliderDistC.disabled = true;
    grpDistC.classList.add('disabled-control');
  } else {
    sliderDistC.disabled = false;
    grpDistC.classList.remove('disabled-control');
  }

  if (numRays > 1) {
    // Multi-ray mode: Angle/Direction slider inactive, Fan Spread active
    sliderTheta.disabled = true;
    grpTheta.classList.add('disabled-control');

    sliderFanSpread.disabled = false;
    grpFanSpread.classList.remove('disabled-control');
  } else {
    // Single-ray mode: Angle/Direction slider active, Fan Spread inactive
    sliderTheta.disabled = false;
    grpTheta.classList.remove('disabled-control');

    sliderFanSpread.disabled = true;
    grpFanSpread.classList.add('disabled-control');
  }
}

updateControlStates();

function calculateRays() {
  const d_G_Gly = parseFloat(sliderDistG.value);
  const x_G_kly = parseFloat(sliderOffsetX.value);
  const d_C_Gly = parseFloat(sliderDistC.value);
  const d_GC_Gly = d_G_Gly - d_C_Gly;
  const M_tril = parseFloat(sliderMass.value);
  const numRays = parseInt(sliderNumRays.value);
  const fanSpreadArcsec = parseFloat(sliderFanSpread.value);
  const thetaInitArcsec = parseFloat(sliderTheta.value);

  const ARCSEC_TO_RAD = Math.PI / (180.0 * 3600.0);
  const d_GC_ly = d_GC_Gly * 1e9;
  const d_C_ly = d_C_Gly * 1e9;
  const x_G_ly = x_G_kly * 1e3;

  const scale_4GM_c2_ly = 0.6243363 * M_tril;

  const rays = [];

  for (let i = 0; i < numRays; i++) {
    let initialAngle = 0;

    if (numRays === 1) {
      initialAngle = thetaInitArcsec;
    } else {
      const frac = (i / (numRays - 1)) - 0.5;
      initialAngle = frac * fanSpreadArcsec;
    }

    const theta_init_rad = initialAngle * ARCSEC_TO_RAD;

    // Horizontal impact parameter b_C at cluster plane
    // Light propagates vertically downwards from Galaxy to Cluster
    const b_C_ly = x_G_ly + d_GC_ly * Math.tan(theta_init_rad);
    const abs_b_C_ly = Math.abs(b_C_ly);

```

```

let alpha_arcsec = 0;
let alpha_rad = 0;
if (M_tril > 0 && abs_b_C_ly > 1e-3) {
  alpha_rad = scale_4GM_c2_ly / abs_b_C_ly;
  alpha_arcsec = alpha_rad / ARCSEC_TO_RAD;
}

// Deflection direction: Gravitational attraction towards central optical axis (x = 0)
let theta_after_rad = theta_init_rad;
if (M_tril > 0) {
  if (b_C_ly > 0) {
    theta_after_rad = theta_init_rad - alpha_rad;
  } else if (b_C_ly < 0) {
    theta_after_rad = theta_init_rad + alpha_rad;
  }
}
const theta_after_arcsec = theta_after_rad / ARCSEC_TO_RAD;

// Horizontal impact parameter b_E at Earth plane (d = 0)
const b_E_ly = b_C_ly + d_C_ly * Math.tan(theta_after_rad);
const abs_b_E_ly = Math.abs(b_E_ly);

rays.push({
  index: i,
  theta_init_arcsec: initialAngle,
  b_C_ly: b_C_ly,
  abs_b_C_ly: abs_b_C_ly,
  b_E_ly: b_E_ly,
  abs_b_E_ly: abs_b_E_ly,
  alpha_arcsec: alpha_arcsec,
  theta_after_arcsec: theta_after_arcsec
});
}

return {
  d_G_Gly,
  x_G_kly,
  d_C_Gly,
  d_GC_Gly,
  M_tril,
  numRays,
  fanSpreadArcsec,
  thetaInitArcsec,
  rays
};
}

function draw() {
  ctx.clearRect(0, 0, width, height);

  const sim = calculateRays();

  // Label Updates
  valDistG.textContent = `${sim.d_G_Gly.toFixed(1)} Gly`;
  valOffsetX.textContent = `${sim.x_G_kly >= 0 ? '+' : ''}${sim.x_G_kly} kly`;
  valDistC.textContent = sim.M_tril === 0 ? "N/A (M=0)" : `${sim.d_C_Gly.toFixed(1)} Gly`;
  valMass.textContent = `${sim.M_tril} × 1012 M⊙`;
  valNumRays.textContent = `${sim.numRays}`;
  valFanSpread.textContent = `${sim.fanSpreadArcsec}`;
  valTheta.textContent = `${sim.thetaInitArcsec >= 0 ? '+' : ''}${sim.thetaInitArcsec.toFixed(1)}°`;

  // Update Numerical Box
  boxDistG.textContent = `${sim.d_G_Gly.toFixed(1)} Gly`;
  boxOffsetX.textContent = `${sim.x_G_kly >= 0 ? '+' : ''}${sim.x_G_kly} kly`;
  boxDistC.textContent = sim.M_tril === 0 ? "N/A" : `${sim.d_C_Gly.toFixed(1)} Gly`;
  boxDistGC.textContent = sim.M_tril === 0 ? "N/A" : `${sim.d_GC_Gly.toFixed(1)} Gly`;
  boxMass.textContent = `${sim.M_tril} × 1012 M⊙`;

  if (sim.numRays === 1) {
    const r = sim.rays[0];
    boxMode.textContent = `Single Ray`;
    boxImpactRange.textContent = `${(r.abs_b_C_ly / 1000.0).toFixed(2)} kly (${Math.round(r.abs_b_C_ly).toLocaleString()} Mpc)`;
    boxEarthImpactRange.textContent = `${(r.abs_b_E_ly / 1000.0).toFixed(2)} kly (${Math.round(r.abs_b_E_ly).toLocaleString()} Mpc)`;
    boxAlphaRange.textContent = `${r.alpha_arcsec.toFixed(2)}''`;
  }
}

```

```

} else {
  boxMode.textContent = `Fan (${sim.numRays} rays)`;

  const bCValues = sim.rays.map(r => r.abs_b_C_ly / 1000.0);
  const bEValues = sim.rays.map(r => r.abs_b_E_ly / 1000.0);
  const alphaValues = sim.rays.map(r => r.alpha_arcsec).filter(a => isFinite(a));

  const minBC = Math.min(...bCValues);
  const maxBC = Math.max(...bCValues);
  const minBE = Math.min(...bEValues);
  const maxBE = Math.max(...bEValues);
  const maxAlpha = Math.max(...alphaValues);

  boxImpactRange.textContent = `${minBC.toFixed(1)} to ${maxBC.toFixed(1)} kly`;
  boxEarthImpactRange.textContent = `${minBE.toFixed(1)} to ${maxBE.toFixed(1)} kly`;
  boxAlphaRange.textContent = `${maxAlpha.toFixed(2)}" max`;
}

// Visual Layout Dimensions (Vertical Field)
const marginY = 50;
const centerX = width / 2;

const earthY = height - marginY; // Earth at Bottom
const drawableHeight = height - 2 * marginY;
const pxPerGly = drawableHeight / CANVAS_TOTAL_GLY;

const galaxyY = earthY - (sim.d_G_Gly * pxPerGly); // Distant Galaxy at Top
const clusterY = earthY - (sim.d_C_Gly * pxPerGly); // Intermediate Cluster

// Uniform Horizontal Physical Scale across Galaxy, Cluster, and Earth:
// 500 kly mapped to 120 pixels visually on screen
const pxPerKly = 120.0 / 500.0;
const galaxyX = centerX + (sim.x_G_kly * pxPerKly);

const ptA = { x: galaxyX, y: galaxyY };

// Starfield Background
ctx.fillStyle = "#ffffff";
stars.forEach(s => {
  ctx.globalAlpha = s.alpha;
  ctx.fillRect(s.x * width, s.y * height, s.size, s.size);
});
ctx.globalAlpha = 1.0;

// Central Optical Axis (0 Gly to 15 Gly full vertical height)
ctx.strokeStyle = "rgba(255, 255, 255, 0.15)";
ctx.lineWidth = 1;
ctx.setLineDash([6, 6]);
ctx.beginPath();
ctx.moveTo(centerX, earthY + 25);
ctx.lineTo(centerX, earthY - CANVAS_TOTAL_GLY * pxPerGly - 25);
ctx.stroke();
ctx.setLineDash([]);

// Cluster Plane Line
if (sim.M_tril > 0) {
  ctx.strokeStyle = "rgba(0, 240, 255, 0.25)";
  ctx.beginPath();
  ctx.moveTo(30, clusterY);
  ctx.lineTo(width - 30, clusterY);
  ctx.stroke();
}

// Earth Plane Line (for impact parameter b_E)
ctx.strokeStyle = "rgba(0, 240, 255, 0.15)";
ctx.beginPath();
ctx.moveTo(30, earthY);
ctx.lineTo(width - 30, earthY);
ctx.stroke();

// Render Light Rays & Virtual Tangents
sim.rays.forEach((ray, idx) => {
  const clusterX = centerX + (ray.b_C_ly / 1000.0) * pxPerKly;
  const earthX = centerX + (ray.b_E_ly / 1000.0) * pxPerKly;

```

```

const ptB = { x: clusterX, y: clusterY };
const ptC = { x: earthX, y: earthY };

// Virtual Tangent Line (Extrapolated straight back from Earth direction up to Galaxy plane)
if (toggleTangent.checked && Math.abs(ray.alpha_arcsec) > 0.1 && sim.M_tril > 0) {
  ctx.strokeStyle = "rgba(239, 68, 68, 0.5)";
  ctx.lineWidth = 1.2;
  ctx.setLineDash([3, 3]);

  // Slope dx/dy of ray arriving at Earth
  const slopeX = (ptC.x - ptB.x) / (ptC.y - ptB.y);
  const virtualX_at_galaxy = ptB.x + slopeX * (galaxyY - clusterY);

  ctx.beginPath();
  ctx.moveTo(ptC.x, ptC.y);
  ctx.lineTo(virtualX_at_galaxy, galaxyY);
  ctx.stroke();
  ctx.setLineDash([]);

  ctx.fillStyle = "rgba(239, 68, 68, 0.8)";
  ctx.beginPath();
  ctx.arc(virtualX_at_galaxy, galaxyY, 3.5, 0, Math.PI * 2);
  ctx.fill();
}

// Actual Light Path (Galaxy -> Cluster -> Earth Plane)
ctx.strokeStyle = "#facc15";
ctx.lineWidth = sim.numRays === 1 ? 2.5 : 1.8;
ctx.shadowColor = "#facc15";
ctx.shadowBlur = sim.numRays === 1 ? 8 : 4;

ctx.beginPath();
ctx.moveTo(ptA.x, ptA.y);
if (sim.M_tril > 0) {
  ctx.lineTo(ptB.x, ptB.y);
}
ctx.lineTo(ptC.x, ptC.y);
ctx.stroke();
ctx.shadowBlur = 0;

// Visual Indicators for Impact Parameters in Single-Ray Mode
if (sim.numRays === 1) {
  // Cluster Impact Parameter (b_C)
  if (sim.M_tril > 0 && Math.abs(ptB.x - centerX) > 3) {
    ctx.strokeStyle = "#00f0ff";
    ctx.lineWidth = 1.5;
    ctx.beginPath();
    ctx.moveTo(centerX, clusterY);
    ctx.lineTo(ptB.x, clusterY);
    ctx.stroke();

    ctx.fillStyle = "#00f0ff";
    ctx.beginPath();
    ctx.arc(ptB.x, clusterY, 3, 0, Math.PI * 2);
    ctx.arc(centerX, clusterY, 3, 0, Math.PI * 2);
    ctx.fill();

    ctx.font = "11px monospace";
    ctx.textAlign = "center";
    ctx.fillText(`b_C = ${ (ray.abs_b_C_ly / 1000).toFixed(1) } kly`, (centerX + ptB.x) / 2, clusterY -
  }

  // Earth Impact Parameter (b_E)
  if (Math.abs(ptC.x - centerX) > 3) {
    ctx.strokeStyle = "#00f0ff";
    ctx.lineWidth = 1.5;
    ctx.beginPath();
    ctx.moveTo(centerX, earthY);
    ctx.lineTo(ptC.x, earthY);
    ctx.stroke();

    ctx.fillStyle = "#00f0ff";
    ctx.beginPath();

```

```

    ctx.arc(ptC.x, earthY, 3, 0, Math.PI * 2);
    ctx.arc(centerX, earthY, 3, 0, Math.PI * 2);
    ctx.fill();

    ctx.font = "11px monospace";
    ctx.textAlign = "center";
    ctx.fillText(`b_E = ${ (ray.abs_b_E_ly / 1000).toFixed(1) } kly`, (centerX + ptC.x) / 2, earthY + 18
  }
}

// Animated Photons
if (togglePhoton.checked) {
  const rayOffset = (idx / sim.numRays);
  const progress = (photonAnimProgress + rayOffset) % 1.0;
  let px = 0, py = 0;

  if (sim.M_tril === 0) {
    px = ptA.x + progress * (ptC.x - ptA.x);
    py = ptA.y + progress * (ptC.y - ptA.y);
  } else {
    if (progress <= 0.5) {
      const t = progress / 0.5;
      px = ptA.x + t * (ptB.x - ptA.x);
      py = ptA.y + t * (ptB.y - ptA.y);
    } else {
      const t = (progress - 0.5) / 0.5;
      px = ptB.x + t * (ptC.x - ptB.x);
      py = ptB.y + t * (ptC.y - ptB.y);
    }
  }

  ctx.fillStyle = "#ffffff";
  ctx.shadowColor = "#00f0ff";
  ctx.shadowBlur = 8;
  ctx.beginPath();
  ctx.arc(px, py, 3.5, 0, Math.PI * 2);
  ctx.fill();
  ctx.shadowBlur = 0;
}
});

if (togglePhoton.checked) {
  photonAnimProgress = (photonAnimProgress + 0.006) % 1.0;
}

// Draw Celestial Objects

// 1. Distant Galaxy (Top)
ctx.save();
ctx.translate(ptA.x, ptA.y);
const galGlow = ctx.createRadialGradient(0, 0, 2, 0, 0, 22);
galGlow.addColorStop(0, "rgba(255, 200, 100, 0.9)");
galGlow.addColorStop(0.5, "rgba(238, 100, 30, 0.3)");
galGlow.addColorStop(1, "rgba(0, 0, 0, 0)");
ctx.fillStyle = galGlow;
ctx.beginPath();
ctx.arc(0, 0, 22, 0, Math.PI * 2);
ctx.fill();

ctx.fillStyle = "#ffd59e";
ctx.beginPath();
ctx.ellipse(0, 0, 14, 5, -0.3, 0, Math.PI * 2);
ctx.fill();
ctx.restore();

ctx.fillStyle = "#e1e7ec";
ctx.font = "12px sans-serif";
ctx.textAlign = "center";
ctx.fillText("Distant Galaxy", ptA.x, ptA.y - 18);

// 2. Intermediate Cluster (Middle)
if (sim.M_tril > 0) {
  ctx.save();
  ctx.translate(centerX, clusterY);

```

```

const haloRadius = 18 + (sim.M_tril / 1000) * 22;
const clusterGlow = ctx.createRadialGradient(0, 0, 2, 0, 0, haloRadius);
clusterGlow.addColorStop(0, "rgba(168, 85, 247, 0.65)");
clusterGlow.addColorStop(0.6, "rgba(126, 34, 206, 0.2)");
clusterGlow.addColorStop(1, "rgba(0, 0, 0, 0)");
ctx.fillStyle = clusterGlow;
ctx.beginPath();
ctx.arc(0, 0, haloRadius, 0, Math.PI * 2);
ctx.fill();

ctx.fillStyle = "ffffff";
ctx.beginPath();
ctx.arc(0, 0, 4, 0, Math.PI * 2);
ctx.arc(-8, -6, 2.5, 0, Math.PI * 2);
ctx.arc(10, 5, 3, 0, Math.PI * 2);
ctx.fill();
ctx.restore();

ctx.fillStyle = "#e1e7ec";
ctx.font = "12px sans-serif";
ctx.textAlign = "left";
ctx.fillText("Intermediate Cluster", centerX + haloRadius + 8, clusterY + 4);
}

// 3. Earth / Observer (Bottom)
ctx.save();
ctx.translate(centerX, earthY);
const earthGrad = ctx.createRadialGradient(-2, -2, 1, 0, 0, 10);
earthGrad.addColorStop(0, "#60a5fa");
earthGrad.addColorStop(0.8, "#1d4ed8");
earthGrad.addColorStop(1, "#1e3a8a");
ctx.fillStyle = earthGrad;
ctx.beginPath();
ctx.arc(0, 0, 9, 0, Math.PI * 2);
ctx.fill();
ctx.restore();

ctx.fillStyle = "#e1e7ec";
ctx.font = "12px sans-serif";
ctx.textAlign = "center";
ctx.fillText("Earth", centerX, earthY + 28);
}

function animate() {
  draw();
  requestAnimationFrame(animate);
}

animate();
</script>
</body>
</html>

```

User prompt: a few graphics tweaks, and we'll be good enough for now. Make the graphic window more of a vertical portrait profile. Let's try 4:3. Continue to allow it to narrow if the screen width is reduced. add a note to the bottom of the graphic window "angles are exaggerated" change the size of the field to 13.5 billion light-years, instead of 15

```

Response: <!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Gravitational Lensing: Vertical Field Simulation</title>
  <style>
    :root {
      --bg-color: #0b0e14;
      --panel-bg: #151922;
      --panel-border: #2a3142;
      --accent-color: #00f0ff;
      --accent-amber: #facc15;
      --accent-red: #ef4444;
      --text-main: #e1e7ec;

```



```
--text-muted: #8c9bae;
}

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial, sans-serif;
  background-color: var(--bg-color);
  color: var(--text-main);
  display: flex;
  flex-direction: column;
  min-height: 100vh;
  overflow-x: hidden;
}

header {
  background-color: var(--panel-bg);
  border-bottom: 1px solid var(--panel-border);
  padding: 0.85rem 1.5rem;
  text-align: center;
}

header h1 {
  font-size: 1.3rem;
  font-weight: 600;
  color: #fff;
}

header p {
  font-size: 0.85rem;
  color: var(--text-muted);
  margin-top: 0.2rem;
}

.main-container {
  display: flex;
  flex: 1;
  flex-direction: column;
  gap: 1rem;
  padding: 1rem;
  max-width: 1200px;
  margin: 0 auto;
  width: 100%;
  align-items: center;
}

@media (min-width: 1024px) {
  .main-container {
    flex-direction: row;
    align-items: flex-start;
  }
}

.canvas-card {
  flex: 1;
  width: 100%;
  background-color: var(--panel-bg);
  border: 1px solid var(--panel-border);
  border-radius: 8px;
  padding: 1rem;
  display: flex;
  flex-direction: column;
  gap: 1rem;
  box-shadow: 0 4px 20px rgba(0, 0, 0, 0.4);
}

.canvas-wrapper {
  position: relative;
  width: 100%;
  max-width: 520px;
}
```

```
    aspect-ratio: 3 / 4; /* Vertical portrait profile (4:3 height to width) */
    margin: 0 auto;
    background-color: #04060a;
    border-radius: 6px;
    overflow: hidden;
    border: 1px solid rgba(255, 255, 255, 0.05);
}

canvas {
    width: 100%;
    height: 100%;
    display: block;
}

.calc-box {
    background: rgba(10, 14, 22, 0.7);
    border: 1px solid var(--panel-border);
    border-radius: 6px;
    padding: 1rem 1.25rem;
    font-family: SFMono-Regular, Consolas, "Liberation Mono", Menlo, monospace;
    font-size: 0.8rem;
    color: #d1d5db;
}

.calc-box h3 {
    font-size: 0.85rem;
    text-transform: uppercase;
    letter-spacing: 0.05em;
    color: var(--accent-color);
    margin-bottom: 0.75rem;
    border-bottom: 1px solid rgba(255, 255, 255, 0.1);
    padding-bottom: 0.4rem;
    font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;
}

.calc-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
    gap: 0.85rem 1.25rem;
}

.calc-item {
    display: flex;
    flex-direction: column;
    gap: 0.25rem;
}

.calc-item .label {
    color: var(--text-muted);
    font-size: 0.75rem;
}

.calc-item .value {
    color: #fff;
    font-weight: 600;
    font-size: 0.85rem;
}

.calc-item .highlight {
    color: var(--accent-amber);
}

.calc-footer {
    margin-top: 0.85rem;
    padding-top: 0.6rem;
    border-top: 1px dashed rgba(255, 255, 255, 0.15);
    display: flex;
    justify-content: space-between;
    align-items: center;
    font-size: 0.75rem;
}

.calc-footer .formula {
    color: #9ca3af;
}
```

```
}

.controls-panel {
  width: 100%;
  max-width: 380px;
  background-color: var(--panel-bg);
  border: 1px solid var(--panel-border);
  border-radius: 8px;
  padding: 1.25rem;
  display: flex;
  flex-direction: column;
  gap: 1.1rem;
  box-shadow: 0 4px 20px rgba(0, 0, 0, 0.4);
}

.control-group {
  display: flex;
  flex-direction: column;
  gap: 0.4rem;
  transition: opacity 0.2s ease;
}

.control-group.disabled-control {
  opacity: 0.35;
}

.control-group label {
  font-size: 0.83rem;
  font-weight: 500;
  color: var(--text-main);
  display: flex;
  justify-content: space-between;
}

.control-group label span {
  color: var(--accent-color);
  font-family: monospace;
}

input[type="range"] {
  -webkit-appearance: none;
  width: 100%;
  height: 6px;
  border-radius: 3px;
  background: #2a3142;
  outline: none;
}

input[type="range"]:disabled {
  cursor: not-allowed;
}

input[type="range"]::-webkit-slider-thumb {
  -webkit-appearance: none;
  width: 16px;
  height: 16px;
  border-radius: 50%;
  background: var(--accent-color);
  cursor: pointer;
  box-shadow: 0 0 8px var(--accent-color);
  transition: transform 0.1s ease;
}

input[type="range"]:disabled::-webkit-slider-thumb {
  background: #6b7280;
  box-shadow: none;
  cursor: not-allowed;
}

input[type="range"]::-webkit-slider-thumb:hover:not(:disabled) {
  transform: scale(1.2);
}

.toggle-row {
```

```

    display: flex;
    align-items: center;
    justify-content: space-between;
    font-size: 0.83rem;
    color: var(--text-main);
  }

  .switch {
    position: relative;
    display: inline-block;
    width: 40px;
    height: 20px;
  }

  .switch input {
    opacity: 0;
    width: 0;
    height: 0;
  }

  .slider {
    position: absolute;
    cursor: pointer;
    top: 0; left: 0; right: 0; bottom: 0;
    background-color: #2a3142;
    transition: .2s;
    border-radius: 20px;
  }

  .slider:before {
    position: absolute;
    content: "";
    height: 14px;
    width: 14px;
    left: 3px;
    bottom: 3px;
    background-color: white;
    transition: .2s;
    border-radius: 50%;
  }

  input:checked + .slider {
    background-color: var(--accent-color);
  }

  input:checked + .slider:before {
    transform: translateX(20px);
  }

  .legend {
    border-top: 1px solid var(--panel-border);
    padding-top: 0.8rem;
    display: flex;
    flex-direction: column;
    gap: 0.4rem;
    font-size: 0.8rem;
  }

  .legend-item {
    display: flex;
    align-items: center;
    gap: 0.6rem;
    color: var(--text-muted);
  }

  .legend-color {
    width: 14px;
    height: 4px;
    border-radius: 2px;
  }
</style>
</head>
<body>

```

125/271

```

</div>

<div class="control-group" id="grpDistC">
  <label for="sliderDistC">
    3. Distance to Cluster (d<sub>C</sub>):
    <span id="valDistC">4.0 Gly</span>
  </label>
  <input type="range" id="sliderDistC" min="1.0" max="12.0" step="0.1" value="4.0">
</div>

<div class="control-group">
  <label for="sliderMass">
    4. Cluster Mass (M):
    <span id="valMass">200 × 1012 M⊙</span>
  </label>
  <input type="range" id="sliderMass" min="0" max="1000" step="5" value="200">
</div>

<div class="control-group">
  <label for="sliderNumRays">
    5. Number of Light Rays:
    <span id="valNumRays">1</span>
  </label>
  <input type="range" id="sliderNumRays" min="1" max="10" step="1" value="1">
</div>

<div class="control-group" id="grpFanSpread">
  <label for="sliderFanSpread">
    6. Ray Fan Spread:
    <span id="valFanSpread">20°</span>
  </label>
  <input type="range" id="sliderFanSpread" min="10" max="40" step="1" value="20">
</div>

<div class="control-group" id="grpTheta">
  <label for="sliderTheta">
    7. Initial Ray Direction (1 ray):
    <span id="valTheta">+15.0°</span>
  </label>
  <input type="range" id="sliderTheta" min="-40.0" max="40.0" step="0.5" value="15.0">
</div>

<div class="toggle-row">
  <span>Animate Photons</span>
  <label class="switch">
    <input type="checkbox" id="togglePhoton" checked>
    <span class="slider"></span>
  </label>
</div>

<div class="toggle-row">
  <span>Show Virtual Tangent Lines</span>
  <label class="switch">
    <input type="checkbox" id="toggleTangent" checked>
    <span class="slider"></span>
  </label>
</div>

<div class="legend">
  <div class="legend-item">
    <div class="legend-color" style="background: #facc15;"></div>
    <span>Actual deflected light rays</span>
  </div>
  <div class="legend-item">
    <div class="legend-color" style="border: 1px dashed #ef4444; background: transparent;"></div>
    <span>Apparent straight-line paths (virtual images)</span>
  </div>
  <div class="legend-item">
    <div class="legend-color" style="background: #00f0ff;"></div>
    <span>Cluster impact parameter (b<sub>C</sub>) & Earth impact (b<sub>E</sub>)</span>
  </div>
</div>
</div>
</div>

```

```

<script>
const canvas = document.getElementById('simCanvas');
const ctx = canvas.getContext('2d');

// Canvas Fixed Total Height Scale (13.5 Billion Light-Years)
const CANVAS_TOTAL_GLY = 13.5;

// Controls
const sliderDistG = document.getElementById('sliderDistG');
const sliderOffsetX = document.getElementById('sliderOffsetX');
const sliderDistC = document.getElementById('sliderDistC');
const sliderMass = document.getElementById('sliderMass');
const sliderNumRays = document.getElementById('sliderNumRays');
const sliderFanSpread = document.getElementById('sliderFanSpread');
const sliderTheta = document.getElementById('sliderTheta');

const grpDistC = document.getElementById('grpDistC');
const grpFanSpread = document.getElementById('grpFanSpread');
const grpTheta = document.getElementById('grpTheta');

const valDistG = document.getElementById('valDistG');
const valOffsetX = document.getElementById('valOffsetX');
const valDistC = document.getElementById('valDistC');
const valMass = document.getElementById('valMass');
const valNumRays = document.getElementById('valNumRays');
const valFanSpread = document.getElementById('valFanSpread');
const valTheta = document.getElementById('valTheta');

const togglePhoton = document.getElementById('togglePhoton');
const toggleTangent = document.getElementById('toggleTangent');

// Calculation Box Elements
const boxDistG = document.getElementById('boxDistG');
const boxOffsetX = document.getElementById('boxOffsetX');
const boxDistC = document.getElementById('boxDistC');
const boxDistGC = document.getElementById('boxDistGC');
const boxMass = document.getElementById('boxMass');
const boxMode = document.getElementById('boxMode');
const boxImpactRange = document.getElementById('boxImpactRange');
const boxEarthImpactRange = document.getElementById('boxEarthImpactRange');
const boxAlphaRange = document.getElementById('boxAlphaRange');

let width = 0;
let height = 0;
let photonAnimProgress = 0;

// Background Stars
const stars = [];
for (let i = 0; i < 220; i++) {
  stars.push({
    x: Math.random(),
    y: Math.random(),
    size: Math.random() * 1.5 + 0.3,
    alpha: Math.random() * 0.7 + 0.3
  });
}

function resizeCanvas() {
  const rect = canvas.parentElement.getBoundingClientRect();
  const dpr = window.devicePixelRatio || 1;
  width = rect.width;
  height = rect.height;

  canvas.width = width * dpr;
  canvas.height = height * dpr;
  ctx.scale(dpr, dpr);
}

window.addEventListener('resize', resizeCanvas);
resizeCanvas();

// Distance constraints: d_G >= d_C + 1.0 Gly
sliderDistG.addEventListener('input', () => {

```

```

    let dG = parseFloat(sliderDistG.value);
    let dC = parseFloat(sliderDistC.value);
    if (dG < dC + 1.0) {
        dC = Math.max(1.0, dG - 1.0);
        sliderDistC.value = dC.toFixed(1);
    }
    updateControlStates();
});

sliderDistC.addEventListener('input', () => {
    let dG = parseFloat(sliderDistG.value);
    let dC = parseFloat(sliderDistC.value);
    if (dC > dG - 1.0) {
        dG = Math.min(13.0, dC + 1.0);
        sliderDistG.value = dG.toFixed(1);
    }
    updateControlStates();
});

sliderMass.addEventListener('input', updateControlStates);
sliderNumRays.addEventListener('input', updateControlStates);

function updateControlStates() {
    const numRays = parseInt(sliderNumRays.value);
    const mass = parseFloat(sliderMass.value);

    // Mass = 0 -> Deactivate Cluster Distance Slider
    if (mass === 0) {
        sliderDistC.disabled = true;
        grpDistC.classList.add('disabled-control');
    } else {
        sliderDistC.disabled = false;
        grpDistC.classList.remove('disabled-control');
    }

    if (numRays > 1) {
        sliderTheta.disabled = true;
        grpTheta.classList.add('disabled-control');

        sliderFanSpread.disabled = false;
        grpFanSpread.classList.remove('disabled-control');
    } else {
        sliderTheta.disabled = false;
        grpTheta.classList.remove('disabled-control');

        sliderFanSpread.disabled = true;
        grpFanSpread.classList.add('disabled-control');
    }
}

updateControlStates();

function calculateRays() {
    const d_G_Gly = parseFloat(sliderDistG.value);
    const x_G_kly = parseFloat(sliderOffsetX.value);
    const d_C_Gly = parseFloat(sliderDistC.value);
    const d_GC_Gly = d_G_Gly - d_C_Gly;
    const M_tril = parseFloat(sliderMass.value);
    const numRays = parseInt(sliderNumRays.value);
    const fanSpreadArcsec = parseFloat(sliderFanSpread.value);
    const thetaInitArcsec = parseFloat(sliderTheta.value);

    const ARCSEC_TO_RAD = Math.PI / (180.0 * 3600.0);
    const d_GC_ly = d_GC_Gly * 1e9;
    const d_C_ly = d_C_Gly * 1e9;
    const x_G_ly = x_G_kly * 1e3;

    const scale_4GM_c2_ly = 0.6243363 * M_tril;

    const rays = [];

    for (let i = 0; i < numRays; i++) {
        let initialAngle = 0;

```



```

    if (numRays === 1) {
      initialAngle = thetaInitArcsec;
    } else {
      const frac = (i / (numRays - 1)) - 0.5;
      initialAngle = frac * fanSpreadArcsec;
    }

    const theta_init_rad = initialAngle * ARCSEC_TO_RAD;

    const b_C_ly = x_G_ly + d_GC_ly * Math.tan(theta_init_rad);
    const abs_b_C_ly = Math.abs(b_C_ly);

    let alpha_arcsec = 0;
    let alpha_rad = 0;
    if (M_tril > 0 && abs_b_C_ly > 1e-3) {
      alpha_rad = scale_4GM_c2_ly / abs_b_C_ly;
      alpha_arcsec = alpha_rad / ARCSEC_TO_RAD;
    }

    let theta_after_rad = theta_init_rad;
    if (M_tril > 0) {
      if (b_C_ly > 0) {
        theta_after_rad = theta_init_rad - alpha_rad;
      } else if (b_C_ly < 0) {
        theta_after_rad = theta_init_rad + alpha_rad;
      }
    }
    const theta_after_arcsec = theta_after_rad / ARCSEC_TO_RAD;

    const b_E_ly = b_C_ly + d_C_ly * Math.tan(theta_after_rad);
    const abs_b_E_ly = Math.abs(b_E_ly);

    rays.push({
      index: i,
      theta_init_arcsec: initialAngle,
      b_C_ly: b_C_ly,
      abs_b_C_ly: abs_b_C_ly,
      b_E_ly: b_E_ly,
      abs_b_E_ly: abs_b_E_ly,
      alpha_arcsec: alpha_arcsec,
      theta_after_arcsec: theta_after_arcsec
    });
  }

  return {
    d_G_Gly,
    x_G_kly,
    d_C_Gly,
    d_GC_Gly,
    M_tril,
    numRays,
    fanSpreadArcsec,
    thetaInitArcsec,
    rays
  };
}

function draw() {
  ctx.clearRect(0, 0, width, height);

  const sim = calculateRays();

  // Label Updates
  valDistG.textContent = `${sim.d_G_Gly.toFixed(1)} Gly`;
  valOffsetX.textContent = `${sim.x_G_kly >= 0 ? '+' : ''}${sim.x_G_kly} kly`;
  valDistC.textContent = sim.M_tril === 0 ? "N/A (M=0)" : `${sim.d_C_Gly.toFixed(1)} Gly`;
  valMass.textContent = `${sim.M_tril} × 1012 M⊙`;
  valNumRays.textContent = `${sim.numRays}`;
  valFanSpread.textContent = `${sim.fanSpreadArcsec}`;
  valTheta.textContent = `${sim.thetaInitArcsec >= 0 ? '+' : ''}${sim.thetaInitArcsec.toFixed(1)}°`;

  // Update Numerical Box
  boxDistG.textContent = `${sim.d_G_Gly.toFixed(1)} Gly`;
  boxOffsetX.textContent = `${sim.x_G_kly >= 0 ? '+' : ''}${sim.x_G_kly} kly`;

```

```

boxDistC.textContent = sim.M_tril === 0 ? "N/A" : `${sim.d_C_Gly.toFixed(1)} Gly`;
boxDistG.textContent = sim.M_tril === 0 ? "N/A" : `${sim.d_GC_Gly.toFixed(1)} Gly`;
boxMass.textContent = `${sim.M_tril} × 1012 M☉`;

if (sim.numRays === 1) {
  const r = sim.rays[0];
  boxMode.textContent = `Single Ray`;
  boxImpactRange.textContent = `${(r.abs_b_C_ly / 1000.0).toFixed(2)} kly (${Math.round(r.abs_b_C_ly).toLocaleString()} ly)`;
  boxEarthImpactRange.textContent = `${(r.abs_b_E_ly / 1000.0).toFixed(2)} kly (${Math.round(r.abs_b_E_ly).toLocaleString()} ly)`;
  boxAlphaRange.textContent = `${r.alpha_arcsec.toFixed(2)}''`;
} else {
  boxMode.textContent = `Fan (${sim.numRays} rays)`;

  const bCValues = sim.rays.map(r => r.abs_b_C_ly / 1000.0);
  const bEValues = sim.rays.map(r => r.abs_b_E_ly / 1000.0);
  const alphaValues = sim.rays.map(r => r.alpha_arcsec).filter(a => isFinite(a));

  const minBC = Math.min(...bCValues);
  const maxBC = Math.max(...bCValues);
  const minBE = Math.min(...bEValues);
  const maxBE = Math.max(...bEValues);
  const maxAlpha = Math.max(...alphaValues);

  boxImpactRange.textContent = `${minBC.toFixed(1)} to ${maxBC.toFixed(1)} kly`;
  boxEarthImpactRange.textContent = `${minBE.toFixed(1)} to ${maxBE.toFixed(1)} kly`;
  boxAlphaRange.textContent = `${maxAlpha.toFixed(2)}'' max`;
}

// Visual Layout Dimensions (Vertical Field)
const topMargin = 45;
const bottomMargin = 55;
const centerX = width / 2;

const earthY = height - bottomMargin; // Earth at Bottom
const drawableHeight = height - topMargin - bottomMargin;
const pxPerGly = drawableHeight / CANVAS_TOTAL_GLY; // Scale for 13.5 Gly total field

const galaxyY = earthY - (sim.d_G_Gly * pxPerGly); // Distant Galaxy at Top
const clusterY = earthY - (sim.d_C_Gly * pxPerGly); // Intermediate Cluster

const pxPerKly = 100.0 / 500.0;
const galaxyX = centerX + (sim.x_G_kly * pxPerKly);

const ptA = { x: galaxyX, y: galaxyY };

// Starfield Background
ctx.fillStyle = "#ffffff";
stars.forEach(s => {
  ctx.globalAlpha = s.alpha;
  ctx.fillRect(s.x * width, s.y * height, s.size, s.size);
});
ctx.globalAlpha = 1.0;

// Central Optical Axis
ctx.strokeStyle = "rgba(255, 255, 255, 0.15)";
ctx.lineWidth = 1;
ctx.setLineDash([6, 6]);
ctx.beginPath();
ctx.moveTo(centerX, earthY + 20);
ctx.lineTo(centerX, earthY - CANVAS_TOTAL_GLY * pxPerGly - 15);
ctx.stroke();
ctx.setLineDash([]);

// Cluster Plane Line
if (sim.M_tril > 0) {
  ctx.strokeStyle = "rgba(0, 240, 255, 0.25)";
  ctx.beginPath();
  ctx.moveTo(20, clusterY);
  ctx.lineTo(width - 20, clusterY);
  ctx.stroke();
}

// Earth Plane Line
ctx.strokeStyle = "rgba(0, 240, 255, 0.15)";

```

```

ctx.beginPath();
ctx.moveTo(20, earthY);
ctx.lineTo(width - 20, earthY);
ctx.stroke();

// Render Light Rays & Virtual Tangents
sim.rays.forEach((ray, idx) => {
  const clusterX = centerX + (ray.b_C_ly / 1000.0) * pxPerKly;
  const earthX = centerX + (ray.b_E_ly / 1000.0) * pxPerKly;

  const ptB = { x: clusterX, y: clusterY };
  const ptC = { x: earthX, y: earthY };

  // Virtual Tangent Line
  if (toggleTangent.checked && Math.abs(ray.alpha_arcsec) > 0.1 && sim.M_tril > 0) {
    ctx.strokeStyle = "rgba(239, 68, 68, 0.5)";
    ctx.lineWidth = 1.2;
    ctx.setLineDash([3, 3]);

    const slopeX = (ptC.x - ptB.x) / (ptC.y - ptB.y);
    const virtualX_at_galaxy = ptB.x + slopeX * (galaxyY - clusterY);

    ctx.beginPath();
    ctx.moveTo(ptC.x, ptC.y);
    ctx.lineTo(virtualX_at_galaxy, galaxyY);
    ctx.stroke();
    ctx.setLineDash([]);

    ctx.fillStyle = "rgba(239, 68, 68, 0.8)";
    ctx.beginPath();
    ctx.arc(virtualX_at_galaxy, galaxyY, 3, 0, Math.PI * 2);
    ctx.fill();
  }

  // Actual Light Path
  ctx.strokeStyle = "#facc15";
  ctx.lineWidth = sim.numRays === 1 ? 2.5 : 1.8;
  ctx.shadowColor = "#facc15";
  ctx.shadowBlur = sim.numRays === 1 ? 6 : 3;

  ctx.beginPath();
  ctx.moveTo(ptA.x, ptA.y);
  if (sim.M_tril > 0) {
    ctx.lineTo(ptB.x, ptB.y);
  }
  ctx.lineTo(ptC.x, ptC.y);
  ctx.stroke();
  ctx.shadowBlur = 0;

  // Visual Indicators for Impact Parameters
  if (sim.numRays === 1) {
    if (sim.M_tril > 0 && Math.abs(ptB.x - centerX) > 3) {
      ctx.strokeStyle = "#00f0ff";
      ctx.lineWidth = 1.2;
      ctx.beginPath();
      ctx.moveTo(centerX, clusterY);
      ctx.lineTo(ptB.x, clusterY);
      ctx.stroke();

      ctx.fillStyle = "#00f0ff";
      ctx.beginPath();
      ctx.arc(ptB.x, clusterY, 2.5, 0, Math.PI * 2);
      ctx.arc(centerX, clusterY, 2.5, 0, Math.PI * 2);
      ctx.fill();

      ctx.font = "10px monospace";
      ctx.textAlign = "center";
      ctx.fillText(`${ray.b_C_ly / 1000}.toFixed(1)} kly`, (centerX + ptB.x) / 2, clusterY -
    }

    if (Math.abs(ptC.x - centerX) > 3) {
      ctx.strokeStyle = "#00f0ff";
      ctx.lineWidth = 1.2;
      ctx.beginPath();

```

```

    ctx.moveTo(centerX, earthY);
    ctx.lineTo(ptC.x, earthY);
    ctx.stroke();

    ctx.fillStyle = "#00f0ff";
    ctx.beginPath();
    ctx.arc(ptC.x, earthY, 2.5, 0, Math.PI * 2);
    ctx.arc(centerX, earthY, 2.5, 0, Math.PI * 2);
    ctx.fill();

    ctx.font = "10px monospace";
    ctx.textAlign = "center";
    ctx.fillText(`b_E = ${ (ray.abs_b_E_ly / 1000).toFixed(1) } kly`, (centerX + ptC.x) / 2, earthY + 16
  }
}

// Animated Photons
if (togglePhoton.checked) {
  const rayOffset = (idx / sim.numRays);
  const progress = (photonAnimProgress + rayOffset) % 1.0;
  let px = 0, py = 0;

  if (sim.M_tril === 0) {
    px = ptA.x + progress * (ptC.x - ptA.x);
    py = ptA.y + progress * (ptC.y - ptA.y);
  } else {
    if (progress <= 0.5) {
      const t = progress / 0.5;
      px = ptA.x + t * (ptB.x - ptA.x);
      py = ptA.y + t * (ptB.y - ptA.y);
    } else {
      const t = (progress - 0.5) / 0.5;
      px = ptB.x + t * (ptC.x - ptB.x);
      py = ptB.y + t * (ptC.y - ptB.y);
    }
  }

  ctx.fillStyle = "#ffffff";
  ctx.shadowColor = "#00f0ff";
  ctx.shadowBlur = 6;
  ctx.beginPath();
  ctx.arc(px, py, 3, 0, Math.PI * 2);
  ctx.fill();
  ctx.shadowBlur = 0;
}
});

if (togglePhoton.checked) {
  photonAnimProgress = (photonAnimProgress + 0.006) % 1.0;
}

// Draw Celestial Objects

// 1. Distant Galaxy (Top)
ctx.save();
ctx.translate(ptA.x, ptA.y);
const galGlow = ctx.createRadialGradient(0, 0, 2, 0, 0, 18);
galGlow.addColorStop(0, "rgba(255, 200, 100, 0.9)");
galGlow.addColorStop(0.5, "rgba(238, 100, 30, 0.3)");
galGlow.addColorStop(1, "rgba(0, 0, 0, 0)");
ctx.fillStyle = galGlow;
ctx.beginPath();
ctx.arc(0, 0, 18, 0, Math.PI * 2);
ctx.fill();

ctx.fillStyle = "#ffd59e";
ctx.beginPath();
ctx.ellipse(0, 0, 12, 4, -0.3, 0, Math.PI * 2);
ctx.fill();
ctx.restore();

ctx.fillStyle = "#e1e7ec";
ctx.font = "11px sans-serif";
ctx.textAlign = "center";

```

```

ctx.fillText("Distant Galaxy", ptA.x, ptA.y - 15);

// 2. Intermediate Cluster (Middle)
if (sim.M_tril > 0) {
  ctx.save();
  ctx.translate(centerX, clusterY);
  const haloRadius = 14 + (sim.M_tril / 1000) * 18;
  const clusterGlow = ctx.createRadialGradient(0, 0, 2, 0, 0, haloRadius);
  clusterGlow.addColorStop(0, "rgba(168, 85, 247, 0.65)");
  clusterGlow.addColorStop(0.6, "rgba(126, 34, 206, 0.2)");
  clusterGlow.addColorStop(1, "rgba(0, 0, 0, 0)");
  ctx.fillStyle = clusterGlow;
  ctx.beginPath();
  ctx.arc(0, 0, haloRadius, 0, Math.PI * 2);
  ctx.fill();

  ctx.fillStyle = "ffffff";
  ctx.beginPath();
  ctx.arc(0, 0, 3.5, 0, Math.PI * 2);
  ctx.arc(-6, -5, 2, 0, Math.PI * 2);
  ctx.arc(8, 4, 2.5, 0, Math.PI * 2);
  ctx.fill();
  ctx.restore();

  ctx.fillStyle = "#e1e7ec";
  ctx.font = "11px sans-serif";
  ctx.textAlign = "left";
  ctx.fillText("Intermediate Cluster", centerX + haloRadius + 6, clusterY + 4);
}

// 3. Earth / Observer (Bottom)
ctx.save();
ctx.translate(centerX, earthY);
const earthGrad = ctx.createRadialGradient(-2, -2, 1, 0, 0, 8);
earthGrad.addColorStop(0, "#60a5fa");
earthGrad.addColorStop(0.8, "#1d4ed8");
earthGrad.addColorStop(1, "#1e3a8a");
ctx.fillStyle = earthGrad;
ctx.beginPath();
ctx.arc(0, 0, 8, 0, Math.PI * 2);
ctx.fill();
ctx.restore();

ctx.fillStyle = "#e1e7ec";
ctx.font = "11px sans-serif";
ctx.textAlign = "center";
ctx.fillText("Earth", centerX, earthY + 26);

// Bottom Note inside Graphic Window
ctx.fillStyle = "rgba(140, 155, 174, 0.75)";
ctx.font = "italic 11px sans-serif";
ctx.textAlign = "center";
ctx.fillText("angles are exaggerated", centerX, height - 12);
}

function animate() {
  draw();
  requestAnimationFrame(animate);
}

animate();
</script>
</body>
</html>

```

User prompt: add a toggle for displaying the impact parameters on the graphic. change the impact parameter labels to just a number, and "thousand light-years" instead of "kly" move the label for the intermediate cluster to beneath the cluster change the cluster mass slider to range from 100 trillion to 1000 trillion solar masses, and add a toggle for the presence of a cluster don't change the speed of the animated photons after they pass the cluster

Response: <!DOCTYPE html>
<html lang="en">

```
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Gravitational Lensing: Vertical Field Simulation</title>
<style>
:root {
  --bg-color: #0b0e14;
  --panel-bg: #151922;
  --panel-border: #2a3142;
  --accent-color: #00f0ff;
  --accent-amber: #facc15;
  --accent-red: #ef4444;
  --text-main: #e1e7ec;
  --text-muted: #8c9bae;
}

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial, sans-serif;
  background-color: var(--bg-color);
  color: var(--text-main);
  display: flex;
  flex-direction: column;
  min-height: 100vh;
  overflow-x: hidden;
}

header {
  background-color: var(--panel-bg);
  border-bottom: 1px solid var(--panel-border);
  padding: 0.85rem 1.5rem;
  text-align: center;
}

header h1 {
  font-size: 1.3rem;
  font-weight: 600;
  color: #fff;
}

header p {
  font-size: 0.85rem;
  color: var(--text-muted);
  margin-top: 0.2rem;
}

.main-container {
  display: flex;
  flex: 1;
  flex-direction: column;
  gap: 1rem;
  padding: 1rem;
  max-width: 1200px;
  margin: 0 auto;
  width: 100%;
  align-items: center;
}

@media (min-width: 1024px) {
  .main-container {
    flex-direction: row;
    align-items: flex-start;
  }
}

.canvas-card {
  flex: 1;
  width: 100%;
  background-color: var(--panel-bg);
```

```

border: 1px solid var(--panel-border);
border-radius: 8px;
padding: 1rem;
display: flex;
flex-direction: column;
gap: 1rem;
box-shadow: 0 4px 20px rgba(0, 0, 0, 0.4);
}

.canvas-wrapper {
  position: relative;
  width: 100%;
  max-width: 520px;
  aspect-ratio: 3 / 4; /* Vertical portrait profile (4:3 height to width) */
  margin: 0 auto;
  background-color: #04060a;
  border-radius: 6px;
  overflow: hidden;
  border: 1px solid rgba(255, 255, 255, 0.05);
}

canvas {
  width: 100%;
  height: 100%;
  display: block;
}

.calc-box {
  background: rgba(10, 14, 22, 0.7);
  border: 1px solid var(--panel-border);
  border-radius: 6px;
  padding: 1rem 1.25rem;
  font-family: SFMono-Regular, Consolas, "Liberation Mono", Menlo, monospace;
  font-size: 0.8rem;
  color: #d1d5db;
}

.calc-box h3 {
  font-size: 0.85rem;
  text-transform: uppercase;
  letter-spacing: 0.05em;
  color: var(--accent-color);
  margin-bottom: 0.75rem;
  border-bottom: 1px solid rgba(255, 255, 255, 0.1);
  padding-bottom: 0.4rem;
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;
}

.calc-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
  gap: 0.85rem 1.25rem;
}

.calc-item {
  display: flex;
  flex-direction: column;
  gap: 0.25rem;
}

.calc-item .label {
  color: var(--text-muted);
  font-size: 0.75rem;
}

.calc-item .value {
  color: #fff;
  font-weight: 600;
  font-size: 0.85rem;
}

.calc-item .highlight {
  color: var(--accent-amber);
}

```

```
.calc-footer {
  margin-top: 0.85rem;
  padding-top: 0.6rem;
  border-top: 1px dashed rgba(255, 255, 255, 0.15);
  display: flex;
  justify-content: space-between;
  align-items: center;
  font-size: 0.75rem;
}

.calc-footer .formula {
  color: #9ca3af;
}

.controls-panel {
  width: 100%;
  max-width: 380px;
  background-color: var(--panel-bg);
  border: 1px solid var(--panel-border);
  border-radius: 8px;
  padding: 1.25rem;
  display: flex;
  flex-direction: column;
  gap: 1.1rem;
  box-shadow: 0 4px 20px rgba(0, 0, 0, 0.4);
}

.control-group {
  display: flex;
  flex-direction: column;
  gap: 0.4rem;
  transition: opacity 0.2s ease;
}

.control-group.disabled-control {
  opacity: 0.35;
}

.control-group label {
  font-size: 0.83rem;
  font-weight: 500;
  color: var(--text-main);
  display: flex;
  justify-content: space-between;
}

.control-group label span {
  color: var(--accent-color);
  font-family: monospace;
}

input[type="range"] {
  -webkit-appearance: none;
  width: 100%;
  height: 6px;
  border-radius: 3px;
  background: #2a3142;
  outline: none;
}

input[type="range"]:disabled {
  cursor: not-allowed;
}

input[type="range"]::-webkit-slider-thumb {
  -webkit-appearance: none;
  width: 16px;
  height: 16px;
  border-radius: 50%;
  background: var(--accent-color);
  cursor: pointer;
  box-shadow: 0 0 8px var(--accent-color);
  transition: transform 0.1s ease;
}
```



```
}

input[type="range"]:disabled::-webkit-slider-thumb {
  background: #6b7280;
  box-shadow: none;
  cursor: not-allowed;
}

input[type="range"]::-webkit-slider-thumb:hover:not(:disabled) {
  transform: scale(1.2);
}

.toggle-row {
  display: flex;
  align-items: center;
  justify-content: space-between;
  font-size: 0.83rem;
  color: var(--text-main);
}

.switch {
  position: relative;
  display: inline-block;
  width: 40px;
  height: 20px;
}

.switch input {
  opacity: 0;
  width: 0;
  height: 0;
}

.slider {
  position: absolute;
  cursor: pointer;
  top: 0; left: 0; right: 0; bottom: 0;
  background-color: #2a3142;
  transition: .2s;
  border-radius: 20px;
}

.slider:before {
  position: absolute;
  content: "";
  height: 14px;
  width: 14px;
  left: 3px;
  bottom: 3px;
  background-color: white;
  transition: .2s;
  border-radius: 50%;
}

input:checked + .slider {
  background-color: var(--accent-color);
}

input:checked + .slider:before {
  transform: translateX(20px);
}

.legend {
  border-top: 1px solid var(--panel-border);
  padding-top: 0.8rem;
  display: flex;
  flex-direction: column;
  gap: 0.4rem;
  font-size: 0.8rem;
}

.legend-item {
  display: flex;
  align-items: center;
}
```

138/271

```

    <span>Enable Intermediate Cluster</span>
    <label class="switch">
      <input type="checkbox" id="toggleCluster" checked>
      <span class="slider"></span>
    </label>
  </div>

  <div class="control-group">
    <label for="sliderDistG">
      1. Distance to Galaxy (d<sub>G</sub>):
      <span id="valDistG">8.0 Gly</span>
    </label>
    <input type="range" id="sliderDistG" min="2.0" max="13.0" step="0.1" value="8.0">
  </div>

  <div class="control-group">
    <label for="sliderOffsetX">
      2. Galaxy Horizontal Offset (x<sub>G</sub>):
      <span id="valOffsetX">0 kly</span>
    </label>
    <input type="range" id="sliderOffsetX" min="-500" max="500" step="10" value="0">
  </div>

  <div class="control-group" id="grpDistC">
    <label for="sliderDistC">
      3. Distance to Cluster (d<sub>C</sub>):
      <span id="valDistC">4.0 Gly</span>
    </label>
    <input type="range" id="sliderDistC" min="1.0" max="12.0" step="0.1" value="4.0">
  </div>

  <div class="control-group" id="grpMass">
    <label for="sliderMass">
      4. Cluster Mass (M):
      <span id="valMass">200 × 1012 M⊙</span>
    </label>
    <input type="range" id="sliderMass" min="100" max="1000" step="10" value="200">
  </div>

  <div class="control-group">
    <label for="sliderNumRays">
      5. Number of Light Rays:
      <span id="valNumRays">1</span>
    </label>
    <input type="range" id="sliderNumRays" min="1" max="10" step="1" value="1">
  </div>

  <div class="control-group" id="grpFanSpread">
    <label for="sliderFanSpread">
      6. Ray Fan Spread:
      <span id="valFanSpread">20°</span>
    </label>
    <input type="range" id="sliderFanSpread" min="10" max="40" step="1" value="20">
  </div>

  <div class="control-group" id="grpTheta">
    <label for="sliderTheta">
      7. Initial Ray Direction (1 ray):
      <span id="valTheta">+15.0°</span>
    </label>
    <input type="range" id="sliderTheta" min="-40.0" max="40.0" step="0.5" value="15.0">
  </div>

  <div class="toggle-row">
    <span>Animate Photons</span>
    <label class="switch">
      <input type="checkbox" id="togglePhoton" checked>
      <span class="slider"></span>
    </label>
  </div>

  <div class="toggle-row">
    <span>Show Virtual Tangent Lines</span>
    <label class="switch">

```

```

        <input type="checkbox" id="toggleTangent" checked>
        <span class="slider"></span>
    </label>
</div>

<div class="toggle-row">
    <span>Show Impact Parameters</span>
    <label class="switch">
        <input type="checkbox" id="toggleImpact" checked>
        <span class="slider"></span>
    </label>
</div>

<div class="legend">
    <div class="legend-item">
        <div class="legend-color" style="background: #facc15;"></div>
        <span>Actual deflected light rays</span>
    </div>
    <div class="legend-item">
        <div class="legend-color" style="border: 1px dashed #ef4444; background: transparent;"></div>
        <span>Apparent straight-line paths (virtual images)</span>
    </div>
    <div class="legend-item">
        <div class="legend-color" style="background: #00f0ff;"></div>
        <span>Cluster impact parameter ( $C$ ) & Earth impact ( $E$ )</span>
    </div>
</div>
</div>
</div>
<script>
const canvas = document.getElementById('simCanvas');
const ctx = canvas.getContext('2d');

// Canvas Fixed Total Height Scale (13.5 Billion Light-Years)
const CANVAS_TOTAL_GLY = 13.5;

// Controls
const toggleCluster = document.getElementById('toggleCluster');
const sliderDistG = document.getElementById('sliderDistG');
const sliderOffsetX = document.getElementById('sliderOffsetX');
const sliderDistC = document.getElementById('sliderDistC');
const sliderMass = document.getElementById('sliderMass');
const sliderNumRays = document.getElementById('sliderNumRays');
const sliderFanSpread = document.getElementById('sliderFanSpread');
const sliderTheta = document.getElementById('sliderTheta');

const grpDistC = document.getElementById('grpDistC');
const grpMass = document.getElementById('grpMass');
const grpFanSpread = document.getElementById('grpFanSpread');
const grpTheta = document.getElementById('grpTheta');

const valDistG = document.getElementById('valDistG');
const valOffsetX = document.getElementById('valOffsetX');
const valDistC = document.getElementById('valDistC');
const valMass = document.getElementById('valMass');
const valNumRays = document.getElementById('valNumRays');
const valFanSpread = document.getElementById('valFanSpread');
const valTheta = document.getElementById('valTheta');

const togglePhoton = document.getElementById('togglePhoton');
const toggleTangent = document.getElementById('toggleTangent');
const toggleImpact = document.getElementById('toggleImpact');

// Calculation Box Elements
const boxDistG = document.getElementById('boxDistG');
const boxOffsetX = document.getElementById('boxOffsetX');
const boxDistC = document.getElementById('boxDistC');
const boxDistGC = document.getElementById('boxDistGC');
const boxMass = document.getElementById('boxMass');
const boxMode = document.getElementById('boxMode');
const boxImpactRange = document.getElementById('boxImpactRange');
const boxEarthImpactRange = document.getElementById('boxEarthImpactRange');
const boxAlphaRange = document.getElementById('boxAlphaRange');

```

```

let width = 0;
let height = 0;
let photonAnimProgress = 0;

// Background Stars
const stars = [];
for (let i = 0; i < 220; i++) {
  stars.push({
    x: Math.random(),
    y: Math.random(),
    size: Math.random() * 1.5 + 0.3,
    alpha: Math.random() * 0.7 + 0.3
  });
}

function resizeCanvas() {
  const rect = canvas.parentElement.getBoundingClientRect();
  const dpr = window.devicePixelRatio || 1;
  width = rect.width;
  height = rect.height;

  canvas.width = width * dpr;
  canvas.height = height * dpr;
  ctx.scale(dpr, dpr);
}

window.addEventListener('resize', resizeCanvas);
resizeCanvas();

// Distance constraints: d_G >= d_C + 1.0 Gly
sliderDistG.addEventListener('input', () => {
  let dG = parseFloat(sliderDistG.value);
  let dC = parseFloat(sliderDistC.value);
  if (dG < dC + 1.0) {
    dC = Math.max(1.0, dG - 1.0);
    sliderDistC.value = dC.toFixed(1);
  }
  updateControlStates();
});

sliderDistC.addEventListener('input', () => {
  let dG = parseFloat(sliderDistG.value);
  let dC = parseFloat(sliderDistC.value);
  if (dC > dG - 1.0) {
    dG = Math.min(13.0, dC + 1.0);
    sliderDistG.value = dG.toFixed(1);
  }
  updateControlStates();
});

toggleCluster.addEventListener('change', updateControlStates);
sliderMass.addEventListener('input', updateControlStates);
sliderNumRays.addEventListener('input', updateControlStates);

function updateControlStates() {
  const numRays = parseInt(sliderNumRays.value);
  const clusterEnabled = toggleCluster.checked;

  if (!clusterEnabled) {
    sliderMass.disabled = true;
    grpMass.classList.add('disabled-control');
    sliderDistC.disabled = true;
    grpDistC.classList.add('disabled-control');
  } else {
    sliderMass.disabled = false;
    grpMass.classList.remove('disabled-control');
    sliderDistC.disabled = false;
    grpDistC.classList.remove('disabled-control');
  }

  if (numRays > 1) {
    sliderTheta.disabled = true;
    grpTheta.classList.add('disabled-control');
  }
}

```

```

    sliderFanSpread.disabled = false;
    grpFanSpread.classList.remove('disabled-control');
  } else {
    sliderTheta.disabled = false;
    grpTheta.classList.remove('disabled-control');

    sliderFanSpread.disabled = true;
    grpFanSpread.classList.add('disabled-control');
  }
}

updateControlStates();

function calculateRays() {
  const clusterEnabled = toggleCluster.checked;
  const d_G_Gly = parseFloat(sliderDistG.value);
  const x_G_kly = parseFloat(sliderOffsetX.value);
  const d_C_Gly = parseFloat(sliderDistC.value);
  const d_GC_Gly = d_G_Gly - d_C_Gly;
  const M_tril = clusterEnabled ? parseFloat(sliderMass.value) : 0;
  const numRays = parseInt(sliderNumRays.value);
  const fanSpreadArcsec = parseFloat(sliderFanSpread.value);
  const thetaInitArcsec = parseFloat(sliderTheta.value);

  const ARCSEC_TO_RAD = Math.PI / (180.0 * 3600.0);
  const d_GC_ly = d_GC_Gly * 1e9;
  const d_C_ly = d_C_Gly * 1e9;
  const x_G_ly = x_G_kly * 1e3;

  const scale_4GM_c2_ly = 0.6243363 * M_tril;

  const rays = [];

  for (let i = 0; i < numRays; i++) {
    let initialAngle = 0;

    if (numRays === 1) {
      initialAngle = thetaInitArcsec;
    } else {
      const frac = (i / (numRays - 1)) - 0.5;
      initialAngle = frac * fanSpreadArcsec;
    }

    const theta_init_rad = initialAngle * ARCSEC_TO_RAD;

    const b_C_ly = x_G_ly + d_GC_ly * Math.tan(theta_init_rad);
    const abs_b_C_ly = Math.abs(b_C_ly);

    let alpha_arcsec = 0;
    let alpha_rad = 0;
    if (M_tril > 0 && abs_b_C_ly > 1e-3) {
      alpha_rad = scale_4GM_c2_ly / abs_b_C_ly;
      alpha_arcsec = alpha_rad / ARCSEC_TO_RAD;
    }

    let theta_after_rad = theta_init_rad;
    if (M_tril > 0) {
      if (b_C_ly > 0) {
        theta_after_rad = theta_init_rad - alpha_rad;
      } else if (b_C_ly < 0) {
        theta_after_rad = theta_init_rad + alpha_rad;
      }
    }
    const theta_after_arcsec = theta_after_rad / ARCSEC_TO_RAD;

    const b_E_ly = b_C_ly + d_C_ly * Math.tan(theta_after_rad);
    const abs_b_E_ly = Math.abs(b_E_ly);

    rays.push({
      index: i,
      theta_init_arcsec: initialAngle,
      b_C_ly: b_C_ly,
      abs_b_C_ly: abs_b_C_ly,

```

```

    b_E_ly: b_E_ly,
    abs_b_E_ly: abs_b_E_ly,
    alpha_arcsec: alpha_arcsec,
    theta_after_arcsec: theta_after_arcsec
  });
}

return {
  clusterEnabled,
  d_G_Gly,
  x_G_kly,
  d_C_Gly,
  d_GC_Gly,
  M_tril,
  numRays,
  fanSpreadArcsec,
  thetaInitArcsec,
  rays
};
}

function draw() {
  ctx.clearRect(0, 0, width, height);

  const sim = calculateRays();

  // Label Updates
  valDistG.textContent = `${sim.d_G_Gly.toFixed(1)} Gly`;
  valOffsetX.textContent = `${sim.x_G_kly >= 0 ? '+' : ''}${sim.x_G_kly} kly`;
  valDistC.textContent = !sim.clusterEnabled ? "N/A" : `${sim.d_C_Gly.toFixed(1)} Gly`;
  valMass.textContent = !sim.clusterEnabled ? "Disabled" : `${sim.M_tril} × 1012 M⊙`;
  valNumRays.textContent = `${sim.numRays}`;
  valFanSpread.textContent = `${sim.fanSpreadArcsec}`;
  valTheta.textContent = `${sim.thetaInitArcsec >= 0 ? '+' : ''}${sim.thetaInitArcsec.toFixed(1)}°`;

  // Update Numerical Box
  boxDistG.textContent = `${sim.d_G_Gly.toFixed(1)} Gly`;
  boxOffsetX.textContent = `${sim.x_G_kly >= 0 ? '+' : ''}${sim.x_G_kly} kly`;
  boxDistC.textContent = !sim.clusterEnabled ? "N/A" : `${sim.d_C_Gly.toFixed(1)} Gly`;
  boxDistGC.textContent = !sim.clusterEnabled ? "N/A" : `${sim.d_GC_Gly.toFixed(1)} Gly`;
  boxMass.textContent = !sim.clusterEnabled ? "None" : `${sim.M_tril} × 1012 M⊙`;

  if (sim.numRays === 1) {
    const r = sim.rays[0];
    boxMode.textContent = `Single Ray`;
    boxImpactRange.textContent = sim.clusterEnabled ? `${(r.abs_b_C_ly / 1000.0).toFixed(2)} kly` : `(${Math.round(r.abs_b_C_ly / 1000.0)} kly)`;
    boxEarthImpactRange.textContent = `${(r.abs_b_E_ly / 1000.0).toFixed(2)} kly` : `(${Math.round(r.abs_b_E_ly / 1000.0)} kly)`;
    boxAlphaRange.textContent = sim.clusterEnabled ? `${r.alpha_arcsec.toFixed(2)}°` : "0.00°";
  } else {
    boxMode.textContent = `Fan (${sim.numRays} rays)`;

    const bCValues = sim.rays.map(r => r.abs_b_C_ly / 1000.0);
    const bEValues = sim.rays.map(r => r.abs_b_E_ly / 1000.0);
    const alphaValues = sim.rays.map(r => r.alpha_arcsec).filter(a => isFinite(a));

    const minBC = Math.min(...bCValues);
    const maxBC = Math.max(...bCValues);
    const minBE = Math.min(...bEValues);
    const maxBE = Math.max(...bEValues);
    const maxAlpha = Math.max(...alphaValues);

    boxImpactRange.textContent = sim.clusterEnabled ? `${minBC.toFixed(1)} to ${maxBC.toFixed(1)} kly` : `${minBC.toFixed(1)} to ${maxBC.toFixed(1)} kly`;
    boxEarthImpactRange.textContent = `${minBE.toFixed(1)} to ${maxBE.toFixed(1)} kly`;
    boxAlphaRange.textContent = sim.clusterEnabled ? `${maxAlpha.toFixed(2)}° max` : "0.00°";
  }

  // Visual Layout Dimensions (Vertical Field)
  const topMargin = 45;
  const bottomMargin = 55;
  const centerX = width / 2;

  const earthY = height - bottomMargin; // Earth at Bottom
  const drawableHeight = height - topMargin - bottomMargin;
  const pxPerGly = drawableHeight / CANVAS_TOTAL_GLY; // Scale for 13.5 Gly total field

```

```

const galaxyY = earthY - (sim.d_G_Gly * pxPerGly); // Distant Galaxy at Top
const clusterY = earthY - (sim.d_C_Gly * pxPerGly); // Intermediate Cluster

const pxPerKly = 100.0 / 500.0;
const galaxyX = centerX + (sim.x_G_kly * pxPerKly);

const ptA = { x: galaxyX, y: galaxyY };

// Starfield Background
ctx.fillStyle = "#ffffff";
stars.forEach(s => {
  ctx.globalAlpha = s.alpha;
  ctx.fillRect(s.x * width, s.y * height, s.size, s.size);
});
ctx.globalAlpha = 1.0;

// Central Optical Axis
ctx.strokeStyle = "rgba(255, 255, 255, 0.15)";
ctx.lineWidth = 1;
ctx.setLineDash([6, 6]);
ctx.beginPath();
ctx.moveTo(centerX, earthY + 20);
ctx.lineTo(centerX, earthY - CANVAS_TOTAL_GLY * pxPerGly - 15);
ctx.stroke();
ctx.setLineDash([]);

// Cluster Plane Line
if (sim.M_tril > 0) {
  ctx.strokeStyle = "rgba(0, 240, 255, 0.25)";
  ctx.beginPath();
  ctx.moveTo(20, clusterY);
  ctx.lineTo(width - 20, clusterY);
  ctx.stroke();
}

// Earth Plane Line
ctx.strokeStyle = "rgba(0, 240, 255, 0.15)";
ctx.beginPath();
ctx.moveTo(20, earthY);
ctx.lineTo(width - 20, earthY);
ctx.stroke();

// Render Light Rays & Virtual Tangents
sim.rays.forEach((ray, idx) => {
  const clusterX = centerX + (ray.b_C_ly / 1000.0) * pxPerKly;
  const earthX = centerX + (ray.b_E_ly / 1000.0) * pxPerKly;

  const ptB = { x: clusterX, y: clusterY };
  const ptC = { x: earthX, y: earthY };

  // Virtual Tangent Line
  if (toggleTangent.checked && Math.abs(ray.alpha_arcsec) > 0.1 && sim.M_tril > 0) {
    ctx.strokeStyle = "rgba(239, 68, 68, 0.5)";
    ctx.lineWidth = 1.2;
    ctx.setLineDash([3, 3]);

    const slopeX = (ptC.x - ptB.x) / (ptC.y - ptB.y);
    const virtualX_at_galaxy = ptB.x + slopeX * (galaxyY - clusterY);

    ctx.beginPath();
    ctx.moveTo(ptC.x, ptC.y);
    ctx.lineTo(virtualX_at_galaxy, galaxyY);
    ctx.stroke();
    ctx.setLineDash([]);

    ctx.fillStyle = "rgba(239, 68, 68, 0.8)";
    ctx.beginPath();
    ctx.arc(virtualX_at_galaxy, galaxyY, 3, 0, Math.PI * 2);
    ctx.fill();
  }

  // Actual Light Path
  ctx.strokeStyle = "#facc15";

```



```

ctx.lineWidth = sim.numRays === 1 ? 2.5 : 1.8;
ctx.shadowColor = "#facc15";
ctx.shadowBlur = sim.numRays === 1 ? 6 : 3;

ctx.beginPath();
ctx.moveTo(ptA.x, ptA.y);
if (sim.M_tril > 0) {
  ctx.lineTo(ptB.x, ptB.y);
}
ctx.lineTo(ptC.x, ptC.y);
ctx.stroke();
ctx.shadowBlur = 0;

// Visual Indicators for Impact Parameters
if (toggleImpact.checked && sim.numRays === 1) {
  // Cluster Impact Parameter (b_C)
  if (sim.M_tril > 0 && Math.abs(ptB.x - centerX) > 3) {
    ctx.strokeStyle = "#00f0ff";
    ctx.lineWidth = 1.2;
    ctx.beginPath();
    ctx.moveTo(centerX, clusterY);
    ctx.lineTo(ptB.x, clusterY);
    ctx.stroke();

    ctx.fillStyle = "#00f0ff";
    ctx.beginPath();
    ctx.arc(ptB.x, clusterY, 2.5, 0, Math.PI * 2);
    ctx.arc(centerX, clusterY, 2.5, 0, Math.PI * 2);
    ctx.fill();

    ctx.font = "10px monospace";
    ctx.textAlign = "center";
    ctx.fillText(`${(ray.abs_b_C_ly / 1000).toFixed(1)} thousand light-years`, (centerX + ptB.x) / 2,
  }

  // Earth Impact Parameter (b_E)
  if (Math.abs(ptC.x - centerX) > 3) {
    ctx.strokeStyle = "#00f0ff";
    ctx.lineWidth = 1.2;
    ctx.beginPath();
    ctx.moveTo(centerX, earthY);
    ctx.lineTo(ptC.x, earthY);
    ctx.stroke();

    ctx.fillStyle = "#00f0ff";
    ctx.beginPath();
    ctx.arc(ptC.x, earthY, 2.5, 0, Math.PI * 2);
    ctx.arc(centerX, earthY, 2.5, 0, Math.PI * 2);
    ctx.fill();

    ctx.font = "10px monospace";
    ctx.textAlign = "center";
    ctx.fillText(`${(ray.abs_b_E_ly / 1000).toFixed(1)} thousand light-years`, (centerX + ptC.x) / 2,
  }
}

// Animated Photons (Constant Speed Along Physical Path)
if (togglePhoton.checked) {
  const rayOffset = (idx / sim.numRays);
  const progress = (photonAnimProgress + rayOffset) % 1.0;
  let px = 0, py = 0;

  if (sim.M_tril === 0) {
    px = ptA.x + progress * (ptC.x - ptA.x);
    py = ptA.y + progress * (ptC.y - ptA.y);
  } else {
    const d1 = Math.hypot(ptB.x - ptA.x, ptB.y - ptA.y);
    const d2 = Math.hypot(ptC.x - ptB.x, ptC.y - ptB.y);
    const dTotal = d1 + d2;
    const f1 = dTotal > 0 ? d1 / dTotal : 0.5;

    if (progress <= f1) {
      const t = f1 > 0 ? progress / f1 : 0;
      px = ptA.x + t * (ptB.x - ptA.x);

```

```

        py = ptA.y + t * (ptB.y - ptA.y);
    } else {
        const t = (1 - f1) > 0 ? (progress - f1) / (1 - f1) : 0;
        px = ptB.x + t * (ptC.x - ptB.x);
        py = ptB.y + t * (ptC.y - ptB.y);
    }
}

ctx.fillStyle = "#ffffff";
ctx.shadowColor = "#00f0ff";
ctx.shadowBlur = 6;
ctx.beginPath();
ctx.arc(px, py, 3, 0, Math.PI * 2);
ctx.fill();
ctx.shadowBlur = 0;
}
});

if (togglePhoton.checked) {
    photonAnimProgress = (photonAnimProgress + 0.006) % 1.0;
}

// Draw Celestial Objects

// 1. Distant Galaxy (Top)
ctx.save();
ctx.translate(ptA.x, ptA.y);
const galGlow = ctx.createRadialGradient(0, 0, 2, 0, 0, 18);
galGlow.addColorStop(0, "rgba(255, 200, 100, 0.9)");
galGlow.addColorStop(0.5, "rgba(238, 100, 30, 0.3)");
galGlow.addColorStop(1, "rgba(0, 0, 0, 0)");
ctx.fillStyle = galGlow;
ctx.beginPath();
ctx.arc(0, 0, 18, 0, Math.PI * 2);
ctx.fill();

ctx.fillStyle = "#ffd59e";
ctx.beginPath();
ctx.ellipse(0, 0, 12, 4, -0.3, 0, Math.PI * 2);
ctx.fill();
ctx.restore();

ctx.fillStyle = "#e1e7ec";
ctx.font = "11px sans-serif";
ctx.textAlign = "center";
ctx.fillText("Distant Galaxy", ptA.x, ptA.y - 15);

// 2. Intermediate Cluster (Middle)
if (sim.M_tril > 0) {
    ctx.save();
    ctx.translate(centerX, clusterY);
    const haloRadius = 14 + (sim.M_tril / 1000) * 18;
    const clusterGlow = ctx.createRadialGradient(0, 0, 2, 0, 0, haloRadius);
    clusterGlow.addColorStop(0, "rgba(168, 85, 247, 0.65)");
    clusterGlow.addColorStop(0.6, "rgba(126, 34, 206, 0.2)");
    clusterGlow.addColorStop(1, "rgba(0, 0, 0, 0)");
    ctx.fillStyle = clusterGlow;
    ctx.beginPath();
    ctx.arc(0, 0, haloRadius, 0, Math.PI * 2);
    ctx.fill();

    ctx.fillStyle = "#ffffff";
    ctx.beginPath();
    ctx.arc(0, 0, 3.5, 0, Math.PI * 2);
    ctx.arc(-6, -5, 2, 0, Math.PI * 2);
    ctx.arc(8, 4, 2.5, 0, Math.PI * 2);
    ctx.fill();
    ctx.restore();

    ctx.fillStyle = "#e1e7ec";
    ctx.font = "11px sans-serif";
    ctx.textAlign = "center";
    ctx.fillText("Intermediate Cluster", centerX, clusterY + haloRadius + 14);
}

```

```

// 3. Earth / Observer (Bottom)
ctx.save();
ctx.translate(centerX, earthY);
const earthGrad = ctx.createRadialGradient(-2, -2, 1, 0, 0, 8);
earthGrad.addColorStop(0, "#60a5fa");
earthGrad.addColorStop(0.8, "#1d4ed8");
earthGrad.addColorStop(1, "#1e3a8a");
ctx.fillStyle = earthGrad;
ctx.beginPath();
ctx.arc(0, 0, 8, 0, Math.PI * 2);
ctx.fill();
ctx.restore();

ctx.fillStyle = "#e1e7ec";
ctx.font = "11px sans-serif";
ctx.textAlign = "center";
ctx.fillText("Earth", centerX, earthY + 26);

// Bottom Note inside Graphic Window
ctx.fillStyle = "rgba(140, 155, 174, 0.75)";
ctx.font = "italic 11px sans-serif";
ctx.textAlign = "center";
ctx.fillText("angles are exaggerated", centerX, height - 12);
}

function animate() {
  draw();
  requestAnimationFrame(animate);
}

animate();
</script>
</body>
</html>

```

User prompt: adjust the position of the cluster impact parameter label so that it does not overlap the ray. So if the initial angle is negative, write the label as left aligned to a point 2 or 3 characters to the right side of the bend. If the initial angle is positive, right-aligned to a point 2 or 3 characters to the left side of the bend. don't let the labels get carried off the edge of the display. add a toggle for showing / turning off the background stars add a toggle to be able to turn off having a cluster (i.e., set the cluster mass to 0) on the sliders, remove the variable names in parentheses. Change the "Gly" labels to "billion light-years". Change the cluster mass label to "trillion solar masses". Change the angle " to "arc-seconds"

```

Response: <!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Gravitational Lensing: Vertical Field Simulation</title>
<style>
  :root {
    --bg-color: #0b0e14;
    --panel-bg: #151922;
    --panel-border: #2a3142;
    --accent-color: #00f0ff;
    --accent-amber: #facc15;
    --accent-red: #ef4444;
    --text-main: #e1e7ec;
    --text-muted: #8c9bae;
  }

  * {
    box-sizing: border-box;
    margin: 0;
    padding: 0;
  }

  body {
    font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial, sans-serif;
    background-color: var(--bg-color);
    color: var(--text-main);
    display: flex;
    flex-direction: column;
  }

```

```
    min-height: 100vh;
    overflow-x: hidden;
  }

  header {
    background-color: var(--panel-bg);
    border-bottom: 1px solid var(--panel-border);
    padding: 0.85rem 1.5rem;
    text-align: center;
  }

  header h1 {
    font-size: 1.3rem;
    font-weight: 600;
    color: #fff;
  }

  header p {
    font-size: 0.85rem;
    color: var(--text-muted);
    margin-top: 0.2rem;
  }

  .main-container {
    display: flex;
    flex: 1;
    flex-direction: column;
    gap: 1rem;
    padding: 1rem;
    max-width: 1200px;
    margin: 0 auto;
    width: 100%;
    align-items: center;
  }

  @media (min-width: 1024px) {
    .main-container {
      flex-direction: row;
      align-items: flex-start;
    }
  }

  .canvas-card {
    flex: 1;
    width: 100%;
    background-color: var(--panel-bg);
    border: 1px solid var(--panel-border);
    border-radius: 8px;
    padding: 1rem;
    display: flex;
    flex-direction: column;
    gap: 1rem;
    box-shadow: 0 4px 20px rgba(0, 0, 0, 0.4);
  }

  .canvas-wrapper {
    position: relative;
    width: 100%;
    max-width: 520px;
    aspect-ratio: 3 / 4; /* Vertical portrait profile (4:3 height to width) */
    margin: 0 auto;
    background-color: #04060a;
    border-radius: 6px;
    overflow: hidden;
    border: 1px solid rgba(255, 255, 255, 0.05);
  }

  canvas {
    width: 100%;
    height: 100%;
    display: block;
  }

  .calc-box {
```

```
background: rgba(10, 14, 22, 0.7);
border: 1px solid var(--panel-border);
border-radius: 6px;
padding: 1rem 1.25rem;
font-family: SFMono-Regular, Consolas, "Liberation Mono", Menlo, monospace;
font-size: 0.8rem;
color: #d1d5db;
}

.calc-box h3 {
  font-size: 0.85rem;
  text-transform: uppercase;
  letter-spacing: 0.05em;
  color: var(--accent-color);
  margin-bottom: 0.75rem;
  border-bottom: 1px solid rgba(255, 255, 255, 0.1);
  padding-bottom: 0.4rem;
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;
}

.calc-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
  gap: 0.85rem 1.25rem;
}

.calc-item {
  display: flex;
  flex-direction: column;
  gap: 0.25rem;
}

.calc-item .label {
  color: var(--text-muted);
  font-size: 0.75rem;
}

.calc-item .value {
  color: #fff;
  font-weight: 600;
  font-size: 0.85rem;
}

.calc-item .highlight {
  color: var(--accent-amber);
}

.calc-footer {
  margin-top: 0.85rem;
  padding-top: 0.6rem;
  border-top: 1px dashed rgba(255, 255, 255, 0.15);
  display: flex;
  justify-content: space-between;
  align-items: center;
  font-size: 0.75rem;
}

.calc-footer .formula {
  color: #9ca3af;
}

.controls-panel {
  width: 100%;
  max-width: 380px;
  background-color: var(--panel-bg);
  border: 1px solid var(--panel-border);
  border-radius: 8px;
  padding: 1.25rem;
  display: flex;
  flex-direction: column;
  gap: 1.1rem;
  box-shadow: 0 4px 20px rgba(0, 0, 0, 0.4);
}
```

```
.control-group {
  display: flex;
  flex-direction: column;
  gap: 0.4rem;
  transition: opacity 0.2s ease;
}

.control-group.disabled-control {
  opacity: 0.35;
}

.control-group label {
  font-size: 0.83rem;
  font-weight: 500;
  color: var(--text-main);
  display: flex;
  justify-content: space-between;
}

.control-group label span {
  color: var(--accent-color);
  font-family: monospace;
}

input[type="range"] {
  -webkit-appearance: none;
  width: 100%;
  height: 6px;
  border-radius: 3px;
  background: #2a3142;
  outline: none;
}

input[type="range"]:disabled {
  cursor: not-allowed;
}

input[type="range"]::-webkit-slider-thumb {
  -webkit-appearance: none;
  width: 16px;
  height: 16px;
  border-radius: 50%;
  background: var(--accent-color);
  cursor: pointer;
  box-shadow: 0 0 8px var(--accent-color);
  transition: transform 0.1s ease;
}

input[type="range"]:disabled::-webkit-slider-thumb {
  background: #6b7280;
  box-shadow: none;
  cursor: not-allowed;
}

input[type="range"]::-webkit-slider-thumb:hover:not(:disabled) {
  transform: scale(1.2);
}

.toggle-row {
  display: flex;
  align-items: center;
  justify-content: space-between;
  font-size: 0.83rem;
  color: var(--text-main);
}

.switch {
  position: relative;
  display: inline-block;
  width: 40px;
  height: 20px;
}

.switch input {
```

```

    opacity: 0;
    width: 0;
    height: 0;
  }

.slider {
  position: absolute;
  cursor: pointer;
  top: 0; left: 0; right: 0; bottom: 0;
  background-color: #2a3142;
  transition: .2s;
  border-radius: 20px;
}

.slider:before {
  position: absolute;
  content: "";
  height: 14px;
  width: 14px;
  left: 3px;
  bottom: 3px;
  background-color: white;
  transition: .2s;
  border-radius: 50%;
}

input:checked + .slider {
  background-color: var(--accent-color);
}

input:checked + .slider:before {
  transform: translateX(20px);
}

.legend {
  border-top: 1px solid var(--panel-border);
  padding-top: 0.8rem;
  display: flex;
  flex-direction: column;
  gap: 0.4rem;
  font-size: 0.8rem;
}

.legend-item {
  display: flex;
  align-items: center;
  gap: 0.6rem;
  color: var(--text-muted);
}

.legend-color {
  width: 14px;
  height: 4px;
  border-radius: 2px;
}
</style>
</head>
<body>

<header>
  <h1>Gravitational Deflection of Light Fan</h1>
  <p>Vertical Propagation View (Distant Galaxy at Top, Earth at Bottom)</p>
</header>

<div class="main-container">
  <div class="canvas-card">
    <div class="canvas-wrapper">
      <canvas id="simCanvas"></canvas>
    </div>

    <!-- Numerical Output Box below Graphic -->
    <div class="calc-box">
      <h3>Numerical Results</h3>
      <div class="calc-grid">

```

```

<div class="calc-item">
  <span class="label">Galaxy Distance:</span>
  <span class="value" id="boxDistG">8.0 billion light-years</span>
</div>
<div class="calc-item">
  <span class="label">Galaxy Offset:</span>
  <span class="value" id="boxOffsetX">0 thousand light-years</span>
</div>
<div class="calc-item">
  <span class="label">Cluster Distance:</span>
  <span class="value" id="boxDistC">4.0 billion light-years</span>
</div>
<div class="calc-item">
  <span class="label">Galaxy-Cluster Distance:</span>
  <span class="value" id="boxDistGC">4.0 billion light-years</span>
</div>
<div class="calc-item">
  <span class="label">Cluster Mass:</span>
  <span class="value" id="boxMass">200 trillion solar masses</span>
</div>
<div class="calc-item">
  <span class="label">Ray Mode:</span>
  <span class="value" id="boxMode">Single Ray</span>
</div>
<div class="calc-item">
  <span class="label">Cluster Impact:</span>
  <span class="value highlight" id="boxImpactRange">290.9 thousand light-years</span>
</div>
<div class="calc-item">
  <span class="label">Earth Impact:</span>
  <span class="value highlight" id="boxEarthImpactRange">581.8 thousand light-years</span>
</div>
<div class="calc-item">
  <span class="label">Deflection Angle:</span>
  <span class="value highlight" id="boxAlphaRange">44.3 arc-seconds</span>
</div>
</div>
<div class="calc-footer">
  <span class="label">Theoretical Formula:</span>
  <span class="formula"> $\alpha = 4GM / (c^2 |b_{\text{C}}|) \quad \&nbsp;\&nbsp;\&nbsp; b_{\text{E}} = b_{\text{C}} + d$ </span>
</div>
</div>

<div class="controls-panel">
  <div class="toggle-row">
    <span>Enable Intermediate Cluster</span>
    <label class="switch">
      <input type="checkbox" id="toggleCluster" checked>
    </label>
  </div>

  <div class="control-group">
    <label for="sliderDistG">
      1. Distance to Galaxy:
      <span id="valDistG">8.0 billion light-years</span>
    </label>
    <input type="range" id="sliderDistG" min="2.0" max="13.0" step="0.1" value="8.0">
  </div>

  <div class="control-group">
    <label for="sliderOffsetX">
      2. Galaxy Horizontal Offset:
      <span id="valOffsetX">0 thousand light-years</span>
    </label>
    <input type="range" id="sliderOffsetX" min="-500" max="500" step="10" value="0">
  </div>

  <div class="control-group" id="grpDistC">
    <label for="sliderDistC">
      3. Distance to Cluster:
      <span id="valDistC">4.0 billion light-years</span>
    </label>
  </div>

```



```

    <input type="range" id="sliderDistC" min="1.0" max="12.0" step="0.1" value="4.0">
</div>

<div class="control-group" id="grpMass">
  <label for="sliderMass">
    4. Cluster Mass:
    <span id="valMass">200 trillion solar masses</span>
  </label>
  <input type="range" id="sliderMass" min="100" max="1000" step="10" value="200">
</div>

<div class="control-group">
  <label for="sliderNumRays">
    5. Number of Light Rays:
    <span id="valNumRays">1</span>
  </label>
  <input type="range" id="sliderNumRays" min="1" max="10" step="1" value="1">
</div>

<div class="control-group" id="grpFanSpread">
  <label for="sliderFanSpread">
    6. Ray Fan Spread:
    <span id="valFanSpread">20 arc-seconds</span>
  </label>
  <input type="range" id="sliderFanSpread" min="10" max="40" step="1" value="20">
</div>

<div class="control-group" id="grpTheta">
  <label for="sliderTheta">
    7. Initial Ray Direction (1 ray):
    <span id="valTheta">+15.0 arc-seconds</span>
  </label>
  <input type="range" id="sliderTheta" min="-40.0" max="40.0" step="0.5" value="15.0">
</div>

<div class="toggle-row">
  <span>Animate Photons</span>
  <label class="switch">
    <input type="checkbox" id="togglePhoton" checked>
    <span class="slider"></span>
  </label>
</div>

<div class="toggle-row">
  <span>Show Virtual Tangent Lines</span>
  <label class="switch">
    <input type="checkbox" id="toggleTangent" checked>
    <span class="slider"></span>
  </label>
</div>

<div class="toggle-row">
  <span>Show Impact Parameters</span>
  <label class="switch">
    <input type="checkbox" id="toggleImpact" checked>
    <span class="slider"></span>
  </label>
</div>

<div class="toggle-row">
  <span>Show Background Stars</span>
  <label class="switch">
    <input type="checkbox" id="toggleStars" checked>
    <span class="slider"></span>
  </label>
</div>

<div class="legend">
  <div class="legend-item">
    <div class="legend-color" style="background: #facc15;"></div>
    <span>Actual deflected light rays</span>
  </div>
  <div class="legend-item">
    <div class="legend-color" style="border: 1px dashed #ef4444; background: transparent;"></div>

```

154/271

```

width = rect.width;
height = rect.height;

canvas.width = width * dpr;
canvas.height = height * dpr;
ctx.scale(dpr, dpr);
}

window.addEventListener('resize', resizeCanvas);
resizeCanvas();

// Distance constraints: d_G >= d_C + 1.0 Gly
sliderDistG.addEventListener('input', () => {
  let dG = parseFloat(sliderDistG.value);
  let dC = parseFloat(sliderDistC.value);
  if (dG < dC + 1.0) {
    dC = Math.max(1.0, dG - 1.0);
    sliderDistC.value = dC.toFixed(1);
  }
  updateControlStates();
});

sliderDistC.addEventListener('input', () => {
  let dG = parseFloat(sliderDistG.value);
  let dC = parseFloat(sliderDistC.value);
  if (dC > dG - 1.0) {
    dG = Math.min(13.0, dC + 1.0);
    sliderDistG.value = dG.toFixed(1);
  }
  updateControlStates();
});

toggleCluster.addEventListener('change', updateControlStates);
sliderMass.addEventListener('input', updateControlStates);
sliderNumRays.addEventListener('input', updateControlStates);

function updateControlStates() {
  const numRays = parseInt(sliderNumRays.value);
  const clusterEnabled = toggleCluster.checked;

  if (!clusterEnabled) {
    sliderMass.disabled = true;
    grpMass.classList.add('disabled-control');
    sliderDistC.disabled = true;
    grpDistC.classList.add('disabled-control');
  } else {
    sliderMass.disabled = false;
    grpMass.classList.remove('disabled-control');
    sliderDistC.disabled = false;
    grpDistC.classList.remove('disabled-control');
  }

  if (numRays > 1) {
    sliderTheta.disabled = true;
    grpTheta.classList.add('disabled-control');

    sliderFanSpread.disabled = false;
    grpFanSpread.classList.remove('disabled-control');
  } else {
    sliderTheta.disabled = false;
    grpTheta.classList.remove('disabled-control');

    sliderFanSpread.disabled = true;
    grpFanSpread.classList.add('disabled-control');
  }
}

updateControlStates();

function calculateRays() {
  const clusterEnabled = toggleCluster.checked;
  const d_G_Gly = parseFloat(sliderDistG.value);
  const x_G_kly = parseFloat(sliderOffsetX.value);
  const d_C_Gly = parseFloat(sliderDistC.value);

```

```

const d_GC_Gly = d_G_Gly - d_C_Gly;
const M_tril = clusterEnabled ? parseFloat(sliderMass.value) : 0;
const numRays = parseInt(sliderNumRays.value);
const fanSpreadArcsec = parseFloat(sliderFanSpread.value);
const thetaInitArcsec = parseFloat(sliderTheta.value);

const ARCSEC_TO_RAD = Math.PI / (180.0 * 3600.0);
const d_GC_ly = d_GC_Gly * 1e9;
const d_C_ly = d_C_Gly * 1e9;
const x_G_ly = x_G_kly * 1e3;

const scale_4GM_c2_ly = 0.6243363 * M_tril;

const rays = [];

for (let i = 0; i < numRays; i++) {
  let initialAngle = 0;

  if (numRays === 1) {
    initialAngle = thetaInitArcsec;
  } else {
    const frac = (i / (numRays - 1)) - 0.5;
    initialAngle = frac * fanSpreadArcsec;
  }

  const theta_init_rad = initialAngle * ARCSEC_TO_RAD;

  const b_C_ly = x_G_ly + d_GC_ly * Math.tan(theta_init_rad);
  const abs_b_C_ly = Math.abs(b_C_ly);

  let alpha_arcsec = 0;
  let alpha_rad = 0;
  if (M_tril > 0 && abs_b_C_ly > 1e-3) {
    alpha_rad = scale_4GM_c2_ly / abs_b_C_ly;
    alpha_arcsec = alpha_rad / ARCSEC_TO_RAD;
  }

  let theta_after_rad = theta_init_rad;
  if (M_tril > 0) {
    if (b_C_ly > 0) {
      theta_after_rad = theta_init_rad - alpha_rad;
    } else if (b_C_ly < 0) {
      theta_after_rad = theta_init_rad + alpha_rad;
    }
  }
  const theta_after_arcsec = theta_after_rad / ARCSEC_TO_RAD;

  const b_E_ly = b_C_ly + d_C_ly * Math.tan(theta_after_rad);
  const abs_b_E_ly = Math.abs(b_E_ly);

  rays.push({
    index: i,
    theta_init_arcsec: initialAngle,
    b_C_ly: b_C_ly,
    abs_b_C_ly: abs_b_C_ly,
    b_E_ly: b_E_ly,
    abs_b_E_ly: abs_b_E_ly,
    alpha_arcsec: alpha_arcsec,
    theta_after_arcsec: theta_after_arcsec
  });
}

return {
  clusterEnabled,
  d_G_Gly,
  x_G_kly,
  d_C_Gly,
  d_GC_Gly,
  M_tril,
  numRays,
  fanSpreadArcsec,
  thetaInitArcsec,
  rays
};

```

```

}

function draw() {
  ctx.clearRect(0, 0, width, height);

  const sim = calculateRays();

  // Label Updates
  valDistG.textContent = `${sim.d_G_Gly.toFixed(1)} billion light-years`;
  valOffsetX.textContent = `${sim.x_G_kly >= 0 ? '+' : ''}${sim.x_G_kly} thousand light-years`;
  valDistC.textContent = `!sim.clusterEnabled ? "N/A" : `${sim.d_C_Gly.toFixed(1)} billion light-years`;
  valMass.textContent = `!sim.clusterEnabled ? "Disabled" : `${sim.M_tril} trillion solar masses`;
  valNumRays.textContent = `${sim.numRays}`;
  valFanSpread.textContent = `${sim.fanSpreadArcsec} arc-seconds`;
  valTheta.textContent = `${sim.thetaInitArcsec >= 0 ? '+' : ''}${sim.thetaInitArcsec.toFixed(1)} arc-sec`

  // Update Numerical Box
  boxDistG.textContent = `${sim.d_G_Gly.toFixed(1)} billion light-years`;
  boxOffsetX.textContent = `${sim.x_G_kly >= 0 ? '+' : ''}${sim.x_G_kly} thousand light-years`;
  boxDistC.textContent = `!sim.clusterEnabled ? "N/A" : `${sim.d_C_Gly.toFixed(1)} billion light-years`;
  boxDistGC.textContent = `!sim.clusterEnabled ? "N/A" : `${sim.d_GC_Gly.toFixed(1)} billion light-years`;
  boxMass.textContent = `!sim.clusterEnabled ? "None" : `${sim.M_tril} trillion solar masses`;

  if (sim.numRays === 1) {
    const r = sim.rays[0];
    boxMode.textContent = `Single Ray`;
    boxImpactRange.textContent = sim.clusterEnabled ? `${(r.abs_b_C_ly / 1000.0).toFixed(2)} thousand light-years`;
    boxEarthImpactRange.textContent = `${(r.abs_b_E_ly / 1000.0).toFixed(2)} thousand light-years`;
    boxAlphaRange.textContent = sim.clusterEnabled ? `${r.alpha_arcsec.toFixed(2)} arc-seconds` : "0.00 arc-seconds";
  } else {
    boxMode.textContent = `Fan (${sim.numRays} rays)`;

    const bCValues = sim.rays.map(r => r.abs_b_C_ly / 1000.0);
    const bEValues = sim.rays.map(r => r.abs_b_E_ly / 1000.0);
    const alphaValues = sim.rays.map(r => r.alpha_arcsec).filter(a => isFinite(a));

    const minBC = Math.min(...bCValues);
    const maxBC = Math.max(...bCValues);
    const minBE = Math.min(...bEValues);
    const maxBE = Math.max(...bEValues);
    const maxAlpha = Math.max(...alphaValues);

    boxImpactRange.textContent = sim.clusterEnabled ? `${minBC.toFixed(1)} to ${maxBC.toFixed(1)} thousand light-years`;
    boxEarthImpactRange.textContent = `${minBE.toFixed(1)} to ${maxBE.toFixed(1)} thousand light-years`;
    boxAlphaRange.textContent = sim.clusterEnabled ? `${maxAlpha.toFixed(2)} arc-seconds max` : "0.00 arc-seconds";
  }

  // Visual Layout Dimensions (Vertical Field)
  const topMargin = 45;
  const bottomMargin = 55;
  const centerX = width / 2;

  const earthY = height - bottomMargin; // Earth at Bottom
  const drawableHeight = height - topMargin - bottomMargin;
  const pxPerGly = drawableHeight / CANVAS_TOTAL_GLY; // Scale for 13.5 Gly total field

  const galaxyY = earthY - (sim.d_G_Gly * pxPerGly); // Distant Galaxy at Top
  const clusterY = earthY - (sim.d_C_Gly * pxPerGly); // Intermediate Cluster

  const pxPerKly = 100.0 / 500.0;
  const galaxyX = centerX + (sim.x_G_kly * pxPerKly);

  const ptA = { x: galaxyX, y: galaxyY };

  // Starfield Background
  if (toggleStars.checked) {
    ctx.fillStyle = "#ffffff";
    stars.forEach(s => {
      ctx.globalAlpha = s.alpha;
      ctx.fillRect(s.x * width, s.y * height, s.size, s.size);
    });
    ctx.globalAlpha = 1.0;
  }
}

```

```

// Central Optical Axis
ctx.strokeStyle = "rgba(255, 255, 255, 0.15)";
ctx.lineWidth = 1;
ctx.setLineDash([6, 6]);
ctx.beginPath();
ctx.moveTo(centerX, earthY + 20);
ctx.lineTo(centerX, earthY - CANVAS_TOTAL_GLY * pxPerGly - 15);
ctx.stroke();
ctx.setLineDash([]);

// Cluster Plane Line
if (sim.clusterEnabled && sim.M_tril > 0) {
  ctx.strokeStyle = "rgba(0, 240, 255, 0.25)";
  ctx.beginPath();
  ctx.moveTo(20, clusterY);
  ctx.lineTo(width - 20, clusterY);
  ctx.stroke();
}

// Earth Plane Line
ctx.strokeStyle = "rgba(0, 240, 255, 0.15)";
ctx.beginPath();
ctx.moveTo(20, earthY);
ctx.lineTo(width - 20, earthY);
ctx.stroke();

// Render Light Rays & Virtual Tangents
sim.rays.forEach((ray, idx) => {
  const clusterX = centerX + (ray.b_C_ly / 1000.0) * pxPerKly;
  const earthX = centerX + (ray.b_E_ly / 1000.0) * pxPerKly;

  const ptB = { x: clusterX, y: clusterY };
  const ptC = { x: earthX, y: earthY };

  // Virtual Tangent Line
  if (toggleTangent.checked && Math.abs(ray.alpha_arcsec) > 0.1 && sim.M_tril > 0) {
    ctx.strokeStyle = "rgba(239, 68, 68, 0.5)";
    ctx.lineWidth = 1.2;
    ctx.setLineDash([3, 3]);

    const slopeX = (ptC.x - ptB.x) / (ptC.y - ptB.y);
    const virtualX_at_galaxy = ptB.x + slopeX * (galaxyY - clusterY);

    ctx.beginPath();
    ctx.moveTo(ptC.x, ptC.y);
    ctx.lineTo(virtualX_at_galaxy, galaxyY);
    ctx.stroke();
    ctx.setLineDash([]);

    ctx.fillStyle = "rgba(239, 68, 68, 0.8)";
    ctx.beginPath();
    ctx.arc(virtualX_at_galaxy, galaxyY, 3, 0, Math.PI * 2);
    ctx.fill();
  }

  // Actual Light Path
  ctx.strokeStyle = "#facc15";
  ctx.lineWidth = sim.numRays === 1 ? 2.5 : 1.8;
  ctx.shadowColor = "#facc15";
  ctx.shadowBlur = sim.numRays === 1 ? 6 : 3;

  ctx.beginPath();
  ctx.moveTo(ptA.x, ptA.y);
  if (sim.M_tril > 0) {
    ctx.lineTo(ptB.x, ptB.y);
  }
  ctx.lineTo(ptC.x, ptC.y);
  ctx.stroke();
  ctx.shadowBlur = 0;

  // Visual Indicators for Impact Parameters
  if (toggleImpact.checked && sim.numRays === 1) {
    // Cluster Impact Parameter
    if (sim.clusterEnabled && sim.M_tril > 0 && Math.abs(ptB.x - centerX) > 3) {

```

```

ctx.strokeStyle = "#00f0ff";
ctx.lineWidth = 1.2;
ctx.beginPath();
ctx.moveTo(centerX, clusterY);
ctx.lineTo(ptB.x, clusterY);
ctx.stroke();

ctx.fillStyle = "#00f0ff";
ctx.beginPath();
ctx.arc(ptB.x, clusterY, 2.5, 0, Math.PI * 2);
ctx.arc(centerX, clusterY, 2.5, 0, Math.PI * 2);
ctx.fill();

ctx.font = "10px monospace";
const clusterLabel = `${(ray.abs_b_C_ly / 1000).toFixed(1)} thousand light-years`;
const textWidth = ctx.measureText(clusterLabel).width;
const charOffset = 18; // ~2-3 character offset from bend

let labelX = 0;
let align = "center";

if (ray.theta_init_arcsec < 0) {
  // Initial angle is negative: left aligned to 2-3 chars to the right side of the bend
  labelX = ptB.x + charOffset;
  align = "left";
  if (labelX + textWidth > width - 10) {
    labelX = width - 10 - textWidth;
  }
  if (labelX < 10) {
    labelX = 10;
  }
} else {
  // Initial angle is positive: right aligned to 2-3 chars to the left side of the bend
  labelX = ptB.x - charOffset;
  align = "right";
  if (labelX - textWidth < 10) {
    labelX = 10 + textWidth;
  }
  if (labelX > width - 10) {
    labelX = width - 10;
  }
}

ctx.textAlign = align;
ctx.fillText(clusterLabel, labelX, clusterY - 6);
}

// Earth Impact Parameter
if (Math.abs(ptC.x - centerX) > 3) {
  ctx.strokeStyle = "#00f0ff";
  ctx.lineWidth = 1.2;
  ctx.beginPath();
  ctx.moveTo(centerX, earthY);
  ctx.lineTo(ptC.x, earthY);
  ctx.stroke();

  ctx.fillStyle = "#00f0ff";
  ctx.beginPath();
  ctx.arc(ptC.x, earthY, 2.5, 0, Math.PI * 2);
  ctx.arc(centerX, earthY, 2.5, 0, Math.PI * 2);
  ctx.fill();

  ctx.font = "10px monospace";
  const earthLabel = `${(ray.abs_b_E_ly / 1000).toFixed(1)} thousand light-years`;
  const earthTextWidth = ctx.measureText(earthLabel).width;
  let earthTextX = (centerX + ptC.x) / 2;

  if (earthTextX - earthTextWidth / 2 < 10) {
    earthTextX = 10 + earthTextWidth / 2;
  } else if (earthTextX + earthTextWidth / 2 > width - 10) {
    earthTextX = width - 10 - earthTextWidth / 2;
  }

  ctx.textAlign = "center";

```

```

    ctx.fillText(earthLabel, earthTextX, earthY + 16);
  }
}

// Animated Photons (Constant Speed Along Physical Path)
if (togglePhoton.checked) {
  const rayOffset = (idx / sim.numRays);
  const progress = (photonAnimProgress + rayOffset) % 1.0;
  let px = 0, py = 0;

  if (sim.M_tril === 0) {
    px = ptA.x + progress * (ptC.x - ptA.x);
    py = ptA.y + progress * (ptC.y - ptA.y);
  } else {
    const d1 = Math.hypot(ptB.x - ptA.x, ptB.y - ptA.y);
    const d2 = Math.hypot(ptC.x - ptB.x, ptC.y - ptB.y);
    const dTotal = d1 + d2;
    const f1 = dTotal > 0 ? d1 / dTotal : 0.5;

    if (progress <= f1) {
      const t = f1 > 0 ? progress / f1 : 0;
      px = ptA.x + t * (ptB.x - ptA.x);
      py = ptA.y + t * (ptB.y - ptA.y);
    } else {
      const t = (1 - f1) > 0 ? (progress - f1) / (1 - f1) : 0;
      px = ptB.x + t * (ptC.x - ptB.x);
      py = ptB.y + t * (ptC.y - ptB.y);
    }
  }

  ctx.fillStyle = "#ffffff";
  ctx.shadowColor = "#00f0ff";
  ctx.shadowBlur = 6;
  ctx.beginPath();
  ctx.arc(px, py, 3, 0, Math.PI * 2);
  ctx.fill();
  ctx.shadowBlur = 0;
}
});

if (togglePhoton.checked) {
  photonAnimProgress = (photonAnimProgress + 0.006) % 1.0;
}

// Draw Celestial Objects

// 1. Distant Galaxy (Top)
ctx.save();
ctx.translate(ptA.x, ptA.y);
const galGlow = ctx.createRadialGradient(0, 0, 2, 0, 0, 18);
galGlow.addColorStop(0, "rgba(255, 200, 100, 0.9)");
galGlow.addColorStop(0.5, "rgba(238, 100, 30, 0.3)");
galGlow.addColorStop(1, "rgba(0, 0, 0, 0)");
ctx.fillStyle = galGlow;
ctx.beginPath();
ctx.arc(0, 0, 18, 0, Math.PI * 2);
ctx.fill();

ctx.fillStyle = "#ffd59e";
ctx.beginPath();
ctx.ellipse(0, 0, 12, 4, -0.3, 0, Math.PI * 2);
ctx.fill();
ctx.restore();

ctx.fillStyle = "#e1e7ec";
ctx.font = "11px sans-serif";
ctx.textAlign = "center";
ctx.fillText("Distant Galaxy", ptA.x, ptA.y - 15);

// 2. Intermediate Cluster (Middle - only rendered if cluster enabled)
if (sim.clusterEnabled && sim.M_tril > 0) {
  ctx.save();
  ctx.translate(centerX, clusterY);
  const haloRadius = 14 + (sim.M_tril / 1000) * 18;

```



```

const clusterGlow = ctx.createRadialGradient(0, 0, 2, 0, 0, haloRadius);
clusterGlow.addColorStop(0, "rgba(168, 85, 247, 0.65)");
clusterGlow.addColorStop(0.6, "rgba(126, 34, 206, 0.2)");
clusterGlow.addColorStop(1, "rgba(0, 0, 0, 0)");
ctx.fillStyle = clusterGlow;
ctx.beginPath();
ctx.arc(0, 0, haloRadius, 0, Math.PI * 2);
ctx.fill();

ctx.fillStyle = "ffffff";
ctx.beginPath();
ctx.arc(0, 0, 3.5, 0, Math.PI * 2);
ctx.arc(-6, -5, 2, 0, Math.PI * 2);
ctx.arc(8, 4, 2.5, 0, Math.PI * 2);
ctx.fill();
ctx.restore();

ctx.fillStyle = "#e1e7ec";
ctx.font = "11px sans-serif";
ctx.textAlign = "center";
ctx.fillText("Intermediate Cluster", centerX, clusterY + haloRadius + 14);
}

// 3. Earth / Observer (Bottom)
ctx.save();
ctx.translate(centerX, earthY);
const earthGrad = ctx.createRadialGradient(-2, -2, 1, 0, 0, 8);
earthGrad.addColorStop(0, "#60a5fa");
earthGrad.addColorStop(0.8, "#1d4ed8");
earthGrad.addColorStop(1, "#1e3a8a");
ctx.fillStyle = earthGrad;
ctx.beginPath();
ctx.arc(0, 0, 8, 0, Math.PI * 2);
ctx.fill();
ctx.restore();

ctx.fillStyle = "#e1e7ec";
ctx.font = "11px sans-serif";
ctx.textAlign = "center";
ctx.fillText("Earth", centerX, earthY + 26);

// Bottom Note inside Graphic Window
ctx.fillStyle = "rgba(140, 155, 174, 0.75)";
ctx.font = "italic 11px sans-serif";
ctx.textAlign = "center";
ctx.fillText("angles are exaggerated", centerX, height - 12);
}

function animate() {
  draw();
  requestAnimationFrame(animate);
}

animate();
</script>
</body>
</html>

```

User prompt: we need to make changes to the mass of the cluster more visually apparent. As the mass increases, add additional galaxies to the cluster, along with the increasingly visible purple color

```

Response: <!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Gravitational Lensing: Vertical Field Simulation</title>
  <style>
    :root {
      --bg-color: #0b0e14;
      --panel-bg: #151922;
      --panel-border: #2a3142;

```

```

--accent-color: #00f0ff;
--accent-amber: #facc15;
--accent-red: #ef4444;
--text-main: #e1e7ec;
--text-muted: #8c9bae;
}

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial, sans-serif;
  background-color: var(--bg-color);
  color: var(--text-main);
  display: flex;
  flex-direction: column;
  min-height: 100vh;
  overflow-x: hidden;
}

header {
  background-color: var(--panel-bg);
  border-bottom: 1px solid var(--panel-border);
  padding: 0.85rem 1.5rem;
  text-align: center;
}

header h1 {
  font-size: 1.3rem;
  font-weight: 600;
  color: #fff;
}

header p {
  font-size: 0.85rem;
  color: var(--text-muted);
  margin-top: 0.2rem;
}

.main-container {
  display: flex;
  flex: 1;
  flex-direction: column;
  gap: 1rem;
  padding: 1rem;
  max-width: 1200px;
  margin: 0 auto;
  width: 100%;
  align-items: center;
}

@media (min-width: 1024px) {
  .main-container {
    flex-direction: row;
    align-items: flex-start;
  }
}

.canvas-card {
  flex: 1;
  width: 100%;
  background-color: var(--panel-bg);
  border: 1px solid var(--panel-border);
  border-radius: 8px;
  padding: 1rem;
  display: flex;
  flex-direction: column;
  gap: 1rem;
  box-shadow: 0 4px 20px rgba(0, 0, 0, 0.4);
}

```

```
.canvas-wrapper {
  position: relative;
  width: 100%;
  max-width: 520px;
  aspect-ratio: 3 / 4; /* Vertical portrait profile (4:3 height to width) */
  margin: 0 auto;
  background-color: #04060a;
  border-radius: 6px;
  overflow: hidden;
  border: 1px solid rgba(255, 255, 255, 0.05);
}

canvas {
  width: 100%;
  height: 100%;
  display: block;
}

.calc-box {
  background: rgba(10, 14, 22, 0.7);
  border: 1px solid var(--panel-border);
  border-radius: 6px;
  padding: 1rem 1.25rem;
  font-family: SFMono-Regular, Consolas, "Liberation Mono", Menlo, monospace;
  font-size: 0.8rem;
  color: #d1d5db;
}

.calc-box h3 {
  font-size: 0.85rem;
  text-transform: uppercase;
  letter-spacing: 0.05em;
  color: var(--accent-color);
  margin-bottom: 0.75rem;
  border-bottom: 1px solid rgba(255, 255, 255, 0.1);
  padding-bottom: 0.4rem;
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;
}

.calc-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
  gap: 0.85rem 1.25rem;
}

.calc-item {
  display: flex;
  flex-direction: column;
  gap: 0.25rem;
}

.calc-item .label {
  color: var(--text-muted);
  font-size: 0.75rem;
}

.calc-item .value {
  color: #fff;
  font-weight: 600;
  font-size: 0.85rem;
}

.calc-item .highlight {
  color: var(--accent-amber);
}

.calc-footer {
  margin-top: 0.85rem;
  padding-top: 0.6rem;
  border-top: 1px dashed rgba(255, 255, 255, 0.15);
  display: flex;
  justify-content: space-between;
  align-items: center;
  font-size: 0.75rem;
}
```

```
}

.calc-footer .formula {
  color: #9ca3af;
}

.controls-panel {
  width: 100%;
  max-width: 380px;
  background-color: var(--panel-bg);
  border: 1px solid var(--panel-border);
  border-radius: 8px;
  padding: 1.25rem;
  display: flex;
  flex-direction: column;
  gap: 1.1rem;
  box-shadow: 0 4px 20px rgba(0, 0, 0, 0.4);
}

.control-group {
  display: flex;
  flex-direction: column;
  gap: 0.4rem;
  transition: opacity 0.2s ease;
}

.control-group.disabled-control {
  opacity: 0.35;
}

.control-group label {
  font-size: 0.83rem;
  font-weight: 500;
  color: var(--text-main);
  display: flex;
  justify-content: space-between;
}

.control-group label span {
  color: var(--accent-color);
  font-family: monospace;
}

input[type="range"] {
  -webkit-appearance: none;
  width: 100%;
  height: 6px;
  border-radius: 3px;
  background: #2a3142;
  outline: none;
}

input[type="range"]:disabled {
  cursor: not-allowed;
}

input[type="range"]::-webkit-slider-thumb {
  -webkit-appearance: none;
  width: 16px;
  height: 16px;
  border-radius: 50%;
  background: var(--accent-color);
  cursor: pointer;
  box-shadow: 0 0 8px var(--accent-color);
  transition: transform 0.1s ease;
}

input[type="range"]:disabled::-webkit-slider-thumb {
  background: #6b7280;
  box-shadow: none;
  cursor: not-allowed;
}

input[type="range"]::-webkit-slider-thumb:hover:not(:disabled) {
```

```
    transform: scale(1.2);
  }

.toggle-row {
  display: flex;
  align-items: center;
  justify-content: space-between;
  font-size: 0.83rem;
  color: var(--text-main);
}

.switch {
  position: relative;
  display: inline-block;
  width: 40px;
  height: 20px;
}

.switch input {
  opacity: 0;
  width: 0;
  height: 0;
}

.slider {
  position: absolute;
  cursor: pointer;
  top: 0; left: 0; right: 0; bottom: 0;
  background-color: #2a3142;
  transition: .2s;
  border-radius: 20px;
}

.slider:before {
  position: absolute;
  content: "";
  height: 14px;
  width: 14px;
  left: 3px;
  bottom: 3px;
  background-color: white;
  transition: .2s;
  border-radius: 50%;
}

input:checked + .slider {
  background-color: var(--accent-color);
}

input:checked + .slider:before {
  transform: translateX(20px);
}

.legend {
  border-top: 1px solid var(--panel-border);
  padding-top: 0.8rem;
  display: flex;
  flex-direction: column;
  gap: 0.4rem;
  font-size: 0.8rem;
}

.legend-item {
  display: flex;
  align-items: center;
  gap: 0.6rem;
  color: var(--text-muted);
}

.legend-color {
  width: 14px;
  height: 4px;
  border-radius: 2px;
}
```

```

</style>
</head>
<body>

<header>
  <h1>Gravitational Deflection of Light Fan</h1>
  <p>Vertical Propagation View (Distant Galaxy at Top, Earth at Bottom)</p>
</header>

<div class="main-container">
  <div class="canvas-card">
    <div class="canvas-wrapper">
      <canvas id="simCanvas"></canvas>
    </div>

    <!-- Numerical Output Box below Graphic -->
    <div class="calc-box">
      <h3>Numerical Results</h3>
      <div class="calc-grid">
        <div class="calc-item">
          <span class="label">Galaxy Distance:</span>
          <span class="value" id="boxDistG">8.0 billion light-years</span>
        </div>
        <div class="calc-item">
          <span class="label">Galaxy Offset:</span>
          <span class="value" id="boxOffsetX">0 thousand light-years</span>
        </div>
        <div class="calc-item">
          <span class="label">Cluster Distance:</span>
          <span class="value" id="boxDistC">4.0 billion light-years</span>
        </div>
        <div class="calc-item">
          <span class="label">Galaxy-Cluster Distance:</span>
          <span class="value" id="boxDistGC">4.0 billion light-years</span>
        </div>
        <div class="calc-item">
          <span class="label">Cluster Mass:</span>
          <span class="value" id="boxMass">200 trillion solar masses</span>
        </div>
        <div class="calc-item">
          <span class="label">Ray Mode:</span>
          <span class="value" id="boxMode">Single Ray</span>
        </div>
        <div class="calc-item">
          <span class="label">Cluster Impact:</span>
          <span class="value highlight" id="boxImpactRange">290.9 thousand light-years</span>
        </div>
        <div class="calc-item">
          <span class="label">Earth Impact:</span>
          <span class="value highlight" id="boxEarthImpactRange">581.8 thousand light-years</span>
        </div>
        <div class="calc-item">
          <span class="label">Deflection Angle:</span>
          <span class="value highlight" id="boxAlphaRange">44.3 arc-seconds</span>
        </div>
      </div>
      <div class="calc-footer">
        <span class="label">Theoretical Formula:</span>
        <span class="formula"> $\hat{\alpha} = 4GM / (c^2 |b_{\text{sub}C}|) \quad \&nbps; | \quad \&nbps; b_{\text{sub}E} = b_{\text{sub}C} + d$ </span>
      </div>
    </div>
  </div>

  <div class="controls-panel">
    <div class="toggle-row">
      <span>Enable Intermediate Cluster</span>
      <label class="switch">
        <input type="checkbox" id="toggleCluster" checked>
      </label>
    </div>

    <div class="control-group">
      <label for="sliderDistG">

```

```

    1. Distance to Galaxy:
    <span id="valDistG">8.0 billion light-years</span>
  </label>
  <input type="range" id="sliderDistG" min="2.0" max="13.0" step="0.1" value="8.0">
</div>

<div class="control-group">
  <label for="sliderOffsetX">
    2. Galaxy Horizontal Offset:
    <span id="valOffsetX">0 thousand light-years</span>
  </label>
  <input type="range" id="sliderOffsetX" min="-500" max="500" step="10" value="0">
</div>

<div class="control-group" id="grpDistC">
  <label for="sliderDistC">
    3. Distance to Cluster:
    <span id="valDistC">4.0 billion light-years</span>
  </label>
  <input type="range" id="sliderDistC" min="1.0" max="12.0" step="0.1" value="4.0">
</div>

<div class="control-group" id="grpMass">
  <label for="sliderMass">
    4. Cluster Mass:
    <span id="valMass">200 trillion solar masses</span>
  </label>
  <input type="range" id="sliderMass" min="100" max="1000" step="10" value="200">
</div>

<div class="control-group">
  <label for="sliderNumRays">
    5. Number of Light Rays:
    <span id="valNumRays">1</span>
  </label>
  <input type="range" id="sliderNumRays" min="1" max="10" step="1" value="1">
</div>

<div class="control-group" id="grpFanSpread">
  <label for="sliderFanSpread">
    6. Ray Fan Spread:
    <span id="valFanSpread">20 arc-seconds</span>
  </label>
  <input type="range" id="sliderFanSpread" min="10" max="40" step="1" value="20">
</div>

<div class="control-group" id="grpTheta">
  <label for="sliderTheta">
    7. Initial Ray Direction (1 ray):
    <span id="valTheta">+15.0 arc-seconds</span>
  </label>
  <input type="range" id="sliderTheta" min="-40.0" max="40.0" step="0.5" value="15.0">
</div>

<div class="toggle-row">
  <span>Animate Photons</span>
  <label class="switch">
    <input type="checkbox" id="togglePhoton" checked>
    <span class="slider"></span>
  </label>
</div>

<div class="toggle-row">
  <span>Show Virtual Tangent Lines</span>
  <label class="switch">
    <input type="checkbox" id="toggleTangent" checked>
    <span class="slider"></span>
  </label>
</div>

<div class="toggle-row">
  <span>Show Impact Parameters</span>
  <label class="switch">
    <input type="checkbox" id="toggleImpact" checked>

```

```

        <span class="slider"></span>
    </label>
</div>

<div class="toggle-row">
    <span>Show Background Stars</span>
    <label class="switch">
        <input type="checkbox" id="toggleStars" checked>
        <span class="slider"></span>
    </label>
</div>

<div class="legend">
    <div class="legend-item">
        <div class="legend-color" style="background: #facc15;"></div>
        <span>Actual deflected light rays</span>
    </div>
    <div class="legend-item">
        <div class="legend-color" style="border: 1px dashed #ef4444; background: transparent;"></div>
        <span>Apparent straight-line paths (virtual images)</span>
    </div>
    <div class="legend-item">
        <div class="legend-color" style="background: #00f0ff;"></div>
        <span>Cluster impact parameter & Earth impact</span>
    </div>
</div>
</div>
</div>

<script>
    const canvas = document.getElementById('simCanvas');
    const ctx = canvas.getContext('2d');

    // Canvas Fixed Total Height Scale (13.5 Billion Light-Years)
    const CANVAS_TOTAL_GLY = 13.5;

    // Controls
    const toggleCluster = document.getElementById('toggleCluster');
    const sliderDistG = document.getElementById('sliderDistG');
    const sliderOffsetX = document.getElementById('sliderOffsetX');
    const sliderDistC = document.getElementById('sliderDistC');
    const sliderMass = document.getElementById('sliderMass');
    const sliderNumRays = document.getElementById('sliderNumRays');
    const sliderFanSpread = document.getElementById('sliderFanSpread');
    const sliderTheta = document.getElementById('sliderTheta');

    const grpDistC = document.getElementById('grpDistC');
    const grpMass = document.getElementById('grpMass');
    const grpFanSpread = document.getElementById('grpFanSpread');
    const grpTheta = document.getElementById('grpTheta');

    const valDistG = document.getElementById('valDistG');
    const valOffsetX = document.getElementById('valOffsetX');
    const valDistC = document.getElementById('valDistC');
    const valMass = document.getElementById('valMass');
    const valNumRays = document.getElementById('valNumRays');
    const valFanSpread = document.getElementById('valFanSpread');
    valTheta = document.getElementById('valTheta');

    const togglePhoton = document.getElementById('togglePhoton');
    const toggleTangent = document.getElementById('toggleTangent');
    const toggleImpact = document.getElementById('toggleImpact');
    const toggleStars = document.getElementById('toggleStars');

    // Calculation Box Elements
    const boxDistG = document.getElementById('boxDistG');
    const boxOffsetX = document.getElementById('boxOffsetX');
    const boxDistC = document.getElementById('boxDistC');
    const boxDistGC = document.getElementById('boxDistGC');
    const boxMass = document.getElementById('boxMass');
    const boxMode = document.getElementById('boxMode');
    const boxImpactRange = document.getElementById('boxImpactRange');
    const boxEarthImpactRange = document.getElementById('boxEarthImpactRange');
    const boxAlphaRange = document.getElementById('boxAlphaRange');

```



```

let width = 0;
let height = 0;
let photonAnimProgress = 0;

// Background Stars
const stars = [];
for (let i = 0; i < 220; i++) {
  stars.push({
    x: Math.random(),
    y: Math.random(),
    size: Math.random() * 1.5 + 0.3,
    alpha: Math.random() * 0.7 + 0.3
  });
}

// Cluster Member Galaxies layout offsets (up to 30 member galaxies)
const clusterGalaxyList = [
  { x: 0, y: 0, r: 3.5 },
  { x: -6, y: -5, r: 2.2 },
  { x: 8, y: 4, r: 2.5 },
  { x: -12, y: 8, r: 2.0 },
  { x: 10, y: -10, r: 2.1 },
  { x: 5, y: -14, r: 1.8 },
  { x: -15, y: -6, r: 2.3 },
  { x: 18, y: -2, r: 1.9 },
  { x: -8, y: 16, r: 2.2 },
  { x: 14, y: 12, r: 1.7 },
  { x: 2, y: 18, r: 2.0 },
  { x: -20, y: 4, r: 1.8 },
  { x: -4, y: -20, r: 1.9 },
  { x: 22, y: -12, r: 1.6 },
  { x: 12, y: -22, r: 1.8 },
  { x: -22, y: -15, r: 1.7 },
  { x: -16, y: 22, r: 1.9 },
  { x: 25, y: 8, r: 1.5 },
  { x: 6, y: 26, r: 1.7 },
  { x: -28, y: -4, r: 1.6 },
  { x: 18, y: 22, r: 1.5 },
  { x: -10, y: -28, r: 1.8 },
  { x: 28, y: -20, r: 1.4 },
  { x: -25, y: 18, r: 1.6 },
  { x: 0, y: -32, r: 1.5 },
  { x: 32, y: 2, r: 1.4 },
  { x: -32, y: -18, r: 1.5 },
  { x: 20, y: -30, r: 1.4 },
  { x: -20, y: -32, r: 1.3 },
  { x: 15, y: 32, r: 1.4 }
];

function resizeCanvas() {
  const rect = canvas.parentElement.getBoundingClientRect();
  const dpr = window.devicePixelRatio || 1;
  width = rect.width;
  height = rect.height;

  canvas.width = width * dpr;
  canvas.height = height * dpr;
  ctx.scale(dpr, dpr);
}

window.addEventListener('resize', resizeCanvas);
resizeCanvas();

// Distance constraints: d_G >= d_C + 1.0 Gly
sliderDistG.addEventListener('input', () => {
  let dG = parseFloat(sliderDistG.value);
  let dC = parseFloat(sliderDistC.value);
  if (dG < dC + 1.0) {
    dC = Math.max(1.0, dG - 1.0);
    sliderDistC.value = dC.toFixed(1);
  }
  updateControlStates();
});

```

```

sliderDistC.addEventListener('input', () => {
  let dG = parseFloat(sliderDistG.value);
  let dC = parseFloat(sliderDistC.value);
  if (dC > dG - 1.0) {
    dG = Math.min(13.0, dC + 1.0);
    sliderDistG.value = dG.toFixed(1);
  }
  updateControlStates();
});

toggleCluster.addEventListener('change', updateControlStates);
sliderMass.addEventListener('input', updateControlStates);
sliderNumRays.addEventListener('input', updateControlStates);

function updateControlStates() {
  const numRays = parseInt(sliderNumRays.value);
  const clusterEnabled = toggleCluster.checked;

  if (!clusterEnabled) {
    sliderMass.disabled = true;
    grpMass.classList.add('disabled-control');
    sliderDistC.disabled = true;
    grpDistC.classList.add('disabled-control');
  } else {
    sliderMass.disabled = false;
    grpMass.classList.remove('disabled-control');
    sliderDistC.disabled = false;
    grpDistC.classList.remove('disabled-control');
  }

  if (numRays > 1) {
    sliderTheta.disabled = true;
    grpTheta.classList.add('disabled-control');

    sliderFanSpread.disabled = false;
    grpFanSpread.classList.remove('disabled-control');
  } else {
    sliderTheta.disabled = false;
    grpTheta.classList.remove('disabled-control');

    sliderFanSpread.disabled = true;
    grpFanSpread.classList.add('disabled-control');
  }
}

updateControlStates();

function calculateRays() {
  const clusterEnabled = toggleCluster.checked;
  const d_G_Gly = parseFloat(sliderDistG.value);
  const x_G_kly = parseFloat(sliderOffsetX.value);
  const d_C_Gly = parseFloat(sliderDistC.value);
  const d_GC_Gly = d_G_Gly - d_C_Gly;
  const M_tril = clusterEnabled ? parseFloat(sliderMass.value) : 0;
  const numRays = parseInt(sliderNumRays.value);
  const fanSpreadArcsec = parseFloat(sliderFanSpread.value);
  const thetaInitArcsec = parseFloat(sliderTheta.value);

  const ARCSEC_TO_RAD = Math.PI / (180.0 * 3600.0);
  const d_GC_ly = d_GC_Gly * 1e9;
  const d_C_ly = d_C_Gly * 1e9;
  const x_G_ly = x_G_kly * 1e3;

  const scale_4GM_c2_ly = 0.6243363 * M_tril;

  const rays = [];

  for (let i = 0; i < numRays; i++) {
    let initialAngle = 0;

    if (numRays === 1) {
      initialAngle = thetaInitArcsec;
    } else {

```

```

    const frac = (i / (numRays - 1)) - 0.5;
    initialAngle = frac * fanSpreadArcsec;
  }

  const theta_init_rad = initialAngle * ARCSEC_TO_RAD;

  const b_C_ly = x_G_ly + d_GC_ly * Math.tan(theta_init_rad);
  const abs_b_C_ly = Math.abs(b_C_ly);

  let alpha_arcsec = 0;
  let alpha_rad = 0;
  if (M_tril > 0 && abs_b_C_ly > 1e-3) {
    alpha_rad = scale_4GM_c2_ly / abs_b_C_ly;
    alpha_arcsec = alpha_rad / ARCSEC_TO_RAD;
  }

  let theta_after_rad = theta_init_rad;
  if (M_tril > 0) {
    if (b_C_ly > 0) {
      theta_after_rad = theta_init_rad - alpha_rad;
    } else if (b_C_ly < 0) {
      theta_after_rad = theta_init_rad + alpha_rad;
    }
  }
  const theta_after_arcsec = theta_after_rad / ARCSEC_TO_RAD;

  const b_E_ly = b_C_ly + d_C_ly * Math.tan(theta_after_rad);
  const abs_b_E_ly = Math.abs(b_E_ly);

  rays.push({
    index: i,
    theta_init_arcsec: initialAngle,
    b_C_ly: b_C_ly,
    abs_b_C_ly: abs_b_C_ly,
    b_E_ly: b_E_ly,
    abs_b_E_ly: abs_b_E_ly,
    alpha_arcsec: alpha_arcsec,
    theta_after_arcsec: theta_after_arcsec
  });
}

return {
  clusterEnabled,
  d_G_Gly,
  x_G_kly,
  d_C_Gly,
  d_GC_Gly,
  M_tril,
  numRays,
  fanSpreadArcsec,
  thetaInitArcsec,
  rays
};
}

function draw() {
  ctx.clearRect(0, 0, width, height);

  const sim = calculateRays();

  // Label Updates
  valDistG.textContent = `${sim.d_G_Gly.toFixed(1)} billion light-years`;
  valOffsetX.textContent = `${sim.x_G_kly >= 0 ? '+' : ''}${sim.x_G_kly} thousand light-years`;
  valDistC.textContent = !sim.clusterEnabled ? "N/A" : `${sim.d_C_Gly.toFixed(1)} billion light-years`;
  valMass.textContent = !sim.clusterEnabled ? "Disabled" : `${sim.M_tril} trillion solar masses`;
  valNumRays.textContent = `${sim.numRays}`;
  valFanSpread.textContent = `${sim.fanSpreadArcsec} arc-seconds`;
  valTheta.textContent = `${sim.thetaInitArcsec >= 0 ? '+' : ''}${sim.thetaInitArcsec.toFixed(1)} arc-sec`

  // Update Numerical Box
  boxDistG.textContent = `${sim.d_G_Gly.toFixed(1)} billion light-years`;
  boxOffsetX.textContent = `${sim.x_G_kly >= 0 ? '+' : ''}${sim.x_G_kly} thousand light-years`;
  boxDistC.textContent = !sim.clusterEnabled ? "N/A" : `${sim.d_C_Gly.toFixed(1)} billion light-years`;
  boxDistGC.textContent = !sim.clusterEnabled ? "N/A" : `${sim.d_GC_Gly.toFixed(1)} billion light-years`;

```

```

boxMass.textContent = !sim.clusterEnabled ? "None" : `${sim.M_tril} trillion solar masses`;

if (sim.numRays === 1) {
  const r = sim.rays[0];
  boxMode.textContent = `Single Ray`;
  boxImpactRange.textContent = sim.clusterEnabled ? `${(r.abs_b_C_ly / 1000.0).toFixed(2)} thousand light`
  boxEarthImpactRange.textContent = `${(r.abs_b_E_ly / 1000.0).toFixed(2)} thousand light-years`;
  boxAlphaRange.textContent = sim.clusterEnabled ? `${r.alpha_arcsec.toFixed(2)} arc-seconds` : "0.00 arc-seconds";
} else {
  boxMode.textContent = `Fan (${sim.numRays} rays)`;

  const bCValues = sim.rays.map(r => r.abs_b_C_ly / 1000.0);
  const bEValues = sim.rays.map(r => r.abs_b_E_ly / 1000.0);
  const alphaValues = sim.rays.map(r => r.alpha_arcsec).filter(a => isFinite(a));

  const minBC = Math.min(...bCValues);
  const maxBC = Math.max(...bCValues);
  const minBE = Math.min(...bEValues);
  const maxBE = Math.max(...bEValues);
  const maxAlpha = Math.max(...alphaValues);

  boxImpactRange.textContent = sim.clusterEnabled ? `${minBC.toFixed(1)} to ${maxBC.toFixed(1)} thousand light-years`
  boxEarthImpactRange.textContent = `${minBE.toFixed(1)} to ${maxBE.toFixed(1)} thousand light-years`;
  boxAlphaRange.textContent = sim.clusterEnabled ? `${maxAlpha.toFixed(2)} arc-seconds max` : "0.00 arc-seconds";
}

// Visual Layout Dimensions (Vertical Field)
const topMargin = 45;
const bottomMargin = 55;
const centerX = width / 2;

const earthY = height - bottomMargin; // Earth at Bottom
const drawableHeight = height - topMargin - bottomMargin;
const pxPerGly = drawableHeight / CANVAS_TOTAL_GLY; // Scale for 13.5 Gly total field

const galaxyY = earthY - (sim.d_G_Gly * pxPerGly); // Distant Galaxy at Top
const clusterY = earthY - (sim.d_C_Gly * pxPerGly); // Intermediate Cluster

const pxPerKly = 100.0 / 500.0;
const galaxyX = centerX + (sim.x_G_kly * pxPerKly);

const ptA = { x: galaxyX, y: galaxyY };

// Starfield Background
if (toggleStars.checked) {
  ctx.fillStyle = "#ffffff";
  stars.forEach(s => {
    ctx.globalAlpha = s.alpha;
    ctx.fillRect(s.x * width, s.y * height, s.size, s.size);
  });
  ctx.globalAlpha = 1.0;
}

// Central Optical Axis
ctx.strokeStyle = "rgba(255, 255, 255, 0.15)";
ctx.lineWidth = 1;
ctx.setLineDash([6, 6]);
ctx.beginPath();
ctx.moveTo(centerX, earthY + 20);
ctx.lineTo(centerX, earthY - CANVAS_TOTAL_GLY * pxPerGly - 15);
ctx.stroke();
ctx.setLineDash([]);

// Cluster Plane Line
if (sim.clusterEnabled && sim.M_tril > 0) {
  ctx.strokeStyle = "rgba(0, 240, 255, 0.25)";
  ctx.beginPath();
  ctx.moveTo(20, clusterY);
  ctx.lineTo(width - 20, clusterY);
  ctx.stroke();
}

// Earth Plane Line
ctx.strokeStyle = "rgba(0, 240, 255, 0.15)";

```

```

ctx.beginPath();
ctx.moveTo(20, earthY);
ctx.lineTo(width - 20, earthY);
ctx.stroke();

// Render Light Rays & Virtual Tangents
sim.rays.forEach((ray, idx) => {
  const clusterX = centerX + (ray.b_C_ly / 1000.0) * pxPerKly;
  const earthX = centerX + (ray.b_E_ly / 1000.0) * pxPerKly;

  const ptB = { x: clusterX, y: clusterY };
  const ptC = { x: earthX, y: earthY };

  // Virtual Tangent Line
  if (toggleTangent.checked && Math.abs(ray.alpha_arcsec) > 0.1 && sim.M_tril > 0) {
    ctx.strokeStyle = "rgba(239, 68, 68, 0.5)";
    ctx.lineWidth = 1.2;
    ctx.setLineDash([3, 3]);

    const slopeX = (ptC.x - ptB.x) / (ptC.y - ptB.y);
    const virtualX_at_galaxy = ptB.x + slopeX * (galaxyY - clusterY);

    ctx.beginPath();
    ctx.moveTo(ptC.x, ptC.y);
    ctx.lineTo(virtualX_at_galaxy, galaxyY);
    ctx.stroke();
    ctx.setLineDash([]);

    ctx.fillStyle = "rgba(239, 68, 68, 0.8)";
    ctx.beginPath();
    ctx.arc(virtualX_at_galaxy, galaxyY, 3, 0, Math.PI * 2);
    ctx.fill();
  }

  // Actual Light Path
  ctx.strokeStyle = "#facc15";
  ctx.lineWidth = sim.numRays === 1 ? 2.5 : 1.8;
  ctx.shadowColor = "#facc15";
  ctx.shadowBlur = sim.numRays === 1 ? 6 : 3;

  ctx.beginPath();
  ctx.moveTo(ptA.x, ptA.y);
  if (sim.M_tril > 0) {
    ctx.lineTo(ptB.x, ptB.y);
  }
  ctx.lineTo(ptC.x, ptC.y);
  ctx.stroke();
  ctx.shadowBlur = 0;

  // Visual Indicators for Impact Parameters
  if (toggleImpact.checked && sim.numRays === 1) {
    // Cluster Impact Parameter
    if (sim.clusterEnabled && sim.M_tril > 0 && Math.abs(ptB.x - centerX) > 3) {
      ctx.strokeStyle = "#00f0ff";
      ctx.lineWidth = 1.2;
      ctx.beginPath();
      ctx.moveTo(centerX, clusterY);
      ctx.lineTo(ptB.x, clusterY);
      ctx.stroke();

      ctx.fillStyle = "#00f0ff";
      ctx.beginPath();
      ctx.arc(ptB.x, clusterY, 2.5, 0, Math.PI * 2);
      ctx.arc(centerX, clusterY, 2.5, 0, Math.PI * 2);
      ctx.fill();

      ctx.font = "10px monospace";
      const clusterLabel = `${(ray.abs_b_C_ly / 1000).toFixed(1)} thousand light-years`;
      const textWidth = ctx.measureText(clusterLabel).width;
      const charOffset = 18; // ~2-3 character offset from bend

      let labelX = 0;
      let align = "center";

```

```

    if (ray.theta_init_arcsec < 0) {
      // Initial angle is negative: left aligned to 2-3 chars to the right side of the bend
      labelX = ptB.x + charOffset;
      align = "left";
      if (labelX + textWidth > width - 10) {
        labelX = width - 10 - textWidth;
      }
      if (labelX < 10) {
        labelX = 10;
      }
    } else {
      // Initial angle is positive: right aligned to 2-3 chars to the left side of the bend
      labelX = ptB.x - charOffset;
      align = "right";
      if (labelX - textWidth < 10) {
        labelX = 10 + textWidth;
      }
      if (labelX > width - 10) {
        labelX = width - 10;
      }
    }

    ctx.textAlign = align;
    ctx.fillText(clusterLabel, labelX, clusterY - 6);
  }

  // Earth Impact Parameter
  if (Math.abs(ptC.x - centerX) > 3) {
    ctx.strokeStyle = "#00f0ff";
    ctx.lineWidth = 1.2;
    ctx.beginPath();
    ctx.moveTo(centerX, earthY);
    ctx.lineTo(ptC.x, earthY);
    ctx.stroke();

    ctx.fillStyle = "#00f0ff";
    ctx.beginPath();
    ctx.arc(ptC.x, earthY, 2.5, 0, Math.PI * 2);
    ctx.arc(centerX, earthY, 2.5, 0, Math.PI * 2);
    ctx.fill();

    ctx.font = "10px monospace";
    const earthLabel = `${(ray.abs_b_E_ly / 1000).toFixed(1)} thousand light-years`;
    const earthTextWidth = ctx.measureText(earthLabel).width;
    let earthTextX = (centerX + ptC.x) / 2;

    if (earthTextX - earthTextWidth / 2 < 10) {
      earthTextX = 10 + earthTextWidth / 2;
    } else if (earthTextX + earthTextWidth / 2 > width - 10) {
      earthTextX = width - 10 - earthTextWidth / 2;
    }

    ctx.textAlign = "center";
    ctx.fillText(earthLabel, earthTextX, earthY + 16);
  }
}

// Animated Photons (Constant Speed Along Physical Path)
if (togglePhoton.checked) {
  const rayOffset = (idx / sim.numRays);
  const progress = (photonAnimProgress + rayOffset) % 1.0;
  let px = 0, py = 0;

  if (sim.M_tril === 0) {
    px = ptA.x + progress * (ptC.x - ptA.x);
    py = ptA.y + progress * (ptC.y - ptA.y);
  } else {
    const d1 = Math.hypot(ptB.x - ptA.x, ptB.y - ptA.y);
    const d2 = Math.hypot(ptC.x - ptB.x, ptC.y - ptB.y);
    const dTotal = d1 + d2;
    const f1 = dTotal > 0 ? d1 / dTotal : 0.5;

    if (progress <= f1) {
      const t = f1 > 0 ? progress / f1 : 0;

```

```

        px = ptA.x + t * (ptB.x - ptA.x);
        py = ptA.y + t * (ptB.y - ptA.y);
    } else {
        const t = (1 - f1) > 0 ? (progress - f1) / (1 - f1) : 0;
        px = ptB.x + t * (ptC.x - ptB.x);
        py = ptB.y + t * (ptC.y - ptB.y);
    }
}

ctx.fillStyle = "#ffffff";
ctx.shadowColor = "#00f0ff";
ctx.shadowBlur = 6;
ctx.beginPath();
ctx.arc(px, py, 3, 0, Math.PI * 2);
ctx.fill();
ctx.shadowBlur = 0;
}
});

if (togglePhoton.checked) {
    photonAnimProgress = (photonAnimProgress + 0.006) % 1.0;
}

// Draw Celestial Objects

// 1. Distant Galaxy (Top)
ctx.save();
ctx.translate(ptA.x, ptA.y);
const galGlow = ctx.createRadialGradient(0, 0, 2, 0, 0, 18);
galGlow.addColorStop(0, "rgba(255, 200, 100, 0.9)");
galGlow.addColorStop(0.5, "rgba(238, 100, 30, 0.3)");
galGlow.addColorStop(1, "rgba(0, 0, 0, 0)");
ctx.fillStyle = galGlow;
ctx.beginPath();
ctx.arc(0, 0, 18, 0, Math.PI * 2);
ctx.fill();

ctx.fillStyle = "#ffd59e";
ctx.beginPath();
ctx.ellipse(0, 0, 12, 4, -0.3, 0, Math.PI * 2);
ctx.fill();
ctx.restore();

ctx.fillStyle = "#e1e7ec";
ctx.font = "11px sans-serif";
ctx.textAlign = "center";
ctx.fillText("Distant Galaxy", ptA.x, ptA.y - 15);

// 2. Intermediate Cluster (Middle - rendered with mass-scaling galaxies & rich purple halo)
if (sim.clusterEnabled && sim.M_tril > 0) {
    ctx.save();
    ctx.translate(centerX, clusterY);

    // Normalized mass ratio (0.0 to 1.0 across 100 to 1000 trillion solar masses)
    const massRatio = Math.max(0, Math.min(1, (sim.M_tril - 100) / 900));

    // Halo radius scales prominently from 16px to 62px
    const haloRadius = 16 + massRatio * 46;

    // Multi-stage radial gradient for increasingly vivid purple dark-matter halo
    const clusterGlow = ctx.createRadialGradient(0, 0, 2, 0, 0, haloRadius);
    const alphaCenter = 0.5 + massRatio * 0.45;
    const alphaMid = 0.25 + massRatio * 0.55;

    clusterGlow.addColorStop(0, `rgba(216, 180, 254, ${alphaCenter})`);
    clusterGlow.addColorStop(0.35, `rgba(168, 85, 247, ${alphaMid})`);
    clusterGlow.addColorStop(0.7, `rgba(126, 34, 206, ${alphaMid * 0.5})`);
    clusterGlow.addColorStop(1, "rgba(0, 0, 0, 0)");

    ctx.fillStyle = clusterGlow;
    ctx.beginPath();
    ctx.arc(0, 0, haloRadius, 0, Math.PI * 2);
    ctx.fill();

```

```

// Dynamically add member galaxies as mass increases (3 at min mass up to 30 at max mass)
const numGalaxies = Math.floor(3 + massRatio * 27);
const spreadScale = 0.6 + massRatio * 0.6;

for (let i = 0; i < numGalaxies; i++) {
  const g = clusterGalaxyList[i];
  const gx = g.x * spreadScale;
  const gy = g.y * spreadScale;

  // Core bright galaxy vs softer peripheral galaxies
  ctx.fillStyle = i === 0 ? "#ffffff" : `rgba(255, 245, 255, ${0.75 + (1 - i / 30) * 0.25})`;
  ctx.beginPath();
  ctx.arc(gx, gy, g.r, 0, Math.PI * 2);
  ctx.fill();

  // Secondary subtle purple glow around individual member galaxies
  if (g.r >= 2.0) {
    ctx.fillStyle = `rgba(192, 132, 252, ${0.3 + massRatio * 0.3})`;
    ctx.beginPath();
    ctx.arc(gx, gy, g.r * 1.8, 0, Math.PI * 2);
    ctx.fill();
  }
}

ctx.restore();

ctx.fillStyle = "#e1e7ec";
ctx.font = "11px sans-serif";
ctx.textAlign = "center";
ctx.fillText("Intermediate Cluster", centerX, clusterY + haloRadius + 14);
}

// 3. Earth / Observer (Bottom)
ctx.save();
ctx.translate(centerX, earthY);
const earthGrad = ctx.createRadialGradient(-2, -2, 1, 0, 0, 8);
earthGrad.addColorStop(0, "#60a5fa");
earthGrad.addColorStop(0.8, "#1d4ed8");
earthGrad.addColorStop(1, "#1e3a8a");
ctx.fillStyle = earthGrad;
ctx.beginPath();
ctx.arc(0, 0, 8, 0, Math.PI * 2);
ctx.fill();
ctx.restore();

ctx.fillStyle = "#e1e7ec";
ctx.font = "11px sans-serif";
ctx.textAlign = "center";
ctx.fillText("Earth", centerX, earthY + 26);

// Bottom Note inside Graphic Window
ctx.fillStyle = "rgba(140, 155, 174, 0.75)";
ctx.font = "italic 11px sans-serif";
ctx.textAlign = "center";
ctx.fillText("angles are exaggerated", centerX, height - 12);
}

function animate() {
  draw();
  requestAnimationFrame(animate);
}

animate();
</script>
</body>
</html>

```

User prompt: Let's try a version that adds a full circle of rays coming from the distant galaxies, for illustrative purposes only. These will be in addition to the rays used with the lensing. Draw them thinner, with lower opacity, and start them at the edge of the current fan, using the same angular spacing as the fan.


```

Response: <!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Gravitational Lensing: Vertical Field Simulation</title>
  <style>
    :root {
      --bg-color: #0b0e14;
      --panel-bg: #151922;
      --panel-border: #2a3142;
      --accent-color: #00f0ff;
      --accent-amber: #facc15;
      --accent-red: #ef4444;
      --text-main: #e1e7ec;
      --text-muted: #8c9bae;
    }

    * {
      box-sizing: border-box;
      margin: 0;
      padding: 0;
    }

    body {
      font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial, sans-serif;
      background-color: var(--bg-color);
      color: var(--text-main);
      display: flex;
      flex-direction: column;
      min-height: 100vh;
      overflow-x: hidden;
    }

    header {
      background-color: var(--panel-bg);
      border-bottom: 1px solid var(--panel-border);
      padding: 0.85rem 1.5rem;
      text-align: center;
    }

    header h1 {
      font-size: 1.3rem;
      font-weight: 600;
      color: #fff;
    }

    header p {
      font-size: 0.85rem;
      color: var(--text-muted);
      margin-top: 0.2rem;
    }

    .main-container {
      display: flex;
      flex: 1;
      flex-direction: column;
      gap: 1rem;
      padding: 1rem;
      max-width: 1200px;
      margin: 0 auto;
      width: 100%;
      align-items: center;
    }

    @media (min-width: 1024px) {
      .main-container {
        flex-direction: row;
        align-items: flex-start;
      }
    }

    .canvas-card {
      flex: 1;

```

```

width: 100%;
background-color: var(--panel-bg);
border: 1px solid var(--panel-border);
border-radius: 8px;
padding: 1rem;
display: flex;
flex-direction: column;
gap: 1rem;
box-shadow: 0 4px 20px rgba(0, 0, 0, 0.4);
}

.canvas-wrapper {
  position: relative;
  width: 100%;
  max-width: 520px;
  aspect-ratio: 3 / 4; /* Vertical portrait profile (4:3 height to width) */
  margin: 0 auto;
  background-color: #04060a;
  border-radius: 6px;
  overflow: hidden;
  border: 1px solid rgba(255, 255, 255, 0.05);
}

canvas {
  width: 100%;
  height: 100%;
  display: block;
}

.calc-box {
  background: rgba(10, 14, 22, 0.7);
  border: 1px solid var(--panel-border);
  border-radius: 6px;
  padding: 1rem 1.25rem;
  font-family: SFMono-Regular, Consolas, "Liberation Mono", Menlo, monospace;
  font-size: 0.8rem;
  color: #d1d5db;
}

.calc-box h3 {
  font-size: 0.85rem;
  text-transform: uppercase;
  letter-spacing: 0.05em;
  color: var(--accent-color);
  margin-bottom: 0.75rem;
  border-bottom: 1px solid rgba(255, 255, 255, 0.1);
  padding-bottom: 0.4rem;
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;
}

.calc-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
  gap: 0.85rem 1.25rem;
}

.calc-item {
  display: flex;
  flex-direction: column;
  gap: 0.25rem;
}

.calc-item .label {
  color: var(--text-muted);
  font-size: 0.75rem;
}

.calc-item .value {
  color: #fff;
  font-weight: 600;
  font-size: 0.85rem;
}

.calc-item .highlight {

```

```
    color: var(--accent-amber);
}

.calc-footer {
  margin-top: 0.85rem;
  padding-top: 0.6rem;
  border-top: 1px dashed rgba(255, 255, 255, 0.15);
  display: flex;
  justify-content: space-between;
  align-items: center;
  font-size: 0.75rem;
}

.calc-footer .formula {
  color: #9ca3af;
}

.controls-panel {
  width: 100%;
  max-width: 380px;
  background-color: var(--panel-bg);
  border: 1px solid var(--panel-border);
  border-radius: 8px;
  padding: 1.25rem;
  display: flex;
  flex-direction: column;
  gap: 1.1rem;
  box-shadow: 0 4px 20px rgba(0, 0, 0, 0.4);
}

.control-group {
  display: flex;
  flex-direction: column;
  gap: 0.4rem;
  transition: opacity 0.2s ease;
}

.control-group.disabled-control {
  opacity: 0.35;
}

.control-group label {
  font-size: 0.83rem;
  font-weight: 500;
  color: var(--text-main);
  display: flex;
  justify-content: space-between;
}

.control-group label span {
  color: var(--accent-color);
  font-family: monospace;
}

input[type="range"] {
  -webkit-appearance: none;
  width: 100%;
  height: 6px;
  border-radius: 3px;
  background: #2a3142;
  outline: none;
}

input[type="range"]:disabled {
  cursor: not-allowed;
}

input[type="range"]::-webkit-slider-thumb {
  -webkit-appearance: none;
  width: 16px;
  height: 16px;
  border-radius: 50%;
  background: var(--accent-color);
  cursor: pointer;
}
```

```

    box-shadow: 0 0 8px var(--accent-color);
    transition: transform 0.1s ease;
}

input[type="range"]:disabled::-webkit-slider-thumb {
    background: #6b7280;
    box-shadow: none;
    cursor: not-allowed;
}

input[type="range"]::-webkit-slider-thumb:hover:not(:disabled) {
    transform: scale(1.2);
}

.toggle-row {
    display: flex;
    align-items: center;
    justify-content: space-between;
    font-size: 0.83rem;
    color: var(--text-main);
}

.switch {
    position: relative;
    display: inline-block;
    width: 40px;
    height: 20px;
}

.switch input {
    opacity: 0;
    width: 0;
    height: 0;
}

.slider {
    position: absolute;
    cursor: pointer;
    top: 0; left: 0; right: 0; bottom: 0;
    background-color: #2a3142;
    transition: .2s;
    border-radius: 20px;
}

.slider:before {
    position: absolute;
    content: "";
    height: 14px;
    width: 14px;
    left: 3px;
    bottom: 3px;
    background-color: white;
    transition: .2s;
    border-radius: 50%;
}

input:checked + .slider {
    background-color: var(--accent-color);
}

input:checked + .slider:before {
    transform: translateX(20px);
}

.legend {
    border-top: 1px solid var(--panel-border);
    padding-top: 0.8rem;
    display: flex;
    flex-direction: column;
    gap: 0.4rem;
    font-size: 0.8rem;
}

.legend-item {

```

```

display: flex;
align-items: center;
gap: 0.6rem;
color: var(--text-muted);
}

.legend-color {
width: 14px;
height: 4px;
border-radius: 2px;
}
</style>
</head>
<body>

<header>
<h1>Gravitational Deflection of Light Fan</h1>
<p>Vertical Propagation View (Distant Galaxy at Top, Earth at Bottom)</p>
</header>

<div class="main-container">
<div class="canvas-card">
<div class="canvas-wrapper">
<canvas id="simCanvas"></canvas>
</div>

<!-- Numerical Output Box below Graphic -->
<div class="calc-box">
<h3>Numerical Results</h3>
<div class="calc-grid">
<div class="calc-item">
<span class="label">Galaxy Distance:</span>
<span class="value" id="boxDistG">8.0 billion light-years</span>
</div>
<div class="calc-item">
<span class="label">Galaxy Offset:</span>
<span class="value" id="boxOffsetX">0 thousand light-years</span>
</div>
<div class="calc-item">
<span class="label">Cluster Distance:</span>
<span class="value" id="boxDistC">4.0 billion light-years</span>
</div>
<div class="calc-item">
<span class="label">Galaxy-Cluster Distance:</span>
<span class="value" id="boxDistGC">4.0 billion light-years</span>
</div>
<div class="calc-item">
<span class="label">Cluster Mass:</span>
<span class="value" id="boxMass">200 trillion solar masses</span>
</div>
<div class="calc-item">
<span class="label">Ray Mode:</span>
<span class="value" id="boxMode">Single Ray</span>
</div>
<div class="calc-item">
<span class="label">Cluster Impact:</span>
<span class="value highlight" id="boxImpactRange">290.9 thousand light-years</span>
</div>
<div class="calc-item">
<span class="label">Earth Impact:</span>
<span class="value highlight" id="boxEarthImpactRange">581.8 thousand light-years</span>
</div>
<div class="calc-item">
<span class="label">Deflection Angle:</span>
<span class="value highlight" id="boxAlphaRange">44.3 arc-seconds</span>
</div>
</div>
<div class="calc-footer">
<span class="label">Theoretical Formula:</span>
<span class="formula"> $\alpha = 4GM / (c^2 |b_{\text{C}}|) \quad \&nbps; \&nbps; b_{\text{E}} = b_{\text{C}} + d$ </span>
</div>
</div>
</div>

```

```

<div class="controls-panel">
  <div class="toggle-row">
    <span>Enable Intermediate Cluster</span>
    <label class="switch">
      <input type="checkbox" id="toggleCluster" checked>
      <span class="slider"></span>
    </label>
  </div>

  <div class="control-group">
    <label for="sliderDistG">
      1. Distance to Galaxy:
      <span id="valDistG">8.0 billion light-years</span>
    </label>
    <input type="range" id="sliderDistG" min="2.0" max="13.0" step="0.1" value="8.0">
  </div>

  <div class="control-group">
    <label for="sliderOffsetX">
      2. Galaxy Horizontal Offset:
      <span id="valOffsetX">0 thousand light-years</span>
    </label>
    <input type="range" id="sliderOffsetX" min="-500" max="500" step="10" value="0">
  </div>

  <div class="control-group" id="grpDistC">
    <label for="sliderDistC">
      3. Distance to Cluster:
      <span id="valDistC">4.0 billion light-years</span>
    </label>
    <input type="range" id="sliderDistC" min="1.0" max="12.0" step="0.1" value="4.0">
  </div>

  <div class="control-group" id="grpMass">
    <label for="sliderMass">
      4. Cluster Mass:
      <span id="valMass">200 trillion solar masses</span>
    </label>
    <input type="range" id="sliderMass" min="100" max="1000" step="10" value="200">
  </div>

  <div class="control-group">
    <label for="sliderNumRays">
      5. Number of Light Rays:
      <span id="valNumRays">1</span>
    </label>
    <input type="range" id="sliderNumRays" min="1" max="10" step="1" value="1">
  </div>

  <div class="control-group" id="grpFanSpread">
    <label for="sliderFanSpread">
      6. Ray Fan Spread:
      <span id="valFanSpread">20 arc-seconds</span>
    </label>
    <input type="range" id="sliderFanSpread" min="10" max="40" step="1" value="20">
  </div>

  <div class="control-group" id="grpTheta">
    <label for="sliderTheta">
      7. Initial Ray Direction (1 ray):
      <span id="valTheta">+15.0 arc-seconds</span>
    </label>
    <input type="range" id="sliderTheta" min="-40.0" max="40.0" step="0.5" value="15.0">
  </div>

  <div class="toggle-row">
    <span>Animate Photons</span>
    <label class="switch">
      <input type="checkbox" id="togglePhoton" checked>
      <span class="slider"></span>
    </label>
  </div>

  <div class="toggle-row">

```

```

    <span>Show Virtual Tangent Lines</span>
    <label class="switch">
      <input type="checkbox" id="toggleTangent" checked>
      <span class="slider"></span>
    </label>
  </div>

  <div class="toggle-row">
    <span>Show Impact Parameters</span>
    <label class="switch">
      <input type="checkbox" id="toggleImpact" checked>
      <span class="slider"></span>
    </label>
  </div>

  <div class="toggle-row">
    <span>Show Background Stars</span>
    <label class="switch">
      <input type="checkbox" id="toggleStars" checked>
      <span class="slider"></span>
    </label>
  </div>

  <div class="toggle-row">
    <span>Show Full Circle Illustrative Rays</span>
    <label class="switch">
      <input type="checkbox" id="toggleFullCircleRays" checked>
      <span class="slider"></span>
    </label>
  </div>

  <div class="legend">
    <div class="legend-item">
      <div class="legend-color" style="background: #facc15;"></div>
      <span>Actual deflected light rays</span>
    </div>
    <div class="legend-item">
      <div class="legend-color" style="background: rgba(250, 204, 21, 0.35);"></div>
      <span>Illustrative full-circle emission rays</span>
    </div>
    <div class="legend-item">
      <div class="legend-color" style="border: 1px dashed #ef4444; background: transparent;"></div>
      <span>Apparent straight-line paths (virtual images)</span>
    </div>
    <div class="legend-item">
      <div class="legend-color" style="background: #00f0ff;"></div>
      <span>Cluster impact parameter & Earth impact</span>
    </div>
  </div>
</div>
</div>

<script>
const canvas = document.getElementById('simCanvas');
const ctx = canvas.getContext('2d');

// Canvas Fixed Total Height Scale (13.5 Billion Light-Years)
const CANVAS_TOTAL_GLY = 13.5;

// Controls
const toggleCluster = document.getElementById('toggleCluster');
const sliderDistG = document.getElementById('sliderDistG');
const sliderOffsetX = document.getElementById('sliderOffsetX');
const sliderDistC = document.getElementById('sliderDistC');
const sliderMass = document.getElementById('sliderMass');
const sliderNumRays = document.getElementById('sliderNumRays');
const sliderFanSpread = document.getElementById('sliderFanSpread');
const sliderTheta = document.getElementById('sliderTheta');

const grpDistC = document.getElementById('grpDistC');
const grpMass = document.getElementById('grpMass');
const grpFanSpread = document.getElementById('grpFanSpread');
const grpTheta = document.getElementById('grpTheta');

```

```

const valDistG = document.getElementById('valDistG');
const valOffsetX = document.getElementById('valOffsetX');
const valDistC = document.getElementById('valDistC');
const valMass = document.getElementById('valMass');
const valNumRays = document.getElementById('valNumRays');
const valFanSpread = document.getElementById('valFanSpread');
const valTheta = document.getElementById('valTheta');

const togglePhoton = document.getElementById('togglePhoton');
const toggleTangent = document.getElementById('toggleTangent');
const toggleImpact = document.getElementById('toggleImpact');
const toggleStars = document.getElementById('toggleStars');
const toggleFullCircleRays = document.getElementById('toggleFullCircleRays');

// Calculation Box Elements
const boxDistG = document.getElementById('boxDistG');
const boxOffsetX = document.getElementById('boxOffsetX');
const boxDistC = document.getElementById('boxDistC');
const boxDistGC = document.getElementById('boxDistGC');
const boxMass = document.getElementById('boxMass');
const boxMode = document.getElementById('boxMode');
const boxImpactRange = document.getElementById('boxImpactRange');
const boxEarthImpactRange = document.getElementById('boxEarthImpactRange');
const boxAlphaRange = document.getElementById('boxAlphaRange');

let width = 0;
let height = 0;
let photonAnimProgress = 0;

// Background Stars
const stars = [];
for (let i = 0; i < 220; i++) {
  stars.push({
    x: Math.random(),
    y: Math.random(),
    size: Math.random() * 1.5 + 0.3,
    alpha: Math.random() * 0.7 + 0.3
  });
}

// Cluster Member Galaxies layout offsets (up to 30 member galaxies)
const clusterGalaxyList = [
  { x: 0, y: 0, r: 3.5 },
  { x: -6, y: -5, r: 2.2 },
  { x: 8, y: 4, r: 2.5 },
  { x: -12, y: 8, r: 2.0 },
  { x: 10, y: -10, r: 2.1 },
  { x: 5, y: -14, r: 1.8 },
  { x: -15, y: -6, r: 2.3 },
  { x: 18, y: -2, r: 1.9 },
  { x: -8, y: 16, r: 2.2 },
  { x: 14, y: 12, r: 1.7 },
  { x: 2, y: 18, r: 2.0 },
  { x: -20, y: 4, r: 1.8 },
  { x: -4, y: -20, r: 1.9 },
  { x: 22, y: -12, r: 1.6 },
  { x: 12, y: -22, r: 1.8 },
  { x: -22, y: -15, r: 1.7 },
  { x: -16, y: 22, r: 1.9 },
  { x: 25, y: 8, r: 1.5 },
  { x: 6, y: 26, r: 1.7 },
  { x: -28, y: -4, r: 1.6 },
  { x: 18, y: 22, r: 1.5 },
  { x: -10, y: -28, r: 1.8 },
  { x: 28, y: -20, r: 1.4 },
  { x: -25, y: 18, r: 1.6 },
  { x: 0, y: -32, r: 1.5 },
  { x: 32, y: 2, r: 1.4 },
  { x: -32, y: -18, r: 1.5 },
  { x: 20, y: -30, r: 1.4 },
  { x: -20, y: -32, r: 1.3 },
  { x: 15, y: 32, r: 1.4 }
];

```



```

function resizeCanvas() {
  const rect = canvas.parentElement.getBoundingClientRect();
  const dpr = window.devicePixelRatio || 1;
  width = rect.width;
  height = rect.height;

  canvas.width = width * dpr;
  canvas.height = height * dpr;
  ctx.scale(dpr, dpr);
}

window.addEventListener('resize', resizeCanvas);
resizeCanvas();

// Distance constraints:  $d_G \geq d_C + 1.0$  Gly
sliderDistG.addEventListener('input', () => {
  let dG = parseFloat(sliderDistG.value);
  let dC = parseFloat(sliderDistC.value);
  if (dG < dC + 1.0) {
    dC = Math.max(1.0, dG - 1.0);
    sliderDistC.value = dC.toFixed(1);
  }
  updateControlStates();
});

sliderDistC.addEventListener('input', () => {
  let dG = parseFloat(sliderDistG.value);
  let dC = parseFloat(sliderDistC.value);
  if (dC > dG - 1.0) {
    dG = Math.min(13.0, dC + 1.0);
    sliderDistG.value = dG.toFixed(1);
  }
  updateControlStates();
});

toggleCluster.addEventListener('change', updateControlStates);
sliderMass.addEventListener('input', updateControlStates);
sliderNumRays.addEventListener('input', updateControlStates);

function updateControlStates() {
  const numRays = parseInt(sliderNumRays.value);
  const clusterEnabled = toggleCluster.checked;

  if (!clusterEnabled) {
    sliderMass.disabled = true;
    grpMass.classList.add('disabled-control');
    sliderDistC.disabled = true;
    grpDistC.classList.add('disabled-control');
  } else {
    sliderMass.disabled = false;
    grpMass.classList.remove('disabled-control');
    sliderDistC.disabled = false;
    grpDistC.classList.remove('disabled-control');
  }

  if (numRays > 1) {
    sliderTheta.disabled = true;
    grpTheta.classList.add('disabled-control');

    sliderFanSpread.disabled = false;
    grpFanSpread.classList.remove('disabled-control');
  } else {
    sliderTheta.disabled = false;
    grpTheta.classList.remove('disabled-control');

    sliderFanSpread.disabled = true;
    grpFanSpread.classList.add('disabled-control');
  }
}

updateControlStates();

function calculateRays() {
  const clusterEnabled = toggleCluster.checked;

```

```

const d_G_Gly = parseFloat(sliderDistG.value);
const x_G_kly = parseFloat(sliderOffsetX.value);
const d_C_Gly = parseFloat(sliderDistC.value);
const d_GC_Gly = d_G_Gly - d_C_Gly;
const M_tril = clusterEnabled ? parseFloat(sliderMass.value) : 0;
const numRays = parseInt(sliderNumRays.value);
const fanSpreadArcsec = parseFloat(sliderFanSpread.value);
const thetaInitArcsec = parseFloat(sliderTheta.value);

const ARCSEC_TO_RAD = Math.PI / (180.0 * 3600.0);
const d_GC_ly = d_GC_Gly * 1e9;
const d_C_ly = d_C_Gly * 1e9;
const x_G_ly = x_G_kly * 1e3;

const scale_4GM_c2_ly = 0.6243363 * M_tril;

const rays = [];

let stepArcsec = 5.0; // default spacing for single ray mode
let minFanAngle = thetaInitArcsec;
let maxFanAngle = thetaInitArcsec;

if (numRays > 1) {
  stepArcsec = fanSpreadArcsec / (numRays - 1);
  minFanAngle = -0.5 * fanSpreadArcsec;
  maxFanAngle = 0.5 * fanSpreadArcsec;
}

for (let i = 0; i < numRays; i++) {
  let initialAngle = 0;

  if (numRays === 1) {
    initialAngle = thetaInitArcsec;
  } else {
    const frac = (i / (numRays - 1)) - 0.5;
    initialAngle = frac * fanSpreadArcsec;
  }

  const theta_init_rad = initialAngle * ARCSEC_TO_RAD;

  const b_C_ly = x_G_ly + d_GC_ly * Math.tan(theta_init_rad);
  const abs_b_C_ly = Math.abs(b_C_ly);

  let alpha_arcsec = 0;
  let alpha_rad = 0;
  if (M_tril > 0 && abs_b_C_ly > 1e-3) {
    alpha_rad = scale_4GM_c2_ly / abs_b_C_ly;
    alpha_arcsec = alpha_rad / ARCSEC_TO_RAD;
  }

  let theta_after_rad = theta_init_rad;
  if (M_tril > 0) {
    if (b_C_ly > 0) {
      theta_after_rad = theta_init_rad - alpha_rad;
    } else if (b_C_ly < 0) {
      theta_after_rad = theta_init_rad + alpha_rad;
    }
  }
  const theta_after_arcsec = theta_after_rad / ARCSEC_TO_RAD;

  const b_E_ly = b_C_ly + d_C_ly * Math.tan(theta_after_rad);
  const abs_b_E_ly = Math.abs(b_E_ly);

  rays.push({
    index: i,
    theta_init_arcsec: initialAngle,
    b_C_ly: b_C_ly,
    abs_b_C_ly: abs_b_C_ly,
    b_E_ly: b_E_ly,
    abs_b_E_ly: abs_b_E_ly,
    alpha_arcsec: alpha_arcsec,
    theta_after_arcsec: theta_after_arcsec
  });
}

```

```

// Generate illustrative rays for the rest of the 360-degree circle
const stepRad = stepArcsec * ARCSEC_TO_RAD;
const numTotalSteps = Math.floor((2 * Math.PI) / stepRad);

const illustrativeAnglesRad = [];
const fanMinRad = (minFanAngle * ARCSEC_TO_RAD);
const fanMaxRad = (maxFanAngle * ARCSEC_TO_RAD);

// Downward direction (towards Earth) is angle = 0 in our relative angle space
for (let k = 0; k < numTotalSteps; k++) {
  let phi = k * stepRad;
  if (phi > Math.PI) phi -= 2 * Math.PI;

  // Convert relative angle phi to angular difference from downward direction
  // Downward vertical propagation corresponds to phi in [-PI, PI]
  // Filter out angles that fall within the active lensing fan range around downward (phi = 0)
  const isInsideFan = (numRays > 1) ? (phi >= fanMinRad - 1e-6 && phi <= fanMaxRad + 1e-6)
    : (Math.abs(phi - (thetaInitArcsec * ARCSEC_TO_RAD)) < 1e-6);

  if (!isInsideFan) {
    illustrativeAnglesRad.push(phi);
  }
}

return {
  clusterEnabled,
  d_G_Gly,
  x_G_kly,
  d_C_Gly,
  d_GC_Gly,
  M_tril,
  numRays,
  fanSpreadArcsec,
  thetaInitArcsec,
  rays,
  illustrativeAnglesRad
};
}

function draw() {
  ctx.clearRect(0, 0, width, height);

  const sim = calculateRays();

  // Label Updates
  valDistG.textContent = `${sim.d_G_Gly.toFixed(1)} billion light-years`;
  valOffsetX.textContent = `${sim.x_G_kly >= 0 ? '+' : ''}${sim.x_G_kly} thousand light-years`;
  valDistC.textContent = !sim.clusterEnabled ? "N/A" : `${sim.d_C_Gly.toFixed(1)} billion light-years`;
  valMass.textContent = !sim.clusterEnabled ? "Disabled" : `${sim.M_tril} trillion solar masses`;
  valNumRays.textContent = `${sim.numRays}`;
  valFanSpread.textContent = `${sim.fanSpreadArcsec} arc-seconds`;
  valTheta.textContent = `${sim.thetaInitArcsec >= 0 ? '+' : ''}${sim.thetaInitArcsec.toFixed(1)} arc-sec`

  // Update Numerical Box
  boxDistG.textContent = `${sim.d_G_Gly.toFixed(1)} billion light-years`;
  boxOffsetX.textContent = `${sim.x_G_kly >= 0 ? '+' : ''}${sim.x_G_kly} thousand light-years`;
  boxDistC.textContent = !sim.clusterEnabled ? "N/A" : `${sim.d_C_Gly.toFixed(1)} billion light-years`;
  boxDistGC.textContent = !sim.clusterEnabled ? "N/A" : `${sim.d_GC_Gly.toFixed(1)} billion light-years`;
  boxMass.textContent = !sim.clusterEnabled ? "None" : `${sim.M_tril} trillion solar masses`;

  if (sim.numRays === 1) {
    const r = sim.rays[0];
    boxMode.textContent = `Single Ray`;
    boxImpactRange.textContent = sim.clusterEnabled ? `${(r.abs_b_C_ly / 1000.0).toFixed(2)} thousand light-years` : `0.00 arc-seconds`;
    boxEarthImpactRange.textContent = `${(r.abs_b_E_ly / 1000.0).toFixed(2)} thousand light-years`;
    boxAlphaRange.textContent = sim.clusterEnabled ? `${r.alpha_arcsec.toFixed(2)} arc-seconds` : `0.00 arc-seconds`;
  } else {
    boxMode.textContent = `Fan (${sim.numRays} rays)`;

    const bCValues = sim.rays.map(r => r.abs_b_C_ly / 1000.0);
    const bEValues = sim.rays.map(r => r.abs_b_E_ly / 1000.0);
    const alphaValues = sim.rays.map(r => r.alpha_arcsec).filter(a => isFinite(a));
  }
}

```

```

const minBC = Math.min(...bCValues);
const maxBC = Math.max(...bCValues);
const minBE = Math.min(...bEValues);
const maxBE = Math.max(...bEValues);
const maxAlpha = Math.max(...alphaValues);

boxImpactRange.textContent = sim.clusterEnabled ? `${minBC.toFixed(1)} to ${maxBC.toFixed(1)} thousand`
boxEarthImpactRange.textContent = `${minBE.toFixed(1)} to ${maxBE.toFixed(1)} thousand light-years`;
boxAlphaRange.textContent = sim.clusterEnabled ? `${maxAlpha.toFixed(2)} arc-seconds max` : "0.00 arc-seconds";
}

// Visual Layout Dimensions (Vertical Field)
const topMargin = 45;
const bottomMargin = 55;
const centerX = width / 2;

const earthY = height - bottomMargin; // Earth at Bottom
const drawableHeight = height - topMargin - bottomMargin;
const pxPerGly = drawableHeight / CANVAS_TOTAL_GLY; // Scale for 13.5 Gly total field

const galaxyY = earthY - (sim.d_G_Gly * pxPerGly); // Distant Galaxy at Top
const clusterY = earthY - (sim.d_C_Gly * pxPerGly); // Intermediate Cluster

const pxPerKly = 100.0 / 500.0;
const galaxyX = centerX + (sim.x_G_kly * pxPerKly);

const ptA = { x: galaxyX, y: galaxyY };

// Starfield Background
if (toggleStars.checked) {
  ctx.fillStyle = "#ffffff";
  stars.forEach(s => {
    ctx.globalAlpha = s.alpha;
    ctx.fillRect(s.x * width, s.y * height, s.size, s.size);
  });
  ctx.globalAlpha = 1.0;
}

// Full Circle Illustrative Emission Rays (Thinner, Lower Opacity)
if (toggleFullCircleRays.checked) {
  ctx.strokeStyle = "rgba(250, 204, 21, 0.28)";
  ctx.lineWidth = 0.75;
  const rayLength = Math.max(width, height) * 1.2;

  sim.illustrativeAnglesRad.forEach(phi => {
    // phi = 0 points straight down (+Y axis)
    const dirX = Math.sin(phi);
    const dirY = Math.cos(phi);

    ctx.beginPath();
    ctx.moveTo(ptA.x, ptA.y);
    ctx.lineTo(ptA.x + dirX * rayLength, ptA.y + dirY * rayLength);
    ctx.stroke();
  });
}

// Central Optical Axis
ctx.strokeStyle = "rgba(255, 255, 255, 0.15)";
ctx.lineWidth = 1;
ctx.setLineDash([6, 6]);
ctx.beginPath();
ctx.moveTo(centerX, earthY + 20);
ctx.lineTo(centerX, earthY - CANVAS_TOTAL_GLY * pxPerGly - 15);
ctx.stroke();
ctx.setLineDash([]);

// Cluster Plane Line
if (sim.clusterEnabled && sim.M_tril > 0) {
  ctx.strokeStyle = "rgba(0, 240, 255, 0.25)";
  ctx.beginPath();
  ctx.moveTo(20, clusterY);
  ctx.lineTo(width - 20, clusterY);
  ctx.stroke();
}

```

```

// Earth Plane Line
ctx.strokeStyle = "rgba(0, 240, 255, 0.15)";
ctx.beginPath();
ctx.moveTo(20, earthY);
ctx.lineTo(width - 20, earthY);
ctx.stroke();

// Render Primary Light Rays & Virtual Tangents
sim.rays.forEach((ray, idx) => {
  const clusterX = centerX + (ray.b_C_ly / 1000.0) * pxPerKly;
  const earthX = centerX + (ray.b_E_ly / 1000.0) * pxPerKly;

  const ptB = { x: clusterX, y: clusterY };
  const ptC = { x: earthX, y: earthY };

  // Virtual Tangent Line
  if (toggleTangent.checked && Math.abs(ray.alpha_arcsec) > 0.1 && sim.M_tril > 0) {
    ctx.strokeStyle = "rgba(239, 68, 68, 0.5)";
    ctx.lineWidth = 1.2;
    ctx.setLineDash([3, 3]);

    const slopeX = (ptC.x - ptB.x) / (ptC.y - ptB.y);
    const virtualX_at_galaxy = ptB.x + slopeX * (galaxyY - clusterY);

    ctx.beginPath();
    ctx.moveTo(ptC.x, ptC.y);
    ctx.lineTo(virtualX_at_galaxy, galaxyY);
    ctx.stroke();
    ctx.setLineDash([]);

    ctx.fillStyle = "rgba(239, 68, 68, 0.8)";
    ctx.beginPath();
    ctx.arc(virtualX_at_galaxy, galaxyY, 3, 0, Math.PI * 2);
    ctx.fill();
  }

  // Actual Lensed Light Path
  ctx.strokeStyle = "#facc15";
  ctx.lineWidth = sim.numRays === 1 ? 2.5 : 1.8;
  ctx.shadowColor = "#facc15";
  ctx.shadowBlur = sim.numRays === 1 ? 6 : 3;

  ctx.beginPath();
  ctx.moveTo(ptA.x, ptA.y);
  if (sim.M_tril > 0) {
    ctx.lineTo(ptB.x, ptB.y);
  }
  ctx.lineTo(ptC.x, ptC.y);
  ctx.stroke();
  ctx.shadowBlur = 0;

  // Visual Indicators for Impact Parameters
  if (toggleImpact.checked && sim.numRays === 1) {
    // Cluster Impact Parameter
    if (sim.clusterEnabled && sim.M_tril > 0 && Math.abs(ptB.x - centerX) > 3) {
      ctx.strokeStyle = "#00f0ff";
      ctx.lineWidth = 1.2;
      ctx.beginPath();
      ctx.moveTo(centerX, clusterY);
      ctx.lineTo(ptB.x, clusterY);
      ctx.stroke();

      ctx.fillStyle = "#00f0ff";
      ctx.beginPath();
      ctx.arc(ptB.x, clusterY, 2.5, 0, Math.PI * 2);
      ctx.arc(centerX, clusterY, 2.5, 0, Math.PI * 2);
      ctx.fill();

      ctx.font = "10px monospace";
      const clusterLabel = `${(ray.abs_b_C_ly / 1000).toFixed(1)} thousand light-years`;
      const textWidth = ctx.measureText(clusterLabel).width;
      const charOffset = 18; // ~2-3 character offset from bend
    }
  }
});

```

```

let labelX = 0;
let align = "center";

if (ray.theta_init_arcsec < 0) {
  // Initial angle is negative: left aligned to 2-3 chars to the right side of the bend
  labelX = ptB.x + charOffset;
  align = "left";
  if (labelX + textWidth > width - 10) {
    labelX = width - 10 - textWidth;
  }
  if (labelX < 10) {
    labelX = 10;
  }
} else {
  // Initial angle is positive: right aligned to 2-3 chars to the left side of the bend
  labelX = ptB.x - charOffset;
  align = "right";
  if (labelX - textWidth < 10) {
    labelX = 10 + textWidth;
  }
  if (labelX > width - 10) {
    labelX = width - 10;
  }
}

ctx.textAlign = align;
ctx.fillText(clusterLabel, labelX, clusterY - 6);
}

// Earth Impact Parameter
if (Math.abs(ptC.x - centerX) > 3) {
  ctx.strokeStyle = "#00f0ff";
  ctx.lineWidth = 1.2;
  ctx.beginPath();
  ctx.moveTo(centerX, earthY);
  ctx.lineTo(ptC.x, earthY);
  ctx.stroke();

  ctx.fillStyle = "#00f0ff";
  ctx.beginPath();
  ctx.arc(ptC.x, earthY, 2.5, 0, Math.PI * 2);
  ctx.arc(centerX, earthY, 2.5, 0, Math.PI * 2);
  ctx.fill();

  ctx.font = "10px monospace";
  const earthLabel = `${(ray.abs_b_E_ly / 1000).toFixed(1)} thousand light-years`;
  const earthTextWidth = ctx.measureText(earthLabel).width;
  let earthTextX = (centerX + ptC.x) / 2;

  if (earthTextX - earthTextWidth / 2 < 10) {
    earthTextX = 10 + earthTextWidth / 2;
  } else if (earthTextX + earthTextWidth / 2 > width - 10) {
    earthTextX = width - 10 - earthTextWidth / 2;
  }

  ctx.textAlign = "center";
  ctx.fillText(earthLabel, earthTextX, earthY + 16);
}
}

// Animated Photons (Constant Speed Along Physical Path)
if (togglePhoton.checked) {
  const rayOffset = (idx / sim.numRays);
  const progress = (photonAnimProgress + rayOffset) % 1.0;
  let px = 0, py = 0;

  if (sim.M_tril === 0) {
    px = ptA.x + progress * (ptC.x - ptA.x);
    py = ptA.y + progress * (ptC.y - ptA.y);
  } else {
    const d1 = Math.hypot(ptB.x - ptA.x, ptB.y - ptA.y);
    const d2 = Math.hypot(ptC.x - ptB.x, ptC.y - ptB.y);
    const dTotal = d1 + d2;
    const f1 = dTotal > 0 ? d1 / dTotal : 0.5;
  }
}

```

```

    if (progress <= f1) {
      const t = f1 > 0 ? progress / f1 : 0;
      px = ptA.x + t * (ptB.x - ptA.x);
      py = ptA.y + t * (ptB.y - ptA.y);
    } else {
      const t = (1 - f1) > 0 ? (progress - f1) / (1 - f1) : 0;
      px = ptB.x + t * (ptC.x - ptB.x);
      py = ptB.y + t * (ptC.y - ptB.y);
    }
  }
}

ctx.fillStyle = "#ffffff";
ctx.shadowColor = "#00f0ff";
ctx.shadowBlur = 6;
ctx.beginPath();
ctx.arc(px, py, 3, 0, Math.PI * 2);
ctx.fill();
ctx.shadowBlur = 0;
}
});

if (togglePhoton.checked) {
  photonAnimProgress = (photonAnimProgress + 0.006) % 1.0;
}

// Draw Celestial Objects

// 1. Distant Galaxy (Top)
ctx.save();
ctx.translate(ptA.x, ptA.y);
const galGlow = ctx.createRadialGradient(0, 0, 2, 0, 0, 18);
galGlow.addColorStop(0, "rgba(255, 200, 100, 0.9)");
galGlow.addColorStop(0.5, "rgba(238, 100, 30, 0.3)");
galGlow.addColorStop(1, "rgba(0, 0, 0, 0)");
ctx.fillStyle = galGlow;
ctx.beginPath();
ctx.arc(0, 0, 18, 0, Math.PI * 2);
ctx.fill();

ctx.fillStyle = "#ffd59e";
ctx.beginPath();
ctx.ellipse(0, 0, 12, 4, -0.3, 0, Math.PI * 2);
ctx.fill();
ctx.restore();

ctx.fillStyle = "#e1e7ec";
ctx.font = "11px sans-serif";
ctx.textAlign = "center";
ctx.fillText("Distant Galaxy", ptA.x, ptA.y - 15);

// 2. Intermediate Cluster (Middle - rendered with mass-scaling galaxies & rich purple halo)
if (sim.clusterEnabled && sim.M_tril > 0) {
  ctx.save();
  ctx.translate(centerX, clusterY);

  // Normalized mass ratio (0.0 to 1.0 across 100 to 1000 trillion solar masses)
  const massRatio = Math.max(0, Math.min(1, (sim.M_tril - 100) / 900));

  // Halo radius scales prominently from 16px to 62px
  const haloRadius = 16 + massRatio * 46;

  // Multi-stage radial gradient for increasingly vivid purple dark-matter halo
  const clusterGlow = ctx.createRadialGradient(0, 0, 2, 0, 0, haloRadius);
  const alphaCenter = 0.5 + massRatio * 0.45;
  const alphaMid = 0.25 + massRatio * 0.55;

  clusterGlow.addColorStop(0, `rgba(216, 180, 254, ${alphaCenter})`);
  clusterGlow.addColorStop(0.35, `rgba(168, 85, 247, ${alphaMid})`);
  clusterGlow.addColorStop(0.7, `rgba(126, 34, 206, ${alphaMid * 0.5})`);
  clusterGlow.addColorStop(1, "rgba(0, 0, 0, 0)");

  ctx.fillStyle = clusterGlow;
  ctx.beginPath();

```

```

    ctx.arc(0, 0, haloRadius, 0, Math.PI * 2);
    ctx.fill();

    // Dynamically add member galaxies as mass increases (3 at min mass up to 30 at max mass)
    const numGalaxies = Math.floor(3 + massRatio * 27);
    const spreadScale = 0.6 + massRatio * 0.6;

    for (let i = 0; i < numGalaxies; i++) {
        const g = clusterGalaxyList[i];
        const gx = g.x * spreadScale;
        const gy = g.y * spreadScale;

        // Core bright galaxy vs softer peripheral galaxies
        ctx.fillStyle = i === 0 ? "#ffffff" : `rgba(255, 245, 255, ${0.75 + (1 - i / 30) * 0.25})`;
        ctx.beginPath();
        ctx.arc(gx, gy, g.r, 0, Math.PI * 2);
        ctx.fill();

        // Secondary subtle purple glow around individual member galaxies
        if (g.r >= 2.0) {
            ctx.fillStyle = `rgba(192, 132, 252, ${0.3 + massRatio * 0.3})`;
            ctx.beginPath();
            ctx.arc(gx, gy, g.r * 1.8, 0, Math.PI * 2);
            ctx.fill();
        }
    }

    ctx.restore();

    ctx.fillStyle = "#e1e7ec";
    ctx.font = "11px sans-serif";
    ctx.textAlign = "center";
    ctx.fillText("Intermediate Cluster", centerX, clusterY + haloRadius + 14);
}

// 3. Earth / Observer (Bottom)
ctx.save();
ctx.translate(centerX, earthY);
const earthGrad = ctx.createRadialGradient(-2, -2, 1, 0, 0, 8);
earthGrad.addColorStop(0, "#60a5fa");
earthGrad.addColorStop(0.8, "#1d4ed8");
earthGrad.addColorStop(1, "#1e3a8a");
ctx.fillStyle = earthGrad;
ctx.beginPath();
ctx.arc(0, 0, 8, 0, Math.PI * 2);
ctx.fill();
ctx.restore();

ctx.fillStyle = "#e1e7ec";
ctx.font = "11px sans-serif";
ctx.textAlign = "center";
ctx.fillText("Earth", centerX, earthY + 26);

// Bottom Note inside Graphic Window
ctx.fillStyle = "rgba(140, 155, 174, 0.75)";
ctx.font = "italic 11px sans-serif";
ctx.textAlign = "center";
ctx.fillText("angles are exaggerated", centerX, height - 12);
}

function animate() {
    draw();
    requestAnimationFrame(animate);
}

animate();
</script>
</body>
</html>

```

User prompt: Hrm, that's not what I was looking for. Remove those last changes.


```

Response: <!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Gravitational Lensing: Vertical Field Simulation</title>
  <style>
    :root {
      --bg-color: #0b0e14;
      --panel-bg: #151922;
      --panel-border: #2a3142;
      --accent-color: #00f0ff;
      --accent-amber: #facc15;
      --accent-red: #ef4444;
      --text-main: #e1e7ec;
      --text-muted: #8c9bae;
    }

    * {
      box-sizing: border-box;
      margin: 0;
      padding: 0;
    }

    body {
      font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial, sans-serif;
      background-color: var(--bg-color);
      color: var(--text-main);
      display: flex;
      flex-direction: column;
      min-height: 100vh;
      overflow-x: hidden;
    }

    header {
      background-color: var(--panel-bg);
      border-bottom: 1px solid var(--panel-border);
      padding: 0.85rem 1.5rem;
      text-align: center;
    }

    header h1 {
      font-size: 1.3rem;
      font-weight: 600;
      color: #fff;
    }

    header p {
      font-size: 0.85rem;
      color: var(--text-muted);
      margin-top: 0.2rem;
    }

    .main-container {
      display: flex;
      flex: 1;
      flex-direction: column;
      gap: 1rem;
      padding: 1rem;
      max-width: 1200px;
      margin: 0 auto;
      width: 100%;
      align-items: center;
    }

    @media (min-width: 1024px) {
      .main-container {
        flex-direction: row;
        align-items: flex-start;
      }
    }

    .canvas-card {
      flex: 1;

```

```
width: 100%;
background-color: var(--panel-bg);
border: 1px solid var(--panel-border);
border-radius: 8px;
padding: 1rem;
display: flex;
flex-direction: column;
gap: 1rem;
box-shadow: 0 4px 20px rgba(0, 0, 0, 0.4);
}

.canvas-wrapper {
  position: relative;
  width: 100%;
  max-width: 520px;
  aspect-ratio: 3 / 4; /* Vertical portrait profile (4:3 height to width) */
  margin: 0 auto;
  background-color: #04060a;
  border-radius: 6px;
  overflow: hidden;
  border: 1px solid rgba(255, 255, 255, 0.05);
}

canvas {
  width: 100%;
  height: 100%;
  display: block;
}

.calc-box {
  background: rgba(10, 14, 22, 0.7);
  border: 1px solid var(--panel-border);
  border-radius: 6px;
  padding: 1rem 1.25rem;
  font-family: SFMono-Regular, Consolas, "Liberation Mono", Menlo, monospace;
  font-size: 0.8rem;
  color: #d1d5db;
}

.calc-box h3 {
  font-size: 0.85rem;
  text-transform: uppercase;
  letter-spacing: 0.05em;
  color: var(--accent-color);
  margin-bottom: 0.75rem;
  border-bottom: 1px solid rgba(255, 255, 255, 0.1);
  padding-bottom: 0.4rem;
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;
}

.calc-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
  gap: 0.85rem 1.25rem;
}

.calc-item {
  display: flex;
  flex-direction: column;
  gap: 0.25rem;
}

.calc-item .label {
  color: var(--text-muted);
  font-size: 0.75rem;
}

.calc-item .value {
  color: #fff;
  font-weight: 600;
  font-size: 0.85rem;
}

.calc-item .highlight {
```

```
    color: var(--accent-amber);
}

.calc-footer {
  margin-top: 0.85rem;
  padding-top: 0.6rem;
  border-top: 1px dashed rgba(255, 255, 255, 0.15);
  display: flex;
  justify-content: space-between;
  align-items: center;
  font-size: 0.75rem;
}

.calc-footer .formula {
  color: #9ca3af;
}

.controls-panel {
  width: 100%;
  max-width: 380px;
  background-color: var(--panel-bg);
  border: 1px solid var(--panel-border);
  border-radius: 8px;
  padding: 1.25rem;
  display: flex;
  flex-direction: column;
  gap: 1.1rem;
  box-shadow: 0 4px 20px rgba(0, 0, 0, 0.4);
}

.control-group {
  display: flex;
  flex-direction: column;
  gap: 0.4rem;
  transition: opacity 0.2s ease;
}

.control-group.disabled-control {
  opacity: 0.35;
}

.control-group label {
  font-size: 0.83rem;
  font-weight: 500;
  color: var(--text-main);
  display: flex;
  justify-content: space-between;
}

.control-group label span {
  color: var(--accent-color);
  font-family: monospace;
}

input[type="range"] {
  -webkit-appearance: none;
  width: 100%;
  height: 6px;
  border-radius: 3px;
  background: #2a3142;
  outline: none;
}

input[type="range"]:disabled {
  cursor: not-allowed;
}

input[type="range"]::-webkit-slider-thumb {
  -webkit-appearance: none;
  width: 16px;
  height: 16px;
  border-radius: 50%;
  background: var(--accent-color);
  cursor: pointer;
}
```

```

    box-shadow: 0 0 8px var(--accent-color);
    transition: transform 0.1s ease;
}

input[type="range"]:disabled::-webkit-slider-thumb {
    background: #6b7280;
    box-shadow: none;
    cursor: not-allowed;
}

input[type="range"]::-webkit-slider-thumb:hover:not(:disabled) {
    transform: scale(1.2);
}

.toggle-row {
    display: flex;
    align-items: center;
    justify-content: space-between;
    font-size: 0.83rem;
    color: var(--text-main);
}

.switch {
    position: relative;
    display: inline-block;
    width: 40px;
    height: 20px;
}

.switch input {
    opacity: 0;
    width: 0;
    height: 0;
}

.slider {
    position: absolute;
    cursor: pointer;
    top: 0; left: 0; right: 0; bottom: 0;
    background-color: #2a3142;
    transition: .2s;
    border-radius: 20px;
}

.slider:before {
    position: absolute;
    content: "";
    height: 14px;
    width: 14px;
    left: 3px;
    bottom: 3px;
    background-color: white;
    transition: .2s;
    border-radius: 50%;
}

input:checked + .slider {
    background-color: var(--accent-color);
}

input:checked + .slider:before {
    transform: translateX(20px);
}

.legend {
    border-top: 1px solid var(--panel-border);
    padding-top: 0.8rem;
    display: flex;
    flex-direction: column;
    gap: 0.4rem;
    font-size: 0.8rem;
}

.legend-item {

```

```

display: flex;
align-items: center;
gap: 0.6rem;
color: var(--text-muted);
}

.legend-color {
width: 14px;
height: 4px;
border-radius: 2px;
}
</style>
</head>
<body>

<header>
<h1>Gravitational Deflection of Light Fan</h1>
<p>Vertical Propagation View (Distant Galaxy at Top, Earth at Bottom)</p>
</header>

<div class="main-container">
<div class="canvas-card">
<div class="canvas-wrapper">
<canvas id="simCanvas"></canvas>
</div>

<!-- Numerical Output Box below Graphic -->
<div class="calc-box">
<h3>Numerical Results</h3>
<div class="calc-grid">
<div class="calc-item">
<span class="label">Galaxy Distance:</span>
<span class="value" id="boxDistG">8.0 billion light-years</span>
</div>
<div class="calc-item">
<span class="label">Galaxy Offset:</span>
<span class="value" id="boxOffsetX">0 thousand light-years</span>
</div>
<div class="calc-item">
<span class="label">Cluster Distance:</span>
<span class="value" id="boxDistC">4.0 billion light-years</span>
</div>
<div class="calc-item">
<span class="label">Galaxy-Cluster Distance:</span>
<span class="value" id="boxDistGC">4.0 billion light-years</span>
</div>
<div class="calc-item">
<span class="label">Cluster Mass:</span>
<span class="value" id="boxMass">200 trillion solar masses</span>
</div>
<div class="calc-item">
<span class="label">Ray Mode:</span>
<span class="value" id="boxMode">Single Ray</span>
</div>
<div class="calc-item">
<span class="label">Cluster Impact:</span>
<span class="value highlight" id="boxImpactRange">290.9 thousand light-years</span>
</div>
<div class="calc-item">
<span class="label">Earth Impact:</span>
<span class="value highlight" id="boxEarthImpactRange">581.8 thousand light-years</span>
</div>
<div class="calc-item">
<span class="label">Deflection Angle:</span>
<span class="value highlight" id="boxAlphaRange">44.3 arc-seconds</span>
</div>
</div>
<div class="calc-footer">
<span class="label">Theoretical Formula:</span>
<span class="formula"> $\alpha = 4GM / (c^2 |b_{\text{C}}|) \quad \text{ ; } \quad b_{\text{E}} = b_{\text{C}} + d$ </span>
</div>
</div>
</div>

```

```

<div class="controls-panel">
  <div class="toggle-row">
    <span>Enable Intermediate Cluster</span>
    <label class="switch">
      <input type="checkbox" id="toggleCluster" checked>
      <span class="slider"></span>
    </label>
  </div>

  <div class="control-group">
    <label for="sliderDistG">
      1. Distance to Galaxy:
      <span id="valDistG">8.0 billion light-years</span>
    </label>
    <input type="range" id="sliderDistG" min="2.0" max="13.0" step="0.1" value="8.0">
  </div>

  <div class="control-group">
    <label for="sliderOffsetX">
      2. Galaxy Horizontal Offset:
      <span id="valOffsetX">0 thousand light-years</span>
    </label>
    <input type="range" id="sliderOffsetX" min="-500" max="500" step="10" value="0">
  </div>

  <div class="control-group" id="grpDistC">
    <label for="sliderDistC">
      3. Distance to Cluster:
      <span id="valDistC">4.0 billion light-years</span>
    </label>
    <input type="range" id="sliderDistC" min="1.0" max="12.0" step="0.1" value="4.0">
  </div>

  <div class="control-group" id="grpMass">
    <label for="sliderMass">
      4. Cluster Mass:
      <span id="valMass">200 trillion solar masses</span>
    </label>
    <input type="range" id="sliderMass" min="100" max="1000" step="10" value="200">
  </div>

  <div class="control-group">
    <label for="sliderNumRays">
      5. Number of Light Rays:
      <span id="valNumRays">1</span>
    </label>
    <input type="range" id="sliderNumRays" min="1" max="10" step="1" value="1">
  </div>

  <div class="control-group" id="grpFanSpread">
    <label for="sliderFanSpread">
      6. Ray Fan Spread:
      <span id="valFanSpread">20 arc-seconds</span>
    </label>
    <input type="range" id="sliderFanSpread" min="10" max="40" step="1" value="20">
  </div>

  <div class="control-group" id="grpTheta">
    <label for="sliderTheta">
      7. Initial Ray Direction (1 ray):
      <span id="valTheta">+15.0 arc-seconds</span>
    </label>
    <input type="range" id="sliderTheta" min="-40.0" max="40.0" step="0.5" value="15.0">
  </div>

  <div class="toggle-row">
    <span>Animate Photons</span>
    <label class="switch">
      <input type="checkbox" id="togglePhoton" checked>
      <span class="slider"></span>
    </label>
  </div>

  <div class="toggle-row">

```

```

    <span>Show Virtual Tangent Lines</span>
    <label class="switch">
      <input type="checkbox" id="toggleTangent" checked>
      <span class="slider"></span>
    </label>
  </div>

  <div class="toggle-row">
    <span>Show Impact Parameters</span>
    <label class="switch">
      <input type="checkbox" id="toggleImpact" checked>
      <span class="slider"></span>
    </label>
  </div>

  <div class="toggle-row">
    <span>Show Background Stars</span>
    <label class="switch">
      <input type="checkbox" id="toggleStars" checked>
      <span class="slider"></span>
    </label>
  </div>

  <div class="legend">
    <div class="legend-item">
      <div class="legend-color" style="background: #facc15;"></div>
      <span>Actual deflected light rays</span>
    </div>
    <div class="legend-item">
      <div class="legend-color" style="border: 1px dashed #ef4444; background: transparent;"></div>
      <span>Apparent straight-line paths (virtual images)</span>
    </div>
    <div class="legend-item">
      <div class="legend-color" style="background: #00f0ff;"></div>
      <span>Cluster impact parameter & Earth impact</span>
    </div>
  </div>
</div>
</div>
</div>
<script>
  const canvas = document.getElementById('simCanvas');
  const ctx = canvas.getContext('2d');

  // Canvas Fixed Total Height Scale (13.5 Billion Light-Years)
  const CANVAS_TOTAL_GLY = 13.5;

  // Controls
  const toggleCluster = document.getElementById('toggleCluster');
  const sliderDistG = document.getElementById('sliderDistG');
  const sliderOffsetX = document.getElementById('sliderOffsetX');
  const sliderDistC = document.getElementById('sliderDistC');
  const sliderMass = document.getElementById('sliderMass');
  const sliderNumRays = document.getElementById('sliderNumRays');
  const sliderFanSpread = document.getElementById('sliderFanSpread');
  const sliderTheta = document.getElementById('sliderTheta');

  const grpDistC = document.getElementById('grpDistC');
  const grpMass = document.getElementById('grpMass');
  const grpFanSpread = document.getElementById('grpFanSpread');
  const grpTheta = document.getElementById('grpTheta');

  const valDistG = document.getElementById('valDistG');
  const valOffsetX = document.getElementById('valOffsetX');
  const valDistC = document.getElementById('valDistC');
  const valMass = document.getElementById('valMass');
  const valNumRays = document.getElementById('valNumRays');
  const valFanSpread = document.getElementById('valFanSpread');
  const valTheta = document.getElementById('valTheta');

  const togglePhoton = document.getElementById('togglePhoton');
  const toggleTangent = document.getElementById('toggleTangent');
  const toggleImpact = document.getElementById('toggleImpact');
  const toggleStars = document.getElementById('toggleStars');

```

```

// Calculation Box Elements
const boxDistG = document.getElementById('boxDistG');
const boxOffsetX = document.getElementById('boxOffsetX');
const boxDistC = document.getElementById('boxDistC');
const boxDistGC = document.getElementById('boxDistGC');
const boxMass = document.getElementById('boxMass');
const boxMode = document.getElementById('boxMode');
const boxImpactRange = document.getElementById('boxImpactRange');
const boxEarthImpactRange = document.getElementById('boxEarthImpactRange');
const boxAlphaRange = document.getElementById('boxAlphaRange');

let width = 0;
let height = 0;
let photonAnimProgress = 0;

// Background Stars
const stars = [];
for (let i = 0; i < 220; i++) {
  stars.push({
    x: Math.random(),
    y: Math.random(),
    size: Math.random() * 1.5 + 0.3,
    alpha: Math.random() * 0.7 + 0.3
  });
}

// Cluster Member Galaxies layout offsets (up to 30 member galaxies)
const clusterGalaxyList = [
  { x: 0, y: 0, r: 3.5 },
  { x: -6, y: -5, r: 2.2 },
  { x: 8, y: 4, r: 2.5 },
  { x: -12, y: 8, r: 2.0 },
  { x: 10, y: -10, r: 2.1 },
  { x: 5, y: -14, r: 1.8 },
  { x: -15, y: -6, r: 2.3 },
  { x: 18, y: -2, r: 1.9 },
  { x: -8, y: 16, r: 2.2 },
  { x: 14, y: 12, r: 1.7 },
  { x: 2, y: 18, r: 2.0 },
  { x: -20, y: 4, r: 1.8 },
  { x: -4, y: -20, r: 1.9 },
  { x: 22, y: -12, r: 1.6 },
  { x: 12, y: -22, r: 1.8 },
  { x: -22, y: -15, r: 1.7 },
  { x: -16, y: 22, r: 1.9 },
  { x: 25, y: 8, r: 1.5 },
  { x: 6, y: 26, r: 1.7 },
  { x: -28, y: -4, r: 1.6 },
  { x: 18, y: 22, r: 1.5 },
  { x: -10, y: -28, r: 1.8 },
  { x: 28, y: -20, r: 1.4 },
  { x: -25, y: 18, r: 1.6 },
  { x: 0, y: -32, r: 1.5 },
  { x: 32, y: 2, r: 1.4 },
  { x: -32, y: -18, r: 1.5 },
  { x: 20, y: -30, r: 1.4 },
  { x: -20, y: -32, r: 1.3 },
  { x: 15, y: 32, r: 1.4 }
];

function resizeCanvas() {
  const rect = canvas.parentElement.getBoundingClientRect();
  const dpr = window.devicePixelRatio || 1;
  width = rect.width;
  height = rect.height;

  canvas.width = width * dpr;
  canvas.height = height * dpr;
  ctx.scale(dpr, dpr);
}

window.addEventListener('resize', resizeCanvas);
resizeCanvas();

```



```

// Distance constraints: d_G >= d_C + 1.0 Gly
sliderDistG.addEventListener('input', () => {
  let dG = parseFloat(sliderDistG.value);
  let dC = parseFloat(sliderDistC.value);
  if (dG < dC + 1.0) {
    dC = Math.max(1.0, dG - 1.0);
    sliderDistC.value = dC.toFixed(1);
  }
  updateControlStates();
});

sliderDistC.addEventListener('input', () => {
  let dG = parseFloat(sliderDistG.value);
  let dC = parseFloat(sliderDistC.value);
  if (dC > dG - 1.0) {
    dG = Math.min(13.0, dC + 1.0);
    sliderDistG.value = dG.toFixed(1);
  }
  updateControlStates();
});

toggleCluster.addEventListener('change', updateControlStates);
sliderMass.addEventListener('input', updateControlStates);
sliderNumRays.addEventListener('input', updateControlStates);

function updateControlStates() {
  const numRays = parseInt(sliderNumRays.value);
  const clusterEnabled = toggleCluster.checked;

  if (!clusterEnabled) {
    sliderMass.disabled = true;
    grpMass.classList.add('disabled-control');
    sliderDistC.disabled = true;
    grpDistC.classList.add('disabled-control');
  } else {
    sliderMass.disabled = false;
    grpMass.classList.remove('disabled-control');
    sliderDistC.disabled = false;
    grpDistC.classList.remove('disabled-control');
  }

  if (numRays > 1) {
    sliderTheta.disabled = true;
    grpTheta.classList.add('disabled-control');

    sliderFanSpread.disabled = false;
    grpFanSpread.classList.remove('disabled-control');
  } else {
    sliderTheta.disabled = false;
    grpTheta.classList.remove('disabled-control');

    sliderFanSpread.disabled = true;
    grpFanSpread.classList.add('disabled-control');
  }
}

updateControlStates();

function calculateRays() {
  const clusterEnabled = toggleCluster.checked;
  const d_G_Gly = parseFloat(sliderDistG.value);
  const x_G_kly = parseFloat(sliderOffsetX.value);
  const d_C_Gly = parseFloat(sliderDistC.value);
  const d_GC_Gly = d_G_Gly - d_C_Gly;
  const M_tril = clusterEnabled ? parseFloat(sliderMass.value) : 0;
  const numRays = parseInt(sliderNumRays.value);
  const fanSpreadArcsec = parseFloat(sliderFanSpread.value);
  const thetaInitArcsec = parseFloat(sliderTheta.value);

  const ARCSEC_TO_RAD = Math.PI / (180.0 * 3600.0);
  const d_GC_ly = d_GC_Gly * 1e9;
  const d_C_ly = d_C_Gly * 1e9;
  const x_G_ly = x_G_kly * 1e3;

```

```

const scale_4GM_c2_ly = 0.6243363 * M_tril;

const rays = [];

for (let i = 0; i < numRays; i++) {
  let initialAngle = 0;

  if (numRays === 1) {
    initialAngle = thetaInitArcsec;
  } else {
    const frac = (i / (numRays - 1)) - 0.5;
    initialAngle = frac * fanSpreadArcsec;
  }

  const theta_init_rad = initialAngle * ARCSEC_TO_RAD;

  const b_C_ly = x_G_ly + d_GC_ly * Math.tan(theta_init_rad);
  const abs_b_C_ly = Math.abs(b_C_ly);

  let alpha_arcsec = 0;
  let alpha_rad = 0;
  if (M_tril > 0 && abs_b_C_ly > 1e-3) {
    alpha_rad = scale_4GM_c2_ly / abs_b_C_ly;
    alpha_arcsec = alpha_rad / ARCSEC_TO_RAD;
  }

  let theta_after_rad = theta_init_rad;
  if (M_tril > 0) {
    if (b_C_ly > 0) {
      theta_after_rad = theta_init_rad - alpha_rad;
    } else if (b_C_ly < 0) {
      theta_after_rad = theta_init_rad + alpha_rad;
    }
  }
  const theta_after_arcsec = theta_after_rad / ARCSEC_TO_RAD;

  const b_E_ly = b_C_ly + d_C_ly * Math.tan(theta_after_rad);
  const abs_b_E_ly = Math.abs(b_E_ly);

  rays.push({
    index: i,
    theta_init_arcsec: initialAngle,
    b_C_ly: b_C_ly,
    abs_b_C_ly: abs_b_C_ly,
    b_E_ly: b_E_ly,
    abs_b_E_ly: abs_b_E_ly,
    alpha_arcsec: alpha_arcsec,
    theta_after_arcsec: theta_after_arcsec
  });
}

return {
  clusterEnabled,
  d_G_Gly,
  x_G_kly,
  d_C_Gly,
  d_GC_Gly,
  M_tril,
  numRays,
  fanSpreadArcsec,
  thetaInitArcsec,
  rays
};
}

function draw() {
  ctx.clearRect(0, 0, width, height);

  const sim = calculateRays();

  // Label Updates
  valDistG.textContent = `${sim.d_G_Gly.toFixed(1)} billion light-years`;
  valOffsetX.textContent = `${sim.x_G_kly >= 0 ? '+' : ''}${sim.x_G_kly} thousand light-years`;

```

```

valDistC.textContent = !sim.clusterEnabled ? "N/A" : `${sim.d_C_Gly.toFixed(1)} billion light-years`;
valMass.textContent = !sim.clusterEnabled ? "Disabled" : `${sim.M_tril} trillion solar masses`;
valNumRays.textContent = `${sim.numRays}`;
valFanSpread.textContent = `${sim.fanSpreadArcsec} arc-seconds`;
valTheta.textContent = `${sim.thetaInitArcsec >= 0 ? '+' : ''}${sim.thetaInitArcsec.toFixed(1)} arc-sec`

// Update Numerical Box
boxDistG.textContent = `${sim.d_G_Gly.toFixed(1)} billion light-years`;
boxOffsetX.textContent = `${sim.x_G_kly >= 0 ? '+' : ''}${sim.x_G_kly} thousand light-years`;
boxDistC.textContent = !sim.clusterEnabled ? "N/A" : `${sim.d_C_Gly.toFixed(1)} billion light-years`;
boxDistGC.textContent = !sim.clusterEnabled ? "N/A" : `${sim.d_GC_Gly.toFixed(1)} billion light-years`;
boxMass.textContent = !sim.clusterEnabled ? "None" : `${sim.M_tril} trillion solar masses`;

if (sim.numRays === 1) {
  const r = sim.rays[0];
  boxMode.textContent = `Single Ray`;
  boxImpactRange.textContent = sim.clusterEnabled ? `${(r.abs_b_C_ly / 1000.0).toFixed(2)} thousand light-years`;
  boxEarthImpactRange.textContent = `${(r.abs_b_E_ly / 1000.0).toFixed(2)} thousand light-years`;
  boxAlphaRange.textContent = sim.clusterEnabled ? `${r.alpha_arcsec.toFixed(2)} arc-seconds` : "0.00 arc-seconds";
} else {
  boxMode.textContent = `Fan (${sim.numRays} rays)`;

  const bCValues = sim.rays.map(r => r.abs_b_C_ly / 1000.0);
  const bEValues = sim.rays.map(r => r.abs_b_E_ly / 1000.0);
  const alphaValues = sim.rays.map(r => r.alpha_arcsec).filter(a => isFinite(a));

  const minBC = Math.min(...bCValues);
  const maxBC = Math.max(...bCValues);
  const minBE = Math.min(...bEValues);
  const maxBE = Math.max(...bEValues);
  const maxAlpha = Math.max(...alphaValues);

  boxImpactRange.textContent = sim.clusterEnabled ? `${minBC.toFixed(1)} to ${maxBC.toFixed(1)} thousand light-years`;
  boxEarthImpactRange.textContent = `${minBE.toFixed(1)} to ${maxBE.toFixed(1)} thousand light-years`;
  boxAlphaRange.textContent = sim.clusterEnabled ? `${maxAlpha.toFixed(2)} arc-seconds max` : "0.00 arc-seconds";
}

// Visual Layout Dimensions (Vertical Field)
const topMargin = 45;
const bottomMargin = 55;
const centerX = width / 2;

const earthY = height - bottomMargin; // Earth at Bottom
const drawableHeight = height - topMargin - bottomMargin;
const pxPerGly = drawableHeight / CANVAS_TOTAL_GLY; // Scale for 13.5 Gly total field

const galaxyY = earthY - (sim.d_G_Gly * pxPerGly); // Distant Galaxy at Top
const clusterY = earthY - (sim.d_C_Gly * pxPerGly); // Intermediate Cluster

const pxPerKly = 100.0 / 500.0;
const galaxyX = centerX + (sim.x_G_kly * pxPerKly);

const ptA = { x: galaxyX, y: galaxyY };

// Starfield Background
if (toggleStars.checked) {
  ctx.fillStyle = "#ffffff";
  stars.forEach(s => {
    ctx.globalAlpha = s.alpha;
    ctx.fillRect(s.x * width, s.y * height, s.size, s.size);
  });
  ctx.globalAlpha = 1.0;
}

// Central Optical Axis
ctx.strokeStyle = "rgba(255, 255, 255, 0.15)";
ctx.lineWidth = 1;
ctx.setLineDash([6, 6]);
ctx.beginPath();
ctx.moveTo(centerX, earthY + 20);
ctx.lineTo(centerX, earthY - CANVAS_TOTAL_GLY * pxPerGly - 15);
ctx.stroke();
ctx.setLineDash([]);

```

```

// Cluster Plane Line
if (sim.clusterEnabled && sim.M_tril > 0) {
  ctx.strokeStyle = "rgba(0, 240, 255, 0.25)";
  ctx.beginPath();
  ctx.moveTo(20, clusterY);
  ctx.lineTo(width - 20, clusterY);
  ctx.stroke();
}

// Earth Plane Line
ctx.strokeStyle = "rgba(0, 240, 255, 0.15)";
ctx.beginPath();
ctx.moveTo(20, earthY);
ctx.lineTo(width - 20, earthY);
ctx.stroke();

// Render Primary Light Rays & Virtual Tangents
sim.rays.forEach((ray, idx) => {
  const clusterX = centerX + (ray.b_C_ly / 1000.0) * pxPerKly;
  const earthX = centerX + (ray.b_E_ly / 1000.0) * pxPerKly;

  const ptB = { x: clusterX, y: clusterY };
  const ptC = { x: earthX, y: earthY };

  // Virtual Tangent Line
  if (toggleTangent.checked && Math.abs(ray.alpha_arcsec) > 0.1 && sim.M_tril > 0) {
    ctx.strokeStyle = "rgba(239, 68, 68, 0.5)";
    ctx.lineWidth = 1.2;
    ctx.setLineDash([3, 3]);

    const slopeX = (ptC.x - ptB.x) / (ptC.y - ptB.y);
    const virtualX_at_galaxy = ptB.x + slopeX * (galaxyY - clusterY);

    ctx.beginPath();
    ctx.moveTo(ptC.x, ptC.y);
    ctx.lineTo(virtualX_at_galaxy, galaxyY);
    ctx.stroke();
    ctx.setLineDash([]);

    ctx.fillStyle = "rgba(239, 68, 68, 0.8)";
    ctx.beginPath();
    ctx.arc(virtualX_at_galaxy, galaxyY, 3, 0, Math.PI * 2);
    ctx.fill();
  }

  // Actual Lensed Light Path
  ctx.strokeStyle = "#facc15";
  ctx.lineWidth = sim.numRays === 1 ? 2.5 : 1.8;
  ctx.shadowColor = "#facc15";
  ctx.shadowBlur = sim.numRays === 1 ? 6 : 3;

  ctx.beginPath();
  ctx.moveTo(ptA.x, ptA.y);
  if (sim.M_tril > 0) {
    ctx.lineTo(ptB.x, ptB.y);
  }
  ctx.lineTo(ptC.x, ptC.y);
  ctx.stroke();
  ctx.shadowBlur = 0;

  // Visual Indicators for Impact Parameters
  if (toggleImpact.checked && sim.numRays === 1) {
    // Cluster Impact Parameter
    if (sim.clusterEnabled && sim.M_tril > 0 && Math.abs(ptB.x - centerX) > 3) {
      ctx.strokeStyle = "#00f0ff";
      ctx.lineWidth = 1.2;
      ctx.beginPath();
      ctx.moveTo(centerX, clusterY);
      ctx.lineTo(ptB.x, clusterY);
      ctx.stroke();

      ctx.fillStyle = "#00f0ff";
      ctx.beginPath();
      ctx.arc(ptB.x, clusterY, 2.5, 0, Math.PI * 2);
    }
  }
}

```

```

ctx.arc(centerX, clusterY, 2.5, 0, Math.PI * 2);
ctx.fill();

ctx.font = "10px monospace";
const clusterLabel = `${(ray.abs_b_C_ly / 1000).toFixed(1)} thousand light-years`;
const textWidth = ctx.measureText(clusterLabel).width;
const charOffset = 18; // ~2-3 character offset from bend

let labelX = 0;
let align = "center";

if (ray.theta_init_arcsec < 0) {
  labelX = ptB.x + charOffset;
  align = "left";
  if (labelX + textWidth > width - 10) {
    labelX = width - 10 - textWidth;
  }
  if (labelX < 10) {
    labelX = 10;
  }
} else {
  labelX = ptB.x - charOffset;
  align = "right";
  if (labelX - textWidth < 10) {
    labelX = 10 + textWidth;
  }
  if (labelX > width - 10) {
    labelX = width - 10;
  }
}

ctx.textAlign = align;
ctx.fillText(clusterLabel, labelX, clusterY - 6);
}

// Earth Impact Parameter
if (Math.abs(ptC.x - centerX) > 3) {
  ctx.strokeStyle = "#00f0ff";
  ctx.lineWidth = 1.2;
  ctx.beginPath();
  ctx.moveTo(centerX, earthY);
  ctx.lineTo(ptC.x, earthY);
  ctx.stroke();

  ctx.fillStyle = "#00f0ff";
  ctx.beginPath();
  ctx.arc(ptC.x, earthY, 2.5, 0, Math.PI * 2);
  ctx.arc(centerX, earthY, 2.5, 0, Math.PI * 2);
  ctx.fill();

  ctx.font = "10px monospace";
  const earthLabel = `${(ray.abs_b_E_ly / 1000).toFixed(1)} thousand light-years`;
  const earthTextWidth = ctx.measureText(earthLabel).width;
  let earthTextX = (centerX + ptC.x) / 2;

  if (earthTextX - earthTextWidth / 2 < 10) {
    earthTextX = 10 + earthTextWidth / 2;
  } else if (earthTextX + earthTextWidth / 2 > width - 10) {
    earthTextX = width - 10 - earthTextWidth / 2;
  }

  ctx.textAlign = "center";
  ctx.fillText(earthLabel, earthTextX, earthY + 16);
}
}

// Animated Photons (Constant Speed Along Physical Path)
if (togglePhoton.checked) {
  const rayOffset = (idx / sim.numRays);
  const progress = (photonAnimProgress + rayOffset) % 1.0;
  let px = 0, py = 0;

  if (sim.M_tril === 0) {
    px = ptA.x + progress * (ptC.x - ptA.x);

```

```

    py = ptA.y + progress * (ptC.y - ptA.y);
  } else {
    const d1 = Math.hypot(ptB.x - ptA.x, ptB.y - ptA.y);
    const d2 = Math.hypot(ptC.x - ptB.x, ptC.y - ptB.y);
    const dTotal = d1 + d2;
    const f1 = dTotal > 0 ? d1 / dTotal : 0.5;

    if (progress <= f1) {
      const t = f1 > 0 ? progress / f1 : 0;
      px = ptA.x + t * (ptB.x - ptA.x);
      py = ptA.y + t * (ptB.y - ptA.y);
    } else {
      const t = (1 - f1) > 0 ? (progress - f1) / (1 - f1) : 0;
      px = ptB.x + t * (ptC.x - ptB.x);
      py = ptB.y + t * (ptC.y - ptB.y);
    }
  }

  ctx.fillStyle = "#ffffff";
  ctx.shadowColor = "#00f0ff";
  ctx.shadowBlur = 6;
  ctx.beginPath();
  ctx.arc(px, py, 3, 0, Math.PI * 2);
  ctx.fill();
  ctx.shadowBlur = 0;
}
});

if (togglePhoton.checked) {
  photonAnimProgress = (photonAnimProgress + 0.006) % 1.0;
}

// Draw Celestial Objects

// 1. Distant Galaxy (Top)
ctx.save();
ctx.translate(ptA.x, ptA.y);
const galGlow = ctx.createRadialGradient(0, 0, 2, 0, 0, 18);
galGlow.addColorStop(0, "rgba(255, 200, 100, 0.9)");
galGlow.addColorStop(0.5, "rgba(238, 100, 30, 0.3)");
galGlow.addColorStop(1, "rgba(0, 0, 0, 0)");
ctx.fillStyle = galGlow;
ctx.beginPath();
ctx.arc(0, 0, 18, 0, Math.PI * 2);
ctx.fill();

ctx.fillStyle = "#ffd59e";
ctx.beginPath();
ctx.ellipse(0, 0, 12, 4, -0.3, 0, Math.PI * 2);
ctx.fill();
ctx.restore();

ctx.fillStyle = "#e1e7ec";
ctx.font = "11px sans-serif";
ctx.textAlign = "center";
ctx.fillText("Distant Galaxy", ptA.x, ptA.y - 15);

// 2. Intermediate Cluster (Middle - rendered with mass-scaling galaxies & rich purple halo)
if (sim.clusterEnabled && sim.M_tril > 0) {
  ctx.save();
  ctx.translate(centerX, clusterY);

  // Normalized mass ratio (0.0 to 1.0 across 100 to 1000 trillion solar masses)
  const massRatio = Math.max(0, Math.min(1, (sim.M_tril - 100) / 900));

  // Halo radius scales prominently from 16px to 62px
  const haloRadius = 16 + massRatio * 46;

  // Multi-stage radial gradient for increasingly vivid purple dark-matter halo
  const clusterGlow = ctx.createRadialGradient(0, 0, 2, 0, 0, haloRadius);
  const alphaCenter = 0.5 + massRatio * 0.45;
  const alphaMid = 0.25 + massRatio * 0.55;

  clusterGlow.addColorStop(0, `rgba(216, 180, 254, ${alphaCenter})`);

```

```

clusterGlow.addColorStop(0.35, `rgba(168, 85, 247, ${alphaMid})`);
clusterGlow.addColorStop(0.7, `rgba(126, 34, 206, ${alphaMid * 0.5})`);
clusterGlow.addColorStop(1, "rgba(0, 0, 0, 0)");

ctx.fillStyle = clusterGlow;
ctx.beginPath();
ctx.arc(0, 0, haloRadius, 0, Math.PI * 2);
ctx.fill();

// Dynamically add member galaxies as mass increases (3 at min mass up to 30 at max mass)
const numGalaxies = Math.floor(3 + massRatio * 27);
const spreadScale = 0.6 + massRatio * 0.6;

for (let i = 0; i < numGalaxies; i++) {
  const g = clusterGalaxyList[i];
  const gx = g.x * spreadScale;
  const gy = g.y * spreadScale;

  // Core bright galaxy vs softer peripheral galaxies
  ctx.fillStyle = i === 0 ? "#ffffff" : `rgba(255, 245, 255, ${0.75 + (1 - i / 30) * 0.25})`;
  ctx.beginPath();
  ctx.arc(gx, gy, g.r, 0, Math.PI * 2);
  ctx.fill();

  // Secondary subtle purple glow around individual member galaxies
  if (g.r >= 2.0) {
    ctx.fillStyle = `rgba(192, 132, 252, ${0.3 + massRatio * 0.3})`;
    ctx.beginPath();
    ctx.arc(gx, gy, g.r * 1.8, 0, Math.PI * 2);
    ctx.fill();
  }
}

ctx.restore();

ctx.fillStyle = "#e1e7ec";
ctx.font = "11px sans-serif";
ctx.textAlign = "center";
ctx.fillText("Intermediate Cluster", centerX, clusterY + haloRadius + 14);
}

// 3. Earth / Observer (Bottom)
ctx.save();
ctx.translate(centerX, earthY);
const earthGrad = ctx.createRadialGradient(-2, -2, 1, 0, 0, 8);
earthGrad.addColorStop(0, "#60a5fa");
earthGrad.addColorStop(0.8, "#1d4ed8");
earthGrad.addColorStop(1, "#1e3a8a");
ctx.fillStyle = earthGrad;
ctx.beginPath();
ctx.arc(0, 0, 8, 0, Math.PI * 2);
ctx.fill();
ctx.restore();

ctx.fillStyle = "#e1e7ec";
ctx.font = "11px sans-serif";
ctx.textAlign = "center";
ctx.fillText("Earth", centerX, earthY + 26);

// Bottom Note inside Graphic Window
ctx.fillStyle = "rgba(140, 155, 174, 0.75)";
ctx.font = "italic 11px sans-serif";
ctx.textAlign = "center";
ctx.fillText("angles are exaggerated", centerX, height - 12);
}

function animate() {
  draw();
  requestAnimationFrame(animate);
}

animate();
</script>

```

```
</body>
</html>
```

User prompt: Now, when showing a fan of rays that are being gravitationally lensed, add a set of non-gravitationally lensed rays to demonstrate that the galaxy is glowing in all directions. Space these non-lensed rays using the same displayed spacing as the lensed fan (so, for example if showing 10 rays spread across 40 arc-seconds, they are being displayed 4.4 degrees apart). Draw them as slightly thinner and with slightly lower opacity than the lensed rays, so that it is clear which rays are part of the lensing model and which are demonstrating that light goes in all directions from the galaxy.

```
Response: <!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Gravitational Lensing: Vertical Field Simulation</title>
  <style>
    :root {
      --bg-color: #0b0e14;
      --panel-bg: #151922;
      --panel-border: #2a3142;
      --accent-color: #00f0ff;
      --accent-amber: #facc15;
      --accent-red: #ef4444;
      --text-main: #e1e7ec;
      --text-muted: #8c9bae;
    }

    * {
      box-sizing: border-box;
      margin: 0;
      padding: 0;
    }

    body {
      font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial, sans-serif;
      background-color: var(--bg-color);
      color: var(--text-main);
      display: flex;
      flex-direction: column;
      min-height: 100vh;
      overflow-x: hidden;
    }

    header {
      background-color: var(--panel-bg);
      border-bottom: 1px solid var(--panel-border);
      padding: 0.85rem 1.5rem;
      text-align: center;
    }

    header h1 {
      font-size: 1.3rem;
      font-weight: 600;
      color: #fff;
    }

    header p {
      font-size: 0.85rem;
      color: var(--text-muted);
      margin-top: 0.2rem;
    }

    .main-container {
      display: flex;
      flex: 1;
      flex-direction: column;
      gap: 1rem;
      padding: 1rem;
      max-width: 1200px;
      margin: 0 auto;
      width: 100%;
      align-items: center;
```



```

}

@media (min-width: 1024px) {
  .main-container {
    flex-direction: row;
    align-items: flex-start;
  }
}

.canvas-card {
  flex: 1;
  width: 100%;
  background-color: var(--panel-bg);
  border: 1px solid var(--panel-border);
  border-radius: 8px;
  padding: 1rem;
  display: flex;
  flex-direction: column;
  gap: 1rem;
  box-shadow: 0 4px 20px rgba(0, 0, 0, 0.4);
}

.canvas-wrapper {
  position: relative;
  width: 100%;
  max-width: 520px;
  aspect-ratio: 3 / 4; /* Vertical portrait profile (4:3 height to width) */
  margin: 0 auto;
  background-color: #04060a;
  border-radius: 6px;
  overflow: hidden;
  border: 1px solid rgba(255, 255, 255, 0.05);
}

canvas {
  width: 100%;
  height: 100%;
  display: block;
}

.calc-box {
  background: rgba(10, 14, 22, 0.7);
  border: 1px solid var(--panel-border);
  border-radius: 6px;
  padding: 1rem 1.25rem;
  font-family: SFMono-Regular, Consolas, "Liberation Mono", Menlo, monospace;
  font-size: 0.8rem;
  color: #d1d5db;
}

.calc-box h3 {
  font-size: 0.85rem;
  text-transform: uppercase;
  letter-spacing: 0.05em;
  color: var(--accent-color);
  margin-bottom: 0.75rem;
  border-bottom: 1px solid rgba(255, 255, 255, 0.1);
  padding-bottom: 0.4rem;
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;
}

.calc-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
  gap: 0.85rem 1.25rem;
}

.calc-item {
  display: flex;
  flex-direction: column;
  gap: 0.25rem;
}

.calc-item .label {

```

```
    color: var(--text-muted);
    font-size: 0.75rem;
}

.calc-item .value {
    color: #fff;
    font-weight: 600;
    font-size: 0.85rem;
}

.calc-item .highlight {
    color: var(--accent-amber);
}

.calc-footer {
    margin-top: 0.85rem;
    padding-top: 0.6rem;
    border-top: 1px dashed rgba(255, 255, 255, 0.15);
    display: flex;
    justify-content: space-between;
    align-items: center;
    font-size: 0.75rem;
}

.calc-footer .formula {
    color: #9ca3af;
}

.controls-panel {
    width: 100%;
    max-width: 380px;
    background-color: var(--panel-bg);
    border: 1px solid var(--panel-border);
    border-radius: 8px;
    padding: 1.25rem;
    display: flex;
    flex-direction: column;
    gap: 1.1rem;
    box-shadow: 0 4px 20px rgba(0, 0, 0, 0.4);
}

.control-group {
    display: flex;
    flex-direction: column;
    gap: 0.4rem;
    transition: opacity 0.2s ease;
}

.control-group.disabled-control {
    opacity: 0.35;
}

.control-group label {
    font-size: 0.83rem;
    font-weight: 500;
    color: var(--text-main);
    display: flex;
    justify-content: space-between;
}

.control-group label span {
    color: var(--accent-color);
    font-family: monospace;
}

input[type="range"] {
    -webkit-appearance: none;
    width: 100%;
    height: 6px;
    border-radius: 3px;
    background: #2a3142;
    outline: none;
}
```

```
input[type="range"]:disabled {
  cursor: not-allowed;
}

input[type="range"]::-webkit-slider-thumb {
  -webkit-appearance: none;
  width: 16px;
  height: 16px;
  border-radius: 50%;
  background: var(--accent-color);
  cursor: pointer;
  box-shadow: 0 0 8px var(--accent-color);
  transition: transform 0.1s ease;
}

input[type="range"]:disabled::-webkit-slider-thumb {
  background: #6b7280;
  box-shadow: none;
  cursor: not-allowed;
}

input[type="range"]::-webkit-slider-thumb:hover:not(:disabled) {
  transform: scale(1.2);
}

.toggle-row {
  display: flex;
  align-items: center;
  justify-content: space-between;
  font-size: 0.83rem;
  color: var(--text-main);
}

.switch {
  position: relative;
  display: inline-block;
  width: 40px;
  height: 20px;
}

.switch input {
  opacity: 0;
  width: 0;
  height: 0;
}

.slider {
  position: absolute;
  cursor: pointer;
  top: 0; left: 0; right: 0; bottom: 0;
  background-color: #2a3142;
  transition: .2s;
  border-radius: 20px;
}

.slider:before {
  position: absolute;
  content: "";
  height: 14px;
  width: 14px;
  left: 3px;
  bottom: 3px;
  background-color: white;
  transition: .2s;
  border-radius: 50%;
}

input:checked + .slider {
  background-color: var(--accent-color);
}

input:checked + .slider:before {
  transform: translateX(20px);
}
```

```

.legend {
  border-top: 1px solid var(--panel-border);
  padding-top: 0.8rem;
  display: flex;
  flex-direction: column;
  gap: 0.4rem;
  font-size: 0.8rem;
}

.legend-item {
  display: flex;
  align-items: center;
  gap: 0.6rem;
  color: var(--text-muted);
}

.legend-color {
  width: 14px;
  height: 4px;
  border-radius: 2px;
}
</style>
</head>
<body>

<header>
  <h1>Gravitational Deflection of Light Fan</h1>
  <p>Vertical Propagation View (Distant Galaxy at Top, Earth at Bottom)</p>
</header>

<div class="main-container">
  <div class="canvas-card">
    <div class="canvas-wrapper">
      <canvas id="simCanvas"></canvas>
    </div>

    <!-- Numerical Output Box below Graphic -->
    <div class="calc-box">
      <h3>Numerical Results</h3>
      <div class="calc-grid">
        <div class="calc-item">
          <span class="label">Galaxy Distance:</span>
          <span class="value" id="boxDistG">8.0 billion light-years</span>
        </div>
        <div class="calc-item">
          <span class="label">Galaxy Offset:</span>
          <span class="value" id="boxOffsetX">0 thousand light-years</span>
        </div>
        <div class="calc-item">
          <span class="label">Cluster Distance:</span>
          <span class="value" id="boxDistC">4.0 billion light-years</span>
        </div>
        <div class="calc-item">
          <span class="label">Galaxy-Cluster Distance:</span>
          <span class="value" id="boxDistGC">4.0 billion light-years</span>
        </div>
        <div class="calc-item">
          <span class="label">Cluster Mass:</span>
          <span class="value" id="boxMass">200 trillion solar masses</span>
        </div>
        <div class="calc-item">
          <span class="label">Ray Mode:</span>
          <span class="value" id="boxMode">Single Ray</span>
        </div>
        <div class="calc-item">
          <span class="label">Cluster Impact:</span>
          <span class="value highlight" id="boxImpactRange">290.9 thousand light-years</span>
        </div>
        <div class="calc-item">
          <span class="label">Earth Impact:</span>
          <span class="value highlight" id="boxEarthImpactRange">581.8 thousand light-years</span>
        </div>
        <div class="calc-item">

```

```

        <span class="label">Deflection Angle:</span>
        <span class="value highlight" id="boxAlphaRange">44.3 arc-seconds</span>
    </div>
</div>
<div class="calc-footer">
    <span class="label">Theoretical Formula:</span>
    <span class="formula"> $\alpha = 4GM / (c^2 |b_{\text{sub}C}|) \ \&nbsp;\&nbsp;\&nbsp; b_{\text{sub}E} = b_{\text{sub}C} + d$ </span>
</div>
</div>
</div>
<div class="controls-panel">
    <div class="toggle-row">
        <span>Enable Intermediate Cluster</span>
        <label class="switch">
            <input type="checkbox" id="toggleCluster" checked="" />
            <span class="slider"></span>
        </label>
    </div>

    <div class="control-group">
        <label for="sliderDistG">
            1. Distance to Galaxy:
            <span id="valDistG">8.0 billion light-years</span>
        </label>
        <input type="range" id="sliderDistG" min="2.0" max="13.0" step="0.1" value="8.0">
    </div>

    <div class="control-group">
        <label for="sliderOffsetX">
            2. Galaxy Horizontal Offset:
            <span id="valOffsetX">0 thousand light-years</span>
        </label>
        <input type="range" id="sliderOffsetX" min="-500" max="500" step="10" value="0">
    </div>

    <div class="control-group" id="grpDistC">
        <label for="sliderDistC">
            3. Distance to Cluster:
            <span id="valDistC">4.0 billion light-years</span>
        </label>
        <input type="range" id="sliderDistC" min="1.0" max="12.0" step="0.1" value="4.0">
    </div>

    <div class="control-group" id="grpMass">
        <label for="sliderMass">
            4. Cluster Mass:
            <span id="valMass">200 trillion solar masses</span>
        </label>
        <input type="range" id="sliderMass" min="100" max="1000" step="10" value="200">
    </div>

    <div class="control-group">
        <label for="sliderNumRays">
            5. Number of Light Rays:
            <span id="valNumRays">1</span>
        </label>
        <input type="range" id="sliderNumRays" min="1" max="10" step="1" value="1">
    </div>

    <div class="control-group" id="grpFanSpread">
        <label for="sliderFanSpread">
            6. Ray Fan Spread:
            <span id="valFanSpread">20 arc-seconds</span>
        </label>
        <input type="range" id="sliderFanSpread" min="10" max="40" step="1" value="20">
    </div>

    <div class="control-group" id="grpTheta">
        <label for="sliderTheta">
            7. Initial Ray Direction (1 ray):
            <span id="valTheta">+15.0 arc-seconds</span>
        </label>
        <input type="range" id="sliderTheta" min="-40.0" max="40.0" step="0.5" value="15.0">
    </div>

```

```

</div>

<div class="toggle-row">
  <span>Animate Photons</span>
  <label class="switch">
    <input type="checkbox" id="togglePhoton" checked>
    <span class="slider"></span>
  </label>
</div>

<div class="toggle-row">
  <span>Show Virtual Tangent Lines</span>
  <label class="switch">
    <input type="checkbox" id="toggleTangent" checked>
    <span class="slider"></span>
  </label>
</div>

<div class="toggle-row">
  <span>Show Impact Parameters</span>
  <label class="switch">
    <input type="checkbox" id="toggleImpact" checked>
    <span class="slider"></span>
  </label>
</div>

<div class="toggle-row">
  <span>Show Background Stars</span>
  <label class="switch">
    <input type="checkbox" id="toggleStars" checked>
    <span class="slider"></span>
  </label>
</div>

<div class="legend">
  <div class="legend-item">
    <div class="legend-color" style="background: #facc15;"></div>
    <span>Actual deflected light rays</span>
  </div>
  <div class="legend-item">
    <div class="legend-color" style="background: rgba(250, 204, 21, 0.35); height: 2px;"></div>
    <span>Non-lensed background light (all directions)</span>
  </div>
  <div class="legend-item">
    <div class="legend-color" style="border: 1px dashed #ef4444; background: transparent;"></div>
    <span>Apparent straight-line paths (virtual images)</span>
  </div>
  <div class="legend-item">
    <div class="legend-color" style="background: #00f0ff;"></div>
    <span>Cluster impact parameter & Earth impact</span>
  </div>
</div>
</div>
</div>

<script>
const canvas = document.getElementById('simCanvas');
const ctx = canvas.getContext('2d');

// Canvas Fixed Total Height Scale (13.5 Billion Light-Years)
const CANVAS_TOTAL_GLY = 13.5;

// Controls
const toggleCluster = document.getElementById('toggleCluster');
const sliderDistG = document.getElementById('sliderDistG');
const sliderOffsetX = document.getElementById('sliderOffsetX');
const sliderDistC = document.getElementById('sliderDistC');
const sliderMass = document.getElementById('sliderMass');
const sliderNumRays = document.getElementById('sliderNumRays');
const sliderFanSpread = document.getElementById('sliderFanSpread');
const sliderTheta = document.getElementById('sliderTheta');

const grpDistC = document.getElementById('grpDistC');
const grpMass = document.getElementById('grpMass');

```

```

const grpFanSpread = document.getElementById('grpFanSpread');
const grpTheta = document.getElementById('grpTheta');

const valDistG = document.getElementById('valDistG');
const valOffsetX = document.getElementById('valOffsetX');
const valDistC = document.getElementById('valDistC');
const valMass = document.getElementById('valMass');
const valNumRays = document.getElementById('valNumRays');
const valFanSpread = document.getElementById('valFanSpread');
const valTheta = document.getElementById('valTheta');

const togglePhoton = document.getElementById('togglePhoton');
const toggleTangent = document.getElementById('toggleTangent');
const toggleImpact = document.getElementById('toggleImpact');
const toggleStars = document.getElementById('toggleStars');

// Calculation Box Elements
const boxDistG = document.getElementById('boxDistG');
const boxOffsetX = document.getElementById('boxOffsetX');
const boxDistC = document.getElementById('boxDistC');
const boxDistGC = document.getElementById('boxDistGC');
const boxMass = document.getElementById('boxMass');
const boxMode = document.getElementById('boxMode');
const boxImpactRange = document.getElementById('boxImpactRange');
const boxEarthImpactRange = document.getElementById('boxEarthImpactRange');
const boxAlphaRange = document.getElementById('boxAlphaRange');

let width = 0;
let height = 0;
let photonAnimProgress = 0;

// Background Stars
const stars = [];
for (let i = 0; i < 220; i++) {
  stars.push({
    x: Math.random(),
    y: Math.random(),
    size: Math.random() * 1.5 + 0.3,
    alpha: Math.random() * 0.7 + 0.3
  });
}

// Cluster Member Galaxies layout offsets (up to 30 member galaxies)
const clusterGalaxyList = [
  { x: 0, y: 0, r: 3.5 },
  { x: -6, y: -5, r: 2.2 },
  { x: 8, y: 4, r: 2.5 },
  { x: -12, y: 8, r: 2.0 },
  { x: 10, y: -10, r: 2.1 },
  { x: 5, y: -14, r: 1.8 },
  { x: -15, y: -6, r: 2.3 },
  { x: 18, y: -2, r: 1.9 },
  { x: -8, y: 16, r: 2.2 },
  { x: 14, y: 12, r: 1.7 },
  { x: 2, y: 18, r: 2.0 },
  { x: -20, y: 4, r: 1.8 },
  { x: -4, y: -20, r: 1.9 },
  { x: 22, y: -12, r: 1.6 },
  { x: 12, y: -22, r: 1.8 },
  { x: -22, y: -15, r: 1.7 },
  { x: -16, y: 22, r: 1.9 },
  { x: 25, y: 8, r: 1.5 },
  { x: 6, y: 26, r: 1.7 },
  { x: -28, y: -4, r: 1.6 },
  { x: 18, y: 22, r: 1.5 },
  { x: -10, y: -28, r: 1.8 },
  { x: 28, y: -20, r: 1.4 },
  { x: -25, y: 18, r: 1.6 },
  { x: 0, y: -32, r: 1.5 },
  { x: 32, y: 2, r: 1.4 },
  { x: -32, y: -18, r: 1.5 },
  { x: 20, y: -30, r: 1.4 },
  { x: -20, y: -32, r: 1.3 },
  { x: 15, y: 32, r: 1.4 }
]

```

```

];

function resizeCanvas() {
  const rect = canvas.parentElement.getBoundingClientRect();
  const dpr = window.devicePixelRatio || 1;
  width = rect.width;
  height = rect.height;

  canvas.width = width * dpr;
  canvas.height = height * dpr;
  ctx.scale(dpr, dpr);
}

window.addEventListener('resize', resizeCanvas);
resizeCanvas();

// Distance constraints: d_G >= d_C + 1.0 Gly
sliderDistG.addEventListener('input', () => {
  let dG = parseFloat(sliderDistG.value);
  let dC = parseFloat(sliderDistC.value);
  if (dG < dC + 1.0) {
    dC = Math.max(1.0, dG - 1.0);
    sliderDistC.value = dC.toFixed(1);
  }
  updateControlStates();
});

sliderDistC.addEventListener('input', () => {
  let dG = parseFloat(sliderDistG.value);
  let dC = parseFloat(sliderDistC.value);
  if (dC > dG - 1.0) {
    dG = Math.min(13.0, dC + 1.0);
    sliderDistG.value = dG.toFixed(1);
  }
  updateControlStates();
});

toggleCluster.addEventListener('change', updateControlStates);
sliderMass.addEventListener('input', updateControlStates);
sliderNumRays.addEventListener('input', updateControlStates);

function updateControlStates() {
  const numRays = parseInt(sliderNumRays.value);
  const clusterEnabled = toggleCluster.checked;

  if (!clusterEnabled) {
    sliderMass.disabled = true;
    grpMass.classList.add('disabled-control');
    sliderDistC.disabled = true;
    grpDistC.classList.add('disabled-control');
  } else {
    sliderMass.disabled = false;
    grpMass.classList.remove('disabled-control');
    sliderDistC.disabled = false;
    grpDistC.classList.remove('disabled-control');
  }

  if (numRays > 1) {
    sliderTheta.disabled = true;
    grpTheta.classList.add('disabled-control');

    sliderFanSpread.disabled = false;
    grpFanSpread.classList.remove('disabled-control');
  } else {
    sliderTheta.disabled = false;
    grpTheta.classList.remove('disabled-control');

    sliderFanSpread.disabled = true;
    grpFanSpread.classList.add('disabled-control');
  }
}

updateControlStates();

```



```

function calculateRays() {
  const clusterEnabled = toggleCluster.checked;
  const d_G_Gly = parseFloat(sliderDistG.value);
  const x_G_kly = parseFloat(sliderOffsetX.value);
  const d_C_Gly = parseFloat(sliderDistC.value);
  const d_GC_Gly = d_G_Gly - d_C_Gly;
  const M_tril = clusterEnabled ? parseFloat(sliderMass.value) : 0;
  const numRays = parseInt(sliderNumRays.value);
  const fanSpreadArcsec = parseFloat(sliderFanSpread.value);
  const thetaInitArcsec = parseFloat(sliderTheta.value);

  const ARCSEC_TO_RAD = Math.PI / (180.0 * 3600.0);
  const d_GC_ly = d_GC_Gly * 1e9;
  const d_C_ly = d_C_Gly * 1e9;
  const x_G_ly = x_G_kly * 1e3;

  const scale_4GM_c2_ly = 0.6243363 * M_tril;

  const rays = [];

  for (let i = 0; i < numRays; i++) {
    let initialAngle = 0;

    if (numRays === 1) {
      initialAngle = thetaInitArcsec;
    } else {
      const frac = (i / (numRays - 1)) - 0.5;
      initialAngle = frac * fanSpreadArcsec;
    }

    const theta_init_rad = initialAngle * ARCSEC_TO_RAD;

    const b_C_ly = x_G_ly + d_GC_ly * Math.tan(theta_init_rad);
    const abs_b_C_ly = Math.abs(b_C_ly);

    let alpha_arcsec = 0;
    let alpha_rad = 0;
    if (M_tril > 0 && abs_b_C_ly > 1e-3) {
      alpha_rad = scale_4GM_c2_ly / abs_b_C_ly;
      alpha_arcsec = alpha_rad / ARCSEC_TO_RAD;
    }

    let theta_after_rad = theta_init_rad;
    if (M_tril > 0) {
      if (b_C_ly > 0) {
        theta_after_rad = theta_init_rad - alpha_rad;
      } else if (b_C_ly < 0) {
        theta_after_rad = theta_init_rad + alpha_rad;
      }
    }
    const theta_after_arcsec = theta_after_rad / ARCSEC_TO_RAD;

    const b_E_ly = b_C_ly + d_C_ly * Math.tan(theta_after_rad);
    const abs_b_E_ly = Math.abs(b_E_ly);

    rays.push({
      index: i,
      theta_init_arcsec: initialAngle,
      b_C_ly: b_C_ly,
      abs_b_C_ly: abs_b_C_ly,
      b_E_ly: b_E_ly,
      abs_b_E_ly: abs_b_E_ly,
      alpha_arcsec: alpha_arcsec,
      theta_after_arcsec: theta_after_arcsec
    });
  }

  return {
    clusterEnabled,
    d_G_Gly,
    x_G_kly,
    d_C_Gly,
    d_GC_Gly,
    M_tril,
  }
}

```

```

    numRays,
    fanSpreadArcsec,
    thetaInitArcsec,
    rays
  };
}

function draw() {
  ctx.clearRect(0, 0, width, height);

  const sim = calculateRays();

  // Label Updates
  valDistG.textContent = `${sim.d_G_Gly.toFixed(1)} billion light-years`;
  valOffsetX.textContent = `${sim.x_G_kly >= 0 ? '+' : ''}${sim.x_G_kly} thousand light-years`;
  valDistC.textContent = !sim.clusterEnabled ? "N/A" : `${sim.d_C_Gly.toFixed(1)} billion light-years`;
  valMass.textContent = !sim.clusterEnabled ? "Disabled" : `${sim.M_tril} trillion solar masses`;
  valNumRays.textContent = `${sim.numRays}`;
  valFanSpread.textContent = `${sim.fanSpreadArcsec} arc-seconds`;
  valTheta.textContent = `${sim.thetaInitArcsec >= 0 ? '+' : ''}${sim.thetaInitArcsec.toFixed(1)} arc-sec`

  // Update Numerical Box
  boxDistG.textContent = `${sim.d_G_Gly.toFixed(1)} billion light-years`;
  boxOffsetX.textContent = `${sim.x_G_kly >= 0 ? '+' : ''}${sim.x_G_kly} thousand light-years`;
  boxDistC.textContent = !sim.clusterEnabled ? "N/A" : `${sim.d_C_Gly.toFixed(1)} billion light-years`;
  boxDistGC.textContent = !sim.clusterEnabled ? "N/A" : `${sim.d_GC_Gly.toFixed(1)} billion light-years`;
  boxMass.textContent = !sim.clusterEnabled ? "None" : `${sim.M_tril} trillion solar masses`;

  if (sim.numRays === 1) {
    const r = sim.rays[0];
    boxMode.textContent = `Single Ray`;
    boxImpactRange.textContent = sim.clusterEnabled ? `${(r.abs_b_C_ly / 1000.0).toFixed(2)} thousand light-years`;
    boxEarthImpactRange.textContent = `${(r.abs_b_E_ly / 1000.0).toFixed(2)} thousand light-years`;
    boxAlphaRange.textContent = sim.clusterEnabled ? `${r.alpha_arcsec.toFixed(2)} arc-seconds` : "0.00 arc-seconds";
  } else {
    boxMode.textContent = `Fan (${sim.numRays} rays)`;

    const bCValues = sim.rays.map(r => r.abs_b_C_ly / 1000.0);
    const bEValues = sim.rays.map(r => r.abs_b_E_ly / 1000.0);
    const alphaValues = sim.rays.map(r => r.alpha_arcsec).filter(a => isFinite(a));

    const minBC = Math.min(...bCValues);
    const maxBC = Math.max(...bCValues);
    const minBE = Math.min(...bEValues);
    const maxBE = Math.max(...bEValues);
    const maxAlpha = Math.max(...alphaValues);

    boxImpactRange.textContent = sim.clusterEnabled ? `${minBC.toFixed(1)} to ${maxBC.toFixed(1)} thousand light-years`;
    boxEarthImpactRange.textContent = `${minBE.toFixed(1)} to ${maxBE.toFixed(1)} thousand light-years`;
    boxAlphaRange.textContent = sim.clusterEnabled ? `${maxAlpha.toFixed(2)} arc-seconds max` : "0.00 arc-seconds";
  }

  // Visual Layout Dimensions (Vertical Field)
  const topMargin = 45;
  const bottomMargin = 55;
  const centerX = width / 2;

  const earthY = height - bottomMargin; // Earth at Bottom
  const drawableHeight = height - topMargin - bottomMargin;
  const pxPerGly = drawableHeight / CANVAS_TOTAL_GLY; // Scale for 13.5 Gly total field

  const galaxyY = earthY - (sim.d_G_Gly * pxPerGly); // Distant Galaxy at Top
  const clusterY = earthY - (sim.d_C_Gly * pxPerGly); // Intermediate Cluster

  const pxPerKly = 100.0 / 500.0;
  const galaxyX = centerX + (sim.x_G_kly * pxPerKly);

  const ptA = { x: galaxyX, y: galaxyY };

  // Starfield Background
  if (toggleStars.checked) {
    ctx.fillStyle = "#ffffff";
    stars.forEach(s => {
      ctx.globalAlpha = s.alpha;
    });
  }
}

```

```

    ctx.fillRect(s.x * width, s.y * height, s.size, s.size);
  });
  ctx.globalAlpha = 1.0;
}

// Central Optical Axis
ctx.strokeStyle = "rgba(255, 255, 255, 0.15)";
ctx.lineWidth = 1;
ctx.setLineDash([6, 6]);
ctx.beginPath();
ctx.moveTo(centerX, earthY + 20);
ctx.lineTo(centerX, earthY - CANVAS_TOTAL_GLY * pxPerGly - 15);
ctx.stroke();
ctx.setLineDash([]);

// Cluster Plane Line
if (sim.clusterEnabled && sim.M_tril > 0) {
  ctx.strokeStyle = "rgba(0, 240, 255, 0.25)";
  ctx.beginPath();
  ctx.moveTo(20, clusterY);
  ctx.lineTo(width - 20, clusterY);
  ctx.stroke();
}

// Earth Plane Line
ctx.strokeStyle = "rgba(0, 240, 255, 0.15)";
ctx.beginPath();
ctx.moveTo(20, earthY);
ctx.lineTo(width - 20, earthY);
ctx.stroke();

// --- NON-LENSED OMNIDIRECTIONAL BACKGROUND RAYS ---
// Determine displayed visual angle step of the lensed fan in canvas coordinates
let phiMin, phiMax, deltaPhi;
const targetY = sim.M_tril > 0 ? clusterY : earthY;

if (sim.numRays > 1) {
  const ray0 = sim.rays[0];
  const rayN = sim.rays[sim.numRays - 1];
  const clusterX0 = centerX + (ray0.b_C_ly / 1000.0) * pxPerKly;
  const clusterXN = centerX + (rayN.b_C_ly / 1000.0) * pxPerKly;

  const phi0 = Math.atan2(targetY - ptA.y, clusterX0 - ptA.x);
  const phiN = Math.atan2(targetY - ptA.y, clusterXN - ptA.x);

  phiMin = Math.min(phi0, phiN);
  phiMax = Math.max(phi0, phiN);
  deltaPhi = Math.abs(phiN - phi0) / (sim.numRays - 1);
} else {
  const ray0 = sim.rays[0];
  const clusterX0 = centerX + (ray0.b_C_ly / 1000.0) * pxPerKly;

  const phi0 = Math.atan2(targetY - ptA.y, clusterX0 - ptA.x);
  phiMin = phi0;
  phiMax = phi0;
  // Default visual angle step for single-ray display (~12 degrees)
  deltaPhi = (12.0 * Math.PI) / 180.0;
}

// Safeguard against tiny deltaPhi
if (deltaPhi < 0.015) deltaPhi = 0.015;

const maxRayLen = Math.max(width, height) * 1.5;

ctx.save();
ctx.strokeStyle = "rgba(250, 204, 21, 0.28)";
ctx.lineWidth = 0.9;

for (let angle = phiMax + deltaPhi; angle < phiMin + 2 * Math.PI - deltaPhi * 0.5; angle += deltaPhi) {
  const endX = ptA.x + Math.cos(angle) * maxRayLen;
  const endY = ptA.y + Math.sin(angle) * maxRayLen;

  ctx.beginPath();
  ctx.moveTo(ptA.x, ptA.y);

```

```

ctx.lineTo(endX, endY);
ctx.stroke();

// Photons traveling along non-lensed rays
if (togglePhoton.checked) {
  const pDist = ((photonAnimProgress * 1.5) % 1.0) * Math.min(width, height) * 0.85;
  const px = ptA.x + Math.cos(angle) * pDist;
  const py = ptA.y + Math.sin(angle) * pDist;

  if (px >= 0 && px <= width && py >= 0 && py <= height) {
    ctx.fillStyle = "rgba(255, 255, 255, 0.4)";
    ctx.beginPath();
    ctx.arc(px, py, 1.5, 0, Math.PI * 2);
    ctx.fill();
  }
}
}
ctx.restore();

// Render Primary Light Rays & Virtual Tangents
sim.rays.forEach((ray, idx) => {
  const clusterX = centerX + (ray.b_C_ly / 1000.0) * pxPerKly;
  const earthX = centerX + (ray.b_E_ly / 1000.0) * pxPerKly;

  const ptB = { x: clusterX, y: clusterY };
  const ptC = { x: earthX, y: earthY };

  // Virtual Tangent Line
  if (toggleTangent.checked && Math.abs(ray.alpha_arcsec) > 0.1 && sim.M_tril > 0) {
    ctx.strokeStyle = "rgba(239, 68, 68, 0.5)";
    ctx.lineWidth = 1.2;
    ctx.setLineDash([3, 3]);

    const slopeX = (ptC.x - ptB.x) / (ptC.y - ptB.y);
    const virtualX_at_galaxy = ptB.x + slopeX * (galaxyY - clusterY);

    ctx.beginPath();
    ctx.moveTo(ptC.x, ptC.y);
    ctx.lineTo(virtualX_at_galaxy, galaxyY);
    ctx.stroke();
    ctx.setLineDash([]);

    ctx.fillStyle = "rgba(239, 68, 68, 0.8)";
    ctx.beginPath();
    ctx.arc(virtualX_at_galaxy, galaxyY, 3, 0, Math.PI * 2);
    ctx.fill();
  }

  // Actual Lensed Light Path
  ctx.strokeStyle = "#facc15";
  ctx.lineWidth = sim.numRays === 1 ? 2.5 : 1.8;
  ctx.shadowColor = "#facc15";
  ctx.shadowBlur = sim.numRays === 1 ? 6 : 3;

  ctx.beginPath();
  ctx.moveTo(ptA.x, ptA.y);
  if (sim.M_tril > 0) {
    ctx.lineTo(ptB.x, ptB.y);
  }
  ctx.lineTo(ptC.x, ptC.y);
  ctx.stroke();
  ctx.shadowBlur = 0;

  // Visual Indicators for Impact Parameters
  if (toggleImpact.checked && sim.numRays === 1) {
    // Cluster Impact Parameter
    if (sim.clusterEnabled && sim.M_tril > 0 && Math.abs(ptB.x - centerX) > 3) {
      ctx.strokeStyle = "#00f0ff";
      ctx.lineWidth = 1.2;
      ctx.beginPath();
      ctx.moveTo(centerX, clusterY);
      ctx.lineTo(ptB.x, clusterY);
      ctx.stroke();
    }
  }
}

```

```

ctx.fillStyle = "#00f0ff";
ctx.beginPath();
ctx.arc(ptB.x, clusterY, 2.5, 0, Math.PI * 2);
ctx.arc(centerX, clusterY, 2.5, 0, Math.PI * 2);
ctx.fill();

ctx.font = "10px monospace";
const clusterLabel = `${(ray.abs_b_C_ly / 1000).toFixed(1)} thousand light-years`;
const textWidth = ctx.measureText(clusterLabel).width;
const charOffset = 18; // ~2-3 character offset from bend

let labelX = 0;
let align = "center";

if (ray.theta_init_arcsec < 0) {
  labelX = ptB.x + charOffset;
  align = "left";
  if (labelX + textWidth > width - 10) {
    labelX = width - 10 - textWidth;
  }
  if (labelX < 10) {
    labelX = 10;
  }
} else {
  labelX = ptB.x - charOffset;
  align = "right";
  if (labelX - textWidth < 10) {
    labelX = 10 + textWidth;
  }
  if (labelX > width - 10) {
    labelX = width - 10;
  }
}

ctx.textAlign = align;
ctx.fillText(clusterLabel, labelX, clusterY - 6);
}

// Earth Impact Parameter
if (Math.abs(ptC.x - centerX) > 3) {
  ctx.strokeStyle = "#00f0ff";
  ctx.lineWidth = 1.2;
  ctx.beginPath();
  ctx.moveTo(centerX, earthY);
  ctx.lineTo(ptC.x, earthY);
  ctx.stroke();

  ctx.fillStyle = "#00f0ff";
  ctx.beginPath();
  ctx.arc(ptC.x, earthY, 2.5, 0, Math.PI * 2);
  ctx.arc(centerX, earthY, 2.5, 0, Math.PI * 2);
  ctx.fill();

  ctx.font = "10px monospace";
  const earthLabel = `${(ray.abs_b_E_ly / 1000).toFixed(1)} thousand light-years`;
  const earthTextWidth = ctx.measureText(earthLabel).width;
  let earthTextX = (centerX + ptC.x) / 2;

  if (earthTextX - earthTextWidth / 2 < 10) {
    earthTextX = 10 + earthTextWidth / 2;
  } else if (earthTextX + earthTextWidth / 2 > width - 10) {
    earthTextX = width - 10 - earthTextWidth / 2;
  }

  ctx.textAlign = "center";
  ctx.fillText(earthLabel, earthTextX, earthY + 16);
}
}

// Animated Photons (Constant Speed Along Physical Path)
if (togglePhoton.checked) {
  const rayOffset = (idx / sim.numRays);
  const progress = (photonAnimProgress + rayOffset) % 1.0;
  let px = 0, py = 0;

```

```

if (sim.M_tril === 0) {
  px = ptA.x + progress * (ptC.x - ptA.x);
  py = ptA.y + progress * (ptC.y - ptA.y);
} else {
  const d1 = Math.hypot(ptB.x - ptA.x, ptB.y - ptA.y);
  const d2 = Math.hypot(ptC.x - ptB.x, ptC.y - ptB.y);
  const dTotal = d1 + d2;
  const f1 = dTotal > 0 ? d1 / dTotal : 0.5;

  if (progress <= f1) {
    const t = f1 > 0 ? progress / f1 : 0;
    px = ptA.x + t * (ptB.x - ptA.x);
    py = ptA.y + t * (ptB.y - ptA.y);
  } else {
    const t = (1 - f1) > 0 ? (progress - f1) / (1 - f1) : 0;
    px = ptB.x + t * (ptC.x - ptB.x);
    py = ptB.y + t * (ptC.y - ptB.y);
  }
}

ctx.fillStyle = "#ffffff";
ctx.shadowColor = "#00f0ff";
ctx.shadowBlur = 6;
ctx.beginPath();
ctx.arc(px, py, 3, 0, Math.PI * 2);
ctx.fill();
ctx.shadowBlur = 0;
}
});

if (togglePhoton.checked) {
  photonAnimProgress = (photonAnimProgress + 0.006) % 1.0;
}

// Draw Celestial Objects

// 1. Distant Galaxy (Top)
ctx.save();
ctx.translate(ptA.x, ptA.y);
const galGlow = ctx.createRadialGradient(0, 0, 2, 0, 0, 18);
galGlow.addColorStop(0, "rgba(255, 200, 100, 0.9)");
galGlow.addColorStop(0.5, "rgba(238, 100, 30, 0.3)");
galGlow.addColorStop(1, "rgba(0, 0, 0, 0)");
ctx.fillStyle = galGlow;
ctx.beginPath();
ctx.arc(0, 0, 18, 0, Math.PI * 2);
ctx.fill();

ctx.fillStyle = "#ffd59e";
ctx.beginPath();
ctx.ellipse(0, 0, 12, 4, -0.3, 0, Math.PI * 2);
ctx.fill();
ctx.restore();

ctx.fillStyle = "#e1e7ec";
ctx.font = "11px sans-serif";
ctx.textAlign = "center";
ctx.fillText("Distant Galaxy", ptA.x, ptA.y - 15);

// 2. Intermediate Cluster (Middle - rendered with mass-scaling galaxies & rich purple halo)
if (sim.clusterEnabled && sim.M_tril > 0) {
  ctx.save();
  ctx.translate(centerX, centerY);

  // Normalized mass ratio (0.0 to 1.0 across 100 to 1000 trillion solar masses)
  const massRatio = Math.max(0, Math.min(1, (sim.M_tril - 100) / 900));

  // Halo radius scales prominently from 16px to 62px
  const haloRadius = 16 + massRatio * 46;

  // Multi-stage radial gradient for increasingly vivid purple dark-matter halo
  const clusterGlow = ctx.createRadialGradient(0, 0, 2, 0, 0, haloRadius);
  const alphaCenter = 0.5 + massRatio * 0.45;

```

```

const alphaMid = 0.25 + massRatio * 0.55;

clusterGlow.addColorStop(0, `rgba(216, 180, 254, ${alphaCenter})`);
clusterGlow.addColorStop(0.35, `rgba(168, 85, 247, ${alphaMid})`);
clusterGlow.addColorStop(0.7, `rgba(126, 34, 206, ${alphaMid * 0.5})`);
clusterGlow.addColorStop(1, "rgba(0, 0, 0, 0)");

ctx.fillStyle = clusterGlow;
ctx.beginPath();
ctx.arc(0, 0, haloRadius, 0, Math.PI * 2);
ctx.fill();

// Dynamically add member galaxies as mass increases (3 at min mass up to 30 at max mass)
const numGalaxies = Math.floor(3 + massRatio * 27);
const spreadScale = 0.6 + massRatio * 0.6;

for (let i = 0; i < numGalaxies; i++) {
  const g = clusterGalaxyList[i];
  const gx = g.x * spreadScale;
  const gy = g.y * spreadScale;

  // Core bright galaxy vs softer peripheral galaxies
  ctx.fillStyle = i === 0 ? "#ffffff" : `rgba(255, 245, 255, ${0.75 + (1 - i / 30) * 0.25})`;
  ctx.beginPath();
  ctx.arc(gx, gy, g.r, 0, Math.PI * 2);
  ctx.fill();

  // Secondary subtle purple glow around individual member galaxies
  if (g.r >= 2.0) {
    ctx.fillStyle = `rgba(192, 132, 252, ${0.3 + massRatio * 0.3})`;
    ctx.beginPath();
    ctx.arc(gx, gy, g.r * 1.8, 0, Math.PI * 2);
    ctx.fill();
  }
}

ctx.restore();

ctx.fillStyle = "#e1e7ec";
ctx.font = "11px sans-serif";
ctx.textAlign = "center";
ctx.fillText("Intermediate Cluster", centerX, clusterY + haloRadius + 14);
}

// 3. Earth / Observer (Bottom)
ctx.save();
ctx.translate(centerX, earthY);
const earthGrad = ctx.createRadialGradient(-2, -2, 1, 0, 0, 8);
earthGrad.addColorStop(0, "#60a5fa");
earthGrad.addColorStop(0.8, "#1d4ed8");
earthGrad.addColorStop(1, "#1e3a8a");
ctx.fillStyle = earthGrad;
ctx.beginPath();
ctx.arc(0, 0, 8, 0, Math.PI * 2);
ctx.fill();
ctx.restore();

ctx.fillStyle = "#e1e7ec";
ctx.font = "11px sans-serif";
ctx.textAlign = "center";
ctx.fillText("Earth", centerX, earthY + 26);

// Bottom Note inside Graphic Window
ctx.fillStyle = "rgba(140, 155, 174, 0.75)";
ctx.font = "italic 11px sans-serif";
ctx.textAlign = "center";
ctx.fillText("angles are exaggerated", centerX, height - 12);
}

function animate() {
  draw();
  requestAnimationFrame(animate);
}

```

```

    animate();
  </script>
</body>
</html>

```

User prompt: *Hrm, let's go back one version, before adding the additional rays.*

```

Response: <!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Gravitational Lensing: Vertical Field Simulation</title>
  <style>
    :root {
      --bg-color: #0b0e14;
      --panel-bg: #151922;
      --panel-border: #2a3142;
      --accent-color: #00f0ff;
      --accent-amber: #facc15;
      --accent-red: #ef4444;
      --text-main: #e1e7ec;
      --text-muted: #8c9bae;
    }

    * {
      box-sizing: border-box;
      margin: 0;
      padding: 0;
    }

    body {
      font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial, sans-serif;
      background-color: var(--bg-color);
      color: var(--text-main);
      display: flex;
      flex-direction: column;
      min-height: 100vh;
      overflow-x: hidden;
    }

    header {
      background-color: var(--panel-bg);
      border-bottom: 1px solid var(--panel-border);
      padding: 0.85rem 1.5rem;
      text-align: center;
    }

    header h1 {
      font-size: 1.3rem;
      font-weight: 600;
      color: #fff;
    }

    header p {
      font-size: 0.85rem;
      color: var(--text-muted);
      margin-top: 0.2rem;
    }

    .main-container {
      display: flex;
      flex: 1;
      flex-direction: column;
      gap: 1rem;
      padding: 1rem;
      max-width: 1200px;
      margin: 0 auto;
      width: 100%;
      align-items: center;
    }
  </style>

```



```
@media (min-width: 1024px) {
  .main-container {
    flex-direction: row;
    align-items: flex-start;
  }
}

.canvas-card {
  flex: 1;
  width: 100%;
  background-color: var(--panel-bg);
  border: 1px solid var(--panel-border);
  border-radius: 8px;
  padding: 1rem;
  display: flex;
  flex-direction: column;
  gap: 1rem;
  box-shadow: 0 4px 20px rgba(0, 0, 0, 0.4);
}

.canvas-wrapper {
  position: relative;
  width: 100%;
  max-width: 520px;
  aspect-ratio: 3 / 4;
  margin: 0 auto;
  background-color: #04060a;
  border-radius: 6px;
  overflow: hidden;
  border: 1px solid rgba(255, 255, 255, 0.05);
}

canvas {
  width: 100%;
  height: 100%;
  display: block;
}

.calc-box {
  background: rgba(10, 14, 22, 0.7);
  border: 1px solid var(--panel-border);
  border-radius: 6px;
  padding: 1rem 1.25rem;
  font-family: SFMono-Regular, Consolas, "Liberation Mono", Menlo, monospace;
  font-size: 0.8rem;
  color: #d1d5db;
}

.calc-box h3 {
  font-size: 0.85rem;
  text-transform: uppercase;
  letter-spacing: 0.05em;
  color: var(--accent-color);
  margin-bottom: 0.75rem;
  border-bottom: 1px solid rgba(255, 255, 255, 0.1);
  padding-bottom: 0.4rem;
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;
}

.calc-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
  gap: 0.85rem 1.25rem;
}

.calc-item {
  display: flex;
  flex-direction: column;
  gap: 0.25rem;
}

.calc-item .label {
  color: var(--text-muted);
  font-size: 0.75rem;
}
```

```
}

.calc-item .value {
  color: #fff;
  font-weight: 600;
  font-size: 0.85rem;
}

.calc-item .highlight {
  color: var(--accent-amber);
}

.calc-footer {
  margin-top: 0.85rem;
  padding-top: 0.6rem;
  border-top: 1px dashed rgba(255, 255, 255, 0.15);
  display: flex;
  justify-content: space-between;
  align-items: center;
  font-size: 0.75rem;
}

.calc-footer .formula {
  color: #9ca3af;
}

.controls-panel {
  width: 100%;
  max-width: 380px;
  background-color: var(--panel-bg);
  border: 1px solid var(--panel-border);
  border-radius: 8px;
  padding: 1.25rem;
  display: flex;
  flex-direction: column;
  gap: 1.1rem;
  box-shadow: 0 4px 20px rgba(0, 0, 0, 0.4);
}

.control-group {
  display: flex;
  flex-direction: column;
  gap: 0.4rem;
  transition: opacity 0.2s ease;
}

.control-group.disabled-control {
  opacity: 0.35;
}

.control-group label {
  font-size: 0.83rem;
  font-weight: 500;
  color: var(--text-main);
  display: flex;
  justify-content: space-between;
}

.control-group label span {
  color: var(--accent-color);
  font-family: monospace;
}

input[type="range"] {
  -webkit-appearance: none;
  width: 100%;
  height: 6px;
  border-radius: 3px;
  background: #2a3142;
  outline: none;
}

input[type="range"]:disabled {
  cursor: not-allowed;
}
```

```
}

input[type="range"]::-webkit-slider-thumb {
  -webkit-appearance: none;
  width: 16px;
  height: 16px;
  border-radius: 50%;
  background: var(--accent-color);
  cursor: pointer;
  box-shadow: 0 0 8px var(--accent-color);
  transition: transform 0.1s ease;
}

input[type="range"]:disabled::-webkit-slider-thumb {
  background: #6b7280;
  box-shadow: none;
  cursor: not-allowed;
}

input[type="range"]::-webkit-slider-thumb:hover:not(:disabled) {
  transform: scale(1.2);
}

.toggle-row {
  display: flex;
  align-items: center;
  justify-content: space-between;
  font-size: 0.83rem;
  color: var(--text-main);
}

.switch {
  position: relative;
  display: inline-block;
  width: 40px;
  height: 20px;
}

.switch input {
  opacity: 0;
  width: 0;
  height: 0;
}

.slider {
  position: absolute;
  cursor: pointer;
  top: 0; left: 0; right: 0; bottom: 0;
  background-color: #2a3142;
  transition: .2s;
  border-radius: 20px;
}

.slider:before {
  position: absolute;
  content: "";
  height: 14px;
  width: 14px;
  left: 3px;
  bottom: 3px;
  background-color: white;
  transition: .2s;
  border-radius: 50%;
}

input:checked + .slider {
  background-color: var(--accent-color);
}

input:checked + .slider:before {
  transform: translateX(20px);
}

.legend {
```

```

border-top: 1px solid var(--panel-border);
padding-top: 0.8rem;
display: flex;
flex-direction: column;
gap: 0.4rem;
font-size: 0.8rem;
}

.legend-item {
display: flex;
align-items: center;
gap: 0.6rem;
color: var(--text-muted);
}

.legend-color {
width: 14px;
height: 4px;
border-radius: 2px;
}
</style>
</head>
<body>

<header>
<h1>Gravitational Deflection of Light Fan</h1>
<p>Vertical Propagation View (Distant Galaxy at Top, Earth at Bottom)</p>
</header>

<div class="main-container">
<div class="canvas-card">
<div class="canvas-wrapper">
<canvas id="simCanvas"></canvas>
</div>

<!-- Numerical Output Box below Graphic -->
<div class="calc-box">
<h3>Numerical Results</h3>
<div class="calc-grid">
<div class="calc-item">
<span class="label">Galaxy Distance:</span>
<span class="value" id="boxDistG">8.0 billion light-years</span>
</div>
<div class="calc-item">
<span class="label">Galaxy Offset:</span>
<span class="value" id="boxOffsetX">0 thousand light-years</span>
</div>
<div class="calc-item">
<span class="label">Cluster Distance:</span>
<span class="value" id="boxDistC">4.0 billion light-years</span>
</div>
<div class="calc-item">
<span class="label">Galaxy-Cluster Distance:</span>
<span class="value" id="boxDistGC">4.0 billion light-years</span>
</div>
<div class="calc-item">
<span class="label">Cluster Mass:</span>
<span class="value" id="boxMass">200 trillion solar masses</span>
</div>
<div class="calc-item">
<span class="label">Ray Mode:</span>
<span class="value" id="boxMode">Single Ray</span>
</div>
<div class="calc-item">
<span class="label">Cluster Impact:</span>
<span class="value highlight" id="boxImpactRange">290.9 thousand light-years</span>
</div>
<div class="calc-item">
<span class="label">Earth Impact:</span>
<span class="value highlight" id="boxEarthImpactRange">581.8 thousand light-years</span>
</div>
<div class="calc-item">
<span class="label">Deflection Angle:</span>
<span class="value highlight" id="boxAlphaRange">44.3 arc-seconds</span>

```

```

    </div>
  </div>
  <div class="calc-footer">
    <span class="label">Theoretical Formula:</span>
    <span class="formula"> $\hat{\alpha} = 4GM / (c^2 |b_{\text{C}}|) \quad \&nbps; \&nbps; b_{\text{E}} = b_{\text{C}} + d$ </span>
  </div>
</div>
</div>

<div class="controls-panel">
  <div class="toggle-row">
    <span>Enable Intermediate Cluster</span>
    <label class="switch">
      <input type="checkbox" id="toggleCluster" checked>
      <span class="slider"></span>
    </label>
  </div>

  <div class="control-group">
    <label for="sliderDistG">
      1. Distance to Galaxy:
      <span id="valDistG">8.0 billion light-years</span>
    </label>
    <input type="range" id="sliderDistG" min="2.0" max="13.0" step="0.1" value="8.0">
  </div>

  <div class="control-group">
    <label for="sliderOffsetX">
      2. Galaxy Horizontal Offset:
      <span id="valOffsetX">0 thousand light-years</span>
    </label>
    <input type="range" id="sliderOffsetX" min="-500" max="500" step="10" value="0">
  </div>

  <div class="control-group" id="grpDistC">
    <label for="sliderDistC">
      3. Distance to Cluster:
      <span id="valDistC">4.0 billion light-years</span>
    </label>
    <input type="range" id="sliderDistC" min="1.0" max="12.0" step="0.1" value="4.0">
  </div>

  <div class="control-group" id="grpMass">
    <label for="sliderMass">
      4. Cluster Mass:
      <span id="valMass">200 trillion solar masses</span>
    </label>
    <input type="range" id="sliderMass" min="100" max="1000" step="10" value="200">
  </div>

  <div class="control-group">
    <label for="sliderNumRays">
      5. Number of Light Rays:
      <span id="valNumRays">1</span>
    </label>
    <input type="range" id="sliderNumRays" min="1" max="10" step="1" value="1">
  </div>

  <div class="control-group" id="grpFanSpread">
    <label for="sliderFanSpread">
      6. Ray Fan Spread:
      <span id="valFanSpread">20 arc-seconds</span>
    </label>
    <input type="range" id="sliderFanSpread" min="10" max="40" step="1" value="20">
  </div>

  <div class="control-group" id="grpTheta">
    <label for="sliderTheta">
      7. Initial Ray Direction (1 ray):
      <span id="valTheta">+15.0 arc-seconds</span>
    </label>
    <input type="range" id="sliderTheta" min="-40.0" max="40.0" step="0.5" value="15.0">
  </div>

```

```

<div class="toggle-row">
  <span>Animate Photons</span>
  <label class="switch">
    <input type="checkbox" id="togglePhoton" checked>
    <span class="slider"></span>
  </label>
</div>

<div class="toggle-row">
  <span>Show Virtual Tangent Lines</span>
  <label class="switch">
    <input type="checkbox" id="toggleTangent" checked>
    <span class="slider"></span>
  </label>
</div>

<div class="toggle-row">
  <span>Show Impact Parameters</span>
  <label class="switch">
    <input type="checkbox" id="toggleImpact" checked>
    <span class="slider"></span>
  </label>
</div>

<div class="toggle-row">
  <span>Show Background Stars</span>
  <label class="switch">
    <input type="checkbox" id="toggleStars" checked>
    <span class="slider"></span>
  </label>
</div>

<div class="legend">
  <div class="legend-item">
    <div class="legend-color" style="background: #facc15;"></div>
    <span>Actual deflected light rays</span>
  </div>
  <div class="legend-item">
    <div class="legend-color" style="border: 1px dashed #ef4444; background: transparent;"></div>
    <span>Apparent straight-line paths (virtual images)</span>
  </div>
  <div class="legend-item">
    <div class="legend-color" style="background: #00f0ff;"></div>
    <span>Cluster impact parameter & Earth impact</span>
  </div>
</div>
</div>
</div>

<script>
const canvas = document.getElementById('simCanvas');
const ctx = canvas.getContext('2d');

// Canvas Fixed Total Height Scale (13.5 Billion Light-Years)
const CANVAS_TOTAL_GLY = 13.5;

// Controls
const toggleCluster = document.getElementById('toggleCluster');
const sliderDistG = document.getElementById('sliderDistG');
const sliderOffsetX = document.getElementById('sliderOffsetX');
const sliderDistC = document.getElementById('sliderDistC');
const sliderMass = document.getElementById('sliderMass');
const sliderNumRays = document.getElementById('sliderNumRays');
const sliderFanSpread = document.getElementById('sliderFanSpread');
const sliderTheta = document.getElementById('sliderTheta');

const grpDistC = document.getElementById('grpDistC');
const grpMass = document.getElementById('grpMass');
const grpFanSpread = document.getElementById('grpFanSpread');
const grpTheta = document.getElementById('grpTheta');

const valDistG = document.getElementById('valDistG');
const valOffsetX = document.getElementById('valOffsetX');
const valDistC = document.getElementById('valDistC');

```

```

const valMass = document.getElementById('valMass');
const valNumRays = document.getElementById('valNumRays');
const valFanSpread = document.getElementById('valFanSpread');
const valTheta = document.getElementById('valTheta');

const togglePhoton = document.getElementById('togglePhoton');
const toggleTangent = document.getElementById('toggleTangent');
const toggleImpact = document.getElementById('toggleImpact');
const toggleStars = document.getElementById('toggleStars');

// Calculation Box Elements
const boxDistG = document.getElementById('boxDistG');
const boxOffsetX = document.getElementById('boxOffsetX');
const boxDistC = document.getElementById('boxDistC');
const boxDistGC = document.getElementById('boxDistGC');
const boxMass = document.getElementById('boxMass');
const boxMode = document.getElementById('boxMode');
const boxImpactRange = document.getElementById('boxImpactRange');
const boxEarthImpactRange = document.getElementById('boxEarthImpactRange');
const boxAlphaRange = document.getElementById('boxAlphaRange');

let width = 0;
let height = 0;
let photonAnimProgress = 0;

// Background Stars
const stars = [];
for (let i = 0; i < 220; i++) {
  stars.push({
    x: Math.random(),
    y: Math.random(),
    size: Math.random() * 1.5 + 0.3,
    alpha: Math.random() * 0.7 + 0.3
  });
}

// Cluster Member Galaxies layout offsets (up to 30 member galaxies)
const clusterGalaxyList = [
  { x: 0, y: 0, r: 3.5 },
  { x: -6, y: -5, r: 2.2 },
  { x: 8, y: 4, r: 2.5 },
  { x: -12, y: 8, r: 2.0 },
  { x: 10, y: -10, r: 2.1 },
  { x: 5, y: -14, r: 1.8 },
  { x: -15, y: -6, r: 2.3 },
  { x: 18, y: -2, r: 1.9 },
  { x: -8, y: 16, r: 2.2 },
  { x: 14, y: 12, r: 1.7 },
  { x: 2, y: 18, r: 2.0 },
  { x: -20, y: 4, r: 1.8 },
  { x: -4, y: -20, r: 1.9 },
  { x: 22, y: -12, r: 1.6 },
  { x: 12, y: -22, r: 1.8 },
  { x: -22, y: -15, r: 1.7 },
  { x: -16, y: 22, r: 1.9 },
  { x: 25, y: 8, r: 1.5 },
  { x: 6, y: 26, r: 1.7 },
  { x: -28, y: -4, r: 1.6 },
  { x: 18, y: 22, r: 1.5 },
  { x: -10, y: -28, r: 1.8 },
  { x: 28, y: -20, r: 1.4 },
  { x: -25, y: 18, r: 1.6 },
  { x: 0, y: -32, r: 1.5 },
  { x: 32, y: 2, r: 1.4 },
  { x: -32, y: -18, r: 1.5 },
  { x: 20, y: -30, r: 1.4 },
  { x: -20, y: -32, r: 1.3 },
  { x: 15, y: 32, r: 1.4 }
];

function resizeCanvas() {
  const rect = canvas.parentElement.getBoundingClientRect();
  const dpr = window.devicePixelRatio || 1;
  width = rect.width;

```

```

    height = rect.height;

    canvas.width = width * dpr;
    canvas.height = height * dpr;
    ctx.scale(dpr, dpr);
}

window.addEventListener('resize', resizeCanvas);
resizeCanvas();

// Distance constraints: d_G >= d_C + 1.0 Gly
sliderDistG.addEventListener('input', () => {
    let dG = parseFloat(sliderDistG.value);
    let dC = parseFloat(sliderDistC.value);
    if (dG < dC + 1.0) {
        dC = Math.max(1.0, dG - 1.0);
        sliderDistC.value = dC.toFixed(1);
    }
    updateControlStates();
});

sliderDistC.addEventListener('input', () => {
    let dG = parseFloat(sliderDistG.value);
    let dC = parseFloat(sliderDistC.value);
    if (dC > dG - 1.0) {
        dG = Math.min(13.0, dC + 1.0);
        sliderDistG.value = dG.toFixed(1);
    }
    updateControlStates();
});

toggleCluster.addEventListener('change', updateControlStates);
sliderMass.addEventListener('input', updateControlStates);
sliderNumRays.addEventListener('input', updateControlStates);

function updateControlStates() {
    const numRays = parseInt(sliderNumRays.value);
    const clusterEnabled = toggleCluster.checked;

    if (!clusterEnabled) {
        sliderMass.disabled = true;
        grpMass.classList.add('disabled-control');
        sliderDistC.disabled = true;
        grpDistC.classList.add('disabled-control');
    } else {
        sliderMass.disabled = false;
        grpMass.classList.remove('disabled-control');
        sliderDistC.disabled = false;
        grpDistC.classList.remove('disabled-control');
    }

    if (numRays > 1) {
        sliderTheta.disabled = true;
        grpTheta.classList.add('disabled-control');

        sliderFanSpread.disabled = false;
        grpFanSpread.classList.remove('disabled-control');
    } else {
        sliderTheta.disabled = false;
        grpTheta.classList.remove('disabled-control');

        sliderFanSpread.disabled = true;
        grpFanSpread.classList.add('disabled-control');
    }
}

updateControlStates();

function calculateRays() {
    const clusterEnabled = toggleCluster.checked;
    const d_G_Gly = parseFloat(sliderDistG.value);
    const x_G_kly = parseFloat(sliderOffsetX.value);
    const d_C_Gly = parseFloat(sliderDistC.value);
    const d_GC_Gly = d_G_Gly - d_C_Gly;

```



```

const M_tril = clusterEnabled ? parseFloat(sliderMass.value) : 0;
const numRays = parseInt(sliderNumRays.value);
const fanSpreadArcsec = parseFloat(sliderFanSpread.value);
const thetaInitArcsec = parseFloat(sliderTheta.value);

const ARCSEC_TO_RAD = Math.PI / (180.0 * 3600.0);
const d_GC_ly = d_GC_Gly * 1e9;
const d_C_ly = d_C_Gly * 1e9;
const x_G_ly = x_G_kly * 1e3;

const scale_4GM_c2_ly = 0.6243363 * M_tril;

const rays = [];

for (let i = 0; i < numRays; i++) {
  let initialAngle = 0;

  if (numRays === 1) {
    initialAngle = thetaInitArcsec;
  } else {
    const frac = (i / (numRays - 1)) - 0.5;
    initialAngle = frac * fanSpreadArcsec;
  }

  const theta_init_rad = initialAngle * ARCSEC_TO_RAD;

  const b_C_ly = x_G_ly + d_GC_ly * Math.tan(theta_init_rad);
  const abs_b_C_ly = Math.abs(b_C_ly);

  let alpha_arcsec = 0;
  let alpha_rad = 0;
  if (M_tril > 0 && abs_b_C_ly > 1e-3) {
    alpha_rad = scale_4GM_c2_ly / abs_b_C_ly;
    alpha_arcsec = alpha_rad / ARCSEC_TO_RAD;
  }

  let theta_after_rad = theta_init_rad;
  if (M_tril > 0) {
    if (b_C_ly > 0) {
      theta_after_rad = theta_init_rad - alpha_rad;
    } else if (b_C_ly < 0) {
      theta_after_rad = theta_init_rad + alpha_rad;
    }
  }
  const theta_after_arcsec = theta_after_rad / ARCSEC_TO_RAD;

  const b_E_ly = b_C_ly + d_C_ly * Math.tan(theta_after_rad);
  const abs_b_E_ly = Math.abs(b_E_ly);

  rays.push({
    index: i,
    theta_init_arcsec: initialAngle,
    b_C_ly: b_C_ly,
    abs_b_C_ly: abs_b_C_ly,
    b_E_ly: b_E_ly,
    abs_b_E_ly: abs_b_E_ly,
    alpha_arcsec: alpha_arcsec,
    theta_after_arcsec: theta_after_arcsec
  });
}

return {
  clusterEnabled,
  d_G_Gly,
  x_G_kly,
  d_C_Gly,
  d_GC_Gly,
  M_tril,
  numRays,
  fanSpreadArcsec,
  thetaInitArcsec,
  rays
};
}

```

```

function draw() {
  ctx.clearRect(0, 0, width, height);

  const sim = calculateRays();

  // Label Updates
  valDistG.textContent = `${sim.d_G_Gly.toFixed(1)} billion light-years`;
  valOffsetX.textContent = `${sim.x_G_kly >= 0 ? '+' : ''}${sim.x_G_kly} thousand light-years`;
  valDistC.textContent = !sim.clusterEnabled ? "N/A" : `${sim.d_C_Gly.toFixed(1)} billion light-years`;
  valMass.textContent = !sim.clusterEnabled ? "Disabled" : `${sim.M_tril} trillion solar masses`;
  valNumRays.textContent = `${sim.numRays}`;
  valFanSpread.textContent = `${sim.fanSpreadArcsec} arc-seconds`;
  valTheta.textContent = `${sim.thetaInitArcsec >= 0 ? '+' : ''}${sim.thetaInitArcsec.toFixed(1)} arc-sec`

  // Update Numerical Box
  boxDistG.textContent = `${sim.d_G_Gly.toFixed(1)} billion light-years`;
  boxOffsetX.textContent = `${sim.x_G_kly >= 0 ? '+' : ''}${sim.x_G_kly} thousand light-years`;
  boxDistC.textContent = !sim.clusterEnabled ? "N/A" : `${sim.d_C_Gly.toFixed(1)} billion light-years`;
  boxDistGC.textContent = !sim.clusterEnabled ? "N/A" : `${sim.d_GC_Gly.toFixed(1)} billion light-years`;
  boxMass.textContent = !sim.clusterEnabled ? "None" : `${sim.M_tril} trillion solar masses`;

  if (sim.numRays === 1) {
    const r = sim.rays[0];
    boxMode.textContent = `Single Ray`;
    boxImpactRange.textContent = sim.clusterEnabled ? `${(r.abs_b_C_ly / 1000.0).toFixed(2)} thousand light-years`;
    boxEarthImpactRange.textContent = `${(r.abs_b_E_ly / 1000.0).toFixed(2)} thousand light-years`;
    boxAlphaRange.textContent = sim.clusterEnabled ? `${r.alpha_arcsec.toFixed(2)} arc-seconds` : "0.00 arc-seconds";
  } else {
    boxMode.textContent = `Fan (${sim.numRays} rays)`;

    const bCValues = sim.rays.map(r => r.abs_b_C_ly / 1000.0);
    const bEValues = sim.rays.map(r => r.abs_b_E_ly / 1000.0);
    const alphaValues = sim.rays.map(r => r.alpha_arcsec).filter(a => isFinite(a));

    const minBC = Math.min(...bCValues);
    const maxBC = Math.max(...bCValues);
    const minBE = Math.min(...bEValues);
    const maxBE = Math.max(...bEValues);
    const maxAlpha = Math.max(...alphaValues);

    boxImpactRange.textContent = sim.clusterEnabled ? `${minBC.toFixed(1)} to ${maxBC.toFixed(1)} thousand light-years`;
    boxEarthImpactRange.textContent = `${minBE.toFixed(1)} to ${maxBE.toFixed(1)} thousand light-years`;
    boxAlphaRange.textContent = sim.clusterEnabled ? `${maxAlpha.toFixed(2)} arc-seconds max` : "0.00 arc-seconds";
  }

  // Visual Layout Dimensions (Vertical Field)
  const topMargin = 45;
  const bottomMargin = 55;
  const centerX = width / 2;

  const earthY = height - bottomMargin;
  const drawableHeight = height - topMargin - bottomMargin;
  const pxPerGly = drawableHeight / CANVAS_TOTAL_GLY;

  const galaxyY = earthY - (sim.d_G_Gly * pxPerGly);
  const clusterY = earthY - (sim.d_C_Gly * pxPerGly);

  const pxPerKly = 100.0 / 500.0;
  const galaxyX = centerX + (sim.x_G_kly * pxPerKly);

  const ptA = { x: galaxyX, y: galaxyY };

  // Starfield Background
  if (toggleStars.checked) {
    ctx.fillStyle = "#ffffff";
    stars.forEach(s => {
      ctx.globalAlpha = s.alpha;
      ctx.fillRect(s.x * width, s.y * height, s.size, s.size);
    });
    ctx.globalAlpha = 1.0;
  }

  // Central Optical Axis

```

```

ctx.strokeStyle = "rgba(255, 255, 255, 0.15)";
ctx.lineWidth = 1;
ctx.setLineDash([6, 6]);
ctx.beginPath();
ctx.moveTo(centerX, earthY + 20);
ctx.lineTo(centerX, earthY - CANVAS_TOTAL_GLY * pxPerGly - 15);
ctx.stroke();
ctx.setLineDash([]);

// Cluster Plane Line
if (sim.clusterEnabled && sim.M_tril > 0) {
  ctx.strokeStyle = "rgba(0, 240, 255, 0.25)";
  ctx.beginPath();
  ctx.moveTo(20, clusterY);
  ctx.lineTo(width - 20, clusterY);
  ctx.stroke();
}

// Earth Plane Line
ctx.strokeStyle = "rgba(0, 240, 255, 0.15)";
ctx.beginPath();
ctx.moveTo(20, earthY);
ctx.lineTo(width - 20, earthY);
ctx.stroke();

// Render Primary Light Rays & Virtual Tangents
sim.rays.forEach((ray, idx) => {
  const clusterX = centerX + (ray.b_C_ly / 1000.0) * pxPerKly;
  const earthX = centerX + (ray.b_E_ly / 1000.0) * pxPerKly;

  const ptB = { x: clusterX, y: clusterY };
  const ptC = { x: earthX, y: earthY };

  // Virtual Tangent Line
  if (toggleTangent.checked && Math.abs(ray.alpha_arcsec) > 0.1 && sim.M_tril > 0) {
    ctx.strokeStyle = "rgba(239, 68, 68, 0.5)";
    ctx.lineWidth = 1.2;
    ctx.setLineDash([3, 3]);

    const slopeX = (ptC.x - ptB.x) / (ptC.y - ptB.y);
    const virtualX_at_galaxy = ptB.x + slopeX * (galaxyY - clusterY);

    ctx.beginPath();
    ctx.moveTo(ptC.x, ptC.y);
    ctx.lineTo(virtualX_at_galaxy, galaxyY);
    ctx.stroke();
    ctx.setLineDash([]);

    ctx.fillStyle = "rgba(239, 68, 68, 0.8)";
    ctx.beginPath();
    ctx.arc(virtualX_at_galaxy, galaxyY, 3, 0, Math.PI * 2);
    ctx.fill();
  }

  // Actual Lensed Light Path
  ctx.strokeStyle = "#facc15";
  ctx.lineWidth = sim.numRays === 1 ? 2.5 : 1.8;
  ctx.shadowColor = "#facc15";
  ctx.shadowBlur = sim.numRays === 1 ? 6 : 3;

  ctx.beginPath();
  ctx.moveTo(ptA.x, ptA.y);
  if (sim.M_tril > 0) {
    ctx.lineTo(ptB.x, ptB.y);
  }
  ctx.lineTo(ptC.x, ptC.y);
  ctx.stroke();
  ctx.shadowBlur = 0;

  // Visual Indicators for Impact Parameters
  if (toggleImpact.checked && sim.numRays === 1) {
    // Cluster Impact Parameter
    if (sim.clusterEnabled && sim.M_tril > 0 && Math.abs(ptB.x - centerX) > 3) {
      ctx.strokeStyle = "#00f0ff";
    }
  }
});

```

```

ctx.lineWidth = 1.2;
ctx.beginPath();
ctx.moveTo(centerX, clusterY);
ctx.lineTo(ptB.x, clusterY);
ctx.stroke();

ctx.fillStyle = "#00f0ff";
ctx.beginPath();
ctx.arc(ptB.x, clusterY, 2.5, 0, Math.PI * 2);
ctx.arc(centerX, clusterY, 2.5, 0, Math.PI * 2);
ctx.fill();

ctx.font = "10px monospace";
const clusterLabel = `${(ray.abs_b_C_ly / 1000).toFixed(1)} thousand light-years`;
const textWidth = ctx.measureText(clusterLabel).width;
const charOffset = 18;

let labelX = 0;
let align = "center";

if (ray.theta_init_arcsec < 0) {
  labelX = ptB.x + charOffset;
  align = "left";
  if (labelX + textWidth > width - 10) {
    labelX = width - 10 - textWidth;
  }
  if (labelX < 10) {
    labelX = 10;
  }
} else {
  labelX = ptB.x - charOffset;
  align = "right";
  if (labelX - textWidth < 10) {
    labelX = 10 + textWidth;
  }
  if (labelX > width - 10) {
    labelX = width - 10;
  }
}

ctx.textAlign = align;
ctx.fillText(clusterLabel, labelX, clusterY - 6);
}

// Earth Impact Parameter
if (Math.abs(ptC.x - centerX) > 3) {
  ctx.strokeStyle = "#00f0ff";
  ctx.lineWidth = 1.2;
  ctx.beginPath();
  ctx.moveTo(centerX, earthY);
  ctx.lineTo(ptC.x, earthY);
  ctx.stroke();

  ctx.fillStyle = "#00f0ff";
  ctx.beginPath();
  ctx.arc(ptC.x, earthY, 2.5, 0, Math.PI * 2);
  ctx.arc(centerX, earthY, 2.5, 0, Math.PI * 2);
  ctx.fill();

  ctx.font = "10px monospace";
  const earthLabel = `${(ray.abs_b_E_ly / 1000).toFixed(1)} thousand light-years`;
  const earthTextWidth = ctx.measureText(earthLabel).width;
  let earthTextX = (centerX + ptC.x) / 2;

  if (earthTextX - earthTextWidth / 2 < 10) {
    earthTextX = 10 + earthTextWidth / 2;
  } else if (earthTextX + earthTextWidth / 2 > width - 10) {
    earthTextX = width - 10 - earthTextWidth / 2;
  }

  ctx.textAlign = "center";
  ctx.fillText(earthLabel, earthTextX, earthY + 16);
}
}

```

```

// Animated Photons
if (togglePhoton.checked) {
  const rayOffset = (idx / sim.numRays);
  const progress = (photonAnimProgress + rayOffset) % 1.0;
  let px = 0, py = 0;

  if (sim.M_tril === 0) {
    px = ptA.x + progress * (ptC.x - ptA.x);
    py = ptA.y + progress * (ptC.y - ptA.y);
  } else {
    const d1 = Math.hypot(ptB.x - ptA.x, ptB.y - ptA.y);
    const d2 = Math.hypot(ptC.x - ptB.x, ptC.y - ptB.y);
    const dTotal = d1 + d2;
    const f1 = dTotal > 0 ? d1 / dTotal : 0.5;

    if (progress <= f1) {
      const t = f1 > 0 ? progress / f1 : 0;
      px = ptA.x + t * (ptB.x - ptA.x);
      py = ptA.y + t * (ptB.y - ptA.y);
    } else {
      const t = (1 - f1) > 0 ? (progress - f1) / (1 - f1) : 0;
      px = ptB.x + t * (ptC.x - ptB.x);
      py = ptB.y + t * (ptC.y - ptB.y);
    }
  }

  ctx.fillStyle = "#ffffff";
  ctx.shadowColor = "#00f0ff";
  ctx.shadowBlur = 6;
  ctx.beginPath();
  ctx.arc(px, py, 3, 0, Math.PI * 2);
  ctx.fill();
  ctx.shadowBlur = 0;
}
});

if (togglePhoton.checked) {
  photonAnimProgress = (photonAnimProgress + 0.006) % 1.0;
}

// Draw Celestial Objects

// 1. Distant Galaxy (Top)
ctx.save();
ctx.translate(ptA.x, ptA.y);
const galGlow = ctx.createRadialGradient(0, 0, 2, 0, 0, 18);
galGlow.addColorStop(0, "rgba(255, 200, 100, 0.9)");
galGlow.addColorStop(0.5, "rgba(238, 100, 30, 0.3)");
galGlow.addColorStop(1, "rgba(0, 0, 0, 0)");
ctx.fillStyle = galGlow;
ctx.beginPath();
ctx.arc(0, 0, 18, 0, Math.PI * 2);
ctx.fill();

ctx.fillStyle = "#ffd59e";
ctx.beginPath();
ctx.ellipse(0, 0, 12, 4, -0.3, 0, Math.PI * 2);
ctx.fill();
ctx.restore();

ctx.fillStyle = "#e1e7ec";
ctx.font = "11px sans-serif";
ctx.textAlign = "center";
ctx.fillText("Distant Galaxy", ptA.x, ptA.y - 15);

// 2. Intermediate Cluster
if (sim.clusterEnabled && sim.M_tril > 0) {
  ctx.save();
  ctx.translate(centerX, clusterY);

  const massRatio = Math.max(0, Math.min(1, (sim.M_tril - 100) / 900));
  const haloRadius = 16 + massRatio * 46;

```

```

const clusterGlow = ctx.createRadialGradient(0, 0, 2, 0, 0, haloRadius);
const alphaCenter = 0.5 + massRatio * 0.45;
const alphaMid = 0.25 + massRatio * 0.55;

clusterGlow.addColorStop(0, `rgba(216, 180, 254, ${alphaCenter})`);
clusterGlow.addColorStop(0.35, `rgba(168, 85, 247, ${alphaMid})`);
clusterGlow.addColorStop(0.7, `rgba(126, 34, 206, ${alphaMid * 0.5})`);
clusterGlow.addColorStop(1, "rgba(0, 0, 0, 0)");

ctx.fillStyle = clusterGlow;
ctx.beginPath();
ctx.arc(0, 0, haloRadius, 0, Math.PI * 2);
ctx.fill();

const numGalaxies = Math.floor(3 + massRatio * 27);
const spreadScale = 0.6 + massRatio * 0.6;

for (let i = 0; i < numGalaxies; i++) {
  const g = clusterGalaxyList[i];
  const gx = g.x * spreadScale;
  const gy = g.y * spreadScale;

  ctx.fillStyle = i === 0 ? "#ffffff" : `rgba(255, 245, 255, ${0.75 + (1 - i / 30) * 0.25})`;
  ctx.beginPath();
  ctx.arc(gx, gy, g.r, 0, Math.PI * 2);
  ctx.fill();

  if (g.r >= 2.0) {
    ctx.fillStyle = `rgba(192, 132, 252, ${0.3 + massRatio * 0.3})`;
    ctx.beginPath();
    ctx.arc(gx, gy, g.r * 1.8, 0, Math.PI * 2);
    ctx.fill();
  }
}

ctx.restore();

ctx.fillStyle = "#e1e7ec";
ctx.font = "11px sans-serif";
ctx.textAlign = "center";
ctx.fillText("Intermediate Cluster", centerX, clusterY + haloRadius + 14);
}

// 3. Earth / Observer (Bottom)
ctx.save();
ctx.translate(centerX, earthY);
const earthGrad = ctx.createRadialGradient(-2, -2, 1, 0, 0, 8);
earthGrad.addColorStop(0, "#60a5fa");
earthGrad.addColorStop(0.8, "#1d4ed8");
earthGrad.addColorStop(1, "#1e3a8a");
ctx.fillStyle = earthGrad;
ctx.beginPath();
ctx.arc(0, 0, 8, 0, Math.PI * 2);
ctx.fill();
ctx.restore();

ctx.fillStyle = "#e1e7ec";
ctx.font = "11px sans-serif";
ctx.textAlign = "center";
ctx.fillText("Earth", centerX, earthY + 26);

// Bottom Note inside Graphic Window
ctx.fillStyle = "rgba(140, 155, 174, 0.75)";
ctx.font = "italic 11px sans-serif";
ctx.textAlign = "center";
ctx.fillText("angles are exaggerated", centerX, height - 12);
}

function animate() {
  draw();
  requestAnimationFrame(animate);
}

animate();

```

```

</script>
</body>
</html>

```

User prompt: L

Response: It looks like that message was cut short. How would you like to adjust or expand the gravitational lensing simulation?

User prompt: Add two input elements: 1) a toggle for enabling manual scaling of the exaggeration factor that is used to display the rays after lensing (default to off) 2) a slider for increasing or decreasing that exaggeration factor. Continue with the current default approach for drawing the rays from the lensing cluster to the Earth's y-position. However, when the manual scaling toggle is activated, it allows the user to scale the displayed impact parameter (values from 0.01 to 1), thus causing the rays to appear at a less extreme angle.

```

Response: <!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Gravitational Lensing: Vertical Field Simulation</title>
  <style>
    :root {
      --bg-color: #0b0e14;
      --panel-bg: #151922;
      --panel-border: #2a3142;
      --accent-color: #00f0ff;
      --accent-amber: #facc15;
      --accent-red: #ef4444;
      --text-main: #e1e7ec;
      --text-muted: #8c9bae;
    }

    * {
      box-sizing: border-box;
      margin: 0;
      padding: 0;
    }

    body {
      font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial, sans-serif;
      background-color: var(--bg-color);
      color: var(--text-main);
      display: flex;
      flex-direction: column;
      min-height: 100vh;
      overflow-x: hidden;
    }

    header {
      background-color: var(--panel-bg);
      border-bottom: 1px solid var(--panel-border);
      padding: 0.85rem 1.5rem;
      text-align: center;
    }

    header h1 {
      font-size: 1.3rem;
      font-weight: 600;
      color: #fff;
    }

    header p {
      font-size: 0.85rem;
      color: var(--text-muted);
      margin-top: 0.2rem;
    }

    .main-container {
      display: flex;
      flex: 1;
      flex-direction: column;
      gap: 1rem;

```

```

padding: 1rem;
max-width: 1200px;
margin: 0 auto;
width: 100%;
align-items: center;
}

@media (min-width: 1024px) {
  .main-container {
    flex-direction: row;
    align-items: flex-start;
  }
}

.canvas-card {
  flex: 1;
  width: 100%;
  background-color: var(--panel-bg);
  border: 1px solid var(--panel-border);
  border-radius: 8px;
  padding: 1rem;
  display: flex;
  flex-direction: column;
  gap: 1rem;
  box-shadow: 0 4px 20px rgba(0, 0, 0, 0.4);
}

.canvas-wrapper {
  position: relative;
  width: 100%;
  max-width: 520px;
  aspect-ratio: 3 / 4;
  margin: 0 auto;
  background-color: #04060a;
  border-radius: 6px;
  overflow: hidden;
  border: 1px solid rgba(255, 255, 255, 0.05);
}

canvas {
  width: 100%;
  height: 100%;
  display: block;
}

.calc-box {
  background: rgba(10, 14, 22, 0.7);
  border: 1px solid var(--panel-border);
  border-radius: 6px;
  padding: 1rem 1.25rem;
  font-family: SFMono-Regular, Consolas, "Liberation Mono", Menlo, monospace;
  font-size: 0.8rem;
  color: #d1d5db;
}

.calc-box h3 {
  font-size: 0.85rem;
  text-transform: uppercase;
  letter-spacing: 0.05em;
  color: var(--accent-color);
  margin-bottom: 0.75rem;
  border-bottom: 1px solid rgba(255, 255, 255, 0.1);
  padding-bottom: 0.4rem;
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;
}

.calc-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
  gap: 0.85rem 1.25rem;
}

.calc-item {
  display: flex;

```



```
    flex-direction: column;
    gap: 0.25rem;
}

.calc-item .label {
    color: var(--text-muted);
    font-size: 0.75rem;
}

.calc-item .value {
    color: #fff;
    font-weight: 600;
    font-size: 0.85rem;
}

.calc-item .highlight {
    color: var(--accent-amber);
}

.calc-footer {
    margin-top: 0.85rem;
    padding-top: 0.6rem;
    border-top: 1px dashed rgba(255, 255, 255, 0.15);
    display: flex;
    justify-content: space-between;
    align-items: center;
    font-size: 0.75rem;
}

.calc-footer .formula {
    color: #9ca3af;
}

.controls-panel {
    width: 100%;
    max-width: 380px;
    background-color: var(--panel-bg);
    border: 1px solid var(--panel-border);
    border-radius: 8px;
    padding: 1.25rem;
    display: flex;
    flex-direction: column;
    gap: 1.1rem;
    box-shadow: 0 4px 20px rgba(0, 0, 0, 0.4);
}

.control-group {
    display: flex;
    flex-direction: column;
    gap: 0.4rem;
    transition: opacity 0.2s ease;
}

.control-group.disabled-control {
    opacity: 0.35;
}

.control-group label {
    font-size: 0.83rem;
    font-weight: 500;
    color: var(--text-main);
    display: flex;
    justify-content: space-between;
}

.control-group label span {
    color: var(--accent-color);
    font-family: monospace;
}

input[type="range"] {
    -webkit-appearance: none;
    width: 100%;
    height: 6px;
}
```

```
border-radius: 3px;
background: #2a3142;
outline: none;
}

input[type="range"]:disabled {
  cursor: not-allowed;
}

input[type="range"]::-webkit-slider-thumb {
  -webkit-appearance: none;
  width: 16px;
  height: 16px;
  border-radius: 50%;
  background: var(--accent-color);
  cursor: pointer;
  box-shadow: 0 0 8px var(--accent-color);
  transition: transform 0.1s ease;
}

input[type="range"]:disabled::-webkit-slider-thumb {
  background: #6b7280;
  box-shadow: none;
  cursor: not-allowed;
}

input[type="range"]::-webkit-slider-thumb:hover:not(:disabled) {
  transform: scale(1.2);
}

.toggle-row {
  display: flex;
  align-items: center;
  justify-content: space-between;
  font-size: 0.83rem;
  color: var(--text-main);
}

.switch {
  position: relative;
  display: inline-block;
  width: 40px;
  height: 20px;
}

.switch input {
  opacity: 0;
  width: 0;
  height: 0;
}

.slider {
  position: absolute;
  cursor: pointer;
  top: 0; left: 0; right: 0; bottom: 0;
  background-color: #2a3142;
  transition: .2s;
  border-radius: 20px;
}

.slider:before {
  position: absolute;
  content: "";
  height: 14px;
  width: 14px;
  left: 3px;
  bottom: 3px;
  background-color: white;
  transition: .2s;
  border-radius: 50%;
}

input:checked + .slider {
  background-color: var(--accent-color);
}
```

```

}

input:checked + .slider:before {
  transform: translateX(20px);
}

.legend {
  border-top: 1px solid var(--panel-border);
  padding-top: 0.8rem;
  display: flex;
  flex-direction: column;
  gap: 0.4rem;
  font-size: 0.8rem;
}

.legend-item {
  display: flex;
  align-items: center;
  gap: 0.6rem;
  color: var(--text-muted);
}

.legend-color {
  width: 14px;
  height: 4px;
  border-radius: 2px;
}
</style>
</head>
<body>

<header>
  <h1>Gravitational Deflection of Light Fan</h1>
  <p>Vertical Propagation View (Distant Galaxy at Top, Earth at Bottom)</p>
</header>

<div class="main-container">
  <div class="canvas-card">
    <div class="canvas-wrapper">
      <canvas id="simCanvas"></canvas>
    </div>

    <!-- Numerical Output Box below Graphic -->
    <div class="calc-box">
      <h3>Numerical Results</h3>
      <div class="calc-grid">
        <div class="calc-item">
          <span class="label">Galaxy Distance:</span>
          <span class="value" id="boxDistG">8.0 billion light-years</span>
        </div>
        <div class="calc-item">
          <span class="label">Galaxy Offset:</span>
          <span class="value" id="boxOffsetX">0 thousand light-years</span>
        </div>
        <div class="calc-item">
          <span class="label">Cluster Distance:</span>
          <span class="value" id="boxDistC">4.0 billion light-years</span>
        </div>
        <div class="calc-item">
          <span class="label">Galaxy-Cluster Distance:</span>
          <span class="value" id="boxDistGC">4.0 billion light-years</span>
        </div>
        <div class="calc-item">
          <span class="label">Cluster Mass:</span>
          <span class="value" id="boxMass">200 trillion solar masses</span>
        </div>
        <div class="calc-item">
          <span class="label">Ray Mode:</span>
          <span class="value" id="boxMode">Single Ray</span>
        </div>
        <div class="calc-item">
          <span class="label">Cluster Impact:</span>
          <span class="value highlight" id="boxImpactRange">290.9 thousand light-years</span>
        </div>
      </div>
    </div>
  </div>
</div>

```

244/271

```

<label for="sliderTheta">
  7. Initial Ray Direction (1 ray):
  <span id="valTheta">+15.0 arc-seconds</span>
</label>
<input type="range" id="sliderTheta" min="-40.0" max="40.0" step="0.5" value="15.0">
</div>

<div class="toggle-row">
  <span>Enable Manual Scaling</span>
  <label class="switch">
    <input type="checkbox" id="toggleManualScale">
    <span class="slider"></span>
  </label>
</div>

<div class="control-group disabled-control" id="grpExaggeration">
  <label for="sliderExaggeration">
    Exaggeration Factor:
    <span id="valExaggeration">1.00x</span>
  </label>
  <input type="range" id="sliderExaggeration" min="0.01" max="1.00" step="0.01" value="1.00" disabled>
</div>

<div class="toggle-row">
  <span>Animate Photons</span>
  <label class="switch">
    <input type="checkbox" id="togglePhoton" checked>
    <span class="slider"></span>
  </label>
</div>

<div class="toggle-row">
  <span>Show Virtual Tangent Lines</span>
  <label class="switch">
    <input type="checkbox" id="toggleTangent" checked>
    <span class="slider"></span>
  </label>
</div>

<div class="toggle-row">
  <span>Show Impact Parameters</span>
  <label class="switch">
    <input type="checkbox" id="toggleImpact" checked>
    <span class="slider"></span>
  </label>
</div>

<div class="toggle-row">
  <span>Show Background Stars</span>
  <label class="switch">
    <input type="checkbox" id="toggleStars" checked>
    <span class="slider"></span>
  </label>
</div>

<div class="legend">
  <div class="legend-item">
    <div class="legend-color" style="background: #facc15;"></div>
    <span>Actual deflected light rays</span>
  </div>
  <div class="legend-item">
    <div class="legend-color" style="border: 1px dashed #ef4444; background: transparent;"></div>
    <span>Apparent straight-line paths (virtual images)</span>
  </div>
  <div class="legend-item">
    <div class="legend-color" style="background: #00f0ff;"></div>
    <span>Cluster impact parameter & Earth impact</span>
  </div>
</div>
</div>
</div>

<script>
  const canvas = document.getElementById('simCanvas');

```

```

const ctx = canvas.getContext('2d');

// Canvas Fixed Total Height Scale (13.5 Billion Light-Years)
const CANVAS_TOTAL_GLY = 13.5;

// Controls
const toggleCluster = document.getElementById('toggleCluster');
const sliderDistG = document.getElementById('sliderDistG');
const sliderOffsetX = document.getElementById('sliderOffsetX');
const sliderDistC = document.getElementById('sliderDistC');
const sliderMass = document.getElementById('sliderMass');
const sliderNumRays = document.getElementById('sliderNumRays');
const sliderFanSpread = document.getElementById('sliderFanSpread');
const sliderTheta = document.getElementById('sliderTheta');
const toggleManualScale = document.getElementById('toggleManualScale');
const sliderExaggeration = document.getElementById('sliderExaggeration');

const grpDistC = document.getElementById('grpDistC');
const grpMass = document.getElementById('grpMass');
const grpFanSpread = document.getElementById('grpFanSpread');
const grpTheta = document.getElementById('grpTheta');
const grpExaggeration = document.getElementById('grpExaggeration');

const valDistG = document.getElementById('valDistG');
const valOffsetX = document.getElementById('valOffsetX');
const valDistC = document.getElementById('valDistC');
const valMass = document.getElementById('valMass');
const valNumRays = document.getElementById('valNumRays');
const valFanSpread = document.getElementById('valFanSpread');
const valTheta = document.getElementById('valTheta');
const valExaggeration = document.getElementById('valExaggeration');

const togglePhoton = document.getElementById('togglePhoton');
const toggleTangent = document.getElementById('toggleTangent');
const toggleImpact = document.getElementById('toggleImpact');
const toggleStars = document.getElementById('toggleStars');

// Calculation Box Elements
const boxDistG = document.getElementById('boxDistG');
const boxOffsetX = document.getElementById('boxOffsetX');
const boxDistC = document.getElementById('boxDistC');
const boxDistGC = document.getElementById('boxDistGC');
const boxMass = document.getElementById('boxMass');
const boxMode = document.getElementById('boxMode');
const boxImpactRange = document.getElementById('boxImpactRange');
const boxEarthImpactRange = document.getElementById('boxEarthImpactRange');
const boxAlphaRange = document.getElementById('boxAlphaRange');

let width = 0;
let height = 0;
let photonAnimProgress = 0;

// Background Stars
const stars = [];
for (let i = 0; i < 220; i++) {
  stars.push({
    x: Math.random(),
    y: Math.random(),
    size: Math.random() * 1.5 + 0.3,
    alpha: Math.random() * 0.7 + 0.3
  });
}

// Cluster Member Galaxies layout offsets (up to 30 member galaxies)
const clusterGalaxyList = [
  { x: 0, y: 0, r: 3.5 },
  { x: -6, y: -5, r: 2.2 },
  { x: 8, y: 4, r: 2.5 },
  { x: -12, y: 8, r: 2.0 },
  { x: 10, y: -10, r: 2.1 },
  { x: 5, y: -14, r: 1.8 },
  { x: -15, y: -6, r: 2.3 },
  { x: 18, y: -2, r: 1.9 },
  { x: -8, y: 16, r: 2.2 },

```

```

    { x: 14, y: 12, r: 1.7 },
    { x: 2, y: 18, r: 2.0 },
    { x: -20, y: 4, r: 1.8 },
    { x: -4, y: -20, r: 1.9 },
    { x: 22, y: -12, r: 1.6 },
    { x: 12, y: -22, r: 1.8 },
    { x: -22, y: -15, r: 1.7 },
    { x: -16, y: 22, r: 1.9 },
    { x: 25, y: 8, r: 1.5 },
    { x: 6, y: 26, r: 1.7 },
    { x: -28, y: -4, r: 1.6 },
    { x: 18, y: 22, r: 1.5 },
    { x: -10, y: -28, r: 1.8 },
    { x: 28, y: -20, r: 1.4 },
    { x: -25, y: 18, r: 1.6 },
    { x: 0, y: -32, r: 1.5 },
    { x: 32, y: 2, r: 1.4 },
    { x: -32, y: -18, r: 1.5 },
    { x: 20, y: -30, r: 1.4 },
    { x: -20, y: -32, r: 1.3 },
    { x: 15, y: 32, r: 1.4 }
  ];

function resizeCanvas() {
  const rect = canvas.parentElement.getBoundingClientRect();
  const dpr = window.devicePixelRatio || 1;
  width = rect.width;
  height = rect.height;

  canvas.width = width * dpr;
  canvas.height = height * dpr;
  ctx.scale(dpr, dpr);
}

window.addEventListener('resize', resizeCanvas);
resizeCanvas();

// Distance constraints: d_G >= d_C + 1.0 Gly
sliderDistG.addEventListener('input', () => {
  let dG = parseFloat(sliderDistG.value);
  let dC = parseFloat(sliderDistC.value);
  if (dG < dC + 1.0) {
    dC = Math.max(1.0, dG - 1.0);
    sliderDistC.value = dC.toFixed(1);
  }
  updateControlStates();
});

sliderDistC.addEventListener('input', () => {
  let dG = parseFloat(sliderDistG.value);
  let dC = parseFloat(sliderDistC.value);
  if (dC > dG - 1.0) {
    dG = Math.min(13.0, dC + 1.0);
    sliderDistG.value = dG.toFixed(1);
  }
  updateControlStates();
});

toggleCluster.addEventListener('change', updateControlStates);
sliderMass.addEventListener('input', updateControlStates);
sliderNumRays.addEventListener('input', updateControlStates);
toggleManualScale.addEventListener('change', updateControlStates);
sliderExaggeration.addEventListener('input', updateControlStates);

function updateControlStates() {
  const numRays = parseInt(sliderNumRays.value);
  const clusterEnabled = toggleCluster.checked;
  const manualScaleEnabled = toggleManualScale.checked;

  if (!clusterEnabled) {
    sliderMass.disabled = true;
    grpMass.classList.add('disabled-control');
    sliderDistC.disabled = true;
    grpDistC.classList.add('disabled-control');
  }
}

```

```

} else {
  sliderMass.disabled = false;
  grpMass.classList.remove('disabled-control');
  sliderDistC.disabled = false;
  grpDistC.classList.remove('disabled-control');
}

if (numRays > 1) {
  sliderTheta.disabled = true;
  grpTheta.classList.add('disabled-control');

  sliderFanSpread.disabled = false;
  grpFanSpread.classList.remove('disabled-control');
} else {
  sliderTheta.disabled = false;
  grpTheta.classList.remove('disabled-control');

  sliderFanSpread.disabled = true;
  grpFanSpread.classList.add('disabled-control');
}

if (!manualScaleEnabled) {
  sliderExaggeration.disabled = true;
  grpExaggeration.classList.add('disabled-control');
} else {
  sliderExaggeration.disabled = false;
  grpExaggeration.classList.remove('disabled-control');
}
}

updateControlStates();

function calculateRays() {
  const clusterEnabled = toggleCluster.checked;
  const d_G_Gly = parseFloat(sliderDistG.value);
  const x_G_kly = parseFloat(sliderOffsetX.value);
  const d_C_Gly = parseFloat(sliderDistC.value);
  const d_GC_Gly = d_G_Gly - d_C_Gly;
  const M_tril = clusterEnabled ? parseFloat(sliderMass.value) : 0;
  const numRays = parseInt(sliderNumRays.value);
  const fanSpreadArcsec = parseFloat(sliderFanSpread.value);
  const thetaInitArcsec = parseFloat(sliderTheta.value);

  const ARCSEC_TO_RAD = Math.PI / (180.0 * 3600.0);
  const d_GC_ly = d_GC_Gly * 1e9;
  const d_C_ly = d_C_Gly * 1e9;
  const x_G_ly = x_G_kly * 1e3;

  const scale_4GM_c2_ly = 0.6243363 * M_tril;

  const rays = [];

  for (let i = 0; i < numRays; i++) {
    let initialAngle = 0;

    if (numRays === 1) {
      initialAngle = thetaInitArcsec;
    } else {
      const frac = (i / (numRays - 1)) - 0.5;
      initialAngle = frac * fanSpreadArcsec;
    }

    const theta_init_rad = initialAngle * ARCSEC_TO_RAD;

    const b_C_ly = x_G_ly + d_GC_ly * Math.tan(theta_init_rad);
    const abs_b_C_ly = Math.abs(b_C_ly);

    let alpha_arcsec = 0;
    let alpha_rad = 0;
    if (M_tril > 0 && abs_b_C_ly > 1e-3) {
      alpha_rad = scale_4GM_c2_ly / abs_b_C_ly;
      alpha_arcsec = alpha_rad / ARCSEC_TO_RAD;
    }
  }
}

```



```

let theta_after_rad = theta_init_rad;
if (M_tril > 0) {
  if (b_C_ly > 0) {
    theta_after_rad = theta_init_rad - alpha_rad;
  } else if (b_C_ly < 0) {
    theta_after_rad = theta_init_rad + alpha_rad;
  }
}
const theta_after_arcsec = theta_after_rad / ARCSEC_TO_RAD;

const b_E_ly = b_C_ly + d_C_ly * Math.tan(theta_after_rad);
const abs_b_E_ly = Math.abs(b_E_ly);

rays.push({
  index: i,
  theta_init_arcsec: initialAngle,
  b_C_ly: b_C_ly,
  abs_b_C_ly: abs_b_C_ly,
  b_E_ly: b_E_ly,
  abs_b_E_ly: abs_b_E_ly,
  alpha_arcsec: alpha_arcsec,
  theta_after_arcsec: theta_after_arcsec
});
}

return {
  clusterEnabled,
  d_G_Gly,
  x_G_kly,
  d_C_Gly,
  d_GC_Gly,
  M_tril,
  numRays,
  fanSpreadArcsec,
  thetaInitArcsec,
  rays
};
}

function draw() {
  ctx.clearRect(0, 0, width, height);

  const sim = calculateRays();

  // Exaggeration Scale Calculation
  const displayScale = toggleManualScale.checked ? parseFloat(sliderExaggeration.value) : 1.0;

  // Label Updates
  valDistG.textContent = `${sim.d_G_Gly.toFixed(1)} billion light-years`;
  valOffsetX.textContent = `${sim.x_G_kly >= 0 ? '+' : ''}${sim.x_G_kly} thousand light-years`;
  valDistC.textContent = !sim.clusterEnabled ? "N/A" : `${sim.d_C_Gly.toFixed(1)} billion light-years`;
  valMass.textContent = !sim.clusterEnabled ? "Disabled" : `${sim.M_tril} trillion solar masses`;
  valNumRays.textContent = `${sim.numRays}`;
  valFanSpread.textContent = `${sim.fanSpreadArcsec} arc-seconds`;
  valTheta.textContent = `${sim.thetaInitArcsec >= 0 ? '+' : ''}${sim.thetaInitArcsec.toFixed(1)} arc-sec`;
  valExaggeration.textContent = `${displayScale.toFixed(2)}x`;

  // Update Numerical Box
  boxDistG.textContent = `${sim.d_G_Gly.toFixed(1)} billion light-years`;
  boxOffsetX.textContent = `${sim.x_G_kly >= 0 ? '+' : ''}${sim.x_G_kly} thousand light-years`;
  boxDistC.textContent = !sim.clusterEnabled ? "N/A" : `${sim.d_C_Gly.toFixed(1)} billion light-years`;
  boxDistGC.textContent = !sim.clusterEnabled ? "N/A" : `${sim.d_GC_Gly.toFixed(1)} billion light-years`;
  boxMass.textContent = !sim.clusterEnabled ? "None" : `${sim.M_tril} trillion solar masses`;

  if (sim.numRays === 1) {
    const r = sim.rays[0];
    boxMode.textContent = `Single Ray`;
    boxImpactRange.textContent = sim.clusterEnabled ? `${(r.abs_b_C_ly / 1000.0).toFixed(2)} thousand light-years` : `0.00 arc-seconds`;
    boxEarthImpactRange.textContent = `${(r.abs_b_E_ly / 1000.0).toFixed(2)} thousand light-years`;
    boxAlphaRange.textContent = sim.clusterEnabled ? `${r.alpha_arcsec.toFixed(2)} arc-seconds` : "0.00 arc-seconds";
  } else {
    boxMode.textContent = `Fan (${sim.numRays} rays)`;
    const bCValues = sim.rays.map(r => r.abs_b_C_ly / 1000.0);
  }
}

```

```

const bEValues = sim.rays.map(r => r.abs_b_E_ly / 1000.0);
const alphaValues = sim.rays.map(r => r.alpha_arcsec).filter(a => isFinite(a));

const minBC = Math.min(...bCValues);
const maxBC = Math.max(...bCValues);
const minBE = Math.min(...bEValues);
const maxBE = Math.max(...bEValues);
const maxAlpha = Math.max(...alphaValues);

boxImpactRange.textContent = sim.clusterEnabled ? `${minBC.toFixed(1)} to ${maxBC.toFixed(1)} thousand` : ``;
boxEarthImpactRange.textContent = `${minBE.toFixed(1)} to ${maxBE.toFixed(1)} thousand light-years`;
boxAlphaRange.textContent = sim.clusterEnabled ? `${maxAlpha.toFixed(2)} arc-seconds max` : "0.00 arc-seconds";
}

// Visual Layout Dimensions (Vertical Field)
const topMargin = 45;
const bottomMargin = 55;
const centerX = width / 2;

const earthY = height - bottomMargin;
const drawableHeight = height - topMargin - bottomMargin;
const pxPerGly = drawableHeight / CANVAS_TOTAL_GLY;

const galaxyY = earthY - (sim.d_G_Gly * pxPerGly);
const clusterY = earthY - (sim.d_C_Gly * pxPerGly);

// Horizontal scale modulated by manual exaggeration scale factor
const pxPerKly = (100.0 / 500.0) * displayScale;
const galaxyX = centerX + (sim.x_G_kly * pxPerKly);

const ptA = { x: galaxyX, y: galaxyY };

// Starfield Background
if (toggleStars.checked) {
  ctx.fillStyle = "#ffffff";
  stars.forEach(s => {
    ctx.globalAlpha = s.alpha;
    ctx.fillRect(s.x * width, s.y * height, s.size, s.size);
  });
  ctx.globalAlpha = 1.0;
}

// Central Optical Axis
ctx.strokeStyle = "rgba(255, 255, 255, 0.15)";
ctx.lineWidth = 1;
ctx.setLineDash([6, 6]);
ctx.beginPath();
ctx.moveTo(centerX, earthY + 20);
ctx.lineTo(centerX, earthY - CANVAS_TOTAL_GLY * pxPerGly - 15);
ctx.stroke();
ctx.setLineDash([]);

// Cluster Plane Line
if (sim.clusterEnabled && sim.M_tril > 0) {
  ctx.strokeStyle = "rgba(0, 240, 255, 0.25)";
  ctx.beginPath();
  ctx.moveTo(20, clusterY);
  ctx.lineTo(width - 20, clusterY);
  ctx.stroke();
}

// Earth Plane Line
ctx.strokeStyle = "rgba(0, 240, 255, 0.15)";
ctx.beginPath();
ctx.moveTo(20, earthY);
ctx.lineTo(width - 20, earthY);
ctx.stroke();

// Render Primary Light Rays & Virtual Tangents
sim.rays.forEach((ray, idx) => {
  const clusterX = centerX + (ray.b_C_ly / 1000.0) * pxPerKly;
  const earthX = centerX + (ray.b_E_ly / 1000.0) * pxPerKly;

  const ptB = { x: clusterX, y: clusterY };

```

```

const ptC = { x: earthX, y: earthY };

// Virtual Tangent Line
if (toggleTangent.checked && Math.abs(ray.alpha_arcsec) > 0.1 && sim.M_tril > 0) {
  ctx.strokeStyle = "rgba(239, 68, 68, 0.5)";
  ctx.lineWidth = 1.2;
  ctx.setLineDash([3, 3]);

  const slopeX = (ptC.x - ptB.x) / (ptC.y - ptB.y);
  const virtualX_at_galaxy = ptB.x + slopeX * (galaxyY - clusterY);

  ctx.beginPath();
  ctx.moveTo(ptC.x, ptC.y);
  ctx.lineTo(virtualX_at_galaxy, galaxyY);
  ctx.stroke();
  ctx.setLineDash([]);

  ctx.fillStyle = "rgba(239, 68, 68, 0.8)";
  ctx.beginPath();
  ctx.arc(virtualX_at_galaxy, galaxyY, 3, 0, Math.PI * 2);
  ctx.fill();
}

// Actual Lensed Light Path
ctx.strokeStyle = "#facc15";
ctx.lineWidth = sim.numRays === 1 ? 2.5 : 1.8;
ctx.shadowColor = "#facc15";
ctx.shadowBlur = sim.numRays === 1 ? 6 : 3;

ctx.beginPath();
ctx.moveTo(ptA.x, ptA.y);
if (sim.M_tril > 0) {
  ctx.lineTo(ptB.x, ptB.y);
}
ctx.lineTo(ptC.x, ptC.y);
ctx.stroke();
ctx.shadowBlur = 0;

// Visual Indicators for Impact Parameters
if (toggleImpact.checked && sim.numRays === 1) {
  // Cluster Impact Parameter
  if (sim.clusterEnabled && sim.M_tril > 0 && Math.abs(ptB.x - centerX) > 3) {
    ctx.strokeStyle = "#00f0ff";
    ctx.lineWidth = 1.2;
    ctx.beginPath();
    ctx.moveTo(centerX, clusterY);
    ctx.lineTo(ptB.x, clusterY);
    ctx.stroke();

    ctx.fillStyle = "#00f0ff";
    ctx.beginPath();
    ctx.arc(ptB.x, clusterY, 2.5, 0, Math.PI * 2);
    ctx.arc(centerX, clusterY, 2.5, 0, Math.PI * 2);
    ctx.fill();

    ctx.font = "10px monospace";
    const clusterLabel = `${(ray.abs_b_C_ly / 1000).toFixed(1)} thousand light-years`;
    const textWidth = ctx.measureText(clusterLabel).width;
    const charOffset = 18;

    let labelX = 0;
    let align = "center";

    if (ray.theta_init_arcsec < 0) {
      labelX = ptB.x + charOffset;
      align = "left";
      if (labelX + textWidth > width - 10) {
        labelX = width - 10 - textWidth;
      }
      if (labelX < 10) {
        labelX = 10;
      }
    } else {
      labelX = ptB.x - charOffset;
    }
  }
}

```

```

    align = "right";
    if (labelX - textWidth < 10) {
        labelX = 10 + textWidth;
    }
    if (labelX > width - 10) {
        labelX = width - 10;
    }
}

ctx.textAlign = align;
ctx.fillText(clusterLabel, labelX, clusterY - 6);
}

// Earth Impact Parameter
if (Math.abs(ptC.x - centerX) > 3) {
    ctx.strokeStyle = "#00f0ff";
    ctx.lineWidth = 1.2;
    ctx.beginPath();
    ctx.moveTo(centerX, earthY);
    ctx.lineTo(ptC.x, earthY);
    ctx.stroke();

    ctx.fillStyle = "#00f0ff";
    ctx.beginPath();
    ctx.arc(ptC.x, earthY, 2.5, 0, Math.PI * 2);
    ctx.arc(centerX, earthY, 2.5, 0, Math.PI * 2);
    ctx.fill();

    ctx.font = "10px monospace";
    const earthLabel = `${(ray.abs_b_E_ly / 1000).toFixed(1)} thousand light-years`;
    const earthTextWidth = ctx.measureText(earthLabel).width;
    let earthTextX = (centerX + ptC.x) / 2;

    if (earthTextX - earthTextWidth / 2 < 10) {
        earthTextX = 10 + earthTextWidth / 2;
    } else if (earthTextX + earthTextWidth / 2 > width - 10) {
        earthTextX = width - 10 - earthTextWidth / 2;
    }

    ctx.textAlign = "center";
    ctx.fillText(earthLabel, earthTextX, earthY + 16);
}
}

// Animated Photons
if (togglePhoton.checked) {
    const rayOffset = (idx / sim.numRays);
    const progress = (photonAnimProgress + rayOffset) % 1.0;
    let px = 0, py = 0;

    if (sim.M_tril === 0) {
        px = ptA.x + progress * (ptC.x - ptA.x);
        py = ptA.y + progress * (ptC.y - ptA.y);
    } else {
        const d1 = Math.hypot(ptB.x - ptA.x, ptB.y - ptA.y);
        const d2 = Math.hypot(ptC.x - ptB.x, ptC.y - ptB.y);
        const dTotal = d1 + d2;
        const f1 = dTotal > 0 ? d1 / dTotal : 0.5;

        if (progress <= f1) {
            const t = f1 > 0 ? progress / f1 : 0;
            px = ptA.x + t * (ptB.x - ptA.x);
            py = ptA.y + t * (ptB.y - ptA.y);
        } else {
            const t = (1 - f1) > 0 ? (progress - f1) / (1 - f1) : 0;
            px = ptB.x + t * (ptC.x - ptB.x);
            py = ptB.y + t * (ptC.y - ptB.y);
        }
    }

    ctx.fillStyle = "#ffffff";
    ctx.shadowColor = "#00f0ff";
    ctx.shadowBlur = 6;
    ctx.beginPath();

```

```

    ctx.arc(px, py, 3, 0, Math.PI * 2);
    ctx.fill();
    ctx.shadowBlur = 0;
  }
});

if (togglePhoton.checked) {
  photonAnimProgress = (photonAnimProgress + 0.006) % 1.0;
}

// Draw Celestial Objects

// 1. Distant Galaxy (Top)
ctx.save();
ctx.translate(ptA.x, ptA.y);
const galGlow = ctx.createRadialGradient(0, 0, 2, 0, 0, 18);
galGlow.addColorStop(0, "rgba(255, 200, 100, 0.9)");
galGlow.addColorStop(0.5, "rgba(238, 100, 30, 0.3)");
galGlow.addColorStop(1, "rgba(0, 0, 0, 0)");
ctx.fillStyle = galGlow;
ctx.beginPath();
ctx.arc(0, 0, 18, 0, Math.PI * 2);
ctx.fill();

ctx.fillStyle = "#ffd59e";
ctx.beginPath();
ctx.ellipse(0, 0, 12, 4, -0.3, 0, Math.PI * 2);
ctx.fill();
ctx.restore();

ctx.fillStyle = "#e1e7ec";
ctx.font = "11px sans-serif";
ctx.textAlign = "center";
ctx.fillText("Distant Galaxy", ptA.x, ptA.y - 15);

// 2. Intermediate Cluster
if (sim.clusterEnabled && sim.M_tril > 0) {
  ctx.save();
  ctx.translate(centerX, clusterY);

  const massRatio = Math.max(0, Math.min(1, (sim.M_tril - 100) / 900));
  const haloRadius = 16 + massRatio * 46;

  const clusterGlow = ctx.createRadialGradient(0, 0, 2, 0, 0, haloRadius);
  const alphaCenter = 0.5 + massRatio * 0.45;
  const alphaMid = 0.25 + massRatio * 0.55;

  clusterGlow.addColorStop(0, `rgba(216, 180, 254, ${alphaCenter})`);
  clusterGlow.addColorStop(0.35, `rgba(168, 85, 247, ${alphaMid})`);
  clusterGlow.addColorStop(0.7, `rgba(126, 34, 206, ${alphaMid * 0.5})`);
  clusterGlow.addColorStop(1, "rgba(0, 0, 0, 0)");

  ctx.fillStyle = clusterGlow;
  ctx.beginPath();
  ctx.arc(0, 0, haloRadius, 0, Math.PI * 2);
  ctx.fill();

  const numGalaxies = Math.floor(3 + massRatio * 27);
  const spreadScale = 0.6 + massRatio * 0.6;

  for (let i = 0; i < numGalaxies; i++) {
    const g = clusterGalaxyList[i];
    const gx = g.x * spreadScale;
    const gy = g.y * spreadScale;

    ctx.fillStyle = i === 0 ? "#ffffff" : `rgba(255, 245, 255, ${0.75 + (1 - i / 30) * 0.25})`;
    ctx.beginPath();
    ctx.arc(gx, gy, g.r, 0, Math.PI * 2);
    ctx.fill();

    if (g.r >= 2.0) {
      ctx.fillStyle = `rgba(192, 132, 252, ${0.3 + massRatio * 0.3})`;
      ctx.beginPath();
      ctx.arc(gx, gy, g.r * 1.8, 0, Math.PI * 2);
    }
  }
}

```

```

        ctx.fill();
    }
}

ctx.restore();

ctx.fillStyle = "#e1e7ec";
ctx.font = "11px sans-serif";
ctx.textAlign = "center";
ctx.fillText("Intermediate Cluster", centerX, clusterY + haloRadius + 14);
}

// 3. Earth / Observer (Bottom)
ctx.save();
ctx.translate(centerX, earthY);
const earthGrad = ctx.createRadialGradient(-2, -2, 1, 0, 0, 8);
earthGrad.addColorStop(0, "#60a5fa");
earthGrad.addColorStop(0.8, "#1d4ed8");
earthGrad.addColorStop(1, "#1e3a8a");
ctx.fillStyle = earthGrad;
ctx.beginPath();
ctx.arc(0, 0, 8, 0, Math.PI * 2);
ctx.fill();
ctx.restore();

ctx.fillStyle = "#e1e7ec";
ctx.font = "11px sans-serif";
ctx.textAlign = "center";
ctx.fillText("Earth", centerX, earthY + 26);

// Bottom Note inside Graphic Window
ctx.fillStyle = "rgba(140, 155, 174, 0.75)";
ctx.font = "italic 11px sans-serif";
ctx.textAlign = "center";
ctx.fillText("angles are exaggerated", centerX, height - 12);
}

function animate() {
    draw();
    requestAnimationFrame(animate);
}

animate();
</script>
</body>
</html>

```

User prompt: apply the manual exaggeration only to the ray after it passes by the cluster, not before. I want to keep the normal exaggerated appearance of the ray on the path from the distant galaxy to the cluster.

Response: <!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Gravitational Lensing: Vertical Field Simulation</title>
<style>
:root {
--bg-color: #0b0e14;
--panel-bg: #151922;
--panel-border: #2a3142;
--accent-color: #00f0ff;
--accent-amber: #facc15;
--accent-red: #ef4444;
--text-main: #e1e7ec;
--text-muted: #8c9bae;
}
* {
box-sizing: border-box;
margin: 0;
padding: 0;

```
}

body {
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial, sans-serif;
  background-color: var(--bg-color);
  color: var(--text-main);
  display: flex;
  flex-direction: column;
  min-height: 100vh;
  overflow-x: hidden;
}

header {
  background-color: var(--panel-bg);
  border-bottom: 1px solid var(--panel-border);
  padding: 0.85rem 1.5rem;
  text-align: center;
}

header h1 {
  font-size: 1.3rem;
  font-weight: 600;
  color: #fff;
}

header p {
  font-size: 0.85rem;
  color: var(--text-muted);
  margin-top: 0.2rem;
}

.main-container {
  display: flex;
  flex: 1;
  flex-direction: column;
  gap: 1rem;
  padding: 1rem;
  max-width: 1200px;
  margin: 0 auto;
  width: 100%;
  align-items: center;
}

@media (min-width: 1024px) {
  .main-container {
    flex-direction: row;
    align-items: flex-start;
  }
}

.canvas-card {
  flex: 1;
  width: 100%;
  background-color: var(--panel-bg);
  border: 1px solid var(--panel-border);
  border-radius: 8px;
  padding: 1rem;
  display: flex;
  flex-direction: column;
  gap: 1rem;
  box-shadow: 0 4px 20px rgba(0, 0, 0, 0.4);
}

.canvas-wrapper {
  position: relative;
  width: 100%;
  max-width: 520px;
  aspect-ratio: 3 / 4;
  margin: 0 auto;
  background-color: #04060a;
  border-radius: 6px;
  overflow: hidden;
  border: 1px solid rgba(255, 255, 255, 0.05);
}
```

```
canvas {
  width: 100%;
  height: 100%;
  display: block;
}

.calc-box {
  background: rgba(10, 14, 22, 0.7);
  border: 1px solid var(--panel-border);
  border-radius: 6px;
  padding: 1rem 1.25rem;
  font-family: SFMono-Regular, Consolas, "Liberation Mono", Menlo, monospace;
  font-size: 0.8rem;
  color: #d1d5db;
}

.calc-box h3 {
  font-size: 0.85rem;
  text-transform: uppercase;
  letter-spacing: 0.05em;
  color: var(--accent-color);
  margin-bottom: 0.75rem;
  border-bottom: 1px solid rgba(255, 255, 255, 0.1);
  padding-bottom: 0.4rem;
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;
}

.calc-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
  gap: 0.85rem 1.25rem;
}

.calc-item {
  display: flex;
  flex-direction: column;
  gap: 0.25rem;
}

.calc-item .label {
  color: var(--text-muted);
  font-size: 0.75rem;
}

.calc-item .value {
  color: #fff;
  font-weight: 600;
  font-size: 0.85rem;
}

.calc-item .highlight {
  color: var(--accent-amber);
}

.calc-footer {
  margin-top: 0.85rem;
  padding-top: 0.6rem;
  border-top: 1px dashed rgba(255, 255, 255, 0.15);
  display: flex;
  justify-content: space-between;
  align-items: center;
  font-size: 0.75rem;
}

.calc-footer .formula {
  color: #9ca3af;
}

.controls-panel {
  width: 100%;
  max-width: 380px;
  background-color: var(--panel-bg);
  border: 1px solid var(--panel-border);
}
```



```
border-radius: 8px;
padding: 1.25rem;
display: flex;
flex-direction: column;
gap: 1.1rem;
box-shadow: 0 4px 20px rgba(0, 0, 0, 0.4);
}

.control-group {
display: flex;
flex-direction: column;
gap: 0.4rem;
transition: opacity 0.2s ease;
}

.control-group.disabled-control {
opacity: 0.35;
}

.control-group label {
font-size: 0.83rem;
font-weight: 500;
color: var(--text-main);
display: flex;
justify-content: space-between;
}

.control-group label span {
color: var(--accent-color);
font-family: monospace;
}

input[type="range"] {
-webkit-appearance: none;
width: 100%;
height: 6px;
border-radius: 3px;
background: #2a3142;
outline: none;
}

input[type="range"]:disabled {
cursor: not-allowed;
}

input[type="range"]::-webkit-slider-thumb {
-webkit-appearance: none;
width: 16px;
height: 16px;
border-radius: 50%;
background: var(--accent-color);
cursor: pointer;
box-shadow: 0 0 8px var(--accent-color);
transition: transform 0.1s ease;
}

input[type="range"]:disabled::-webkit-slider-thumb {
background: #6b7280;
box-shadow: none;
cursor: not-allowed;
}

input[type="range"]::-webkit-slider-thumb:hover:not(:disabled) {
transform: scale(1.2);
}

.toggle-row {
display: flex;
align-items: center;
justify-content: space-between;
font-size: 0.83rem;
color: var(--text-main);
}
```

```

.switch {
  position: relative;
  display: inline-block;
  width: 40px;
  height: 20px;
}

.switch input {
  opacity: 0;
  width: 0;
  height: 0;
}

.slider {
  position: absolute;
  cursor: pointer;
  top: 0; left: 0; right: 0; bottom: 0;
  background-color: #2a3142;
  transition: .2s;
  border-radius: 20px;
}

.slider:before {
  position: absolute;
  content: "";
  height: 14px;
  width: 14px;
  left: 3px;
  bottom: 3px;
  background-color: white;
  transition: .2s;
  border-radius: 50%;
}

input:checked + .slider {
  background-color: var(--accent-color);
}

input:checked + .slider:before {
  transform: translateX(20px);
}

.legend {
  border-top: 1px solid var(--panel-border);
  padding-top: 0.8rem;
  display: flex;
  flex-direction: column;
  gap: 0.4rem;
  font-size: 0.8rem;
}

.legend-item {
  display: flex;
  align-items: center;
  gap: 0.6rem;
  color: var(--text-muted);
}

.legend-color {
  width: 14px;
  height: 4px;
  border-radius: 2px;
}
</style>
</head>
<body>

<header>
  <h1>Gravitational Deflection of Light Fan</h1>
  <p>Vertical Propagation View (Distant Galaxy at Top, Earth at Bottom)</p>
</header>

<div class="main-container">
  <div class="canvas-card">

```

```

<div class="canvas-wrapper">
  <canvas id="simCanvas"></canvas>
</div>

<!-- Numerical Output Box below Graphic -->
<div class="calc-box">
  <h3>Numerical Results</h3>
  <div class="calc-grid">
    <div class="calc-item">
      <span class="label">Galaxy Distance:</span>
      <span class="value" id="boxDistG">8.0 billion light-years</span>
    </div>
    <div class="calc-item">
      <span class="label">Galaxy Offset:</span>
      <span class="value" id="boxOffsetX">0 thousand light-years</span>
    </div>
    <div class="calc-item">
      <span class="label">Cluster Distance:</span>
      <span class="value" id="boxDistC">4.0 billion light-years</span>
    </div>
    <div class="calc-item">
      <span class="label">Galaxy-Cluster Distance:</span>
      <span class="value" id="boxDistGC">4.0 billion light-years</span>
    </div>
    <div class="calc-item">
      <span class="label">Cluster Mass:</span>
      <span class="value" id="boxMass">200 trillion solar masses</span>
    </div>
    <div class="calc-item">
      <span class="label">Ray Mode:</span>
      <span class="value" id="boxMode">Single Ray</span>
    </div>
    <div class="calc-item">
      <span class="label">Cluster Impact:</span>
      <span class="value highlight" id="boxImpactRange">290.9 thousand light-years</span>
    </div>
    <div class="calc-item">
      <span class="label">Earth Impact:</span>
      <span class="value highlight" id="boxEarthImpactRange">581.8 thousand light-years</span>
    </div>
    <div class="calc-item">
      <span class="label">Deflection Angle:</span>
      <span class="value highlight" id="boxAlphaRange">44.3 arc-seconds</span>
    </div>
  </div>
  <div class="calc-footer">
    <span class="label">Theoretical Formula:</span>
    <span class="formula"> $\hat{\alpha} = 4G M / (c^2 |b_{sub>C</sub>}|) \ \&nbsp;\&nbsp;\&nbsp; b_{sub>E</sub>} = b_{sub>C</sub>} + d_{$ </span>
  </div>
</div>

<div class="controls-panel">
  <div class="toggle-row">
    <span>Enable Intermediate Cluster</span>
    <label class="switch">
      <input type="checkbox" id="toggleCluster" checked>
    </label>
  </div>

  <div class="control-group">
    <label for="sliderDistG">
      1. Distance to Galaxy:
      <span id="valDistG">8.0 billion light-years</span>
    </label>
    <input type="range" id="sliderDistG" min="2.0" max="13.0" step="0.1" value="8.0">
  </div>

  <div class="control-group">
    <label for="sliderOffsetX">
      2. Galaxy Horizontal Offset:
      <span id="valOffsetX">0 thousand light-years</span>
    </label>
  </div>
</div>

```

```

    <input type="range" id="sliderOffsetX" min="-500" max="500" step="10" value="0">
</div>

<div class="control-group" id="grpDistC">
  <label for="sliderDistC">
    3. Distance to Cluster:
    <span id="valDistC">4.0 billion light-years</span>
  </label>
  <input type="range" id="sliderDistC" min="1.0" max="12.0" step="0.1" value="4.0">
</div>

<div class="control-group" id="grpMass">
  <label for="sliderMass">
    4. Cluster Mass:
    <span id="valMass">200 trillion solar masses</span>
  </label>
  <input type="range" id="sliderMass" min="100" max="1000" step="10" value="200">
</div>

<div class="control-group">
  <label for="sliderNumRays">
    5. Number of Light Rays:
    <span id="valNumRays">1</span>
  </label>
  <input type="range" id="sliderNumRays" min="1" max="10" step="1" value="1">
</div>

<div class="control-group" id="grpFanSpread">
  <label for="sliderFanSpread">
    6. Ray Fan Spread:
    <span id="valFanSpread">20 arc-seconds</span>
  </label>
  <input type="range" id="sliderFanSpread" min="10" max="40" step="1" value="20">
</div>

<div class="control-group" id="grpTheta">
  <label for="sliderTheta">
    7. Initial Ray Direction (1 ray):
    <span id="valTheta">+15.0 arc-seconds</span>
  </label>
  <input type="range" id="sliderTheta" min="-40.0" max="40.0" step="0.5" value="15.0">
</div>

<div class="toggle-row">
  <span>Enable Post-Lensing Scale</span>
  <label class="switch">
    <input type="checkbox" id="toggleManualScale">
    <span class="slider"></span>
  </label>
</div>

<div class="control-group disabled-control" id="grpExaggeration">
  <label for="sliderExaggeration">
    Post-Lensing Scale Factor:
    <span id="valExaggeration">1.00x</span>
  </label>
  <input type="range" id="sliderExaggeration" min="0.01" max="1.00" step="0.01" value="1.00" disabled>
</div>

<div class="toggle-row">
  <span>Animate Photons</span>
  <label class="switch">
    <input type="checkbox" id="togglePhoton" checked>
    <span class="slider"></span>
  </label>
</div>

<div class="toggle-row">
  <span>Show Virtual Tangent Lines</span>
  <label class="switch">
    <input type="checkbox" id="toggleTangent" checked>
    <span class="slider"></span>
  </label>
</div>

```

```

<div class="toggle-row">
  <span>Show Impact Parameters</span>
  <label class="switch">
    <input type="checkbox" id="toggleImpact" checked>
    <span class="slider"></span>
  </label>
</div>

<div class="toggle-row">
  <span>Show Background Stars</span>
  <label class="switch">
    <input type="checkbox" id="toggleStars" checked>
    <span class="slider"></span>
  </label>
</div>

<div class="legend">
  <div class="legend-item">
    <div class="legend-color" style="background: #facc15;"></div>
    <span>Actual deflected light rays</span>
  </div>
  <div class="legend-item">
    <div class="legend-color" style="border: 1px dashed #ef4444; background: transparent;"></div>
    <span>Apparent straight-line paths (virtual images)</span>
  </div>
  <div class="legend-item">
    <div class="legend-color" style="background: #00f0ff;"></div>
    <span>Cluster impact parameter & Earth impact</span>
  </div>
</div>
</div>
</div>

<script>
const canvas = document.getElementById('simCanvas');
const ctx = canvas.getContext('2d');

// Canvas Fixed Total Height Scale (13.5 Billion Light-Years)
const CANVAS_TOTAL_GLY = 13.5;

// Controls
const toggleCluster = document.getElementById('toggleCluster');
const sliderDistG = document.getElementById('sliderDistG');
const sliderOffsetX = document.getElementById('sliderOffsetX');
const sliderDistC = document.getElementById('sliderDistC');
const sliderMass = document.getElementById('sliderMass');
const sliderNumRays = document.getElementById('sliderNumRays');
const sliderFanSpread = document.getElementById('sliderFanSpread');
const sliderTheta = document.getElementById('sliderTheta');
const toggleManualScale = document.getElementById('toggleManualScale');
const sliderExaggeration = document.getElementById('sliderExaggeration');

const grpDistC = document.getElementById('grpDistC');
const grpMass = document.getElementById('grpMass');
const grpFanSpread = document.getElementById('grpFanSpread');
const grpTheta = document.getElementById('grpTheta');
const grpExaggeration = document.getElementById('grpExaggeration');

const valDistG = document.getElementById('valDistG');
const valOffsetX = document.getElementById('valOffsetX');
const valDistC = document.getElementById('valDistC');
const valMass = document.getElementById('valMass');
const valNumRays = document.getElementById('valNumRays');
const valFanSpread = document.getElementById('valFanSpread');
const valTheta = document.getElementById('valTheta');
const valExaggeration = document.getElementById('valExaggeration');

const togglePhoton = document.getElementById('togglePhoton');
const toggleTangent = document.getElementById('toggleTangent');
const toggleImpact = document.getElementById('toggleImpact');
const toggleStars = document.getElementById('toggleStars');

// Calculation Box Elements

```

```

const boxDistG = document.getElementById('boxDistG');
const boxOffsetX = document.getElementById('boxOffsetX');
const boxDistC = document.getElementById('boxDistC');
const boxDistGC = document.getElementById('boxDistGC');
const boxMass = document.getElementById('boxMass');
const boxMode = document.getElementById('boxMode');
const boxImpactRange = document.getElementById('boxImpactRange');
const boxEarthImpactRange = document.getElementById('boxEarthImpactRange');
const boxAlphaRange = document.getElementById('boxAlphaRange');

let width = 0;
let height = 0;
let photonAnimProgress = 0;

// Background Stars
const stars = [];
for (let i = 0; i < 220; i++) {
  stars.push({
    x: Math.random(),
    y: Math.random(),
    size: Math.random() * 1.5 + 0.3,
    alpha: Math.random() * 0.7 + 0.3
  });
}

// Cluster Member Galaxies layout offsets (up to 30 member galaxies)
const clusterGalaxyList = [
  { x: 0, y: 0, r: 3.5 },
  { x: -6, y: -5, r: 2.2 },
  { x: 8, y: 4, r: 2.5 },
  { x: -12, y: 8, r: 2.0 },
  { x: 10, y: -10, r: 2.1 },
  { x: 5, y: -14, r: 1.8 },
  { x: -15, y: -6, r: 2.3 },
  { x: 18, y: -2, r: 1.9 },
  { x: -8, y: 16, r: 2.2 },
  { x: 14, y: 12, r: 1.7 },
  { x: 2, y: 18, r: 2.0 },
  { x: -20, y: 4, r: 1.8 },
  { x: -4, y: -20, r: 1.9 },
  { x: 22, y: -12, r: 1.6 },
  { x: 12, y: -22, r: 1.8 },
  { x: -22, y: -15, r: 1.7 },
  { x: -16, y: 22, r: 1.9 },
  { x: 25, y: 8, r: 1.5 },
  { x: 6, y: 26, r: 1.7 },
  { x: -28, y: -4, r: 1.6 },
  { x: 18, y: 22, r: 1.5 },
  { x: -10, y: -28, r: 1.8 },
  { x: 28, y: -20, r: 1.4 },
  { x: -25, y: 18, r: 1.6 },
  { x: 0, y: -32, r: 1.5 },
  { x: 32, y: 2, r: 1.4 },
  { x: -32, y: -18, r: 1.5 },
  { x: 20, y: -30, r: 1.4 },
  { x: -20, y: -32, r: 1.3 },
  { x: 15, y: 32, r: 1.4 }
];

function resizeCanvas() {
  const rect = canvas.parentElement.getBoundingClientRect();
  const dpr = window.devicePixelRatio || 1;
  width = rect.width;
  height = rect.height;

  canvas.width = width * dpr;
  canvas.height = height * dpr;
  ctx.scale(dpr, dpr);
}

window.addEventListener('resize', resizeCanvas);
resizeCanvas();

// Distance constraints: d_G >= d_C + 1.0 Gly

```

```

sliderDistG.addEventListener('input', () => {
  let dG = parseFloat(sliderDistG.value);
  let dC = parseFloat(sliderDistC.value);
  if (dG < dC + 1.0) {
    dC = Math.max(1.0, dG - 1.0);
    sliderDistC.value = dC.toFixed(1);
  }
  updateControlStates();
});

sliderDistC.addEventListener('input', () => {
  let dG = parseFloat(sliderDistG.value);
  let dC = parseFloat(sliderDistC.value);
  if (dC > dG - 1.0) {
    dG = Math.min(13.0, dC + 1.0);
    sliderDistG.value = dG.toFixed(1);
  }
  updateControlStates();
});

toggleCluster.addEventListener('change', updateControlStates);
sliderMass.addEventListener('input', updateControlStates);
sliderNumRays.addEventListener('input', updateControlStates);
toggleManualScale.addEventListener('change', updateControlStates);
sliderExaggeration.addEventListener('input', updateControlStates);

function updateControlStates() {
  const numRays = parseInt(sliderNumRays.value);
  const clusterEnabled = toggleCluster.checked;
  const manualScaleEnabled = toggleManualScale.checked;

  if (!clusterEnabled) {
    sliderMass.disabled = true;
    grpMass.classList.add('disabled-control');
    sliderDistC.disabled = true;
    grpDistC.classList.add('disabled-control');
  } else {
    sliderMass.disabled = false;
    grpMass.classList.remove('disabled-control');
    sliderDistC.disabled = false;
    grpDistC.classList.remove('disabled-control');
  }

  if (numRays > 1) {
    sliderTheta.disabled = true;
    grpTheta.classList.add('disabled-control');

    sliderFanSpread.disabled = false;
    grpFanSpread.classList.remove('disabled-control');
  } else {
    sliderTheta.disabled = false;
    grpTheta.classList.remove('disabled-control');

    sliderFanSpread.disabled = true;
    grpFanSpread.classList.add('disabled-control');
  }

  if (!manualScaleEnabled) {
    sliderExaggeration.disabled = true;
    grpExaggeration.classList.add('disabled-control');
  } else {
    sliderExaggeration.disabled = false;
    grpExaggeration.classList.remove('disabled-control');
  }
}

updateControlStates();

function calculateRays() {
  const clusterEnabled = toggleCluster.checked;
  const d_G_Gly = parseFloat(sliderDistG.value);
  const x_G_kly = parseFloat(sliderOffsetX.value);
  const d_C_Gly = parseFloat(sliderDistC.value);
  const d_GC_Gly = d_G_Gly - d_C_Gly;

```

```

const M_tril = clusterEnabled ? parseFloat(sliderMass.value) : 0;
const numRays = parseInt(sliderNumRays.value);
const fanSpreadArcsec = parseFloat(sliderFanSpread.value);
const thetaInitArcsec = parseFloat(sliderTheta.value);

const ARCSEC_TO_RAD = Math.PI / (180.0 * 3600.0);
const d_GC_ly = d_GC_Gly * 1e9;
const d_C_ly = d_C_Gly * 1e9;
const x_G_ly = x_G_kly * 1e3;

const scale_4GM_c2_ly = 0.6243363 * M_tril;

const rays = [];

for (let i = 0; i < numRays; i++) {
  let initialAngle = 0;

  if (numRays === 1) {
    initialAngle = thetaInitArcsec;
  } else {
    const frac = (i / (numRays - 1)) - 0.5;
    initialAngle = frac * fanSpreadArcsec;
  }

  const theta_init_rad = initialAngle * ARCSEC_TO_RAD;

  const b_C_ly = x_G_ly + d_GC_ly * Math.tan(theta_init_rad);
  const abs_b_C_ly = Math.abs(b_C_ly);

  let alpha_arcsec = 0;
  let alpha_rad = 0;
  if (M_tril > 0 && abs_b_C_ly > 1e-3) {
    alpha_rad = scale_4GM_c2_ly / abs_b_C_ly;
    alpha_arcsec = alpha_rad / ARCSEC_TO_RAD;
  }

  let theta_after_rad = theta_init_rad;
  if (M_tril > 0) {
    if (b_C_ly > 0) {
      theta_after_rad = theta_init_rad - alpha_rad;
    } else if (b_C_ly < 0) {
      theta_after_rad = theta_init_rad + alpha_rad;
    }
  }
  const theta_after_arcsec = theta_after_rad / ARCSEC_TO_RAD;

  const b_E_ly = b_C_ly + d_C_ly * Math.tan(theta_after_rad);
  const abs_b_E_ly = Math.abs(b_E_ly);

  rays.push({
    index: i,
    theta_init_arcsec: initialAngle,
    b_C_ly: b_C_ly,
    abs_b_C_ly: abs_b_C_ly,
    b_E_ly: b_E_ly,
    abs_b_E_ly: abs_b_E_ly,
    alpha_arcsec: alpha_arcsec,
    theta_after_arcsec: theta_after_arcsec
  });
}

return {
  clusterEnabled,
  d_G_Gly,
  x_G_kly,
  d_C_Gly,
  d_GC_Gly,
  M_tril,
  numRays,
  fanSpreadArcsec,
  thetaInitArcsec,
  rays
};
}

```



```

function draw() {
  ctx.clearRect(0, 0, width, height);

  const sim = calculateRays();

  // Post-lensing manual exaggeration scale (affects only Cluster -> Earth ray path)
  const postLensingScale = toggleManualScale.checked ? parseFloat(sliderExaggeration.value) : 1.0;

  // Label Updates
  valDistG.textContent = `${sim.d_G_Gly.toFixed(1)} billion light-years`;
  valOffsetX.textContent = `${sim.x_G_kly >= 0 ? '+' : ''}${sim.x_G_kly} thousand light-years`;
  valDistC.textContent = !sim.clusterEnabled ? "N/A" : `${sim.d_C_Gly.toFixed(1)} billion light-years`;
  valMass.textContent = !sim.clusterEnabled ? "Disabled" : `${sim.M_tril} trillion solar masses`;
  valNumRays.textContent = `${sim.numRays}`;
  valFanSpread.textContent = `${sim.fanSpreadArcsec} arc-seconds`;
  valTheta.textContent = `${sim.thetaInitArcsec >= 0 ? '+' : ''}${sim.thetaInitArcsec.toFixed(1)} arc-sec`;
  valExaggeration.textContent = `${postLensingScale.toFixed(2)}x`;

  // Update Numerical Box
  boxDistG.textContent = `${sim.d_G_Gly.toFixed(1)} billion light-years`;
  boxOffsetX.textContent = `${sim.x_G_kly >= 0 ? '+' : ''}${sim.x_G_kly} thousand light-years`;
  boxDistC.textContent = !sim.clusterEnabled ? "N/A" : `${sim.d_C_Gly.toFixed(1)} billion light-years`;
  boxDistGC.textContent = !sim.clusterEnabled ? "N/A" : `${sim.d_GC_Gly.toFixed(1)} billion light-years`;
  boxMass.textContent = !sim.clusterEnabled ? "None" : `${sim.M_tril} trillion solar masses`;

  if (sim.numRays === 1) {
    const r = sim.rays[0];
    boxMode.textContent = `Single Ray`;
    boxImpactRange.textContent = sim.clusterEnabled ? `${(r.abs_b_C_ly / 1000.0).toFixed(2)} thousand light-years` : `0.00 thousand light-years`;
    boxEarthImpactRange.textContent = `${(r.abs_b_E_ly / 1000.0).toFixed(2)} thousand light-years`;
    boxAlphaRange.textContent = sim.clusterEnabled ? `${r.alpha_arcsec.toFixed(2)} arc-seconds` : "0.00 arc-seconds";
  } else {
    boxMode.textContent = `Fan (${sim.numRays} rays)`;

    const bCValues = sim.rays.map(r => r.abs_b_C_ly / 1000.0);
    const bEValues = sim.rays.map(r => r.abs_b_E_ly / 1000.0);
    const alphaValues = sim.rays.map(r => r.alpha_arcsec).filter(a => isFinite(a));

    const minBC = Math.min(...bCValues);
    const maxBC = Math.max(...bCValues);
    const minBE = Math.min(...bEValues);
    const maxBE = Math.max(...bEValues);
    const maxAlpha = Math.max(...alphaValues);

    boxImpactRange.textContent = sim.clusterEnabled ? `${minBC.toFixed(1)} to ${maxBC.toFixed(1)} thousand light-years` : `0.00 to 0.00 thousand light-years`;
    boxEarthImpactRange.textContent = `${minBE.toFixed(1)} to ${maxBE.toFixed(1)} thousand light-years`;
    boxAlphaRange.textContent = sim.clusterEnabled ? `${maxAlpha.toFixed(2)} arc-seconds max` : "0.00 arc-seconds max";
  }

  // Visual Layout Dimensions (Vertical Field)
  const topMargin = 45;
  const bottomMargin = 55;
  const centerX = width / 2;

  const earthY = height - bottomMargin;
  const drawableHeight = height - topMargin - bottomMargin;
  const pxPerGly = drawableHeight / CANVAS_TOTAL_GLY;

  const galaxyY = earthY - (sim.d_G_Gly * pxPerGly);
  const clusterY = earthY - (sim.d_C_Gly * pxPerGly);

  // Base exaggerated scale for pre-lensing path (Galaxy -> Cluster)
  const pxPerKlyBase = 100.0 / 500.0;
  const galaxyX = centerX + (sim.x_G_kly * pxPerKlyBase);

  const ptA = { x: galaxyX, y: galaxyY };

  // Starfield Background
  if (toggleStars.checked) {
    ctx.fillStyle = "#ffffff";
    stars.forEach(s => {
      ctx.globalAlpha = s.alpha;
      ctx.fillRect(s.x * width, s.y * height, s.size, s.size);
    });
  }
}

```

```

});
ctx.globalAlpha = 1.0;
}

// Central Optical Axis
ctx.strokeStyle = "rgba(255, 255, 255, 0.15)";
ctx.lineWidth = 1;
ctx.setLineDash([6, 6]);
ctx.beginPath();
ctx.moveTo(centerX, earthY + 20);
ctx.lineTo(centerX, earthY - CANVAS_TOTAL_GLY * pxPerGly - 15);
ctx.stroke();
ctx.setLineDash([]);

// Cluster Plane Line
if (sim.clusterEnabled && sim.M_tril > 0) {
  ctx.strokeStyle = "rgba(0, 240, 255, 0.25)";
  ctx.beginPath();
  ctx.moveTo(20, clusterY);
  ctx.lineTo(width - 20, clusterY);
  ctx.stroke();
}

// Earth Plane Line
ctx.strokeStyle = "rgba(0, 240, 255, 0.15)";
ctx.beginPath();
ctx.moveTo(20, earthY);
ctx.lineTo(width - 20, earthY);
ctx.stroke();

// Render Primary Light Rays & Virtual Tangents
sim.rays.forEach((ray, idx) => {
  // Path before Cluster uses the standard exaggerated horizontal scale
  const clusterX = centerX + (ray.b_C_ly / 1000.0) * pxPerKlyBase;

  // Path after Cluster applies the manual exaggeration scale factor to the displacement
  const delta_b_kly = (ray.b_E_ly - ray.b_C_ly) / 1000.0;
  const earthX = clusterX + (delta_b_kly * pxPerKlyBase * postLensingScale);

  const ptB = { x: clusterX, y: clusterY };
  const ptC = { x: earthX, y: earthY };

  // Virtual Tangent Line
  if (toggleTangent.checked && Math.abs(ray.alpha_arcsec) > 0.1 && sim.M_tril > 0) {
    ctx.strokeStyle = "rgba(239, 68, 68, 0.5)";
    ctx.lineWidth = 1.2;
    ctx.setLineDash([3, 3]);

    const slopeX = (ptC.x - ptB.x) / (ptC.y - ptB.y);
    const virtualX_at_galaxy = ptB.x + slopeX * (galaxyY - clusterY);

    ctx.beginPath();
    ctx.moveTo(ptC.x, ptC.y);
    ctx.lineTo(virtualX_at_galaxy, galaxyY);
    ctx.stroke();
    ctx.setLineDash([]);

    ctx.fillStyle = "rgba(239, 68, 68, 0.8)";
    ctx.beginPath();
    ctx.arc(virtualX_at_galaxy, galaxyY, 3, 0, Math.PI * 2);
    ctx.fill();
  }

  // Actual Lensed Light Path
  ctx.strokeStyle = "#facc15";
  ctx.lineWidth = sim.numRays === 1 ? 2.5 : 1.8;
  ctx.shadowColor = "#facc15";
  ctx.shadowBlur = sim.numRays === 1 ? 6 : 3;

  ctx.beginPath();
  ctx.moveTo(ptA.x, ptA.y);
  if (sim.M_tril > 0) {
    ctx.lineTo(ptB.x, ptB.y);
  }
}

```

```

ctx.lineTo(ptC.x, ptC.y);
ctx.stroke();
ctx.shadowBlur = 0;

// Visual Indicators for Impact Parameters
if (toggleImpact.checked && sim.numRays === 1) {
  // Cluster Impact Parameter
  if (sim.clusterEnabled && sim.M_tril > 0 && Math.abs(ptB.x - centerX) > 3) {
    ctx.strokeStyle = "#00f0ff";
    ctx.lineWidth = 1.2;
    ctx.beginPath();
    ctx.moveTo(centerX, clusterY);
    ctx.lineTo(ptB.x, clusterY);
    ctx.stroke();

    ctx.fillStyle = "#00f0ff";
    ctx.beginPath();
    ctx.arc(ptB.x, clusterY, 2.5, 0, Math.PI * 2);
    ctx.arc(centerX, clusterY, 2.5, 0, Math.PI * 2);
    ctx.fill();

    ctx.font = "10px monospace";
    const clusterLabel = `${(ray.abs_b_C_ly / 1000).toFixed(1)} thousand light-years`;
    const textWidth = ctx.measureText(clusterLabel).width;
    const charOffset = 18;

    let labelX = 0;
    let align = "center";

    if (ray.theta_init_arcsec < 0) {
      labelX = ptB.x + charOffset;
      align = "left";
      if (labelX + textWidth > width - 10) {
        labelX = width - 10 - textWidth;
      }
      if (labelX < 10) {
        labelX = 10;
      }
    } else {
      labelX = ptB.x - charOffset;
      align = "right";
      if (labelX - textWidth < 10) {
        labelX = 10 + textWidth;
      }
      if (labelX > width - 10) {
        labelX = width - 10;
      }
    }

    ctx.textAlign = align;
    ctx.fillText(clusterLabel, labelX, clusterY - 6);
  }

  // Earth Impact Parameter
  if (Math.abs(ptC.x - centerX) > 3) {
    ctx.strokeStyle = "#00f0ff";
    ctx.lineWidth = 1.2;
    ctx.beginPath();
    ctx.moveTo(centerX, earthY);
    ctx.lineTo(ptC.x, earthY);
    ctx.stroke();

    ctx.fillStyle = "#00f0ff";
    ctx.beginPath();
    ctx.arc(ptC.x, earthY, 2.5, 0, Math.PI * 2);
    ctx.arc(centerX, earthY, 2.5, 0, Math.PI * 2);
    ctx.fill();

    ctx.font = "10px monospace";
    const earthLabel = `${(ray.abs_b_E_ly / 1000).toFixed(1)} thousand light-years`;
    const earthTextWidth = ctx.measureText(earthLabel).width;
    let earthTextX = (centerX + ptC.x) / 2;

    if (earthTextX - earthTextWidth / 2 < 10) {

```

```

    earthTextX = 10 + earthTextWidth / 2;
  } else if (earthTextX + earthTextWidth / 2 > width - 10) {
    earthTextX = width - 10 - earthTextWidth / 2;
  }

  ctx.textAlign = "center";
  ctx.fillText(earthLabel, earthTextX, earthY + 16);
}

// Animated Photons
if (togglePhoton.checked) {
  const rayOffset = (idx / sim.numRays);
  const progress = (photonAnimProgress + rayOffset) % 1.0;
  let px = 0, py = 0;

  if (sim.M_tril === 0) {
    px = ptA.x + progress * (ptC.x - ptA.x);
    py = ptA.y + progress * (ptC.y - ptA.y);
  } else {
    const d1 = Math.hypot(ptB.x - ptA.x, ptB.y - ptA.y);
    const d2 = Math.hypot(ptC.x - ptB.x, ptC.y - ptB.y);
    const dTotal = d1 + d2;
    const f1 = dTotal > 0 ? d1 / dTotal : 0.5;

    if (progress <= f1) {
      const t = f1 > 0 ? progress / f1 : 0;
      px = ptA.x + t * (ptB.x - ptA.x);
      py = ptA.y + t * (ptB.y - ptA.y);
    } else {
      const t = (1 - f1) > 0 ? (progress - f1) / (1 - f1) : 0;
      px = ptB.x + t * (ptC.x - ptB.x);
      py = ptB.y + t * (ptC.y - ptB.y);
    }
  }

  ctx.fillStyle = "#ffffff";
  ctx.shadowColor = "#00f0ff";
  ctx.shadowBlur = 6;
  ctx.beginPath();
  ctx.arc(px, py, 3, 0, Math.PI * 2);
  ctx.fill();
  ctx.shadowBlur = 0;
}
});

if (togglePhoton.checked) {
  photonAnimProgress = (photonAnimProgress + 0.006) % 1.0;
}

// Draw Celestial Objects

// 1. Distant Galaxy (Top)
ctx.save();
ctx.translate(ptA.x, ptA.y);
const galGlow = ctx.createRadialGradient(0, 0, 2, 0, 0, 18);
galGlow.addColorStop(0, "rgba(255, 200, 100, 0.9)");
galGlow.addColorStop(0.5, "rgba(238, 100, 30, 0.3)");
galGlow.addColorStop(1, "rgba(0, 0, 0, 0)");
ctx.fillStyle = galGlow;
ctx.beginPath();
ctx.arc(0, 0, 18, 0, Math.PI * 2);
ctx.fill();

ctx.fillStyle = "#ffd59e";
ctx.beginPath();
ctx.ellipse(0, 0, 12, 4, -0.3, 0, Math.PI * 2);
ctx.fill();
ctx.restore();

ctx.fillStyle = "#e1e7ec";
ctx.font = "11px sans-serif";
ctx.textAlign = "center";
ctx.fillText("Distant Galaxy", ptA.x, ptA.y - 15);

```

```

// 2. Intermediate Cluster
if (sim.clusterEnabled && sim.M_tril > 0) {
  ctx.save();
  ctx.translate(centerX, clusterY);

  const massRatio = Math.max(0, Math.min(1, (sim.M_tril - 100) / 900));
  const haloRadius = 16 + massRatio * 46;

  const clusterGlow = ctx.createRadialGradient(0, 0, 2, 0, 0, haloRadius);
  const alphaCenter = 0.5 + massRatio * 0.45;
  const alphaMid = 0.25 + massRatio * 0.55;

  clusterGlow.addColorStop(0, `rgba(216, 180, 254, ${alphaCenter})`);
  clusterGlow.addColorStop(0.35, `rgba(168, 85, 247, ${alphaMid})`);
  clusterGlow.addColorStop(0.7, `rgba(126, 34, 206, ${alphaMid * 0.5})`);
  clusterGlow.addColorStop(1, "rgba(0, 0, 0, 0)");

  ctx.fillStyle = clusterGlow;
  ctx.beginPath();
  ctx.arc(0, 0, haloRadius, 0, Math.PI * 2);
  ctx.fill();

  const numGalaxies = Math.floor(3 + massRatio * 27);
  const spreadScale = 0.6 + massRatio * 0.6;

  for (let i = 0; i < numGalaxies; i++) {
    const g = clusterGalaxyList[i];
    const gx = g.x * spreadScale;
    const gy = g.y * spreadScale;

    ctx.fillStyle = i === 0 ? "#ffffff" : `rgba(255, 245, 255, ${0.75 + (1 - i / 30) * 0.25})`;
    ctx.beginPath();
    ctx.arc(gx, gy, g.r, 0, Math.PI * 2);
    ctx.fill();

    if (g.r >= 2.0) {
      ctx.fillStyle = `rgba(192, 132, 252, ${0.3 + massRatio * 0.3})`;
      ctx.beginPath();
      ctx.arc(gx, gy, g.r * 1.8, 0, Math.PI * 2);
      ctx.fill();
    }
  }

  ctx.restore();

  ctx.fillStyle = "#e1e7ec";
  ctx.font = "11px sans-serif";
  ctx.textAlign = "center";
  ctx.fillText("Intermediate Cluster", centerX, clusterY + haloRadius + 14);
}

// 3. Earth / Observer (Bottom)
ctx.save();
ctx.translate(centerX, earthY);
const earthGrad = ctx.createRadialGradient(-2, -2, 1, 0, 0, 8);
earthGrad.addColorStop(0, "#60a5fa");
earthGrad.addColorStop(0.8, "#1d4ed8");
earthGrad.addColorStop(1, "#1e3a8a");
ctx.fillStyle = earthGrad;
ctx.beginPath();
ctx.arc(0, 0, 8, 0, Math.PI * 2);
ctx.fill();
ctx.restore();

ctx.fillStyle = "#e1e7ec";
ctx.font = "11px sans-serif";
ctx.textAlign = "center";
ctx.fillText("Earth", centerX, earthY + 26);

// Bottom Note inside Graphic Window
ctx.fillStyle = "rgba(140, 155, 174, 0.75)";
ctx.font = "italic 11px sans-serif";
ctx.textAlign = "center";

```

```

    ctx.fillText("angles are exaggerated", centerX, height - 12);
}

function animate() {
    draw();
    requestAnimationFrame(animate);
}

animate();
</script>
</body>
</html>

```

User prompt: I would like to download a record of my prompts and your typed responses, but not the HTML+Javascript code outputs. Is that possible?

Response: There is no single built-in button in the interface that automatically exports your conversation history while filtering out code blocks. Standard export features (like *Export to Docs* or downloading data via *Google Takeout*) will include the entire output, including the HTML/JavaScript blocks.

However, **you can easily achieve this using a few practical workarounds:**

Option 1: Have Me Generate the Text Transcript (Easiest)

Since I have access to our current conversation transcript, I can compile and format a clean record of your prompts and my conversational responses, excluding the code blocks.

If you would like me to do this right now, just reply with **"Generate transcript"**, and I will output it in text format for you to copy and save.

Option 2: Use a Browser Console Script (Automatic Download)

If you want to quickly extract text from the page in your browser without code blocks, you can run a simple 2-line snippet in your browser's Developer Tools:

1. Press **F12** (or right-click anywhere on the page and select **Inspect**).
2. Click the **Console** tab at the top.
3. Copy and paste the following snippet, then press **Enter**:

```

// Copy all conversation text except code blocks to clipboard
let clone = document.body.cloneNode(true);
clone.querySelectorAll('pre, code').forEach(el => el.remove());
navigator.clipboard.writeText(clone.innerText);
alert("Conversation text (without code) copied to clipboard!");

```

This strips out all `<pre>` and `<code>` blocks from the current page view and copies all the text directly to your clipboard so you can paste it into Notepad, Word, or Google Docs.

Option 3: Clean an Exported File

If you use the built-in **Export to Google Docs** feature:

1. Export the chat to Google Docs.
 2. Open the document and use **Find and Replace** (Ctrl+H or Cmd+H).
 3. If you want to remove code blocks manually, you can select and delete the gray code containers, or use a short script if it's a long document.
-

User prompt: Generate transcript

Response: Here is the clean text transcript of our conversation up to this point, with all HTML/JavaScript code blocks removed:

Conversation Transcript

User: L

Gemini: It looks like that message was cut short. How would you like to adjust or expand the gravitational lensing simulation?

User: Add two input elements:

1. a toggle for enabling manual scaling of the exaggeration factor that is used to display the rays after lensing (default to off)

2. a slider for increasing or decreasing that exaggeration factor.

Continue with the current default approach for drawing the rays from the lensing cluster to the Earth's y-position. However, when the manual scaling toggle is activated, it allows the user to scale the displayed impact parameter (values from 0.01 to 1), thus causing the rays to appear at a less extreme angle.

Gemini: [HTML/JavaScript Code Output]

User: apply the manual exaggeration only to the ray after it passes by the cluster, not before. I want to keep the normal exaggerated appearance of the ray on the path from the distant galaxy to the cluster.

Gemini: [HTML/JavaScript Code Output]

User: I would like to download a record of my prompts and your typed responses, but not the HTML+Javascript code outputs. Is that possible?

Gemini: There is no single built-in button in the interface that automatically exports your conversation history while filtering out code blocks. Standard export features (like Export to Docs or downloading data via Google Takeout) will include the entire output, including the HTML/JavaScript blocks.

However, **you can easily achieve this using a few practical workarounds:**

Option 1: Have Me Generate the Text Transcript (Easiest)

Since I have access to our current conversation transcript, I can compile and format a clean record of your prompts and my conversational responses, excluding the code blocks.

If you would like me to do this right now, just reply with **"Generate transcript"**, and I will output it in text format for you to copy and save.

Option 2: Use a Browser Console Script (Automatic Download)

If you want to quickly extract text from the page in your browser without code blocks, you can run a simple 2-line snippet in your browser's Developer Tools:

1. Press **F12** (or right-click anywhere on the page and select **Inspect**).
2. Click the **Console** tab at the top.
3. Copy and paste the following snippet, then press **Enter**:

```
// Copy all conversation text except code blocks to clipboard
let clone = document.body.cloneNode(true);
clone.querySelectorAll('pre, code').forEach(el => el.remove());
navigator.clipboard.writeText(clone.innerText);
alert("Conversation text (without code) copied to clipboard!");
```

This strips out all `<pre>` and `<code>` blocks from the current page view and copies all the text directly to your clipboard so you can paste it into Notepad, Word, or Google Docs.

Option 3: Clean an Exported File

If you use the built-in **Export to Google Docs** feature:

1. Export the chat to Google Docs.
2. Open the document and use **Find and Replace** (Ctrl+H or Cmd+H).
3. If you want to remove code blocks manually, you can select and delete the gray code containers, or use a short script if it's a long document.

User: Generate transcript
